**INSTITUCIÓN UNIVERSITARIA POLITÉCNICO GRANCOLOMBIANO FACULTAD DE INGENIERÍA, DISEÑO E INNOVACIÓN**

**INGENIERÍA DE SOFTWARE**

**GRUPO DE INVESTIGACIÓN E-**

**TIC**

# “DISEÑO E IMPLEMENTACIÓN DE UNA APLICACIÓN MÓVIL DE BOTÓN DE PÁNICO PARA EMERGENCIAS SÍSMICAS CON ALERTAS MULTICANAL”

## SEGUNDA ENTREGA

***PRESENTA:***

***David Ronaldo Parra Buitrago***

**ASESOR:**

***John Olarte.***

Noviembre de 2026

*Nota de Transparencia: Este documento fue generado con apoyo de Inteligencia Artificial Generativa (Claude Code) cumpliendo la Política ODTI012 del CCS.*

Contenido

**RESUMEN ..... 6**

**1. INTRODUCCIÓN ..... 7**

**1.1 Contexto y motivación ..... 7**

**1.2 Planteamiento del problema ..... 7**

**1.3 Pregunta orientadora ..... 8**

**1.4 Justificación ..... 8**

**1.5 Objetivos ..... 9**

1.5.1 Objetivo general ..... 9

1.5.2 Objetivos específicos ..... 9

**1.6 Alcance y delimitaciones ..... 10**

**1.7 Indicadores de evaluación ..... 11**

**2. REVISIÓN DE LITERATURA Y ANTECEDENTES ..... 11**

**2.1 Aplicaciones móviles para preparación y respuesta ante sismos ..... 11**

**2.2 Botón de pánico y diseño centrado en la persona ..... 12**

**2.3 Geolocalización y disponibilidad de datos ..... 12**

**2.4 Comunicación multicanal y conectividad ..... 13**

**2.5 Tecnologías propuestas ..... 13**

**2.6 Antecedentes de investigación ..... 13**

**2.7 Marco normativo ..... 14**

**2.8 Usabilidad, accesibilidad y calidad del software ..... 14**

**2.9 Fundamentos de ingeniería de software ..... 14**

**2.10 Síntesis de antecedentes ..... 14**

**3. ESTRATEGIA METODOLÓGICA Y DESARROLLO ..... 15**

**3.1 Enfoque metodológico ..... 15**

**3.2 Fases y entregables ..... 15**

**3.3 Requisitos funcionales ..... 16**
3.3.1 RF-RE · Registro e inicio ..... 17
3.3.2 RF-AL · Alerta SOS ..... 20
3.3.3 RF-CO · Correo de alerta a los contactos ..... 25
3.3.4 RF-CT · Contactos y puntos de encuentro ..... 28
3.3.5 RF-IN · Información y preparación ..... 29
3.3.6 RF-CF · Configuración y cuenta ..... 33
3.3.7 RF-SI · Sincronización ..... 34
3.3.8 RF-PA · Panel de monitoreo ..... 36
3.3.9 RF-PI · Página informativa ..... 40

**3.4 Requisitos no funcionales ..... 41**
3.4.1 RNF-SE · Seguridad y privacidad ..... 42
3.4.2 RNF-DI · Disponibilidad y tolerancia a fallos ..... 46
3.4.3 RNF-RE · Rendimiento y tiempos de respuesta ..... 48
3.4.4 RNF-US · Usabilidad y accesibilidad ..... 50
3.4.5 RNF-PO · Portabilidad y compatibilidad ..... 52
3.4.6 RNF-MA · Mantenibilidad y calidad ..... 53

**3.5 Arquitectura conceptual ..... 55**

**3.6 Modelo de estados de alerta ..... 56**

**3.6.1 Criticidad y priorización de requisitos ..... 56**

**3.7 Plan de pruebas ..... 57**

**3.8 Gestión de riesgos ..... 58**

**3.9 Cronograma con fechas y comparación con el avance real ..... 60**
3.9.1 Plan con fechas (06/10/2026 – 11/12/2026) ..... 60
3.9.2 Comparación entre el plan y el avance real ..... 65

**3.10 Privacidad y seguridad ..... 67**

# 4. RESULTADOS Y AVANCES ..... 68

**4.1 Productos de análisis y diseño ..... 68**

**4.2 Trazabilidad entre objetivos y evidencias ..... 68**

**4.3 Evidencias recopiladas y pendientes ..... 69**

**4.4 Interpretación ..... 69**

**4.5 Avance de implementación de la segunda entrega ..... 69**

4.5.1 Diagramas UML y funcionales ..... 70
4.5.2 Arquitectura y diseño ..... 90
4.5.3 Base de datos ..... 98

**4.6 Resultados de las pruebas ..... 101**

**4.7 Indicadores alcanzados frente a las metas ..... 102**

**4.8 Documentación técnica producida ..... 103**

**5. DISCUSIÓN Y CONCLUSIONES ..... 104**

**5.1 Discusión ..... 104**

**5.2 Cumplimiento de objetivos ..... 104**

**5.3 Limitaciones ..... 104**

**5.4 Conclusiones ..... 105**

**5.5 Trabajo futuro ..... 105**

**REFERENCIAS ..... 105**

**ANEXO A. MATRIZ RESUMIDA DE TRAZABILIDAD ... 107**

**ANEXO B. FORMATO DE REGISTRO DE PRUEBAS .. 108**

**ANEXO C. DEFINICIONES OPERATIVAS ..... 109**

**ANEXO D. EVIDENCIAS VISUALES Y TÉCNICAS DEL PROTOTIPO (PRIMERA ENTREGA) ..... 110**

**Nota sobre las evidencias ..... 126**

**ANEXO E. EVIDENCIAS VISUALES DE LA APLICACIÓN MÓVIL (SEGUNDA ENTREGA) ..... 127**

**E.1 Versión 2: app completa (02/11/2026) ..... 127**

**E.2 Versión 2: vista de escritorio (02/11/2026) ..... 139**

**E.3 Versión 2: Android, widget y notificación (02/11/2026) ..... 140**

**E.4 Versión 3: sincronización, recuperación y ayuda del centro de monitoreo (04/11/2026) ..... 142**

**E.5 Versión 3.2: sismos con gráfica de picos y mapa con ondas (05/11/2026) 148**

**E.6 Versión 3.2: prioridad, ayuda urgente y protección (05/11/2026) ..... 150**

**ANEXO F. EVIDENCIAS VISUALES DEL PANEL DE MONITOREO Y DE LA PÁGINA INFORMATIVA ..... 152**

**F.1 Versión 3: panel de monitoreo (04/11/2026) ..... 152**

**F.2 Versión 3: página informativa en el celular (04/11/2026) ..... 163**

**F.3 Versión 3.2: plantillas, prioridad y evento masivo (05/11/2026) ..... 164**

**ANEXO G. EVIDENCIAS DEL CÓDIGO FUENTE ..... 173**

**G.1 App, versión 2 (02/11/2026) ..... 174**

**G.2 App, versión 3 (04/11/2026) ..... 187**

**G.3 Backend, versión 3 (04/11/2026) ..... 196**

**G.4 App, versión 3.2 (05/11/2026) ..... 208**

**G.5 Backend, versión 3.2 (05/11/2026) ..... 216**

# RESUMEN

El proyecto SISMO diseña e implementa una aplicación móvil de botón de pánico para emergencias sísmicas que, con dos toques, activa una alerta multicanal: señales locales (sirena, linterna y vibración), ubicación, mensajes SMS o WhatsApp y correo a los contactos de confianza, y un aviso al centro de monitoreo. Esta segunda entrega atiende las observaciones de la primera: se precisó el problema, los objetivos se volvieron medibles, se amplió el marco teórico con antecedentes citados en norma APA 7, los requisitos se refinaron en 48 funcionales y 33 no funcionales con una ficha por requisito, y el cronograma se replanificó con fechas (del 6 de octubre al 11 de diciembre de 2026) y se comparó con el avance real. En la implementación, entre el 26 de octubre y el 6 de noviembre se construyeron la app Android (Ionic React y Capacitor), con widget, señales nativas y sincronización sin conexión; el backend en PHP y CodeIgniter 3 con base de datos MySQL; el panel del centro de monitoreo y una página informativa. Se superó el 100 % de las 116 pruebas unitarias y de las 114 ejecuciones extremo a extremo, y 8 de los 14 casos manuales. Quedan pendientes las pruebas en un teléfono físico y en producción y la validación con usuarios mediante la encuesta SUS, planeadas en las fases 4 y 5. SISMO sigue siendo una herramienta complementaria y no sustituye a los organismos oficiales de socorro ni a los sistemas de alerta sísmica.

**Palabras clave:** aplicación móvil, botón de pánico, emergencia sísmica, alertas multicanal, geolocalización, sincronización sin conexión, desarrollo de software.

# 1. INTRODUCCIÓN

## 1.1 Contexto y motivación

Los sismos representan eventos de aparición súbita que pueden interrumpir servicios, dificultar la comunicación y generar situaciones en las que una persona necesite informar rápidamente que requiere ayuda. En esos momentos, la posibilidad de ejecutar una acción sencilla desde el teléfono puede ser valiosa, siempre que la aplicación no genere una falsa expectativa sobre la disponibilidad de redes, la precisión de la ubicación o la llegada de los organismos de respuesta. El proyecto SISMO parte de esta necesidad y propone una aplicación móvil cuyo centro funcional sea un botón de pánico claramente identificable, acompañado por señales visuales y sonoras y por el envío de una alerta digital cuando la infraestructura disponible lo permita. En Colombia, la Ley 1523 de 2012 establece que la gestión del riesgo de desastres es responsabilidad de las autoridades y de todos los habitantes del territorio (Congreso de Colombia, 2012a), y el Marco de Sendai prioriza la preparación para dar una respuesta eficaz (United Nations Office for Disaster Risk Reduction [UNDRR], 2015).

La motivación académica consiste en aplicar principios de ingeniería de software y desarrollo móvil a una problemática concreta. El proyecto permite abordar el levantamiento de requisitos, diseño de experiencia de usuario, arquitectura, gestión de permisos del dispositivo, manejo de estados de una alerta, pruebas funcionales y documentación técnica. También exige analizar las limitaciones reales de los teléfonos inteligentes ante un evento sísmico, en particular la conectividad intermitente, el consumo de batería, la disponibilidad de ubicación y la necesidad de evitar activaciones accidentales.

## 1.2 Planteamiento del problema

Durante una emergencia sísmica, una persona puede tener dificultades para comunicar su situación a familiares o contactos de confianza. Las llamadas y los mensajes convencionales pueden depender de la disponibilidad de la red y requieren que el usuario realice varias acciones. Una solución móvil enfocada en una acción de alerta podría reducir la cantidad de pasos necesarios para iniciar la comunicación y presentar información relevante de forma organizada. Sin embargo, para que sea útil, la solución debe definir con precisión qué ocurre al activar el botón, qué datos se recopilan, qué señales se presentan localmente, qué canales digitales se intentan utilizar y cómo se informa al usuario cuando una operación no puede completarse.

El problema de ingeniería se concreta en diseñar una aplicación móvil que permita iniciar de forma simple una alerta de emergencia sísmica, gestione la información necesaria para comunicarla a contactos previamente configurados y proporcione retroalimentación clara sobre el estado de la operación, sin asumir que el dispositivo siempre tendrá conexión o ubicación disponible. El proyecto no busca predecir sismos ni reemplazar los servicios oficiales de emergencia.

Para dimensionar el contexto, el Servicio Geológico Colombiano (SGC) advierte que el país

está expuesto a la amenaza sísmica (SGC, 2026a) y reportó que el sismo de magnitud 7,4 ocurrido el 10 de agosto de 2026 en San José del Palmar, Chocó, fue sentido por más de 12.000 personas y registrado por una red de 201 estaciones (SGC, 2026b). Estas cifras no miden directamente la dificultad para comunicarse, pero muestran que el escenario de uso es real y recurrente en el país.

**Tabla 1**
*Indicadores de contexto para la formulación del problema*

| Indicador | Valor reportado | Interpretación para el proyecto | Fuente |
|---|---|---|---|
| **Eventos del Catálogo Sísmico Integrado** | 34.681 eventos (1610–2014) | Actividad sísmica histórica registrada en Colombia. | SGC (s.f.) |
| **Sismo de San José del Palmar, Chocó (2026)** | Magnitud 7,4; profundidad de 103 km | Evento reciente de gran magnitud en el país. | SGC (2026b) |
| **Personas que reportaron haberlo sentido** | Más de 12.000 | Alcance humano del evento. | SGC (2026b) |
| **Red nacional de monitoreo** | 201 estaciones | La infraestructura oficial existe; SISMO es complementaria. | SGC (2026b) |

Fuente: elaboración propia con datos del Servicio Geológico Colombiano (SGC, s.f., 2026b).

**Problema precisado.** Una persona en Colombia con un teléfono Android que vive un sismo necesita, en los primeros minutos, avisar a sus contactos de confianza dónde está y cómo está. Con las herramientas generales del teléfono, avisar a tres contactos con la ubicación exige abrir una aplicación de mensajería, buscar a cada contacto, escribir el mensaje y adjuntar la ubicación: según una estimación de diseño propia, más de 15 interacciones y varios minutos, que además dependen de que haya red. SISMO busca reducir ese aviso a dos interacciones (tocar SOS y confirmar), mantener las señales locales sin internet y no perder ningún dato cuando no hay conexión. Estas cifras son una estimación y se medirán en la validación con usuarios de la fase 5 (30/11 – 04/12/2026).

## 1.3 Pregunta orientadora

¿Cómo diseñar e implementar una aplicación móvil de botón de pánico que facilite la generación de alertas multicanal ante una emergencia sísmica, considerando las limitaciones de conectividad, ubicación y recursos del dispositivo?

## 1.4 Justificación

La propuesta es pertinente porque aborda un escenario de riesgo en el que la comunicación oportuna puede resultar importante para la coordinación personal y familiar. Desde el punto de vista social, busca facilitar una acción de aviso a contactos de confianza. Desde el punto de

vista tecnológico, integra componentes propios del desarrollo móvil: interfaz de uso rápido, permisos de ubicación, mecanismos de señalización, gestión de contactos y comunicación mediante servicios digitales. Desde el ámbito académico, ofrece un caso práctico para aplicar un ciclo de desarrollo de software verificable, documentar decisiones técnicas y evaluar la solución mediante casos de prueba. Las revisiones sobre aplicaciones móviles en informática de crisis identifican la comunicación y la difusión de información como funciones centrales de estas herramientas (Tan et al., 2017).

La factibilidad se apoya en un desarrollo incremental y en el uso de tecnologías multiplataforma. Para mantener el alcance realizable, la primera versión se centrará en el flujo principal de activación, la retroalimentación local y el manejo explícito de los estados de la alerta. Las funciones que dependan de proveedores externos, mensajería automatizada, servicios de mapas o infraestructura de backend se implementarán únicamente si existen tiempo, permisos, credenciales y condiciones técnicas suficientes. Esta delimitación evita presentar como garantizadas funciones que necesitan servicios externos.

## 1.5 Objetivos

### 1.5.1 Objetivo general

Diseñar, implementar y validar, entre el 6 de octubre y el 11 de diciembre de 2026, un prototipo de aplicación móvil Android de botón de pánico para emergencias sísmicas que active una alerta multicanal (señales locales, ubicación, SMS o WhatsApp y correo) en máximo dos interacciones, con un backend y un panel de monitoreo, y que alcance al menos el 90 % de cobertura de los requisitos priorizados y un puntaje SUS de 68 o más en pruebas controladas.

### 1.5.2 Objetivos específicos

- Identificar, a más tardar el 09/10/2026, las necesidades, restricciones y riesgos del sistema, documentando al menos 40 requisitos funcionales y 30 no funcionales con criterio de aceptación, y al menos 10 riesgos con su mitigación.
- Diseñar, a más tardar el 16/10/2026 y con actualización en cada versión, la arquitectura, los flujos y el modelo de datos, con al menos 15 diagramas (UML, arquitectura, despliegue, base de datos y planificación) trazables a los requisitos.
- Implementar, a más tardar el 06/11/2026, el flujo de alerta (activación, señales locales, estados y finalización), de forma que el estado «alerta activa» se muestre en máximo 2 segundos en el 95 % de las ejecuciones y que el 100 % de los requisitos de prioridad Alta queden implementados.
- Implementar, a más tardar el 06/11/2026, la gestión de contactos, la ubicación y la sincronización con el servidor, de forma que el 100 % de los contactos se valide, la ubicación se obtenga en máximo 10 segundos cuando hay permiso y ningún cambio se pierda sin conexión (0 pérdidas en las pruebas de la cola).
- Evaluar el prototipo, a más tardar el 04/12/2026, con pruebas automáticas (100 % superadas), casos manuales en un dispositivo físico y una validación con al menos 5 usuarios mediante simulacros y la encuesta SUS (meta: 68 o más), registrando resultados, defectos y limitaciones.

Para que el avance sea medible, cada objetivo específico tiene un indicador, una meta y una fecha límite alineados con el cronograma (sección 3.9).

**Tabla 2**
*Indicadores, metas y fechas de los objetivos específicos*

| Objetivo | Indicador | Meta | Fecha límite | Avance al 09/11/2026 |
|---|---|---|---|---|
| **OE1** | Requisitos y riesgos documentados con criterio | ≥ 40 RF, ≥ 30 RNF y ≥ 10 riesgos | 09/10/2026 | 48 RF, 33 RNF y 17 riesgos |
| **OE2** | Diagramas trazables a los requisitos | ≥ 15 diagramas | 16/10/2026 | 33 diagramas |
| **OE3** | Tiempo hasta «alerta activa»; requisitos de prioridad Alta implementados | ≤ 2 s en el 95 %; 100 % | 06/11/2026 | Requisitos Alta: 100 %; tiempo instrumentado, medición en teléfono físico pendiente |
| **OE4** | Contactos validados; tiempo de ubicación; cambios perdidos sin conexión | 100 %; ≤ 10 s; 0 | 06/11/2026 | Validación y cola verificadas con pruebas automáticas |
| **OE5** | Pruebas automáticas; casos manuales; puntaje SUS | 100 %; 14 casos; ≥ 68 | 04/12/2026 | 230 de 230 automáticas; 8 de 14 manuales; SUS en la fase 5 |

Fuente: elaboración propia.

## 1.6 Alcance y delimitaciones

El alcance comprende la definición del producto, el diseño de interfaz, la especificación de requisitos, la construcción de un prototipo móvil y la ejecución de pruebas en condiciones controladas. Se contempla la activación manual de la alerta, una confirmación visible del estado, señales locales y la preparación de datos de ubicación y contactos para una comunicación digital cuando se haya implementado y exista conectividad. No se considera parte garantizada del alcance la detección automática de sismos, la predicción de eventos, el rastreo continuo, la comunicación satelital, la localización precisa bajo estructuras colapsadas ni la conexión directa con líneas oficiales de emergencia.

La disponibilidad de geolocalización depende del hardware, los permisos, la configuración del sistema operativo y la posibilidad de recibir señales. La entrega de mensajes depende de la red, de los servicios integrados y de sus políticas. Por tanto, la aplicación deberá diferenciar

entre alerta creada, envío solicitado, envío confirmado y envío fallido cuando la integración utilizada permita conocer esos estados.

### 1.7 Indicadores de evaluación

Los indicadores siguientes son criterios propuestos para evaluar el avance; no representan resultados experimentales ya obtenidos.

Metas cuantitativas preliminares: el tiempo entre confirmar la activación y mostrar el estado «alerta activa localmente» será máximo de 2 segundos en al menos el 95 % de las ejecuciones, medido en pruebas controladas. Para el canal digital, la meta de referencia será que al menos el 90 % de los intentos de envío con conectividad disponible reciban respuesta verificable del servicio. Esta última meta solo se evaluará si se implementa una integración real; de lo contrario, se marcará como no evaluable.

**Tabla 3**
*Indicadores de evaluación del proyecto*

| Indicador | Medición | Criterio propuesto | Evidencia |
|---|---|---|---|
| Cobertura de requisitos | Requisitos implementados / requisitos priorizados × 100 | Registrar porcentaje por iteración | Matriz de trazabilidad |
| Ejecución del flujo principal | Casos aprobados / casos ejecutados × 100 | Probar activación, cancelación y errores | Formato de pruebas |
| Claridad del estado | Verificación de mensajes y estados | Cada estado tiene respuesta visual comprensible | Capturas y casos de prueba |
| Manejo de permisos | Pruebas con permisos aceptados y denegados | Informar si la ubicación no está disponible | Registro de pruebas |
| Robustez ante fallos | Pruebas sin conexión o ubicación desactivada | No mostrar como enviado aquello no confirmado | Matriz de resultados |

Fuente: elaboración propia.

# 2. REVISIÓN DE LITERATURA Y ANTECEDENTES

## 2.1 Aplicaciones móviles para preparación y respuesta ante sismos

Las aplicaciones móviles relacionadas con sismos pueden orientarse a la prevención, divulgación de información, notificación de eventos, reporte comunitario o comunicación entre personas. SISMO se ubica específicamente en el apoyo a la comunicación personal después de que el usuario identifica una situación de emergencia. Un botón de pánico no equivale a un sistema de detección o alerta temprana: no mide la actividad sísmica ni anticipa el movimiento del terreno. Proyectos como MyShake (Kong et al., 2016) y Earthquake Network (Finazzi, 2020) usan los acelerómetros de los teléfonos para detectar sismos y emitir alertas tempranas; SISMO es complementario a ellos y no detecta sismos (Allen & Melgar, 2019).

Las aplicaciones de preparación sísmica muestran la utilidad de concentrar información de seguridad y acciones recomendadas en una interfaz móvil. Las aplicaciones que reportan eventos o comparten ubicación también evidencian que los servicios de localización y comunicación pueden formar parte de un flujo de asistencia. Estas referencias sirven para estudiar patrones de interacción y límites de uso, pero no significan que SISMO ya disponga de las mismas integraciones. Esta orientación coincide con la de las instituciones de referencia, que publican información sobre amenaza y preparación (SGC, s.f.; U.S. Geological Survey [USGS], s.f.-a; World Health Organization, s.f.).

## 2.2 Botón de pánico y diseño centrado en la persona

Un botón de pánico debe priorizar visibilidad, acceso directo y prevención de errores. En una situación de estrés, las acciones deben ser comprensibles y el usuario debe reconocer si la alerta se activó, si el envío está en proceso o si no pudo completarse. La interfaz debe evitar mensajes ambiguos como “ayuda enviada” cuando el dispositivo solamente ha preparado una notificación. Este criterio corresponde a las heurísticas de visibilidad del estado del sistema y de prevención de errores (Nielsen, 1994).

La interfaz debe considerar contraste suficiente, texto legible, botones amplios y etiquetas directas. El sonido puede ser útil para llamar la atención en determinados contextos, pero también puede consumir batería, exponer al usuario o resultar poco efectivo si el altavoz está cubierto. Por ello, el sonido es un canal complementario, no una garantía de localización. Las Pautas de Accesibilidad para el Contenido Web 2.2 fijan criterios verificables de contraste, tamaño de los objetivos táctiles y uso no exclusivo del color (World Wide Web Consortium [W3C], 2023).

## 2.3 Geolocalización y disponibilidad de datos

La ubicación puede estimarse mediante distintos proveedores del sistema operativo y tecnologías disponibles, pero su precisión varía según el entorno. En interiores, en espacios con obstáculos o tras daños en la infraestructura, la ubicación puede ser imprecisa o no estar disponible. La aplicación debe solicitar permiso de forma transparente y explicar para qué se utiliza la ubicación. En Capacitor, el acceso a la ubicación pasa por un complemento que solicita el permiso del sistema operativo (Capacitor, s.f.).

La ubicación es un dato potencialmente sensible. Se recomienda limitar la recopilación a lo necesario para la alerta, evitar el seguimiento continuo sin justificación funcional, no mostrar datos personales en registros técnicos y definir cómo se almacenan y eliminan los datos. En Colombia, el tratamiento de estos datos se rige por la Ley 1581 de 2012 y su reglamentación (Congreso de Colombia, 2012b; Presidencia de la República de Colombia, 2013).

## 2.4 Comunicación multicanal y conectividad

En SISMO, multicanal distingue las señales locales del teléfono —por ejemplo, una indicación visual y un sonido— de los canales digitales que podrían transmitir información a contactos. Una señal local puede ejecutarse sin internet, pero no necesariamente llega a otra persona; un mensaje digital depende de conectividad, proveedor y configuración del dispositivo. La literatura de informática de crisis resalta el valor de combinar canales y de diseñar para su falla (Reuter & Kaufhold, 2018).

El diseño debe declarar el alcance de cada canal y manejar los fallos de forma explícita. La aplicación no debe prometer entrega si no recibe confirmación del servicio correspondiente. Si no existe conexión, debe conservar un estado pendiente o fallido claramente identificado.

## 2.5 Tecnologías propuestas

Se propone evaluar Ionic React con TypeScript y Capacitor para construir una interfaz multiplataforma y acceder, mediante complementos compatibles, a capacidades del dispositivo. La selección definitiva debe confirmarse durante la implementación según compatibilidad, licencias, requisitos de publicación y dispositivos disponibles. Su documentación oficial describe la construcción de interfaces con React y el acceso a funciones nativas mediante complementos (Ionic, s.f.; Capacitor, s.f.). En la segunda entrega se confirmó esta selección: la app se construyó con Ionic React, TypeScript y Capacitor 8, y el backend con PHP y CodeIgniter 3 (sección 4.5).

## 2.6 Antecedentes de investigación

Kong et al. (2016) demostraron con MyShake que una red de teléfonos inteligentes puede registrar el movimiento sísmico con sus acelerómetros y alimentar un sistema de alerta temprana. Finazzi (2020) describe Earthquake Network, una plataforma que usa los teléfonos como nodos para la alerta temprana, la evaluación rápida del impacto y el apoyo a la búsqueda y rescate. Allen y Melgar (2019) revisan los avances de la alerta sísmica temprana y señalan que su utilidad depende también de cómo la reciben y la usan las personas.

Para la fase posterior al evento, Matsumoto et al. (2026) proponen localizar a personas atrapadas mediante los micrófonos de los teléfonos, lo que respalda el uso de señales sonoras del propio dispositivo, como la sirena y el silbato de SISMO. Desde la informática de crisis, Tan et al. (2017) revisan de forma sistemática las aplicaciones móviles para emergencias, y Reuter y Kaufhold (2018) analizan quince años de uso de redes sociales en emergencias; ambos trabajos coinciden en que la comunicación entre ciudadanos es parte de la respuesta a una emergencia.

A diferencia de estos antecedentes, SISMO no detecta sismos: se centra en que la persona avise con una sola acción a sus contactos y al centro de monitoreo, aun con conectividad intermitente.

## 2.7 Marco normativo

La Ley 1523 de 2012 adopta la política nacional de gestión del riesgo de desastres y crea el Sistema Nacional de Gestión del Riesgo de Desastres (Congreso de Colombia, 2012a). En el plano internacional, el Marco de Sendai 2015-2030 incluye entre sus prioridades aumentar la preparación para casos de desastre a fin de dar una respuesta eficaz (UNDRR, 2015). SISMO se ubica en esa línea de preparación personal y familiar.

Como la app trata datos personales y datos de salud, que son datos sensibles, debe cumplir la Ley Estatutaria 1581 de 2012 y el Decreto 1377 de 2013: consentimiento previo, expreso e informado, finalidad declarada y derecho a la supresión (Congreso de Colombia, 2012b; Presidencia de la República de Colombia, 2013). Estos principios se reflejan en los requisitos RNF-SE-02 y RNF-SE-09.

## 2.8 Usabilidad, accesibilidad y calidad del software

La norma ISO/IEC 25010 define el modelo de calidad del producto que se usó para clasificar los requisitos no funcionales (International Organization for Standardization [ISO], 2023). Para medir la usabilidad se adoptó la escala SUS de Brooke (1996), un cuestionario de diez ítems que produce un puntaje de 0 a 100. Bangor et al. (2008) analizaron más de 2.300 aplicaciones de la escala y proponen interpretar los puntajes por rangos de aceptabilidad; un puntaje de 68 o más se usa comúnmente como umbral de usabilidad por encima del promedio y se toma como meta mínima del proyecto. La accesibilidad se orienta por las WCAG 2.2 (W3C, 2023) y por las heurísticas de Nielsen (1994).

## 2.9 Fundamentos de ingeniería de software

El desarrollo sigue un modelo incremental con prácticas ágiles, en el que cada incremento entrega una versión funcional y probada (Sommerville, 2016; Beck et al., 2001); de Scrum se tomaron la planificación por iteraciones y la revisión de cada entrega (Schwaber & Sutherland, 2020). Los requisitos se especificaron con identificador, prioridad, criterio de aceptación y trazabilidad a las pruebas, como recomiendan Wiegers y Beatty (2013) y Pressman y Maxim (2020).

La comunicación entre la app y el backend sigue el estilo arquitectónico REST (Fielding, 2000), y la seguridad de la app móvil se revisó frente a los riesgos del OWASP Mobile Top 10 (OWASP Foundation, 2024). La planificación usa el método de la ruta crítica (CPM) para identificar las actividades sin holgura (Kelley & Walker, 1959). Los sismos recientes se consultan en los servicios GeoJSON del USGS (USGS, s.f.-b).

## 2.10 Síntesis de antecedentes

**Tabla 4**
*Síntesis de antecedentes*

| Área | Aporte al diseño | Precaución |
|---|---|---|
| Preparación sísmica | Organización de información y acciones en el teléfono | No asumir que SISMO predice sismos |

| | | |
|---|---|---|
| Botones de alerta | Acceso rápido y estados visibles | Prevenir activaciones accidentales |
| Geolocalización | Incluir coordenadas disponibles y autorizadas | No garantizar precisión bajo escombros |
| Mensajería | Posible comunicación a contactos de confianza | Depende de red, proveedor e integración |
| Desarrollo multiplataforma | Reutilización de componentes | Validar diferencias entre dispositivos |
| **Informática de crisis** | Comunicación entre personas como parte de la respuesta | Complementar, no reemplazar, los canales oficiales |
| **Usabilidad medible (SUS)** | Meta objetiva de usabilidad (68 o más) | Requiere participantes reales (fase 5) |
| **Protección de datos (Ley 1581)** | Consentimiento y supresión de los datos | Aprobación formal del tratamiento pendiente (fase 4) |

Fuente: elaboración propia.

La oportunidad del proyecto no consiste en reemplazar soluciones oficiales, sino en explorar un flujo de alerta personal que integre activación rápida, retroalimentación local y preparación de información para comunicar una emergencia.

# 3. ESTRATEGIA METODOLÓGICA Y DESARROLLO

## 3.1 Enfoque metodológico

Se propone un enfoque aplicado de desarrollo incremental. Cada iteración parte de requisitos priorizados, produce un componente o flujo funcional y termina con pruebas y registro de hallazgos. Las funciones dependientes de servicios externos se marcarán como pendientes hasta comprobar su integración. Este enfoque corresponde al desarrollo incremental descrito por Sommerville (2016) y a los principios del desarrollo ágil (Beck et al., 2001).

## 3.2 Fases y entregables

**Tabla 5**

*Fases, entregables, semanas y fechas*

| Fase | Actividades | Entregables | Criterio de salida | Semanas y fechas |
|---|---|---|---|---|
| Análisis | Identificar escenarios, necesidades, riesgos y restricciones | Problema, alcance y requisitos | Requisitos priorizados | 1–2 · 06/10 – 09/10/2026 |
| Diseño | Definir navegación, interfaz, datos y | Diagramas, prototipos y modelo preliminar | Flujos revisados | 3–4 · 12/10 – 16/10/2026 |

| | | | | |
|---|---|---|---|---|
| | arquitectura | | | |
| Construcción base | Configurar proyecto móvil y componentes | Proyecto ejecutable y navegación | Pantallas previstas operativas | 5–6 · 19/10 – 23/10/2026 |
| Función de alerta | Implementar activación, estados y señalización local | Flujo principal | Casos básicos registrados | 7–8 · 19/10 – 30/10/2026 |
| Capacidades del dispositivo | Evaluar permisos y ubicación; preparar contactos | Módulos según viabilidad | Errores controlados | 9–10 · 26/10 – 30/10/2026 |
| Integración y pruebas | Probar condiciones normales y de fallo | Matriz de pruebas | Resultados documentados | 11–12 · 26/10 – 06/11/2026 |
| Documentación | Consolidar guía, decisiones y conclusiones | Informe y manual | Documentación coherente | 13–14 · 05/11 – 11/12/2026 |

Fuente: elaboración propia.

## 3.3 Requisitos funcionales

En la primera entrega los requisitos se plantearon de forma general (RF-01 a RF-10). En la segunda entrega se refinaron en 48 requisitos funcionales agrupados en nueve módulos. Cada uno tiene una tabla con identificador RF-XX-NN, actor, prioridad, descripción, criterio de aceptación, caso de prueba que lo verifica, estado y requisito de origen. Las prioridades son: **Alta**, indispensable en una emergencia; **Media**, preparación y soporte; y **Baja**, comodidad.

Los actores son la persona usuaria (PU), el operador del centro de monitoreo (OP), el visitante de la página informativa (VI) y el propio sistema (SIS). Los casos con prefijo CP-E son pruebas automáticas extremo a extremo; los CP-M, casos manuales; y los archivos terminados en .test.ts, pruebas unitarias (sección 4.6).

**Tabla 6**

*Correspondencia entre los requisitos funcionales de la primera entrega y los requisitos detallados*

| Código | Requisito general (primera entrega) | Requisitos detallados (segunda entrega) |
|---|---|---|
| **RF-01** | Pantalla principal con acceso visible al botón de pánico | RF-RE-01, RF-AL-01 |
| **RF-02** | Iniciar la alerta mediante una acción explícita | RF-AL-01, RF-AL-02 |
| **RF-03** | Mostrar confirmación visual del estado | RF-AL-03, RF-AL-08, RF-RE-04 |
| **RF-04** | Señal sonora local cuando sea compatible | RF-AL-03, RF-AL-07, RF-CF-01 |

| Código | Requisito general (primera entrega) | Requisitos detallados (segunda entrega) |
|---|---|---|
| **RF-05** | Solicitar autorización para la ubicación | RF-RE-02, RF-AL-04, RF-CT-03 |
| **RF-06** | Configurar contactos de confianza | RF-RE-02, RF-RE-03, RF-CT-01 |
| **RF-07** | Diferenciar estados de envío | RF-AL-06, RF-CO-01 a RF-CO-06 |
| **RF-08** | Informar falta de conectividad o fallo del servicio | RF-RE-05, RF-SI-01 a RF-SI-03, RF-CO-05 |
| **RF-09** | Consultar instrucciones y limitaciones | RF-IN-03, RF-AL-10, RF-PA-08 |
| **RF-10** | Cancelar o finalizar la señal local | RF-AL-02, RF-AL-08 |
| **Nuevos** | Surgieron en el análisis de la segunda entrega | RF-AL-05, RF-AL-09, RF-CT-02, RF-IN-01, RF-IN-02, RF-IN-04 a RF-IN-07, RF-CF-02, RF-CF-03, RF-SI-04, RF-PA-01 a RF-PA-07, RF-PI-01, RF-PI-02 |

Fuente: elaboración propia.

### 3.3.1 RF-RE · Registro e inicio

**Tabla 7**
*Especificación del requisito funcional RF-RE-01*

| **RF-RE-01 · Pantalla de carga y enrutamiento inicial** | |
|---|---|
| **Módulo** | RF-RE · Registro e inicio |
| **Actor** | Persona usuaria (PU) |
| **Prioridad** | Media |
| **Descripción** | El sistema debe mostrar una pantalla de carga al abrir la app y llevar a la Bienvenida si la persona no está registrada, o a Inicio si ya lo está. |
| **Criterio de aceptación** | La pantalla de carga dura entre 0,6 y 1,5 s y nunca tapa una alerta activa; después abre Bienvenida o Inicio según el estado del registro. |
| **Verificación** | CP-E-07 (las 17 rutas cargan sin error). |
| **Estado al 09/11/2026** | Implementado y verificado. |
| **Origen (primera entrega)** | RF-01 |

Fuente: elaboración propia.

**Tabla 8**
*Especificación del requisito funcional RF-RE-02*

| RF-RE-02 · Registro obligatorio en cuatro pasos | |
|---|---|
| **Módulo** | RF-RE · Registro e inicio |
| **Actor** | Persona usuaria (PU) |
| **Prioridad** | Alta |
| **Descripción** | El sistema debe exigir un registro en 4 pasos antes de usar la app: nombre de usuario, ubicación (GPS), al menos un punto de encuentro y al menos un contacto de emergencia. |
| **Criterio de aceptación** | El botón «Siguiente» queda bloqueado hasta completar el dato obligatorio de cada paso; el SOS sigue disponible durante el registro. |
| **Verificación** | CP-E-01, CP-E-02 y CP-E-05. |
| **Estado al 09/11/2026** | Implementado y verificado. |
| **Origen (primera entrega)** | RF-05, RF-06 |

Fuente: elaboración propia.

**Tabla 9**
*Especificación del requisito funcional RF-RE-03*

| RF-RE-03 · Contactos desde la agenda y edición en el registro | |
|---|---|
| **Módulo** | RF-RE · Registro e inicio |
| **Actor** | Persona usuaria (PU) |
| **Prioridad** | Media |
| **Descripción** | El sistema debe permitir agregar contactos desde la agenda del teléfono (donde el dispositivo lo permita), y editar o eliminar los contactos agregados durante el registro. |
| **Criterio de aceptación** | El contacto elegido en la agenda precarga nombre y teléfono; editar o eliminar un contacto se refleja de inmediato en el paso. |
| **Verificación** | CP-E-03 y CP-E-04. |
| **Estado al 09/11/2026** | Implementado y verificado. |
| **Origen (primera entrega)** | RF-06 |

Fuente: elaboración propia.

**Tabla 10**
*Especificación del requisito funcional RF-RE-04*

| RF-RE-04 · Datos obligatorios faltantes en rojo | |
|---|---|
| **Módulo** | RF-RE · Registro e inicio |
| **Actor** | Persona usuaria (PU) |
| **Prioridad** | Media |
| **Descripción** | El sistema debe marcar en rojo los datos obligatorios que falten y permitir completarlos después. |
| **Criterio de aceptación** | Inicio muestra un aviso rojo y cada tarjeta con datos faltantes tiene borde rojo y enlace para completarla; cada acción destructiva pide confirmación. |
| **Verificación** | CP-E-05 y CP-E-06. |
| **Estado al 09/11/2026** | Implementado y verificado. |
| **Origen (primera entrega)** | RF-03 |

Fuente: elaboración propia.

**Tabla 11**
*Especificación del requisito funcional RF-RE-05*

| RF-RE-05 · Registro del dispositivo en el servidor | |
|---|---|
| **Módulo** | RF-RE · Registro e inicio |
| **Actor** | Sistema (SIS) |
| **Prioridad** | Alta |
| **Descripción** | Al terminar el registro, el sistema debe registrar el dispositivo en el servidor (sin contraseña, con un token) y subir todos los datos; sin conexión, debe dejarlos en cola. |
| **Criterio de aceptación** | POST /registro responde 201 con el UUID del dispositivo y el token; en la base de datos solo queda el SHA-256 del token; sin red los datos quedan en la cola. |
| **Verificación** | CP-M-01 y syncService.test.ts. |
| **Estado al 09/11/2026** | Implementado y verificado. |
| **Origen (primera entrega)** | RF-08 |

Fuente: elaboración propia.

### 3.3.2 RF-AL · Alerta SOS

**Tabla 12**
*Especificación del requisito funcional RF-AL-01*

| RF-AL-01 · Activación del SOS por varios medios | |
|---|---|
| **Módulo** | RF-AL · Alerta SOS |
| **Actor** | Persona usuaria (PU) |
| **Prioridad** | Alta |
| **Descripción** | El sistema debe permitir activar la alerta SOS desde el botón de la app, el widget de Android (doble toque), el acceso rápido o la detección automática de caídas. |
| **Criterio de aceptación** | Cualquiera de los cuatro medios abre la confirmación de la alerta; la detección de caídas pregunta «¿Está bien?» antes de pedir ayuda. |
| **Verificación** | CP-E-09, CP-E-22 y CP-M-13. |
| **Estado al 09/11/2026** | Implementado; verificado en el emulador Android 16. Falta la prueba en un teléfono físico (fase 4). |
| **Origen (primera entrega)** | RF-01, RF-02 |

Fuente: elaboración propia.

**Tabla 13**
*Especificación del requisito funcional RF-AL-02*

| RF-AL-02 · Tipo de evento y confirmación | |
|---|---|
| **Módulo** | RF-AL · Alerta SOS |
| **Actor** | Persona usuaria (PU) |
| **Prioridad** | Alta |
| **Descripción** | El sistema debe pedir el tipo de evento (catálogo administrable: sismo, otra emergencia, prueba…) y confirmar antes de activar, para evitar activaciones accidentales. |
| **Criterio de aceptación** | No se crea ninguna alerta sin confirmar; cancelar la confirmación no deja registro en el historial. |
| **Verificación** | CP-E-09 y CP-E-11. |
| **Estado al 09/11/2026** | Implementado y verificado. |
| **Origen (primera entrega)** | RF-02, RF-10 |

Fuente: elaboración propia.

**Tabla 14**
*Especificación del requisito funcional RF-AL-03*

| RF-AL-03 · Señales locales de la alerta | |
|---|---|
| **Módulo** | RF-AL · Alerta SOS |
| **Actor** | Sistema (SIS) |
| **Prioridad** | Alta |
| **Descripción** | Al activarse, el sistema debe emitir sirena (o silbato), destello y linterna en código SOS, vibración en patrón SOS y mantener la pantalla encendida, incluso con la pantalla bloqueada en Android. |
| **Criterio de aceptación** | Las señales arrancan al activar la alerta, se pausan en el modo atrapado y se detienen al finalizarla. |
| **Verificación** | CP-E-18, CP-E-28 y CP-M-13. |
| **Estado al 09/11/2026** | Implementado; verificado en el emulador Android 16. Falta la prueba en un teléfono físico (fase 4). |
| **Origen (primera entrega)** | RF-03, RF-04 |

Fuente: elaboración propia.

**Tabla 15**
*Especificación del requisito funcional RF-AL-04*

| RF-AL-04 · Ubicación afinada y actualizada | |
|---|---|
| **Módulo** | RF-AL · Alerta SOS |
| **Actor** | Sistema (SIS) |
| **Prioridad** | Alta |
| **Descripción** | El sistema debe afinar la ubicación GPS (hasta 10 s) y actualizarla cada 2 minutos (parámetro administrable) mientras la alerta esté activa. |
| **Criterio de aceptación** | La alerta incluye la ubicación aunque no hubiera una guardada; con el permiso denegado se informa la limitación y la alerta continúa. |
| **Verificación** | CP-E-13, CP-E-14 y CP-E-17. |
| **Estado al 09/11/2026** | Implementado y verificado. |
| **Origen (primera entrega)** | RF-05 |

Fuente: elaboración propia.

**Tabla 16**
*Especificación del requisito funcional RF-AL-05*

| RF-AL-05 · Llamada al número de emergencia | |
|---|---|
| **Módulo** | RF-AL · Alerta SOS |
| **Actor** | Persona usuaria (PU) |
| **Prioridad** | Alta |
| **Descripción** | El sistema debe ofrecer el botón «Llamar al {número de emergencia}» (parámetro administrable, por defecto 123). |
| **Criterio de aceptación** | El botón abre un enlace tel: con el número configurado en el servidor (123 por defecto). |
| **Verificación** | CP-E-16. |
| **Estado al 09/11/2026** | Implementado y verificado. |
| **Origen (primera entrega)** | Nuevo |

Fuente: elaboración propia.

**Tabla 17**
*Especificación del requisito funcional RF-AL-06*

| RF-AL-06 · Mensaje SMS o WhatsApp a los contactos | |
|---|---|
| **Módulo** | RF-AL · Alerta SOS |
| **Actor** | Persona usuaria (PU) |
| **Prioridad** | Alta |
| **Descripción** | El sistema debe permitir enviar a los contactos un SMS o WhatsApp con la ubicación, los puntos de encuentro, la ficha médica (si se comparte) y la batería. |
| **Criterio de aceptación** | El mensaje incluye los datos disponibles y la app nunca lo muestra como «enviado» si no hay confirmación del canal. |
| **Verificación** | smsService.test.ts (22 pruebas) y CP-E-10. |
| **Estado al 09/11/2026** | Implementado y verificado. |
| **Origen (primera entrega)** | RF-07 |

Fuente: elaboración propia.

**Tabla 18**
*Especificación del requisito funcional RF-AL-07*

| RF-AL-07 · Ahorro de batería, modo atrapado y silbato | |
|---|---|
| **Módulo** | RF-AL · Alerta SOS |
| **Actor** | Persona usuaria (PU) |
| **Prioridad** | Alta |
| **Descripción** | El sistema debe activar el ahorro de batería cuando la carga baje del umbral configurado, y ofrecer el modo atrapado y el silbato de rescate. |
| **Criterio de aceptación** | Bajo el umbral (20 % por defecto) la alerta pasa sola al modo ahorro y avisa; la opción «Nunca» lo desactiva. |
| **Verificación** | CP-E-18, CP-E-44, CP-E-45 y CP-E-46. |
| **Estado al 09/11/2026** | Implementado y verificado. |
| **Origen (primera entrega)** | RF-04 |

Fuente: elaboración propia.

**Tabla 19**
*Especificación del requisito funcional RF-AL-08*

| RF-AL-08 · Finalizar la alerta y guardarla en el historial | |
|---|---|
| **Módulo** | RF-AL · Alerta SOS |
| **Actor** | Persona usuaria (PU) |
| **Prioridad** | Alta |
| **Descripción** | El sistema debe permitir finalizar la alerta y guardarla en el historial con sus métricas (tiempos de confirmación, sirena, linterna, vibración y SMS). |
| **Criterio de aceptación** | Al finalizar, la alerta aparece en el historial con sus tiempos de activación. |
| **Verificación** | CP-E-09, CP-E-16 y metricsService.test.ts. |
| **Estado al 09/11/2026** | Implementado y verificado. |
| **Origen (primera entrega)** | RF-03, RF-10 |

Fuente: elaboración propia.

**Tabla 20**
*Especificación del requisito funcional RF-AL-09*

| RF-AL-09 · Aviso «Estoy bien» | |
|---|---|
| **Módulo** | RF-AL · Alerta SOS |
| **Actor** | Persona usuaria (PU) |
| **Prioridad** | Alta |
| **Descripción** | El sistema debe permitir avisar «Estoy bien» por SMS o WhatsApp con la ubicación actual, el punto de encuentro y la batería. |
| **Criterio de aceptación** | Solo se envía a contactos reales; la persona elige SMS o WhatsApp por contacto. |
| **Verificación** | CP-E-26 y CP-E-39. |
| **Estado al 09/11/2026** | Implementado y verificado. |
| **Origen (primera entrega)** | Nuevo |

Fuente: elaboración propia.

**Tabla 21**
*Especificación del requisito funcional RF-AL-10*

| RF-AL-10 · Lista «Después del sismo» | |
|---|---|
| **Módulo** | RF-AL · Alerta SOS |
| **Actor** | Persona usuaria (PU) |
| **Prioridad** | Media |
| **Descripción** | El sistema debe mostrar la lista «Después del sismo» (catálogo administrable) y guardar el avance por clave de paso. |
| **Criterio de aceptación** | El avance se guarda por clave de paso y cada nueva alerta empieza la lista vacía. |
| **Verificación** | CP-E-36. |
| **Estado al 09/11/2026** | Implementado y verificado. |
| **Origen (primera entrega)** | RF-09 |

Fuente: elaboración propia.

### 3.3.3 RF-CO · Correo de alerta a los contactos

**Tabla 22**
*Especificación del requisito funcional RF-CO-01*

| RF-CO-01 · Pregunta de correo con cuenta regresiva | |
|---|---|
| **Módulo** | RF-CO · Correo de alerta a los contactos |
| **Actor** | Persona usuaria (PU) |
| **Prioridad** | Alta |
| **Descripción** | Al activar una alerta real, el sistema debe preguntar «¿Enviar correo a sus contactos?» con una cuenta regresiva de 20 s. |
| **Criterio de aceptación** | El diálogo aparece al activar una alerta real y muestra la cuenta regresiva de 20 s sin salirse de la pantalla. |
| **Verificación** | CP-E-29. |
| **Estado al 09/11/2026** | Implementado y verificado. |
| **Origen (primera entrega)** | RF-07 |

Fuente: elaboración propia.

**Tabla 23**
*Especificación del requisito funcional RF-CO-02*

| RF-CO-02 · Envío automático o cancelación del correo | |
|---|---|
| **Módulo** | RF-CO · Correo de alerta a los contactos |
| **Actor** | Persona usuaria (PU) |
| **Prioridad** | Alta |
| **Descripción** | Si la persona toca «Sí, enviar ahora» o no responde en 20 s, el sistema debe enviar el correo; con «No enviar», debe cancelarlo y permitir reintentar. |
| **Criterio de aceptación** | «Sí» o 20 s sin respuesta envían el correo; «No enviar» deja el estado «cancelado» con opción de reintentar. |
| **Verificación** | CP-E-29 y CP-E-30. |
| **Estado al 09/11/2026** | Implementado y verificado. |
| **Origen (primera entrega)** | RF-07 |

Fuente: elaboración propia.

**Tabla 24**
*Especificación del requisito funcional RF-CO-03*

| RF-CO-03 · Destinatarios y contenido del correo | |
|---|---|
| **Módulo** | RF-CO · Correo de alerta a los contactos |
| **Actor** | Sistema (SIS) |
| **Prioridad** | Alta |
| **Descripción** | El correo debe llegar a los contactos reales que tengan correo e incluir el tipo de evento, la hora, la ubicación, los puntos de encuentro, la ficha médica (si se comparte) y la batería. |
| **Criterio de aceptación** | Solo se incluyen contactos reales con correo válido y el cuerpo contiene todos los datos disponibles de la alerta. |
| **Verificación** | emailService.test.ts y CP-M-12. |
| **Estado al 09/11/2026** | Implementado; cubierto por pruebas automáticas. Falta la prueba con el servicio de correo en producción (fase 4). |
| **Origen (primera entrega)** | RF-07 |

Fuente: elaboración propia.

**Tabla 25**
*Especificación del requisito funcional RF-CO-04*

| RF-CO-04 · Servicio de correo sin credenciales en el teléfono | |
|---|---|
| **Módulo** | RF-CO · Correo de alerta a los contactos |
| **Actor** | Sistema (SIS) |
| **Prioridad** | Alta |
| **Descripción** | La app debe enviar la solicitud al servicio de correo de alertas (POST JSON, sin credenciales en el teléfono); el servicio valida destinatarios, aplica límites y envía por el proveedor de correo. |
| **Criterio de aceptación** | La app solo conoce la URL del servicio; las credenciales del proveedor no aparecen en el código de la app. |
| **Verificación** | emailService.test.ts, revisión de código y CP-M-12. |
| **Estado al 09/11/2026** | Implementado; falta la prueba de punta a punta en producción (fase 4). |
| **Origen (primera entrega)** | RF-07 |

Fuente: elaboración propia.

**Tabla 26**
*Especificación del requisito funcional RF-CO-05*

| RF-CO-05 · Reintentos y estados del correo | |
|---|---|
| **Módulo** | RF-CO · Correo de alerta a los contactos |
| **Actor** | Sistema (SIS) |
| **Prioridad** | Media |
| **Descripción** | El sistema debe reintentar 2 veces ante fallos de red o 5xx (espera exponencial) y mostrar el estado: enviado, cancelado, fallido, sin destinatarios o sin servidor. |
| **Criterio de aceptación** | Ante red caída o error 5xx hay 2 reintentos con espera exponencial; un 4xx no se reintenta; el estado final siempre es visible. |
| **Verificación** | emailService.test.ts y CP-E-30. |
| **Estado al 09/11/2026** | Implementado y verificado. |
| **Origen (primera entrega)** | RF-07, RF-08 |

Fuente: elaboración propia.

**Tabla 27**
*Especificación del requisito funcional RF-CO-06*

| RF-CO-06 · Sin correo en simulacros ni sin destinatarios | |
|---|---|
| **Módulo** | RF-CO · Correo de alerta a los contactos |
| **Actor** | Sistema (SIS) |
| **Prioridad** | Media |
| **Descripción** | El sistema no debe preguntar por el correo en los simulacros ni si ningún contacto real tiene correo. |
| **Criterio de aceptación** | En un simulacro, o sin contactos reales con correo, el diálogo no aparece. |
| **Verificación** | CP-E-31. |
| **Estado al 09/11/2026** | Implementado y verificado. |
| **Origen (primera entrega)** | RF-07 |

Fuente: elaboración propia.

### 3.3.4 RF-CT · Contactos y puntos de encuentro

**Tabla 28**
*Especificación del requisito funcional RF-CT-01*

| RF-CT-01 · Gestión de contactos de emergencia | |
|---|---|
| **Módulo** | RF-CT · Contactos y puntos de encuentro |
| **Actor** | Persona usuaria (PU) |
| **Prioridad** | Alta |
| **Descripción** | El sistema debe permitir crear, editar y eliminar contactos de emergencia con nombre, teléfono, correo (validado) y relación (catálogo administrable). |
| **Criterio de aceptación** | Se rechazan teléfonos y correos inválidos; altas, ediciones y borrados persisten al recargar la app. |
| **Verificación** | CP-E-12. |
| **Estado al 09/11/2026** | Implementado y verificado. |
| **Origen (primera entrega)** | RF-06 |

Fuente: elaboración propia.

**Tabla 29**
*Especificación del requisito funcional RF-CT-02*

| RF-CT-02 · Puntos de encuentro | |
|---|---|
| **Módulo** | RF-CT · Contactos y puntos de encuentro |
| **Actor** | Persona usuaria (PU) |
| **Prioridad** | Alta |
| **Descripción** | El sistema debe permitir hasta N puntos de encuentro (parámetro administrable, por defecto 5) desde la ubicación actual, el mapa o una dirección. |
| **Criterio de aceptación** | Un punto se puede agregar desde la ubicación, el mapa o una dirección; no se permite superar el máximo configurado. |
| **Verificación** | CP-E-37, CP-E-41 y CP-E-42. |
| **Estado al 09/11/2026** | Implementado y verificado. |
| **Origen (primera entrega)** | Nuevo |

Fuente: elaboración propia.

**Tabla 30**
*Especificación del requisito funcional RF-CT-03*

| RF-CT-03 · Mapa, brújula y ruta al punto de encuentro | |
|---|---|
| **Módulo** | RF-CT · Contactos y puntos de encuentro |
| **Actor** | Persona usuaria (PU) |
| **Prioridad** | Media |
| **Descripción** | El sistema debe mostrar la ubicación en un mapa real (OpenStreetMap), la brújula sin internet hacia el punto más cercano y la ruta (a pie, carro/moto o bicicleta), con enlace a Waze o Google Maps. |
| **Criterio de aceptación** | Se muestran la distancia, la dirección y el tiempo estimado por modo; la brújula funciona sin internet. |
| **Verificación** | CP-E-38, CP-E-41, compassService.test.ts y routeService.test.ts. |
| **Estado al 09/11/2026** | Implementado y verificado. |
| **Origen (primera entrega)** | RF-05 |

Fuente: elaboración propia.

### 3.3.5 RF-IN · Información y preparación

**Tabla 31**
*Especificación del requisito funcional RF-IN-01*

| RF-IN-01 · Ficha médica con consentimiento | |
|---|---|
| **Módulo** | RF-IN · Información y preparación |
| **Actor** | Persona usuaria (PU) |
| **Prioridad** | Alta |
| **Descripción** | El sistema debe permitir registrar una ficha médica (sangre, alergias, medicamentos, condiciones, notas) y elegir si se comparte en la alerta y el SMS. |
| **Criterio de aceptación** | La ficha se comparte solo con el consentimiento activo; al desactivarlo desaparece de la alerta y del SMS. |
| **Verificación** | CP-E-24 y smsService.test.ts. |
| **Estado al 09/11/2026** | Implementado y verificado. |
| **Origen (primera entrega)** | Nuevo |

Fuente: elaboración propia.

**Tabla 32**
*Especificación del requisito funcional RF-IN-02*

| RF-IN-02 · QR médico sin internet | |
|---|---|
| **Módulo** | RF-IN · Información y preparación |
| **Actor** | Persona usuaria (PU) |
| **Prioridad** | Alta |
| **Descripción** | El sistema debe generar un QR médico con la ficha en texto plano, legible por cualquier lector de QR sin internet ni la app. |
| **Criterio de aceptación** | El QR aparece en la ficha y en la alerta, y un lector genérico muestra la ficha sin conexión. |
| **Verificación** | CP-E-32. |
| **Estado al 09/11/2026** | Implementado y verificado. |
| **Origen (primera entrega)** | Nuevo |

Fuente: elaboración propia.

**Tabla 33**
*Especificación del requisito funcional RF-IN-03*

| RF-IN-03 · Guía de emergencia con lectura en voz alta | |
|---|---|
| **Módulo** | RF-IN · Información y preparación |
| **Actor** | Persona usuaria (PU) |
| **Prioridad** | Media |
| **Descripción** | El sistema debe mostrar la guía de emergencia (antes, durante, después) con lectura en voz alta; el contenido es administrable. |
| **Criterio de aceptación** | La guía se carga desde el catálogo y el botón de audio lee el texto en el idioma elegido. |
| **Verificación** | CP-E-07 y CP-M-11. |
| **Estado al 09/11/2026** | Implementado y verificado. |
| **Origen (primera entrega)** | RF-09 |

Fuente: elaboración propia.

**Tabla 34**
*Especificación del requisito funcional RF-IN-04*

| RF-IN-04 · Sismos recientes | |
|---|---|
| **Módulo** | RF-IN · Información y preparación |
| **Actor** | Persona usuaria (PU) |
| **Prioridad** | Media |
| **Descripción** | El sistema debe mostrar los sismos recientes de Colombia (USGS) de las últimas 24 h y 7 días, y los cercanos a la persona. |
| **Criterio de aceptación** | Sin ubicación muestra solo los sismos de Colombia; con ubicación agrega los cercanos; ante un fallo muestra un mensaje y permite reintentar. |
| **Verificación** | CP-E-23 y quakeService.test.ts. |
| **Estado al 09/11/2026** | Implementado y verificado. |
| **Origen (primera entrega)** | Nuevo |

Fuente: elaboración propia.

**Tabla 35**
*Especificación del requisito funcional RF-IN-05*

| RF-IN-05 · Mochila de emergencia | |
|---|---|
| **Módulo** | RF-IN · Información y preparación |
| **Actor** | Persona usuaria (PU) |
| **Prioridad** | Media |
| **Descripción** | El sistema debe permitir gestionar la mochila de emergencia (ítems sugeridos del catálogo, vencimientos y revisión periódica). |
| **Criterio de aceptación** | La lista base viene del catálogo y los ítems vencidos o por vencer generan un aviso en Inicio. |
| **Verificación** | CP-E-21 y kitService.test.ts. |
| **Estado al 09/11/2026** | Implementado y verificado. |
| **Origen (primera entrega)** | Nuevo |

Fuente: elaboración propia.

**Tabla 36**
*Especificación del requisito funcional RF-IN-06*

| RF-IN-06 · Simulacros, encuesta SUS y estadísticas | |
|---|---|
| **Módulo** | RF-IN · Información y preparación |
| **Actor** | Persona usuaria (PU) |
| **Prioridad** | Media |
| **Descripción** | El sistema debe ofrecer simulacros con cronómetro, la encuesta de usabilidad SUS y estadísticas (con exportación CSV). |
| **Criterio de aceptación** | Se guarda el tiempo de reacción del simulacro, se calcula el puntaje SUS (0–100) y el CSV no contiene datos personales. |
| **Verificación** | CP-E-19, CP-E-20 y CP-E-25. |
| **Estado al 09/11/2026** | Implementado y verificado. |
| **Origen (primera entrega)** | Nuevo |

Fuente: elaboración propia.

**Tabla 37**
*Especificación del requisito funcional RF-IN-07*

| RF-IN-07 · Recordatorio de simulacro | |
|---|---|
| **Módulo** | RF-IN · Información y preparación |
| **Actor** | Sistema (SIS) |
| **Prioridad** | Baja |
| **Descripción** | El sistema debe recordar hacer un simulacro si nunca se hizo o pasaron 90 días. |
| **Criterio de aceptación** | Si nunca se hizo un simulacro o pasaron 90 días, Inicio lo recuerda y lleva al simulacro. |
| **Verificación** | CP-E-40 y drillReminderService.test.ts. |
| **Estado al 09/11/2026** | Implementado y verificado. |
| **Origen (primera entrega)** | Nuevo |

Fuente: elaboración propia.

### 3.3.6 RF-CF · Configuración y cuenta

**Tabla 38**
*Especificación del requisito funcional RF-CF-01*

| RF-CF-01 · Preferencias y accesibilidad | |
|---|---|
| **Módulo** | RF-CF · Configuración y cuenta |
| **Actor** | Persona usuaria (PU) |
| **Prioridad** | Media |
| **Descripción** | El sistema debe permitir elegir idioma (español / inglés), tema (sistema, claro, oscuro), sonido de alarma, señales, detección de caídas, umbral de ahorro de batería y opciones de accesibilidad (alto contraste, texto grande, reducir movimiento). |
| **Criterio de aceptación** | Cada preferencia persiste tras recargar y el idioma y el tema se aplican en toda la app. |
| **Verificación** | CP-E-15, CP-E-34 y CP-E-35. |
| **Estado al 09/11/2026** | Implementado y verificado. |
| **Origen (primera entrega)** | RF-04 |

Fuente: elaboración propia.

**Tabla 39**
*Especificación del requisito funcional RF-CF-02*

| RF-CF-02 · Código de recuperación de la cuenta | |
|---|---|
| **Módulo** | RF-CF · Configuración y cuenta |
| **Actor** | Persona usuaria (PU) |
| **Prioridad** | Alta |
| **Descripción** | El sistema debe permitir crear un código de recuperación (XXXX-XXXX-XXXX) y recuperar todos los datos en un teléfono nuevo con ese código; el teléfono anterior pierde el acceso. |
| **Criterio de aceptación** | El código es de un solo uso; el teléfono nuevo recibe un token nuevo y el anterior recibe 401. |
| **Verificación** | CP-M-06 y syncService.test.ts. |
| **Estado al 09/11/2026** | Implementado y verificado. |
| **Origen (primera entrega)** | Nuevo |

Fuente: elaboración propia.

**Tabla 40**
*Especificación del requisito funcional RF-CF-03*

| RF-CF-03 · Eliminar mi cuenta | |
|---|---|
| **Módulo** | RF-CF · Configuración y cuenta |
| **Actor** | Persona usuaria (PU) |
| **Prioridad** | Alta |
| **Descripción** | El sistema debe permitir «Eliminar mi cuenta»: borrado lógico en cascada, supresión de datos personales y de salud, y revocación del token y del código; después la persona puede registrarse de nuevo. |
| **Criterio de aceptación** | Todos los registros quedan con activo = 0, los datos personales se vacían, el token deja de servir y la persona puede registrarse otra vez. |
| **Verificación** | CP-M-07 y syncService.test.ts. |
| **Estado al 09/11/2026** | Implementado y verificado. |
| **Origen (primera entrega)** | Nuevo |

Fuente: elaboración propia.

### 3.3.7 RF-SI · Sincronización

**Tabla 41**
*Especificación del requisito funcional RF-SI-01*

| RF-SI-01 · Guardado local primero y subida por recurso | |
|---|---|
| **Módulo** | RF-SI · Sincronización |
| **Actor** | Sistema (SIS) |
| **Prioridad** | Alta |
| **Descripción** | Todo cambio debe guardarse primero en el teléfono y subirse al servidor por recurso (ajustes, contactos, alertas, puntos, encuestas, kit, ficha médica). |
| **Criterio de aceptación** | Cada cambio genera un PUT o DELETE por diferencias y nunca se suben datos de demostración. |
| **Verificación** | syncService.test.ts. |
| **Estado al 09/11/2026** | Implementado y verificado. |
| **Origen (primera entrega)** | RF-08 |

Fuente: elaboración propia.

**Tabla 42**
*Especificación del requisito funcional RF-SI-02*

| RF-SI-02 · Cola persistente sin conexión | |
|---|---|
| **Módulo** | RF-SI · Sincronización |
| **Actor** | Sistema (SIS) |
| **Prioridad** | Alta |
| **Descripción** | Sin conexión, los cambios deben esperar en una cola persistente y subirse solos al volver la red, con reintentos; si el token deja de ser válido, la app debe registrarse de nuevo y subir todo. |
| **Criterio de aceptación** | Los cambios hechos sin red se suben en orden al volver la conexión, sin perder ninguno. |
| **Verificación** | syncService.test.ts y CP-E-33. |
| **Estado al 09/11/2026** | Implementado y verificado. |
| **Origen (primera entrega)** | RF-08 |

Fuente: elaboración propia.

**Tabla 43**
*Especificación del requisito funcional RF-SI-03*

| RF-SI-03 · Aviso de sin conexión y cambios pendientes | |
|---|---|
| **Módulo** | RF-SI · Sincronización |
| **Actor** | Sistema (SIS) |
| **Prioridad** | Media |
| **Descripción** | El sistema debe avisar «Sin conexión: sus datos se guardarán apenas tenga conexión» y mostrar los cambios pendientes. |
| **Criterio de aceptación** | El aviso aparece en la alerta y en Inicio, con el número de cambios pendientes. |
| **Verificación** | CP-E-33. |
| **Estado al 09/11/2026** | Implementado y verificado. |
| **Origen (primera entrega)** | RF-08 |

Fuente: elaboración propia.

**Tabla 44**
*Especificación del requisito funcional RF-SI-04*

| RF-SI-04 · Catálogos con copia local | |
|---|---|
| **Módulo** | RF-SI · Sincronización |
| **Actor** | Sistema (SIS) |
| **Prioridad** | Media |
| **Descripción** | La app debe descargar los catálogos al abrirse y usar la última copia (o los valores por defecto) sin conexión. |
| **Criterio de aceptación** | Sin conexión se usa la última copia o los valores por defecto; se rechazan respuestas sin versión. |
| **Verificación** | catalogService.test.ts. |
| **Estado al 09/11/2026** | Implementado y verificado. |
| **Origen (primera entrega)** | Nuevo |

Fuente: elaboración propia.

### 3.3.8 RF-PA · Panel de monitoreo

**Tabla 45**
*Especificación del requisito funcional RF-PA-01*

| RF-PA-01 · Acceso restringido al panel | |
|---|---|
| **Módulo** | RF-PA · Panel de monitoreo |
| **Actor** | Operador del centro de monitoreo (OP) |
| **Prioridad** | Alta |
| **Descripción** | Solo deben entrar al panel las personas con sesión iniciada y correo autorizado en la configuración del servidor. |
| **Criterio de aceptación** | Sin sesión redirige al inicio de sesión; con sesión sin autorización responde 403; con autorización abre el panel. |
| **Verificación** | CP-M-08. |
| **Estado al 09/11/2026** | Implementado y verificado. |
| **Origen (primera entrega)** | Nuevo |

Fuente: elaboración propia.

**Tabla 46**
*Especificación del requisito funcional RF-PA-02*

| RF-PA-02 · Indicadores, alertas y mapa casi en tiempo real | |
|---|---|
| **Módulo** | RF-PA · Panel de monitoreo |
| **Actor** | Operador del centro de monitoreo (OP) |
| **Prioridad** | Alta |
| **Descripción** | El panel debe mostrar indicadores, alertas activas y un mapa en tiempo real, y actualizarse solo casi en tiempo real (pulso cada 3 s, sin loader); cada consulta iniciada por el operador muestra un loader. |
| **Criterio de aceptación** | Una alerta nueva aparece en unos 3 s sin loader; las consultas del operador sí muestran loader. |
| **Verificación** | CP-M-09. |
| **Estado al 09/11/2026** | Implementado; falta la prueba con una alerta desde un teléfono real y varios operadores (fase 4). |
| **Origen (primera entrega)** | Nuevo |

Fuente: elaboración propia.

**Tabla 47**
*Especificación del requisito funcional RF-PA-03*

| RF-PA-03 · Avisos de alertas nuevas | |
|---|---|
| **Módulo** | RF-PA · Panel de monitoreo |
| **Actor** | Operador del centro de monitoreo (OP) |
| **Prioridad** | Alta |
| **Descripción** | El panel debe avisar las alertas nuevas con banner, sonido y notificación del sistema (si los avisos están activos; la preferencia se recuerda). |
| **Criterio de aceptación** | Cada alerta nueva genera banner, sonido y notificación; la preferencia de avisos se conserva entre sesiones. |
| **Verificación** | CP-M-09. |
| **Estado al 09/11/2026** | Implementado; falta la prueba con un teléfono real (fase 4). |
| **Origen (primera entrega)** | Nuevo |

Fuente: elaboración propia.

**Tabla 48**
*Especificación del requisito funcional RF-PA-04*

| RF-PA-04 · Registro de la ayuda brindada | |
|---|---|
| **Módulo** | RF-PA · Panel de monitoreo |
| **Actor** | Operador del centro de monitoreo (OP) |
| **Prioridad** | Alta |
| **Descripción** | El operador debe poder registrar la ayuda (En atención, Ayuda enviada, Resuelta) con un mensaje; la persona lo ve como aviso fijo en la app (consulta cada 15 s). |
| **Criterio de aceptación** | El estado y el mensaje del operador llegan a la app como aviso fijo en un máximo de 15 s. |
| **Verificación** | CP-M-10. |
| **Estado al 09/11/2026** | Parcial: verificado con un servidor simulado; falta la prueba de punta a punta con el panel y un teléfono real. |
| **Origen (primera entrega)** | Nuevo |

Fuente: elaboración propia.

**Tabla 49**
*Especificación del requisito funcional RF-PA-05*

| RF-PA-05 · Detalle de la persona con bitácora | |
|---|---|
| **Módulo** | RF-PA · Panel de monitoreo |
| **Actor** | Operador del centro de monitoreo (OP) |
| **Prioridad** | Alta |
| **Descripción** | El operador debe poder ver el detalle de la persona: contactos, ficha médica descifrada, puntos de encuentro, alertas, kit, encuestas y bitácora de ayuda. Cada consulta queda en el log. |
| **Criterio de aceptación** | El detalle muestra todos los datos de la persona y cada consulta queda registrada en el log del servidor. |
| **Verificación** | Revisión manual en desarrollo (evidencia visual en el Anexo F). |
| **Estado al 09/11/2026** | Implementado. |
| **Origen (primera entrega)** | Nuevo |

Fuente: elaboración propia.

**Tabla 50**
*Especificación del requisito funcional RF-PA-06*

| RF-PA-06 · Historial, usuarios y sismos en el panel | |
|---|---|
| **Módulo** | RF-PA · Panel de monitoreo |
| **Actor** | Operador del centro de monitoreo (OP) |
| **Prioridad** | Media |
| **Descripción** | El panel debe mostrar el historial de alertas, la lista de usuarios con buscador y los sismos recientes (24 h / 7 días) en tabla y mapa. |
| **Criterio de aceptación** | Las tablas cargan con buscador y los sismos se ven en tabla y mapa para 24 h y 7 días. |
| **Verificación** | Revisión manual en desarrollo (evidencia visual en el Anexo F). |
| **Estado al 09/11/2026** | Implementado. |
| **Origen (primera entrega)** | Nuevo |

Fuente: elaboración propia.

**Tabla 51**
*Especificación del requisito funcional RF-PA-07*

| RF-PA-07 · Código de recuperación temporal | |
|---|---|
| **Módulo** | RF-PA · Panel de monitoreo |
| **Actor** | Operador del centro de monitoreo (OP) |
| **Prioridad** | Media |
| **Descripción** | El operador debe poder generar un código de recuperación temporal (48 h) para una persona que perdió el teléfono. |
| **Criterio de aceptación** | El código generado vence a las 48 h y es de un solo uso. |
| **Verificación** | Revisión manual junto con CP-M-06. |
| **Estado al 09/11/2026** | Implementado. |
| **Origen (primera entrega)** | Nuevo |

Fuente: elaboración propia.

**Tabla 52**
*Especificación del requisito funcional RF-PA-08*

| RF-PA-08 · Administración de catálogos, guía y parámetros | |
|---|---|
| **Módulo** | RF-PA · Panel de monitoreo |
| **Actor** | Operador del centro de monitoreo (OP) |
| **Prioridad** | Media |
| **Descripción** | El operador debe poder administrar catálogos (tipos de evento, parentescos, kit, pasos post-sismo, tipos de sangre), la guía de emergencia (es/en) y los parámetros. Los tipos de evento sismo y emergencia_general no se pueden ocultar. |
| **Criterio de aceptación** | Un cambio en el panel aparece en la app al reabrirla; los tipos sismo y emergencia_general no se pueden ocultar. |
| **Verificación** | CP-M-11. |
| **Estado al 09/11/2026** | Implementado; GET /catalogos verificado. Falta editar desde el panel con sesión (fase 4). |
| **Origen (primera entrega)** | RF-09 |

Fuente: elaboración propia.

### 3.3.9 RF-PI · Página informativa

**Tabla 53**
*Especificación del requisito funcional RF-PI-01*

| RF-PI-01 · Página informativa pública | |
|---|---|
| **Módulo** | RF-PI · Página informativa |
| **Actor** | Visitante de la página informativa (VI) |
| **Prioridad** | Baja |
| **Descripción** | El sistema debe publicar una página informativa sin inicio de sesión (Sismo_info) con beneficios, funciones, capturas, lo que está pendiente y cómo funciona una alerta. |
| **Criterio de aceptación** | GET /Sismo_info responde 200 sin sesión y presenta las secciones previstas. |
| **Verificación** | CP-M-14. |
| **Estado al 09/11/2026** | Implementado y verificado. |
| **Origen (primera entrega)** | Nuevo |

Fuente: elaboración propia.

**Tabla 54**
*Especificación del requisito funcional RF-PI-02*

| RF-PI-02 · Botones condicionados de la página | |
|---|---|
| **Módulo** | RF-PI · Página informativa |
| **Actor** | Visitante de la página informativa (VI) |
| **Prioridad** | Baja |
| **Descripción** | La página debe mostrar «Ir al panel» solo a los operadores autorizados y «Solicitar una demostración» solo si está configurada una URL https. |
| **Criterio de aceptación** | Un visitante sin sesión no ve «Ir al panel»; el botón de demostración solo aparece con una URL https configurada. |
| **Verificación** | CP-M-14. |
| **Estado al 09/11/2026** | Implementado y verificado. |
| **Origen (primera entrega)** | Nuevo |

Fuente: elaboración propia.

## 3.4 Requisitos no funcionales

Los diez requisitos no funcionales de la primera entrega se refinaron en 33, agrupados en seis categorías y clasificados según los atributos de calidad de la norma ISO/IEC 25010 (ISO, 2023). Cada tabla indica la métrica o el criterio, cómo lo cumple la implementación, la forma de verificarlo y su estado.

**Tabla 55**
*Correspondencia entre los requisitos no funcionales de la primera entrega y los requisitos detallados*

| Código | Atributo (primera entrega) | Requisitos detallados (segunda entrega) |
|---|---|---|
| **RNF-01** | Usabilidad | RNF-US-01, RNF-US-02, RNF-US-05 |
| **RNF-02** | Accesibilidad | RNF-US-03 |
| **RNF-03** | Fiabilidad | RNF-DI-01 a RNF-DI-05 |
| **RNF-04** | Privacidad | RNF-SE-09 |
| **RNF-05** | Seguridad | RNF-SE-01 a RNF-SE-08, RNF-MA-05 |
| **RNF-06** | Compatibilidad | RNF-PO-01 a RNF-PO-03 |
| **RNF-07** | Mantenibilidad | RNF-MA-01 a RNF-MA-06 |
| **RNF-08** | Rendimiento | RNF-RE-01 a RNF-RE-05 |
| **RNF-09** | Transparencia | RNF-US-04 |
| **RNF-10** | Disponibilidad | RNF-DI-01, RNF-DI-04 |

Fuente: elaboración propia.

### 3.4.1 RNF-SE · Seguridad y privacidad

**Tabla 56**
*Especificación del requisito no funcional RNF-SE-01*

| RNF-SE-01 · Las credenciales del dispositivo no se guardan en claro | |
|---|---|
| **Atributo de calidad** | Seguridad (confidencialidad, integridad, autenticidad) (ISO/IEC 25010) |
| **Requisito** | Las credenciales del dispositivo no se guardan en claro. |
| **Métrica o criterio** | Ningún token legible en la base de datos. |
| **Cómo se cumple** | Token aleatorio entregado una sola vez; en la base de datos solo queda su SHA-256. |
| **Verificación** | CP-M-01. |
| **Estado al 09/11/2026** | Cumple. |
| **Origen (primera entrega)** | RNF-05 |

Fuente: elaboración propia.

**Tabla 57**
*Especificación del requisito no funcional RNF-SE-02*

| RNF-SE-02 · Los datos de salud se guardan cifrados | |
|---|---|
| **Atributo de calidad** | Seguridad (confidencialidad, integridad, autenticidad) (ISO/IEC 25010) |
| **Requisito** | Los datos de salud se guardan cifrados. |
| **Métrica o criterio** | La ficha médica es ilegible en la base de datos; sin llave no se guarda nada. |
| **Cómo se cumple** | Ficha médica cifrada con AES-256-GCM (IV aleatorio y etiqueta de autenticación). La llave está en una variable de entorno, nunca en el código ni en la base de datos. Sin llave, la API responde 503. |
| **Verificación** | CP-M-05. |
| **Estado al 09/11/2026** | Implementado; el 503 sin llave está verificado. Falta repetir la prueba con la llave en producción (fase 4). |
| **Origen (primera entrega)** | RNF-05 |

Fuente: elaboración propia.

**Tabla 58**
*Especificación del requisito no funcional RNF-SE-03*

| RNF-SE-03 · Ningún secreto en el código de la app | |
|---|---|
| **Atributo de calidad** | Seguridad (confidencialidad, integridad, autenticidad) (ISO/IEC 25010) |
| **Requisito** | Ningún secreto en el código de la app. |
| **Métrica o criterio** | Cero credenciales en el código fuente y en el APK. |
| **Cómo se cumple** | La app solo conoce URLs (API y servicio de correo); las credenciales del proveedor de correo viven en el servicio. |
| **Verificación** | Revisión de código. |
| **Estado al 09/11/2026** | Cumple. |
| **Origen (primera entrega)** | RNF-05 |

Fuente: elaboración propia.

**Tabla 59**
*Especificación del requisito no funcional RNF-SE-04*

| RNF-SE-04 · Los códigos de recuperación resisten la fuerza bruta | |
|---|---|
| **Atributo de calidad** | Seguridad (confidencialidad, integridad, autenticidad) (ISO/IEC 25010) |
| **Requisito** | Los códigos de recuperación resisten la fuerza bruta. |
| **Métrica o criterio** | 12 caracteres, un solo uso y 1 s de espera ante un código inválido. |
| **Cómo se cumple** | 12 caracteres sin letras ambiguas, de un solo uso, guardados como SHA-256; un código inválido responde con 1 s de espera. El del panel vence en 48 h. |
| **Verificación** | CP-M-06. |
| **Estado al 09/11/2026** | Cumple. |
| **Origen (primera entrega)** | RNF-05 |

Fuente: elaboración propia.

**Tabla 60**
*Especificación del requisito no funcional RNF-SE-05*

| RNF-SE-05 · El panel exige identidad y protege sus acciones | |
|---|---|
| **Atributo de calidad** | Seguridad (confidencialidad, integridad, autenticidad) (ISO/IEC 25010) |
| **Requisito** | El panel exige identidad y protege sus acciones. |
| **Métrica o criterio** | Todo POST del panel sin token CSRF se rechaza. |
| **Cómo se cumple** | Sesión iniciada y correo en la lista de administradores; todo POST exige la cabecera X-CSRF-Token. |
| **Verificación** | CP-M-08. |
| **Estado al 09/11/2026** | Cumple. |
| **Origen (primera entrega)** | RNF-05 |

Fuente: elaboración propia.

**Tabla 61**
*Especificación del requisito no funcional RNF-SE-06*

| RNF-SE-06 · Protección contra inyección y XSS | |
|---|---|
| **Atributo de calidad** | Seguridad (confidencialidad, integridad, autenticidad) (ISO/IEC 25010) |
| **Requisito** | Protección contra inyección y XSS. |
| **Métrica o criterio** | Cero consultas SQL concatenadas; los datos se pintan como texto. |
| **Cómo se cumple** | Consultas con el Query Builder de CodeIgniter (valores escapados); validación central de cada campo (tipos, longitudes, expresiones regulares, listas cerradas); en el panel los datos de la app se pintan con textContent. |
| **Verificación** | CP-M-03. |
| **Estado al 09/11/2026** | Cumple. |
| **Origen (primera entrega)** | RNF-05 |

Fuente: elaboración propia.

**Tabla 62**
*Especificación del requisito no funcional RNF-SE-07*

| RNF-SE-07 · Límites de entrada | |
|---|---|
| **Atributo de calidad** | Seguridad (confidencialidad, integridad, autenticidad) (ISO/IEC 25010) |
| **Requisito** | Límites de entrada. |
| **Métrica o criterio** | Cuerpo JSON de máximo 64 KB (413 si se supera). |
| **Cómo se cumple** | Cuerpo JSON máximo de 64 KB; identificadores y claves de catálogo validados con expresiones regulares. |
| **Verificación** | CP-M-03. |
| **Estado al 09/11/2026** | Cumple. |
| **Origen (primera entrega)** | RNF-05 |

Fuente: elaboración propia.

**Tabla 63**
*Especificación del requisito no funcional RNF-SE-08*

| RNF-SE-08 · Origen de las peticiones controlado | |
|---|---|
| **Atributo de calidad** | Seguridad (confidencialidad, integridad, autenticidad) (ISO/IEC 25010) |
| **Requisito** | Origen de las peticiones controlado. |
| **Métrica o criterio** | Un origen no permitido es bloqueado. |
| **Cómo se cumple** | CORS con lista de orígenes configurable (o los de Capacitor y Vite local). |
| **Verificación** | CP-M-04. |
| **Estado al 09/11/2026** | Cumple. |
| **Origen (primera entrega)** | RNF-05 |

Fuente: elaboración propia.

**Tabla 64**
*Especificación del requisito no funcional RNF-SE-09*

| RNF-SE-09 · Tratamiento de datos personales (Ley 1581 de 2012) | |
|---|---|
| **Atributo de calidad** | Seguridad (confidencialidad, integridad, autenticidad) (ISO/IEC 25010) |
| **Requisito** | Tratamiento de datos personales (Ley 1581 de 2012). |
| **Métrica o criterio** | Consentimiento explícito y supresión total al eliminar la cuenta. |
| **Cómo se cumple** | Consentimiento explícito para compartir la ficha médica; «Eliminar mi cuenta» suprime nombre, contactos, ficha, ubicaciones, puntos, notas y mensajes; solo quedan métricas anónimas. |
| **Verificación** | CP-M-07 y CP-E-24. |
| **Estado al 09/11/2026** | Cumple; la aprobación formal del tratamiento de datos está planeada en la fase 4. |
| **Origen (primera entrega)** | RNF-04 |

Fuente: elaboración propia.

### 3.4.2 RNF-DI · Disponibilidad y tolerancia a fallos

**Tabla 65**
*Especificación del requisito no funcional RNF-DI-01*

| RNF-DI-01 · La alerta local funciona sin internet | |
|---|---|
| **Atributo de calidad** | Fiabilidad (disponibilidad, tolerancia a fallos, recuperabilidad) (ISO/IEC 25010) |
| **Requisito** | La alerta local funciona sin internet. |
| **Métrica o criterio** | Sirena, linterna, vibración, brújula, guía y QR operan en modo avión. |
| **Cómo se cumple** | Sirena, linterna, vibración, brújula, guía y QR médico funcionan en el teléfono sin red. |
| **Verificación** | CP-E-33 y pruebas e2e sin conexión. |
| **Estado al 09/11/2026** | Cumple. |
| **Origen (primera entrega)** | RNF-03, RNF-10 |

Fuente: elaboración propia.

**Tabla 66**
*Especificación del requisito no funcional RNF-DI-02*

| RNF-DI-02 · Ningún dato se pierde sin conexión | |
|---|---|
| **Atributo de calidad** | Fiabilidad (disponibilidad, tolerancia a fallos, recuperabilidad) (ISO/IEC 25010) |
| **Requisito** | Ningún dato se pierde sin conexión. |
| **Métrica o criterio** | Cero cambios perdidos en las pruebas de cola. |
| **Cómo se cumple** | Primero se guarda en el teléfono; los cambios esperan en una cola persistente y se suben al volver la red, con reintentos y espera creciente. |
| **Verificación** | syncService.test.ts. |
| **Estado al 09/11/2026** | Cumple. |
| **Origen (primera entrega)** | RNF-03 |

Fuente: elaboración propia.

**Tabla 67**
*Especificación del requisito no funcional RNF-DI-03*

| RNF-DI-03 · Las escrituras se pueden repetir sin duplicar | |
|---|---|
| **Atributo de calidad** | Fiabilidad (disponibilidad, tolerancia a fallos, recuperabilidad) (ISO/IEC 25010) |
| **Requisito** | Las escrituras se pueden repetir sin duplicar. |
| **Métrica o criterio** | Repetir un PUT deja una sola fila. |
| **Cómo se cumple** | Cada registro lleva el UUID de la app con índice único y la API hace upsert. |
| **Verificación** | CP-M-02. |
| **Estado al 09/11/2026** | Cumple. |
| **Origen (primera entrega)** | RNF-03 |

Fuente: elaboración propia.

**Tabla 68**
*Especificación del requisito no funcional RNF-DI-04*

| RNF-DI-04 · La app sigue funcionando si un servicio externo falla | |
|---|---|
| **Atributo de calidad** | Fiabilidad (disponibilidad, tolerancia a fallos, recuperabilidad) (ISO/IEC 25010) |
| **Requisito** | La app sigue funcionando si un servicio externo falla. |
| **Métrica o criterio** | Ningún fallo externo bloquea la app. |
| **Cómo se cumple** | Catálogos: copia local y valores por defecto. Sismos: mensaje de error y reintento. Correo: estado «fallido» con reintento. Mapa: aviso en lugar de quedar gris. |
| **Verificación** | catalogService.test.ts, emailService.test.ts y quakeService.test.ts. |
| **Estado al 09/11/2026** | Cumple. |
| **Origen (primera entrega)** | RNF-03, RNF-10 |

Fuente: elaboración propia.

**Tabla 69**
*Especificación del requisito no funcional RNF-DI-05*

| RNF-DI-05 · Un error de pantalla no deja la app en blanco | |
|---|---|
| **Atributo de calidad** | Fiabilidad (disponibilidad, tolerancia a fallos, recuperabilidad) (ISO/IEC 25010) |
| **Requisito** | Un error de pantalla no deja la app en blanco. |
| **Métrica o criterio** | Ante un error se ve la pantalla de recuperación con el número de emergencia. |
| **Cómo se cumple** | Pantalla de error con opción de recargar y el número de emergencia visible. |
| **Verificación** | Revisión de código. |
| **Estado al 09/11/2026** | Cumple. |
| **Origen (primera entrega)** | RNF-03 |

Fuente: elaboración propia.

### 3.4.3 RNF-RE · Rendimiento y tiempos de respuesta

**Tabla 70**
*Especificación del requisito no funcional RNF-RE-01*

| RNF-RE-01 · La pantalla de carga no retrasa el uso de la app | |
|---|---|
| **Atributo de calidad** | Eficiencia de desempeño (comportamiento temporal) (ISO/IEC 25010) |
| **Requisito** | La pantalla de carga no retrasa el uso de la app. |
| **Métrica o criterio** | Mínimo 600 ms y máximo 1,5 s. |
| **Cómo se cumple** | La pantalla de carga se cierra entre 0,6 y 1,5 s y nunca tapa una alerta activa. |
| **Verificación** | CP-E-07. |
| **Estado al 09/11/2026** | Cumple. |
| **Origen (primera entrega)** | RNF-08 |

Fuente: elaboración propia.

**Tabla 71**
*Especificación del requisito no funcional RNF-RE-02*

| RNF-RE-02 · La alerta del panel llega rápido al operador | |
|---|---|
| **Atributo de calidad** | Eficiencia de desempeño (comportamiento temporal) (ISO/IEC 25010) |
| **Requisito** | La alerta del panel llega rápido al operador. |
| **Métrica o criterio** | Una alerta nueva aparece en unos 3 s. |
| **Cómo se cumple** | Pulso cada 3 s; respaldo completo cada 60 s; detalle de la persona al abrirla. |
| **Verificación** | CP-M-09. |
| **Estado al 09/11/2026** | Implementado; falta la medición con un teléfono real (fase 4). |
| **Origen (primera entrega)** | RNF-08 |

Fuente: elaboración propia.

**Tabla 72**
*Especificación del requisito no funcional RNF-RE-03*

| RNF-RE-03 · La ayuda del operador llega rápido a la persona | |
|---|---|
| **Atributo de calidad** | Eficiencia de desempeño (comportamiento temporal) (ISO/IEC 25010) |
| **Requisito** | La ayuda del operador llega rápido a la persona. |
| **Métrica o criterio** | Máximo 15 s. |
| **Cómo se cumple** | La app consulta el estado de la atención cada 15 s mientras la alerta está activa. |
| **Verificación** | CP-M-10. |
| **Estado al 09/11/2026** | Parcial: verificado con un servidor simulado. |
| **Origen (primera entrega)** | RNF-08 |

Fuente: elaboración propia.

**Tabla 73**
*Especificación del requisito no funcional RNF-RE-04*

| RNF-RE-04 · El catálogo de sismos no satura el servicio externo | |
|---|---|
| **Atributo de calidad** | Eficiencia de desempeño (comportamiento temporal) (ISO/IEC 25010) |
| **Requisito** | El catálogo de sismos no satura el servicio externo. |
| **Métrica o criterio** | Caché de 5 minutos. |
| **Cómo se cumple** | Caché de 5 minutos en el backend para el servicio del USGS. |
| **Verificación** | Revisión de código. |
| **Estado al 09/11/2026** | Cumple. |
| **Origen (primera entrega)** | RNF-08 |

Fuente: elaboración propia.

**Tabla 74**
*Especificación del requisito no funcional RNF-RE-05*

| RNF-RE-05 · La ubicación de la alerta es precisa sin demorar el aviso | |
|---|---|
| **Atributo de calidad** | Eficiencia de desempeño (comportamiento temporal) (ISO/IEC 25010) |
| **Requisito** | La ubicación de la alerta es precisa sin demorar el aviso. |
| **Métrica o criterio** | Afinado de hasta 10 s; actualización cada 2 min. |
| **Cómo se cumple** | Afinado de GPS de hasta 10 s y actualización cada 2 minutos. |
| **Verificación** | CP-E-17. |
| **Estado al 09/11/2026** | Cumple. |
| **Origen (primera entrega)** | RNF-08 |

Fuente: elaboración propia.

### 3.4.4 RNF-US · Usabilidad y accesibilidad

**Tabla 75**
*Especificación del requisito no funcional RNF-US-01*

| RNF-US-01 · Se puede usar con nervios y con una mano | |
|---|---|
| **Atributo de calidad** | Usabilidad (capacidad de interacción) (ISO/IEC 25010) |
| **Requisito** | Se puede usar con nervios y con una mano. |
| **Métrica o criterio** | La alerta se activa con dos toques (SOS y confirmar). |
| **Cómo se cumple** | Botón SOS grande, confirmación de un toque, textos cortos y barra de navegación inferior. |
| **Verificación** | Pruebas e2e en vista móvil (390 × 844). |
| **Estado al 09/11/2026** | Cumple. |
| **Origen (primera entrega)** | RNF-01 |

Fuente: elaboración propia.

**Tabla 76**
*Especificación del requisito no funcional RNF-US-02*

| RNF-US-02 · Bilingüe | |
|---|---|
| **Atributo de calidad** | Usabilidad (capacidad de interacción) (ISO/IEC 25010) |
| **Requisito** | Bilingüe. |
| **Métrica o criterio** | 100 % de las pantallas en español e inglés. |
| **Cómo se cumple** | Español e inglés en toda la app; catálogos con texto en ambos idiomas. |
| **Verificación** | CP-E-34. |
| **Estado al 09/11/2026** | Cumple. |
| **Origen (primera entrega)** | RNF-01 |

Fuente: elaboración propia.

**Tabla 77**
*Especificación del requisito no funcional RNF-US-03*

| RNF-US-03 · Accesible | |
|---|---|
| **Atributo de calidad** | Usabilidad (capacidad de interacción) (ISO/IEC 25010) |
| **Requisito** | Accesible. |
| **Métrica o criterio** | Opciones de alto contraste, texto grande y reducir movimiento persistentes. |
| **Cómo se cumple** | Alto contraste, texto grande, reducir movimiento, tema oscuro, lectura en voz alta de la guía y etiquetas ARIA. |
| **Verificación** | CP-E-15 y CP-E-35. |
| **Estado al 09/11/2026** | Cumple. |
| **Origen (primera entrega)** | RNF-02 |

Fuente: elaboración propia.

**Tabla 78**
*Especificación del requisito no funcional RNF-US-04*

| RNF-US-04 · Mensajes claros, nunca engañosos | |
|---|---|
| **Atributo de calidad** | Usabilidad (capacidad de interacción) (ISO/IEC 25010) |
| **Requisito** | Mensajes claros, nunca engañosos. |
| **Métrica o criterio** | Ningún «enviado» sin confirmación del canal. |
| **Cómo se cumple** | La app nunca dice que un SMS o correo salió si no es así; muestra los avisos de sin conexión, los cambios pendientes y el estado del correo. |
| **Verificación** | CP-E-10, CP-E-29 y CP-E-30. |
| **Estado al 09/11/2026** | Cumple. |
| **Origen (primera entrega)** | RNF-09 |

Fuente: elaboración propia.

**Tabla 79**
*Especificación del requisito no funcional RNF-US-05*

| RNF-US-05 · Usabilidad medible | |
|---|---|
| **Atributo de calidad** | Usabilidad (capacidad de interacción) (ISO/IEC 25010) |
| **Requisito** | Usabilidad medible. |
| **Métrica o criterio** | Puntaje SUS de 68 o más en la validación con usuarios. |
| **Cómo se cumple** | Encuesta SUS integrada (puntaje 0–100) y métricas de tiempo de reacción en simulacros. |
| **Verificación** | CP-E-20. |
| **Estado al 09/11/2026** | Instrumento implementado; la validación con usuarios está planeada en la fase 5. |
| **Origen (primera entrega)** | RNF-01 |

Fuente: elaboración propia.

### 3.4.5 RNF-PO · Portabilidad y compatibilidad

**Tabla 80**
*Especificación del requisito no funcional RNF-PO-01*

| RNF-PO-01 · Plataformas de la app | |
|---|---|
| **Atributo de calidad** | Portabilidad y compatibilidad (flexibilidad) (ISO/IEC 25010) |
| **Requisito** | Plataformas de la app. |
| **Métrica o criterio** | APK para Android y build web para navegadores modernos. |
| **Cómo se cumple** | Android (Capacitor 8, APK) y navegadores modernos (build web de Vite). iOS queda fuera del alcance actual. |
| **Verificación** | Pruebas e2e (móvil y escritorio) y APK generado en la integración continua. |
| **Estado al 09/11/2026** | Cumple; falta la prueba en un teléfono físico (fase 4). |
| **Origen (primera entrega)** | RNF-06 |

Fuente: elaboración propia.

**Tabla 81**
*Especificación del requisito no funcional RNF-PO-02*

| RNF-PO-02 · Plataforma del backend | |
|---|---|
| **Atributo de calidad** | Portabilidad y compatibilidad (flexibilidad) (ISO/IEC 25010) |
| **Requisito** | Plataforma del backend. |
| **Métrica o criterio** | PHP 7.3, CodeIgniter 3 y MySQL / MariaDB. |
| **Cómo se cumple** | PHP 7.3 y CodeIgniter 3 sobre Apache; MySQL / MariaDB (InnoDB, utf8mb4). |
| **Verificación** | Ambiente de desarrollo. |
| **Estado al 09/11/2026** | Cumple. |
| **Origen (primera entrega)** | RNF-06 |

Fuente: elaboración propia.

**Tabla 82**
*Especificación del requisito no funcional RNF-PO-03*

| RNF-PO-03 · Diseño adaptable | |
|---|---|
| **Atributo de calidad** | Portabilidad y compatibilidad (flexibilidad) (ISO/IEC 25010) |
| **Requisito** | Diseño adaptable. |
| **Métrica o criterio** | Sin scroll horizontal desde 390 px. |
| **Cómo se cumple** | App en vista móvil y escritorio; panel y página informativa sin scroll horizontal desde 390 px. |
| **Verificación** | Pruebas e2e en escritorio y revisión visual. |
| **Estado al 09/11/2026** | Cumple. |
| **Origen (primera entrega)** | RNF-06 |

Fuente: elaboración propia.

### 3.4.6 RNF-MA · Mantenibilidad y calidad

**Tabla 83**
*Especificación del requisito no funcional RNF-MA-01*

| RNF-MA-01 · Código tipado y revisado automáticamente | |
|---|---|
| **Atributo de calidad** | Mantenibilidad (modularidad, analizabilidad, capacidad de prueba) (ISO/IEC 25010) |
| **Requisito** | Código tipado y revisado automáticamente. |
| **Métrica o criterio** | Cero errores de ESLint y de TypeScript. |
| **Cómo se cumple** | TypeScript estricto, ESLint y tsc en cada cambio. |
| **Verificación** | ESLint y tsc en la integración continua. |
| **Estado al 09/11/2026** | Cumple. |
| **Origen (primera entrega)** | RNF-07 |

Fuente: elaboración propia.

**Tabla 84**
*Especificación del requisito no funcional RNF-MA-02*

| RNF-MA-02 · Pruebas automáticas en integración continua | |
|---|---|
| **Atributo de calidad** | Mantenibilidad (modularidad, analizabilidad, capacidad de prueba) (ISO/IEC 25010) |
| **Requisito** | Pruebas automáticas en integración continua. |
| **Métrica o criterio** | 100 % de las pruebas superadas en cada cambio. |
| **Cómo se cumple** | GitHub Actions: lint, tipos, pruebas unitarias (Vitest), e2e (Playwright, móvil y escritorio) y APK de depuración. |
| **Verificación** | Informe de resultados de pruebas (116 unitarias y 114 e2e). |
| **Estado al 09/11/2026** | Cumple. |
| **Origen (primera entrega)** | RNF-07 |

Fuente: elaboración propia.

**Tabla 85**
*Especificación del requisito no funcional RNF-MA-03*

| RNF-MA-03 · Contenido de negocio sin «quemar» en el código | |
|---|---|
| **Atributo de calidad** | Mantenibilidad (modularidad, analizabilidad, capacidad de prueba) (ISO/IEC 25010) |
| **Requisito** | Contenido de negocio sin «quemar» en el código. |
| **Métrica o criterio** | Catálogos, guía y parámetros editables sin publicar otra versión. |
| **Cómo se cumple** | Catálogos, guía y parámetros administrables desde el panel. |
| **Verificación** | CP-M-11. |
| **Estado al 09/11/2026** | Implementado; falta editar desde el panel con sesión (fase 4). |
| **Origen (primera entrega)** | RNF-07 |

Fuente: elaboración propia.

**Tabla 86**
*Especificación del requisito no funcional RNF-MA-04*

| RNF-MA-04 · Convención de base de datos | |
|---|---|
| **Atributo de calidad** | Mantenibilidad (modularidad, analizabilidad, capacidad de prueba) (ISO/IEC 25010) |
| **Requisito** | Convención de base de datos. |
| **Métrica o criterio** | 100 % de las tablas cumplen la convención. |
| **Cómo se cumple** | Tablas en plural y snake_case, todo comentado, claves foráneas con prefijo id_, auditoría completa y activo como único borrado. |
| **Verificación** | Revisión del script SQL. |
| **Estado al 09/11/2026** | Cumple. |
| **Origen (primera entrega)** | RNF-07 |

Fuente: elaboración propia.

**Tabla 87**
*Especificación del requisito no funcional RNF-MA-05*

| RNF-MA-05 · Trazabilidad de operaciones | |
|---|---|
| **Atributo de calidad** | Mantenibilidad (modularidad, analizabilidad, capacidad de prueba) (ISO/IEC 25010) |
| **Requisito** | Trazabilidad de operaciones. |
| **Métrica o criterio** | Toda petición tiene un execution_id (UUID v4). |
| **Cómo se cumple** | Logs JSON con execution_id por petición, también en la cabecera X-Execution-Id; el panel registra las consultas de datos sensibles, las atenciones y los códigos generados. |
| **Verificación** | Revisión de logs. |
| **Estado al 09/11/2026** | Cumple. |
| **Origen (primera entrega)** | RNF-05, RNF-07 |

Fuente: elaboración propia.

**Tabla 88**
*Especificación del requisito no funcional RNF-MA-06*

| RNF-MA-06 · Script de base de datos reproducible | |
|---|---|
| **Atributo de calidad** | Mantenibilidad (modularidad, analizabilidad, capacidad de prueba) (ISO/IEC 25010) |
| **Requisito** | Script de base de datos reproducible. |
| **Métrica o criterio** | El script se puede ejecutar varias veces sin error. |
| **Cómo se cumple** | Un único script SQL idempotente (CREATE TABLE IF NOT EXISTS, ALTER … MODIFY, INSERT IGNORE). |
| **Verificación** | Reejecución del script en desarrollo. |
| **Estado al 09/11/2026** | Cumple. |
| **Origen (primera entrega)** | RNF-07 |

Fuente: elaboración propia.

## 3.5 Arquitectura conceptual

La arquitectura separa presentación, lógica de alerta, adaptadores del dispositivo y servicios externos. La presentación contiene pantalla principal y estados; la lógica controla transiciones y validaciones; los adaptadores encapsulan geolocalización y señalización local; los servicios externos, si se implementan, gestionan la comunicación digital.

**Tabla 89**

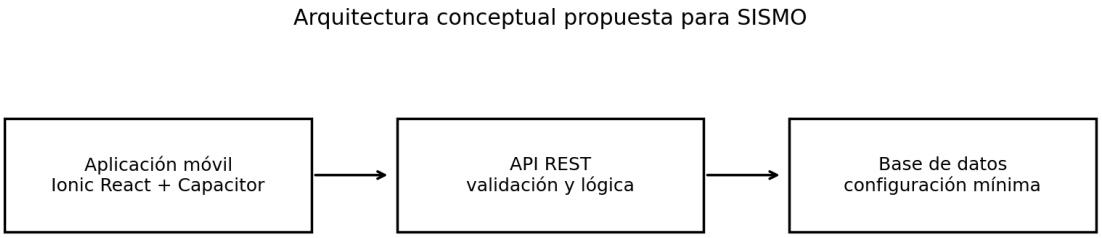
*Componentes de la arquitectura*

| Componente | Responsabilidad | Dependencias |
|---|---|---|
| Interfaz móvil | Mostrar botón, configuración, estados y mensajes | Ionic React |
| Gestor de alertas | Validar activación y coordinar acciones | Interfaz y adaptadores |
| Adaptador de ubicación | Solicitar permisos y obtener coordenadas disponibles | Capacidades nativas |
| Señalización local | Gestionar sonido y confirmación visual | Dispositivo |
| Gestor de contactos | Validar y preparar contactos | Almacenamiento definido |
| Adaptador de comunicación | Solicitar envío por canal integrado | Conectividad y proveedor |
| Registro técnico | Registrar eventos no sensibles | Política de privacidad |

Fuente: elaboración propia.

En la primera entrega esta arquitectura era conceptual y la API REST y la base de datos eran opcionales. En la segunda entrega se implementaron: la app (Ionic React con Capacitor 8) se comunica con una API REST en PHP y CodeIgniter 3 que guarda los datos en MySQL; un panel web permite al centro de monitoreo atender las alertas, y un servicio de correo envía los avisos a los contactos. La arquitectura implementada, los componentes, el despliegue y el modelo de datos se presentan en la sección 4.5.

**Figura 1**
*Arquitectura conceptual propuesta para SISMO (primera entrega)*




Fuente: elaboración propia.

## 3.6 Modelo de estados de alerta

**Tabla 90**
*Modelo de estados de la alerta*

| Estado | Descripción | Transición esperada |
|---|---|---|
| Inactiva | No existe alerta en curso | El usuario activa el botón |
| Preparando | Se valida acción y se recopilan datos disponibles | Se completa validación o se informa limitación |
| Activa localmente | Se confirma activación y se ejecutan señales locales | Se solicita canal digital o continúa localmente |
| Enviando | Servicio digital procesa solicitud | Confirmación o fallo |
| Confirmada | Servicio devuelve confirmación verificable | Se informa resultado |
| Pendiente/fallida | No se pudo confirmar envío | Se informa limitación |
| Finalizada | El usuario detiene el flujo | Regresa a inactiva |

Fuente: elaboración propia.

### 3.6.1 Criticidad y priorización de requisitos

La criticidad se clasifica según el efecto que tendría la ausencia de cada función sobre el objetivo central de SISMO. Es una valoración de diseño para orientar el desarrollo; deberá revisarse con las pruebas y no representa una medición de usuarios.

**Tabla 91**
*Criticidad y priorización de los requisitos detallados*

| Nivel | Requisitos asociados | Justificación | Tratamiento |
|---|---|---|---|

| Nivel | Requisitos asociados | Justificación | Tratamiento |
|---|---|---|---|
| **Crítica** | RF-AL-01 a RF-AL-04, RF-AL-08, RF-RE-02, RF-SI-01, RF-SI-02, RNF-DI-01, RNF-US-04 | Sin ellos no existe la alerta, se perderían datos o se informaría un estado falso. | Implementados en las versiones 1.0 a 2.1 y cubiertos por pruebas automáticas. |
| **Alta** | RF-AL-05 a RF-AL-07, RF-AL-09, RF-CO-01 a RF-CO-04, RF-CT-01, RF-CT-02, RF-IN-01, RF-IN-02, RF-RE-05, RF-CF-02, RF-CF-03, RF-PA-01 a RF-PA-05, RNF-SE-01 a RNF-SE-09 | Amplían el alcance de la alerta, protegen los datos y permiten la atención del centro de monitoreo. | Implementados; faltan las pruebas en producción y en un teléfono físico. |
| **Media** | RF-RE-01, RF-RE-03, RF-RE-04, RF-AL-10, RF-CO-05, RF-CO-06, RF-CT-03, RF-IN-03 a RF-IN-06, RF-CF-01, RF-SI-03, RF-SI-04, RF-PA-06 a RF-PA-08 | Preparación, soporte y administración del contenido. | Implementados después del flujo principal. |
| **Baja** | RF-IN-07, RF-PI-01, RF-PI-02 | Comodidad y divulgación. | Implementados en la versión 3.2. |
| **Transversal** | RNF-RE, RNF-US, RNF-PO y RNF-MA | Aplican a todas las funciones. | Revisión en cada versión y en la integración continua. |

Fuente: elaboración propia.

## 3.7 Plan de pruebas

**Tabla 92**
*Casos de prueba de la primera entrega*

| ID | Escenario | Resultado esperado | Evidencia |
|---|---|---|---|
| CP-01 | Abrir aplicación | Pantalla principal sin errores | Dispositivo y resultado |
| CP-02 | Activar botón | Interfaz refleja el estado | Captura y observación |
| CP-03 | Activar/detener sonido | Señal local controlable | Resultado |
| CP-04 | Denegar ubicación | Limitación informada | Captura |
| CP-05 | Permitir ubicación | Se informa disponibilidad de coordenadas | Resultado sin coordenadas reales |
| CP-06 | Desactivar internet | Canal digital no confirmado | Captura |
| CP-07 | Fallo de servicio | No se declara entrega | Mensaje y registro |
| CP-08 | Contacto inválido | Se solicita corrección | Entrada de prueba |

| | | | |
|---|---|---|---|
| CP-09 | Pulsaciones repetidas | No se generan estados inconsistentes | Secuencia |
| CP-10 | Revisar legibilidad | Mensajes comprensibles | Lista de verificación |

Matriz básica de dispositivos y condiciones de red. Ejecutar las pruebas únicamente en dispositivos propios o autorizados, usando contactos ficticios y sin emitir alertas reales.

**Tabla 93**
*Matriz de dispositivos y condiciones de red*

| Entorno | Condición | Pruebas principales | Evidencia esperada |
|---|---|---|---|
| **Android físico (versión disponible)** | Wi-Fi estable | Inicio, activación local, sonido, navegación y persistencia | Capturas y resultado por caso |
| **Android físico o emulador** | Datos móviles | Permisos, ubicación disponible y cambio de red | Registro de permisos y estados |
| **Navegador de escritorio** | Wi-Fi estable | Diseño adaptable, navegación, formularios y validaciones | Capturas de las vistas |
| **Entorno de prueba** | Modo avión / sin internet | No marcar el envío como confirmado; mantener claro el estado local | Captura de estado pendiente/fallido |
| **Android físico o emulador** | Ubicación denegada | Explicar la limitación y permitir continuar sin ubicación | Captura y registro CP-04 |
| **Dos tamaños de pantalla** | Red estable y desconectada | Legibilidad, botones, desbordamientos y mensajes | Lista de verificación visual |

Fuente: elaboración propia.

En la segunda entrega este plan se amplió y se automatizó: 16 archivos de pruebas unitarias con Vitest (116 pruebas), 57 escenarios extremo a extremo con Playwright ejecutados en vista móvil (390 × 844) y de escritorio (114 ejecuciones) y 14 casos manuales para el backend, el panel y el dispositivo. Las pruebas automáticas corren en la integración continua (GitHub Actions) con cada cambio. Los casos CP-01 a CP-10 de la primera entrega quedaron cubiertos por los nuevos casos; los resultados están en la sección 4.6.

## 3.8 Gestión de riesgos

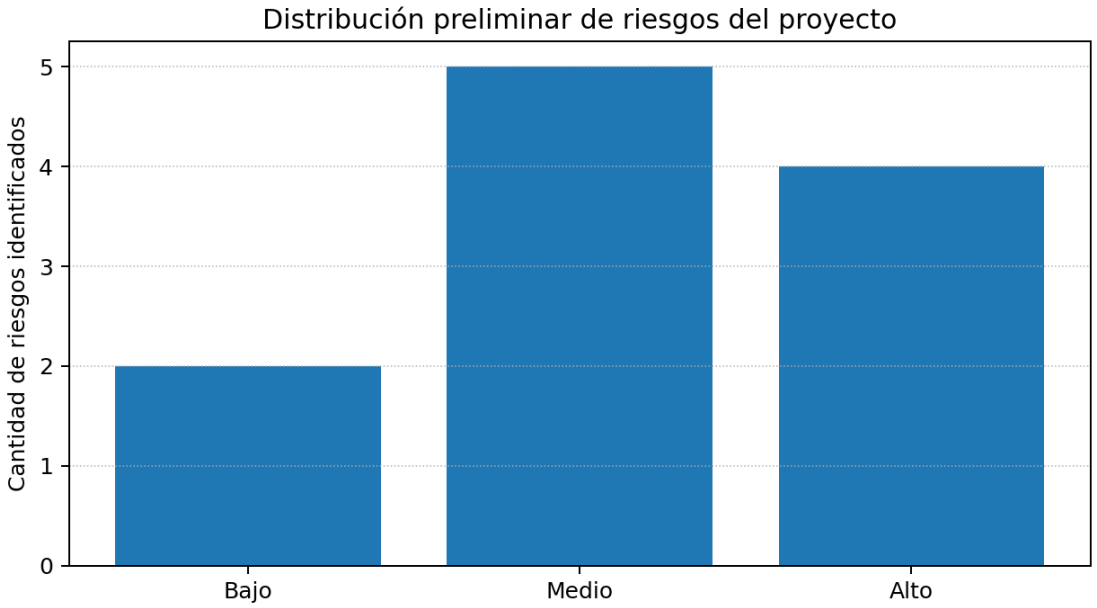
**Tabla 94**
*Riesgos del proyecto*

| Riesgo | Probabilidad | Impacto | Mitigación |
|---|---|---|---|

| | | | |
|---|---|---|---|
| Ausencia de conectividad | Alta | Alto | Estados locales y aviso de envío no confirmado. |
| Ubicación imprecisa | Media-alta | Alto | Continuar sin coordenadas e informar limitación. |
| Activación accidental | Media | Medio-alto | Prevenir errores sin añadir pasos innecesarios. |
| Restricciones de servicios externos | Media | Alto | Priorizar versión base y aislar integraciones. |
| Permisos denegados | Media | Medio | Explicar propósito y manejar denegación. |
| Consumo de batería | Media | Medio | Limitar señales y evitar seguimiento continuo. |
| Exposición de datos | Baja-media | Alto | Minimizar datos y proteger almacenamiento. |
| Incompatibilidad de dispositivos | Media | Medio | Definir equipos objetivo y ejecutar pruebas. |
| Confusión con servicio oficial | Media | Alto | Advertencias claras sobre alcance. |
| Tiempo insuficiente | Media | Medio-alto | Priorizar requisitos esenciales y documentar pendientes. |

Fuente: elaboración propia.

**Figura 2**
*Priorización preliminar de riesgos del proyecto*




Fuente: elaboración propia; valoración de planificación.

En la segunda entrega el registro se amplió a 17 riesgos (R-01 a R-17), con seguimiento por versión. Los riesgos de ausencia de conectividad y de exposición de datos se mitigaron con la cola de sincronización sin conexión y el cifrado de la ficha médica; siguen abiertos los que dependen de pruebas en producción y en un teléfono físico, que se atienden en la fase 4 del cronograma.

## 3.9 Cronograma con fechas y comparación con el avance real

El cronograma de 16 semanas de la primera entrega no tenía fechas. En esta entrega se replanificó con las fechas reales del periodo académico, del martes 6 de octubre al viernes 11 de diciembre de 2026 (49 días hábiles, de lunes a viernes), en cinco fases, y se comparó con el avance real.

### 3.9.1 Plan con fechas (06/10/2026 – 11/12/2026)

La tabla siguiente presenta cada actividad con sus fechas y su estado al 9 de noviembre de 2026. Las fases 1 a 3 ya se ejecutaron y respetan las fechas de las entregas (versión 1 el 23/10, versión 2 el 02/11 y versión 3 el 04/11); las fases 4 y 5 son el plan para lo que falta.

**Tabla 95**
*Cronograma con fechas por actividad*

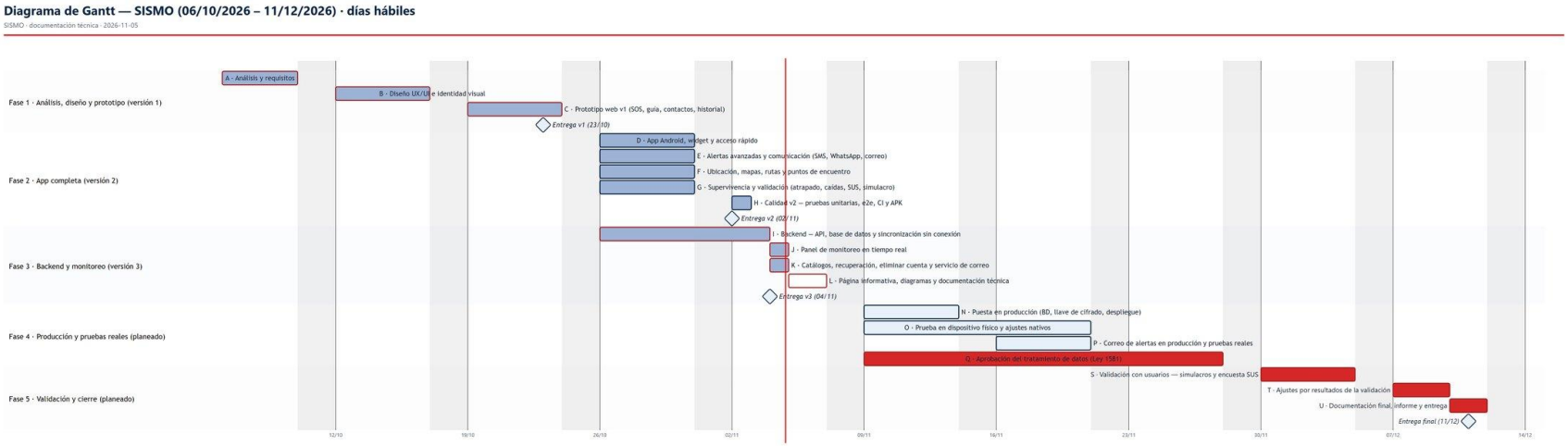
| Act. | Actividad | Fase | Fechas (2026) | Días hábiles | Estado al 09/11 |
|---|---|---|---|---|---|
| A | Análisis y requisitos | 1 | 06/10 – 09/10 | 4 | Terminada |
| B | Diseño UX/UI e identidad visual | 1 | 12/10 – 16/10 | 5 | Terminada |

| Act. | Actividad | Fase | Fechas (2026) | Días hábiles | Estado al 09/11 |
|---|---|---|---|---|---|
| C | Prototipo web v1 (SOS, guía, contactos, historial) | 1 | 19/10 – 23/10 | 5 | Terminada |
| D | App Android, widget y acceso rápido | 2 | 26/10 – 30/10 | 5 | Terminada |
| E | Alertas avanzadas y comunicación (SMS, WhatsApp, correo) | 2 | 26/10 – 30/10 | 5 | Terminada |
| F | Ubicación, mapas, rutas y puntos de encuentro | 2 | 26/10 – 30/10 | 5 | Terminada |
| G | Supervivencia y validación (atrapado, caídas, SUS, simulacro) | 2 | 26/10 – 30/10 | 5 | Terminada |
| H | Calidad: pruebas unitarias, e2e, integración continua y APK | 2 | 02/11 | 1 | Terminada |
| I | Backend: API, base de datos y sincronización sin conexión | 3 | 26/10 – 03/11 | 7 | Terminada |
| J | Panel de monitoreo en tiempo real | 3 | 04/11 | 1 | Terminada |
| K | Catálogos, recuperación, eliminar cuenta y servicio de correo | 3 | 04/11 | 1 | Terminada |
| L | Página informativa, diagramas y documentación técnica | 3 | 05/11 – 06/11 | 2 | Terminada |
| N | Puesta en producción (base de datos, llave de cifrado, despliegue) | 4 | 09/11 – 13/11 | 5 | En curso |
| O | Prueba en dispositivo físico y ajustes nativos | 4 | 09/11 – 20/11 | 10 | En curso |
| P | Correo de alertas en producción y pruebas reales | 4 | 16/11 – 20/11 | 5 | Planeada |
| Q | Aprobación del tratamiento de datos (Ley 1581) | 4 | 09/11 – 27/11 | 15 | En curso |
| S | Validación con usuarios: simulacros y encuesta SUS | 5 | 30/11 – 04/12 | 5 | Planeada |
| T | Ajustes por resultados de la validación | 5 | 07/12 – 09/12 | 3 | Planeada |
| U | Documentación final, informe y entrega | 5 | 10/12 – 11/12 | 2 | Planeada |

Fuente: elaboración propia.

**Figura 3**

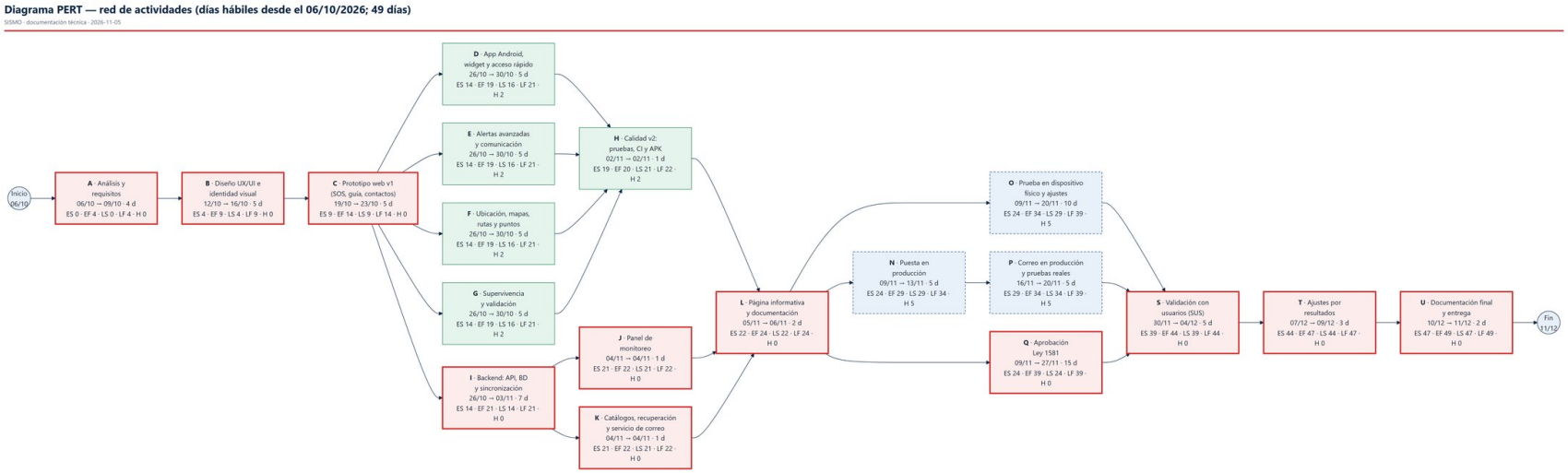
*Diagrama de Gantt del proyecto SISMO (06/10/2026 – 11/12/2026, días hábiles)*




Fuente: elaboración propia.

**Figura 4**

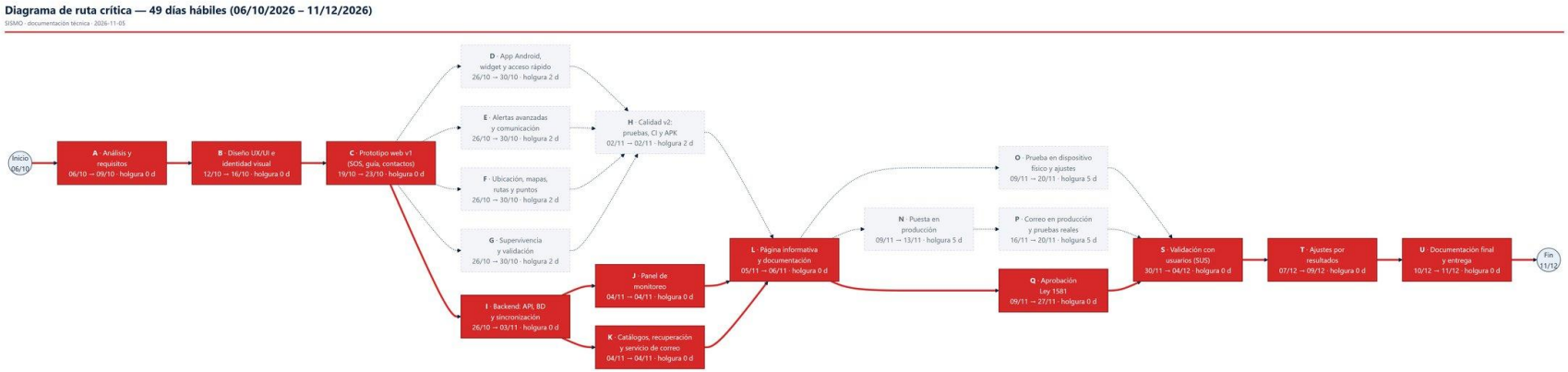
*Red PERT con inicio y fin tempranos y tardíos y holgura por actividad*




Fuente: elaboración propia.

**Figura 5**

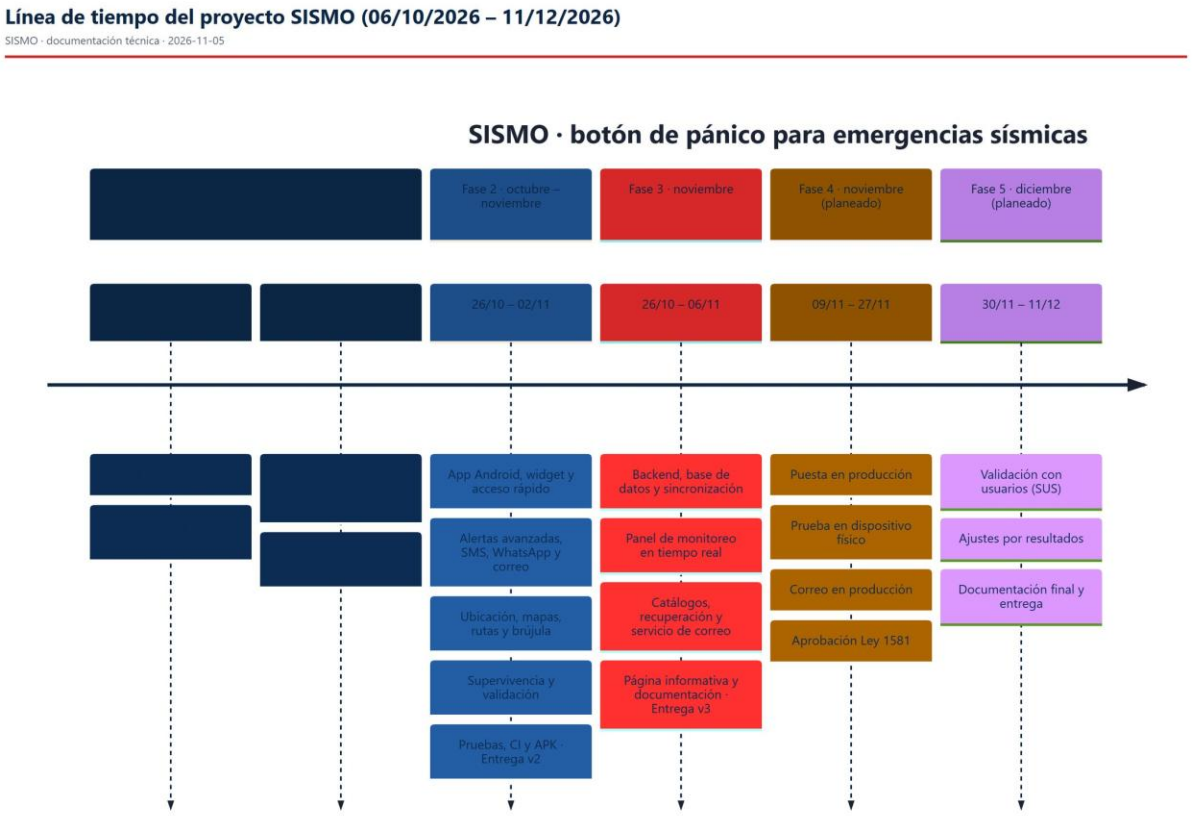
*Ruta crítica del proyecto (método CPM)*




Fuente: elaboración propia.

La ruta crítica es A → B → C → I → J/K → L → Q → S → T → U (49 días hábiles): un retraso en cualquiera de estas actividades mueve la entrega final del 11/12/2026. Las actividades D a H se ejecutaron en paralelo con la I y tienen holgura. Los hitos son la entrega de la versión 1 (23/10), la versión 2 (02/11), la versión 3 (04/11) y la entrega final (11/12).

**Figura 6**
*Línea de tiempo por versión y próximos pasos*




Fuente: elaboración propia.

### 3.9.2 Comparación entre el plan y el avance real

La revisión de la primera entrega tomó como fecha de corte el 26 de octubre de 2026, cuando el plan esperaba tener en curso las semanas 5 y 6 (configuración del proyecto, estructura y navegación). La tabla siguiente compara cada actividad del cronograma original con su ejecución real y con su estado en esa fecha y al 9 de noviembre de 2026.

Al 26 de octubre las semanas 5 y 6 ya estaban terminadas: el 23 de octubre se entregó la versión 1, un prototipo web con el proyecto configurado, la navegación entre pantallas, el botón SOS, la guía, los contactos y el historial. Las capturas fechadas del 24 de octubre del Anexo D son la evidencia de ese avance. Desde entonces el proyecto va adelantado frente al plan original: las actividades de las semanas 7 a 12 se terminaron entre el 26 de octubre y el 6 de noviembre porque se trabajaron en paralelo (actividades D a I).

**Tabla 96**
*Comparación del cronograma de 16 semanas con la ejecución real*

| Actividad (plan original) | Semanas | Ejecución real (2026) | Estado al 26/10/2026 | Estado al 09/11/2026 |
|---|---|---|---|---|
| **Análisis y delimitación** | 1–2 | 06/10 – 09/10 (A) | Terminada | Terminada |
| **Diseño de experiencia y arquitectura** | 3–4 | 12/10 – 16/10 (B) | Terminada | Terminada; actualizada con 33 diagramas |
| **Configuración del proyecto** | 5–6 | 19/10 – 23/10 (C) | Terminada (versión 1, Anexo D) | Terminada |
| **Botón y estados** | 7–8 | 19/10 – 30/10 (C y E) | Flujo local terminado; alertas avanzadas en curso | Terminada |
| **Ubicación y contactos** | 9–10 | 26/10 – 30/10 (F) | En curso | Terminada |
| **Integraciones viables** | 11–12 | 26/10 – 04/11 (I, J y K) | En curso (backend iniciado) | Terminada; correo en producción en la fase 4 |
| **Pruebas y correcciones** | 13–14 | 02/11 (H) y 09/11 – 04/12 (O, P y S) | No iniciada | En curso: 230 pruebas automáticas superadas; faltan las pruebas en teléfono físico, en producción y con usuarios |
| **Documentación y cierre** | 15–16 | 05/11 – 06/11 (L) y 10/12 – 11/12 (U) | No iniciada | En curso: documentación técnica y diagramas terminados; falta el informe final |

Fuente: elaboración propia.

El repositorio de la aplicación se publicó el 2 de noviembre de 2026 con el trabajo acumulado de las fases 1 y 2, por lo que la evidencia del avance al 26 de octubre son las capturas fechadas del Anexo D. Desde esa fecha cada versión queda registrada en el historial Git.

**Tabla 97**
*Hitos verificables en el repositorio de la aplicación*

| Commit | Fecha | Descripción |
|---|---|---|
| **dfab238** | 02/11/2026 | Prototipo de botón de pánico (web y Android) |
| **968615a** | 02/11/2026 | Modo atrapado, silbato, detección de caídas, SUS, mochila, Vitest e integración continua |

| Commit | Fecha | Descripción |
|---|---|---|
| 003ed95 | 02/11/2026 | Registro obligatorio: nombre, ubicación, punto de encuentro y contacto |
| a9c8c39 | 02/11/2026 | Widget SOS con doble toque (1x1 y 2x2) |
| ef9bc54 | 04/11/2026 | Backend y base de datos con sincronización sin conexión, catálogos, recuperación y eliminación de cuenta |
| 74f8328 | 04/11/2026 | Documentación gráfica (33 diagramas) y documentación técnica y funcional |
| ff15227 | 04/11/2026 | Planificación con fechas: 06/10/2026 – 11/12/2026 en 5 fases |
| 8f32c20 | 05/11/2026 | Ayuda urgente, mensajes con formato, sismos con gráfica e Inicio que avisa qué falta |

Fuente: historial Git del repositorio Sismo_front (rama main).

## 3.10 Privacidad y seguridad

El prototipo debe utilizar datos de prueba y evitar divulgar ubicaciones reales. Si se almacenan contactos, debe definirse la finalidad, el acceso y el tiempo de conservación. Las pruebas serán controladas, sin enviar alertas falsas a servicios oficiales. Como medidas mínimas: (1) solicitar permiso de ubicación solo cuando el usuario active la función que lo necesita; (2) explicar en pantalla para qué se utilizarán las coordenadas y los contactos; (3) utilizar contactos ficticios durante las pruebas; (4) guardar únicamente los datos necesarios y limitar el acceso al dispositivo de prueba; (5) no incluir teléfonos ni coordenadas en logs o capturas públicas; (6) permitir eliminar contactos e historial de demostración; (7) evitar seguimiento continuo en segundo plano; (8) no transmitir datos a terceros sin integración explícita y consentimiento; y (9) documentar la eliminación de datos al cerrar la prueba.

Para la versión de demostración, la información permanecerá localmente solo mientras sea necesaria para la prueba y podrá borrarse desde Configuración. Cualquier integración futura deberá definir consentimiento, finalidad, seguridad del transporte, periodo de conservación y procedimiento de eliminación antes de utilizar datos reales.

# 4. RESULTADOS Y AVANCES

En la primera entrega los resultados eran productos de formulación y diseño. En esta segunda entrega se suman los resultados de la implementación: la app móvil (web y Android), el backend con su base de datos, el panel de monitoreo, la página informativa, las pruebas automáticas y manuales y la documentación técnica. Las funciones que aún dependen de pruebas en producción, en un teléfono físico o con usuarios se reportan como pendientes, no como aprobadas.

## 4.1 Productos de análisis y diseño

**Tabla 98**
*Productos de análisis y diseño*

| Producto | Contenido | Verificación |
|---|---|---|
| Problema | Contexto, necesidad y delimitaciones | Coherencia con objetivos |
| Requisitos | Requisitos funcionales y no funcionales | Matriz de trazabilidad |
| Arquitectura | Interfaz, lógica, adaptadores y servicios | Revisión de dependencias |
| Estados | Estados y transiciones de alerta | Casos normales y de error |
| Plan de pruebas | Escenarios de permisos, conexión y activación | Registro de ejecución |
| Riesgos | Riesgos técnicos, privacidad y planificación | Seguimiento por iteración |
| Cronograma | 16 semanas replanificadas con fechas (06/10 – 11/12/2026) | Comparación con el avance real (sección 3.9.2) |

Fuente: elaboración propia.

## 4.2 Trazabilidad entre objetivos y evidencias

**Tabla 99**
*Trazabilidad entre objetivos, evidencias y avance*

| Objetivo | Evidencia | Avance estimado al 09/11/2026 |
|---|---|---|
| **OE1 · Identificar necesidades** | 48 RF y 33 RNF con una tabla por requisito (secciones 3.3 y 3.4) y 17 riesgos | 100 % |
| **OE2 · Definir la arquitectura** | 33 diagramas: UML, arquitectura, base de datos y planificación (secciones 3.9 y 4.5) | 100 % |
| **OE3 · Implementar el flujo de alerta** | Versiones 1.0 a 3.2, Anexos E y G, casos CP-E-09 a CP-E-28 | 90 % (falta la medición en un teléfono físico) |

| Objetivo | Evidencia | Avance estimado al 09/11/2026 |
|---|---|---|
| **OE4 · Contactos, ubicación y sincronización** | Registro obligatorio, cola sin conexión, backend y panel (Anexos E, F y G) | 90 % (faltan las pruebas en producción) |
| **OE5 · Evaluar el prototipo** | 116 pruebas unitarias y 114 e2e superadas; 8 de 14 casos manuales | 50 % (faltan casos manuales y la validación con usuarios) |

Fuente: elaboración propia.

## 4.3 Evidencias recopiladas y pendientes

- Recopiladas: capturas fechadas de la app (02/11, 04/11 y 05/11/2026) con datos ficticios (Anexo E).
- Recopiladas: capturas del emulador Android 16 con el widget, la alerta desde el widget y la notificación (Anexo E).
- Recopiladas: capturas del panel de monitoreo y de la página informativa (Anexo F).
- Recopiladas: capturas del código fuente de la app y del backend (Anexo G).
- Recopilados: resultados de 116 pruebas unitarias y de 114 ejecuciones e2e, y 7 defectos corregidos (sección 4.6).
- Recopilado: historial de versiones en el repositorio Git (sección 3.9.2).
- Pendientes: pruebas en un teléfono físico, en producción (correo y cifrado) y validación con usuarios (fases 4 y 5).

## 4.4 Interpretación

La evaluación deberá contrastar resultados reales con criterios de aceptación. Si un canal digital no se integra, se reportará pendiente, no aprobado. La calidad también se valorará por la claridad de las limitaciones, el tratamiento de errores, la protección de datos y la trazabilidad.

## 4.5 Avance de implementación de la segunda entrega

Entre el 26 de octubre y el 6 de noviembre de 2026 se ejecutaron las fases 2 y 3 del cronograma. La app pasó de un prototipo web a una app Android completa con widget, señales nativas, correo de alerta, mapas y sincronización sin conexión; además se construyeron el backend, la base de datos, el panel del centro de monitoreo y la página informativa.

**Tabla 100**

*Tecnologías implementadas por capa*

| Capa | Tecnología | Uso en SISMO |
|---|---|---|
| **App móvil** | Ionic React, React y TypeScript; compilación con Vite | Pantallas, navegación (17 rutas), estado y lógica de la alerta |
| **Acceso nativo** | Capacitor 8 y código nativo de Android | Ubicación, linterna, vibración, servicio con la pantalla bloqueada, widget y acceso rápido |

| Capa | Tecnología | Uso en SISMO |
|---|---|---|
| **Mapas** | Leaflet y OpenStreetMap | Ubicación, puntos de encuentro y ruta |
| **Backend** | PHP 7.3 y CodeIgniter 3 (API REST con token por dispositivo) | Registro, sincronización, recuperación, eliminación de cuenta y panel |
| **Base de datos** | MySQL / MariaDB (InnoDB, utf8mb4) | 13 tablas del sistema en un script SQL idempotente |
| **Servicios externos** | USGS (GeoJSON) y servicio de correo de alertas | Sismos recientes y correo a los contactos |
| **Calidad** | Vitest, Playwright, ESLint, TypeScript y GitHub Actions | Pruebas unitarias y e2e, revisión estática y APK con cada cambio |

Fuente: elaboración propia.

**Tabla 101**
*Versiones entregadas y funcionalidades por versión*

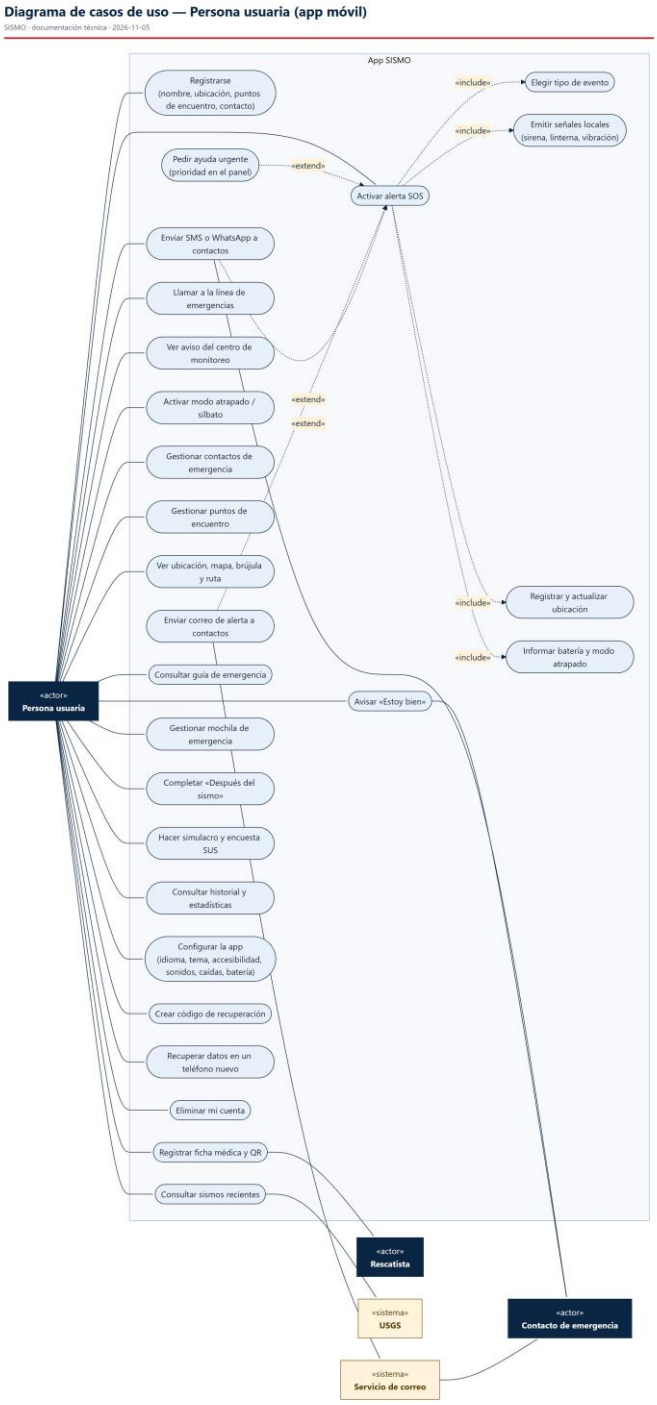
| Versión | Fechas (2026) | Actividad | Funcionalidades principales |
|---|---|---|---|
| **1.0** | 19/10 – 23/10 | Fase 1 · C | Prototipo web del botón de pánico: SOS, guía, contactos e historial (entrega v1) |
| **1.1** | 26/10 – 30/10 | Fase 2 · D | App Android, widget SOS con doble toque y acceso rápido |
| **1.2** | 26/10 – 30/10 | Fase 2 · E | Señales nativas, botón 123, SMS y WhatsApp, «Estoy bien» y correo de alerta |
| **1.3** | 26/10 – 30/10 | Fase 2 · F | Mapa real, brújula, rutas y varios puntos de encuentro |
| **1.4** | 26/10 – 30/10 | Fase 2 · G | Modo atrapado, silbato, detección de caídas, simulacro, encuesta SUS y mochila |
| **2.0** | 02/11 | Fase 2 · H | Pruebas unitarias y e2e, integración continua y APK (entrega v2) |
| **2.1** | 26/10 – 03/11 | Fase 3 · I | Backend, base de datos, sincronización sin conexión y ficha médica cifrada |
| **3.0** | 04/11 | Fase 3 · J y K | Panel de monitoreo, catálogos administrables, recuperación, eliminar cuenta y servicio de correo (entrega v3) |
| **3.2** | 05/11 – 06/11 | Fase 3 · L | Ayuda urgente y prioridad de alertas, plantillas de mensajes, evento masivo, sismos con gráfica y mapa, página informativa y documentación |

Fuente: elaboración propia con base en el registro de cambios del proyecto.

### 4.5.1 Diagramas UML y funcionales

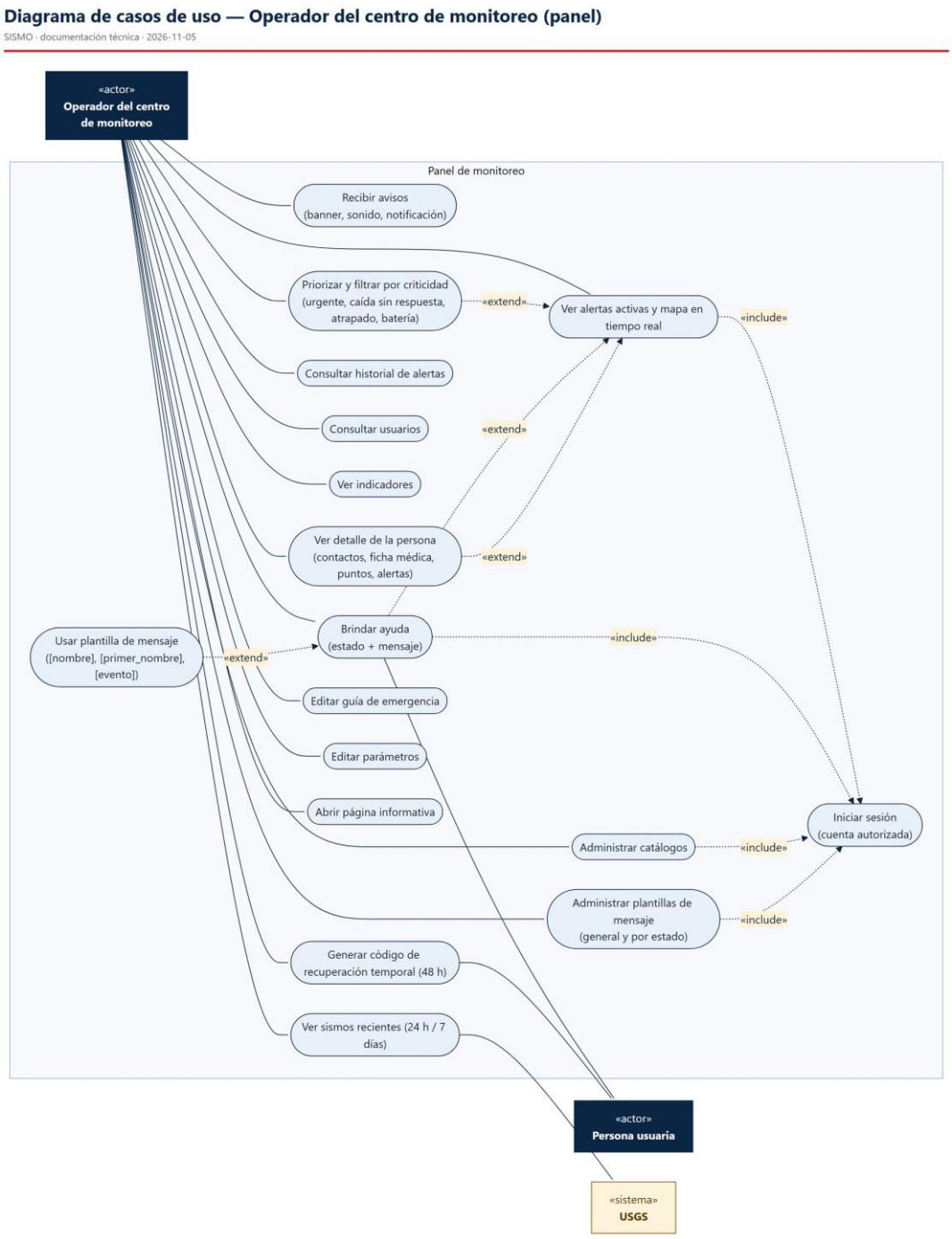
Los diagramas se elaboraron en Mermaid y se actualizan con cada versión. Incluyen casos de uso por actor, actividades, secuencia, clases, estados y flujos de proceso.

**Figura 7**
*Casos de uso de la persona usuaria*



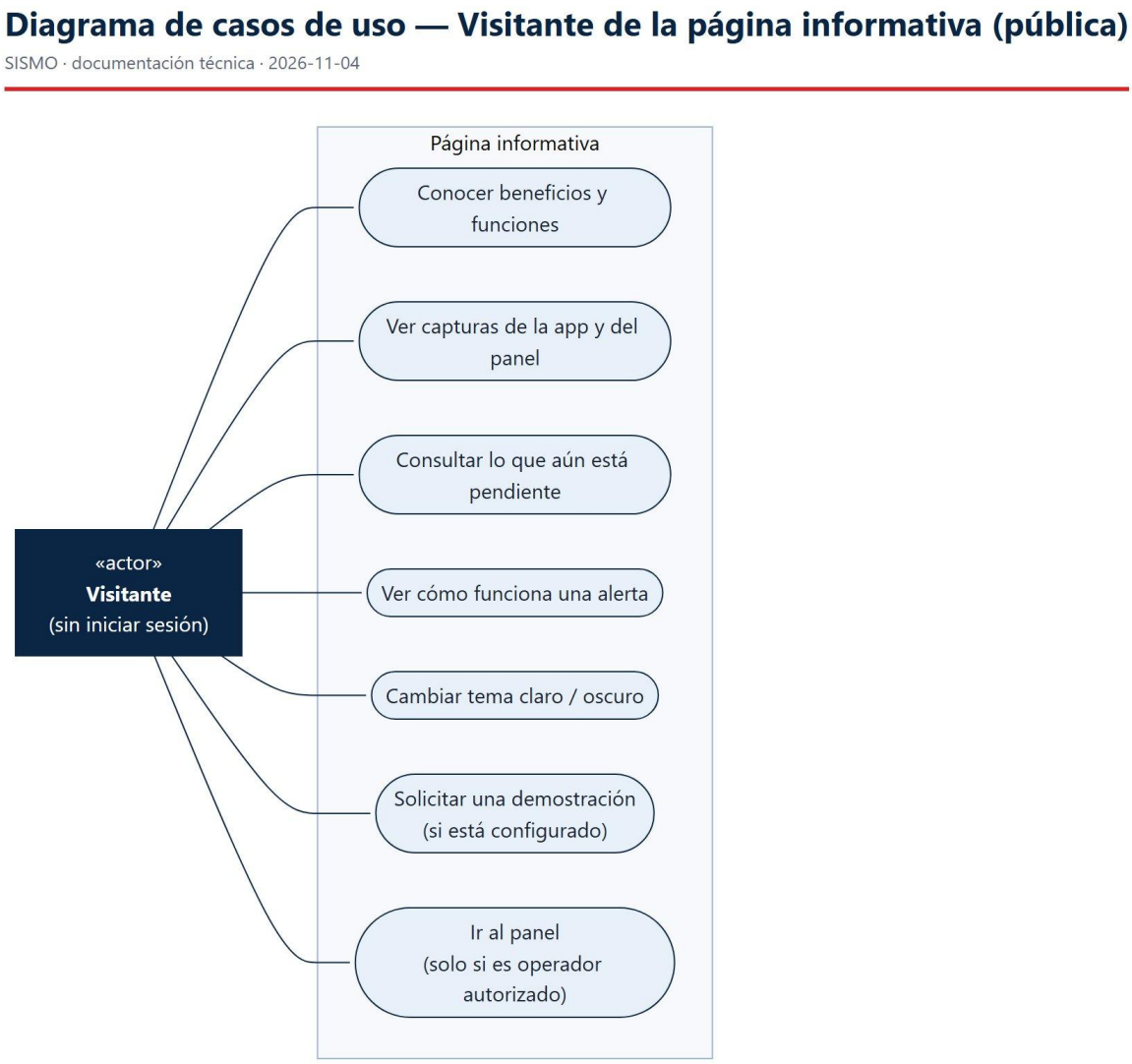

Fuente: elaboración propia.

**Figura 8**
*Casos de uso del operador del centro de monitoreo*



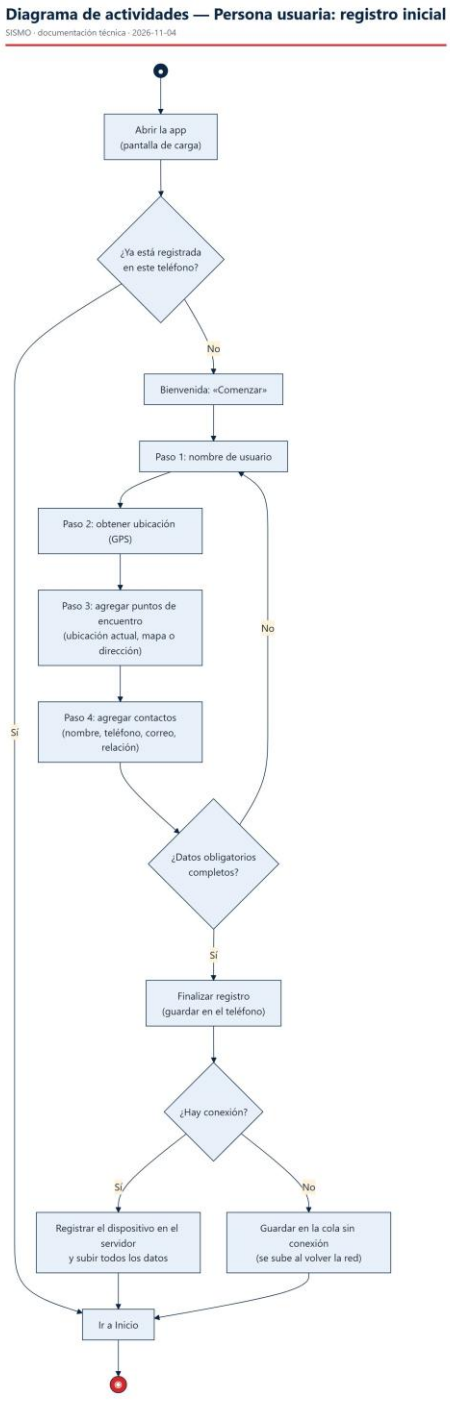

Fuente: elaboración propia.

**Figura 9**
*Casos de uso del visitante de la página informativa*




Fuente: elaboración propia.

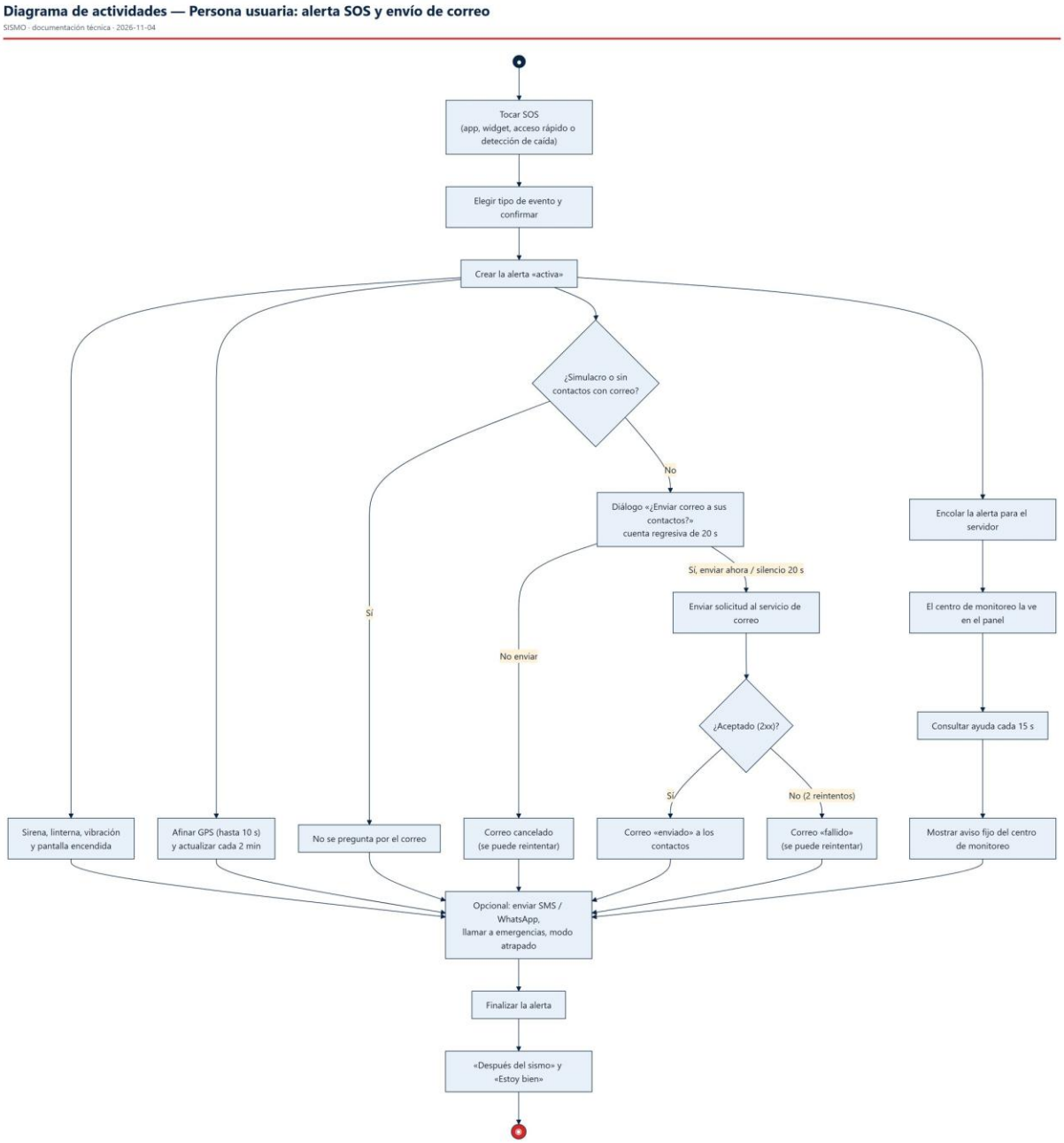
**Figura 10**
*Diagrama de actividades: registro inicial de la persona*




Fuente: elaboración propia.

**Figura 11**

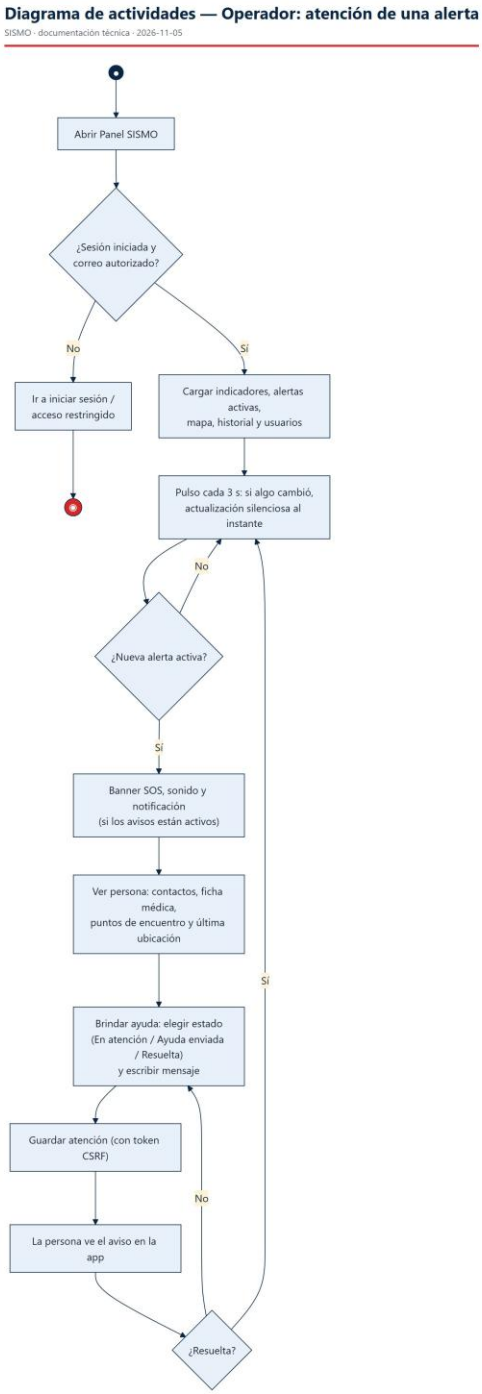
*Diagrama de actividades: alerta SOS y correo a los contactos*




Fuente: elaboración propia.

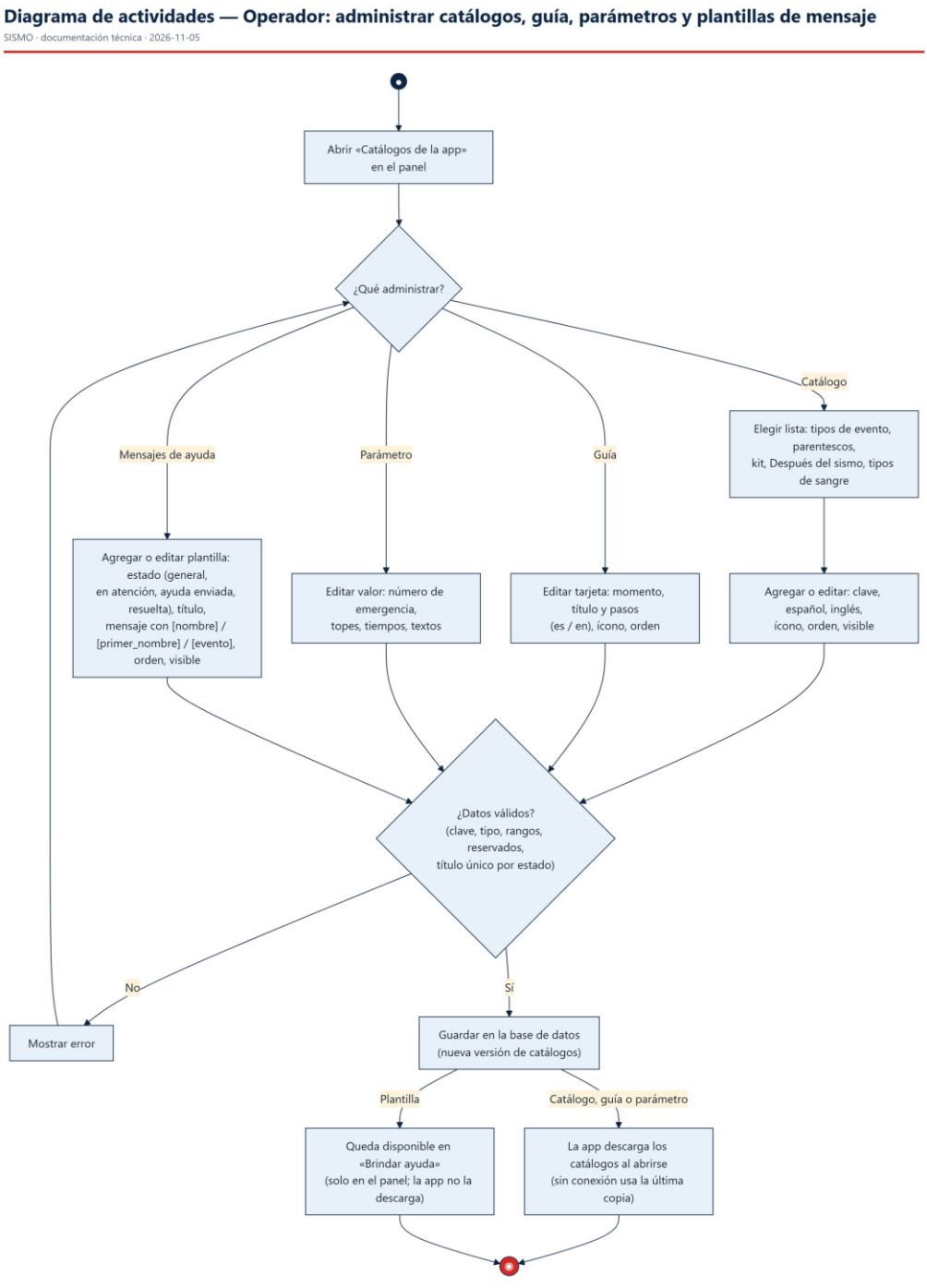
**Figura 12**

*Diagrama de actividades: atención de una alerta por el operador*



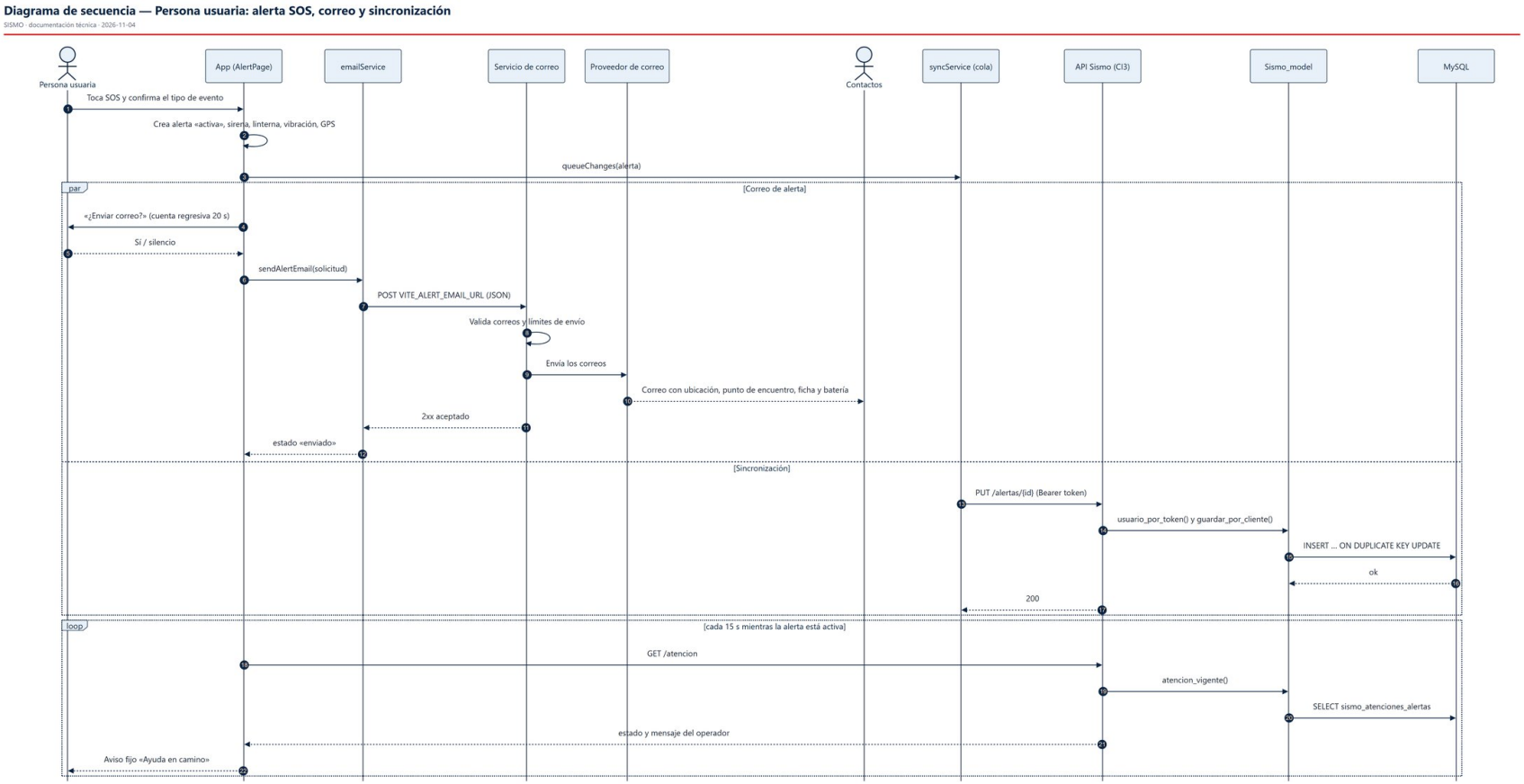

Fuente: elaboración propia.

**Figura 13**
*Diagrama de actividades: administración del contenido por el operador*




Fuente: elaboración propia.

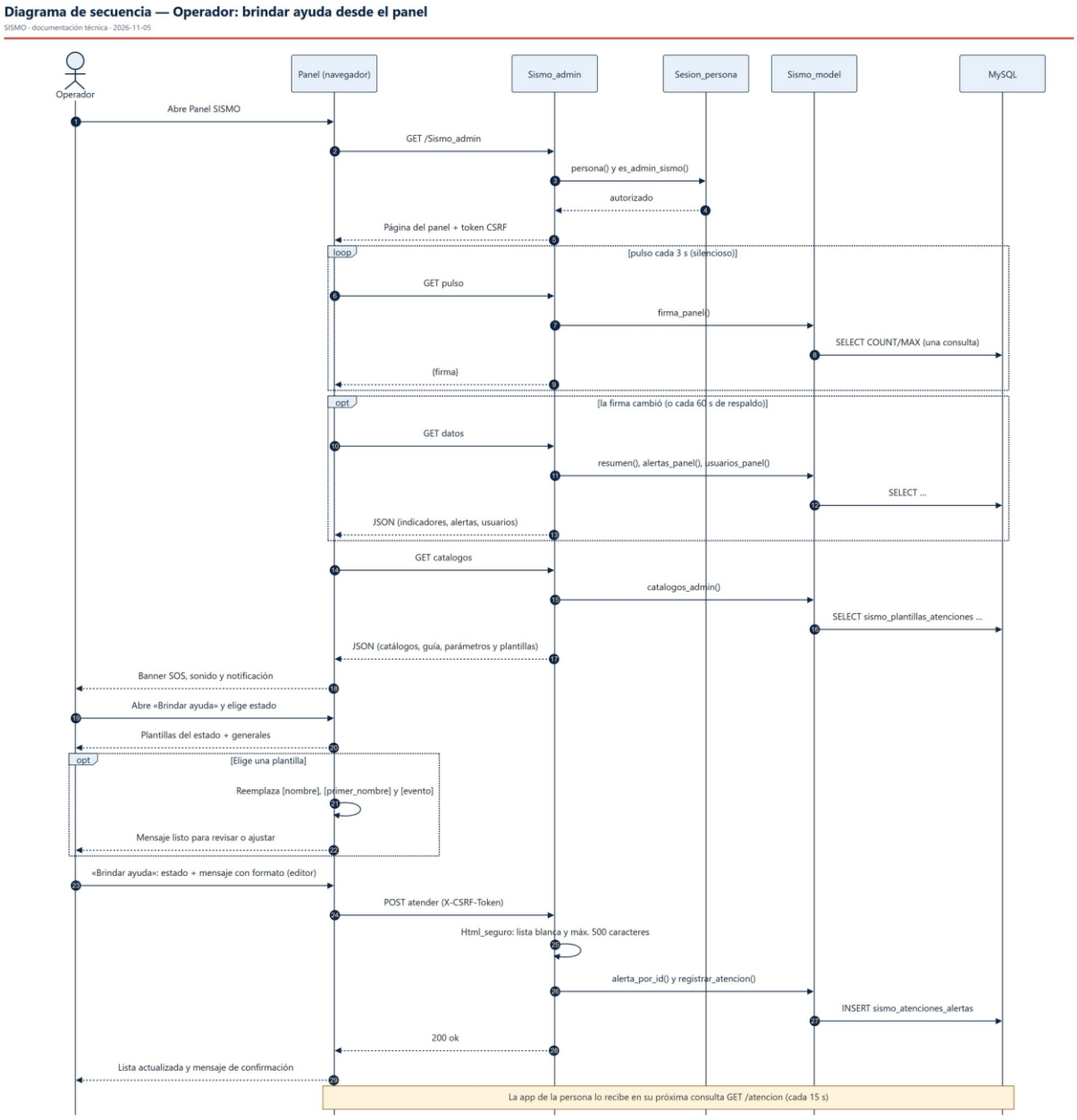
**Figura 14**
*Diagrama de secuencia: alerta SOS y correo*




Fuente: elaboración propia.

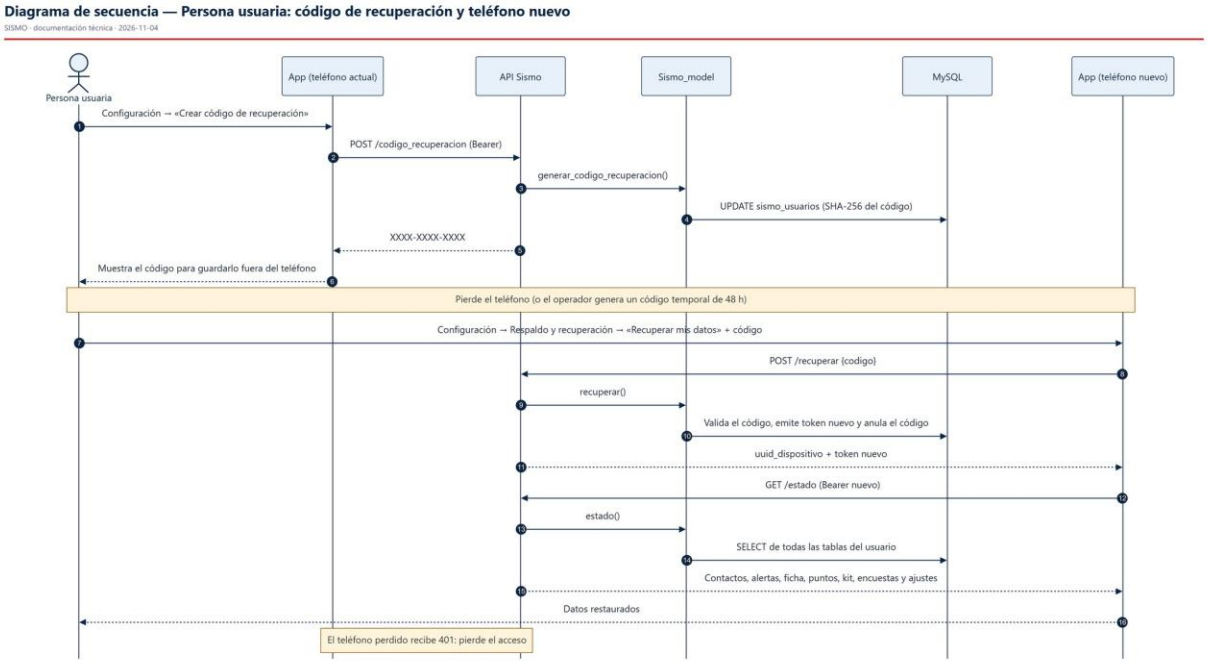
**Figura 15**

*Diagrama de secuencia: el operador brinda ayuda*



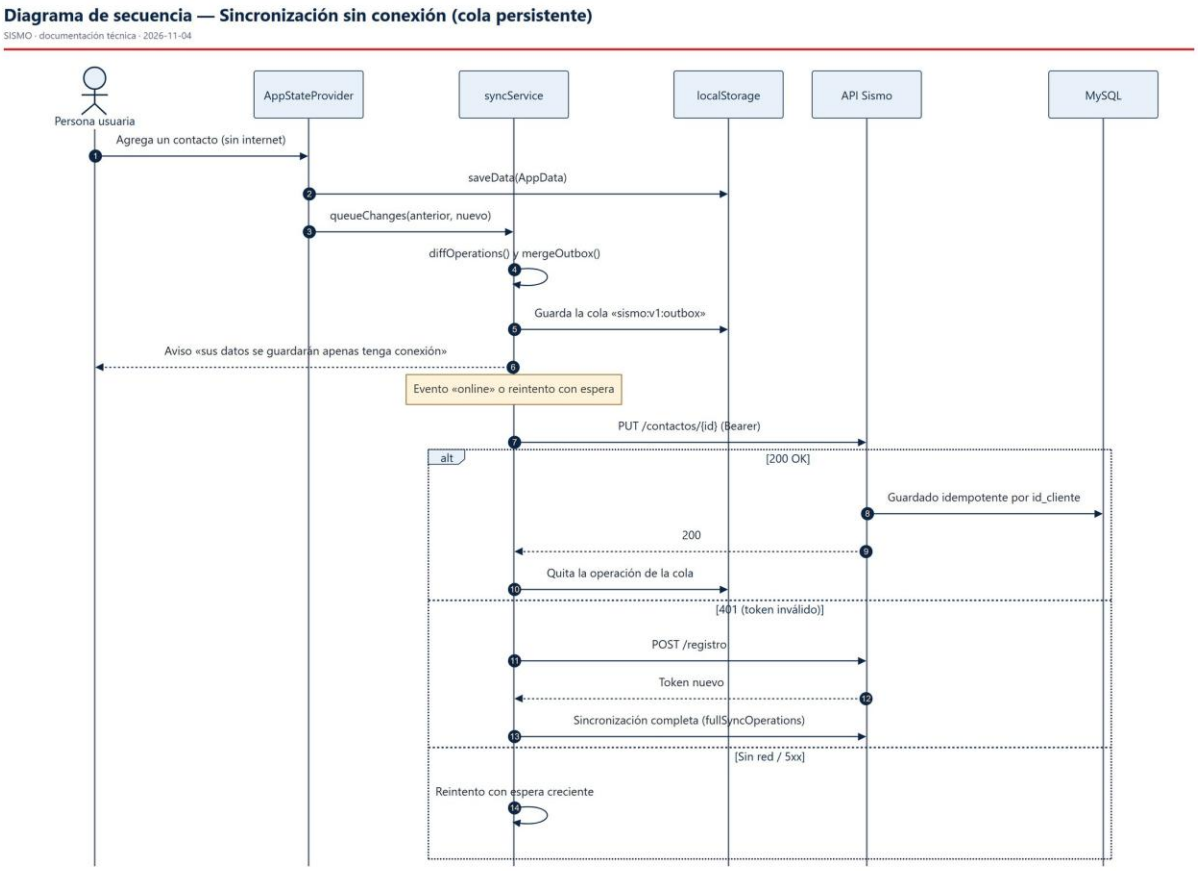

Fuente: elaboración propia.

**Figura 16**
*Diagrama de secuencia: recuperación de la cuenta*



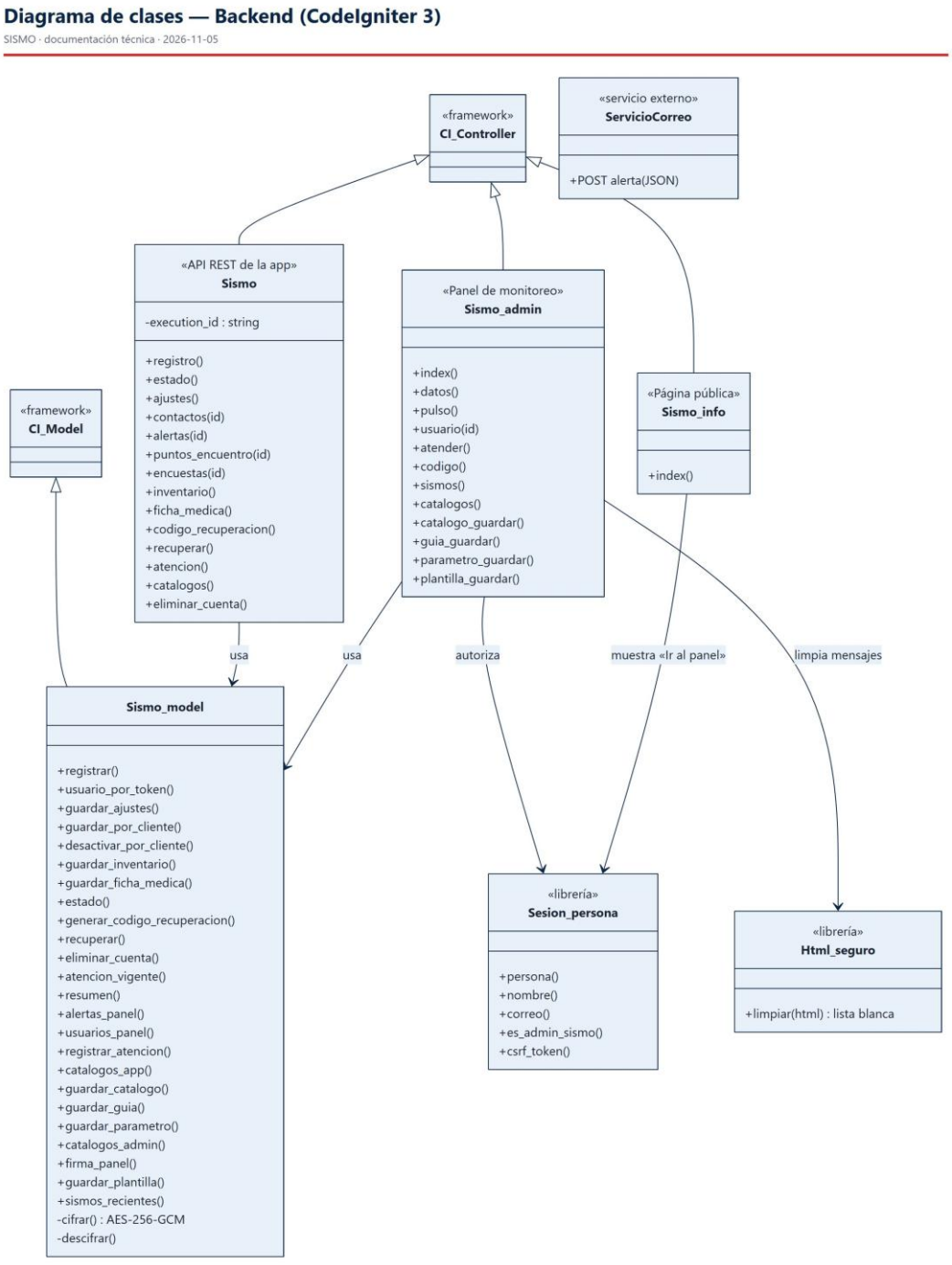

Fuente: elaboración propia.

**Figura 17**
*Diagrama de secuencia: sincronización sin conexión*



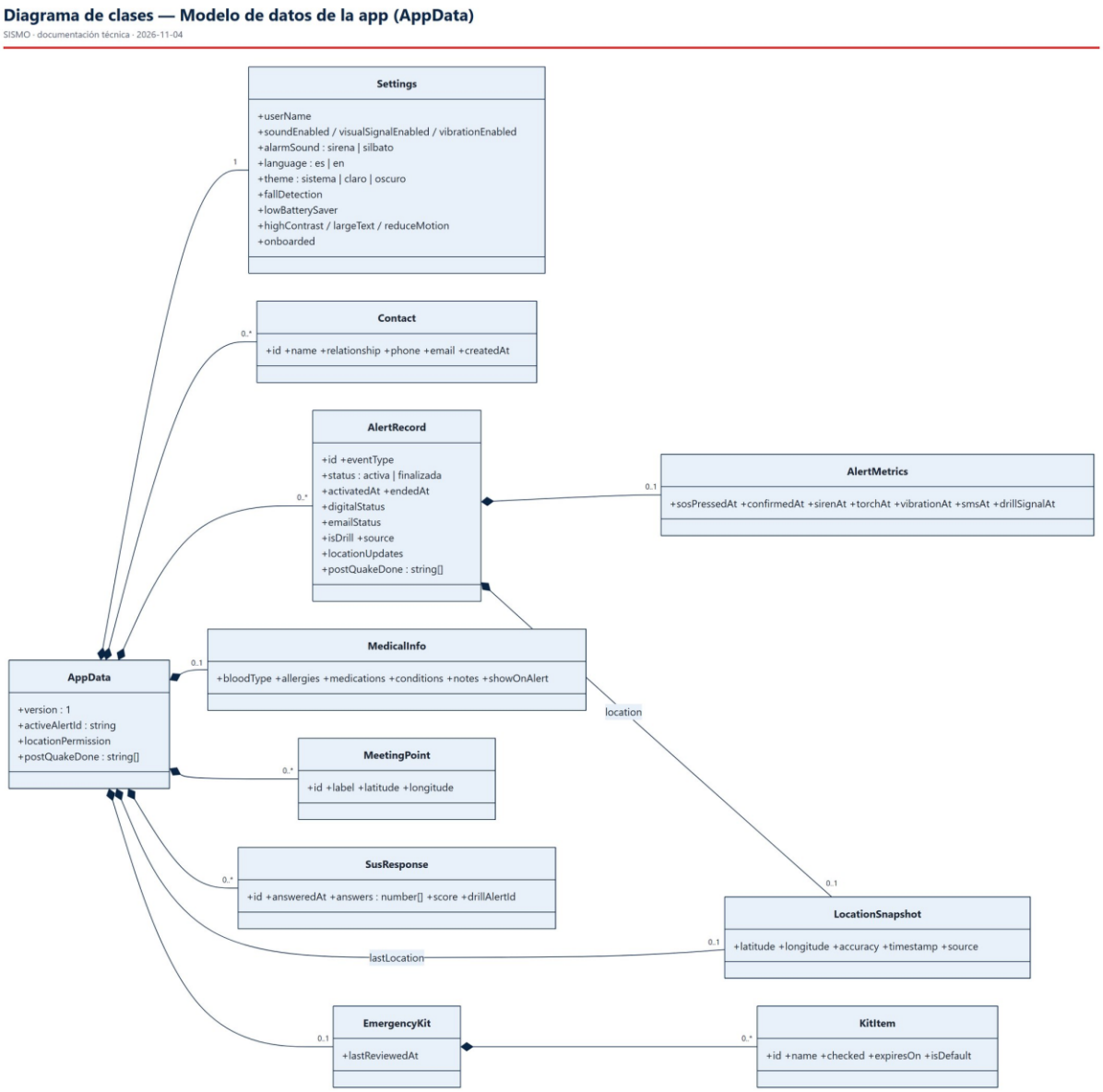

Fuente: elaboración propia.

**Figura 18**
*Diagrama de clases del backend*



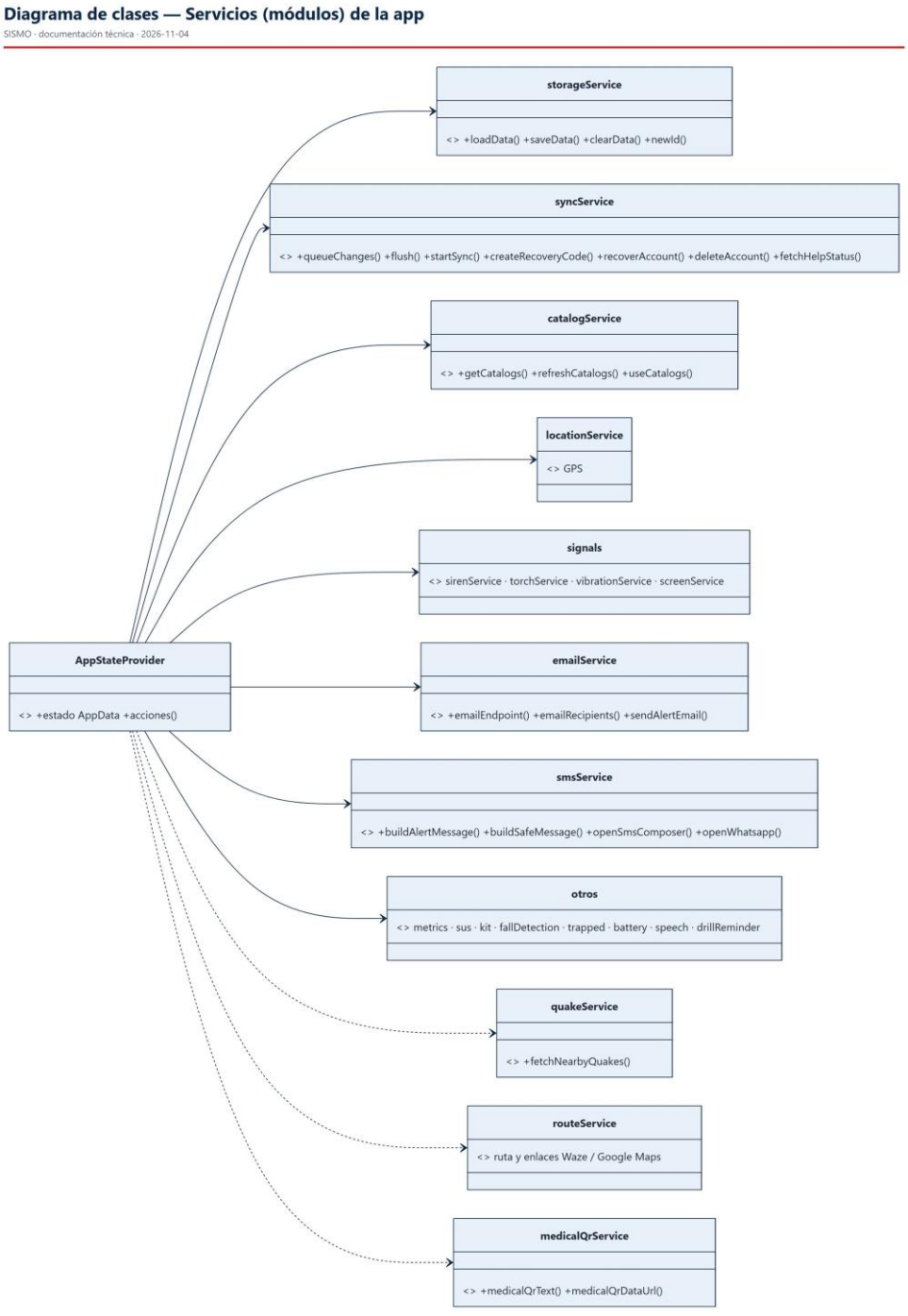

Fuente: elaboración propia.

**Figura 19**
*Diagrama de clases del modelo de datos de la app*



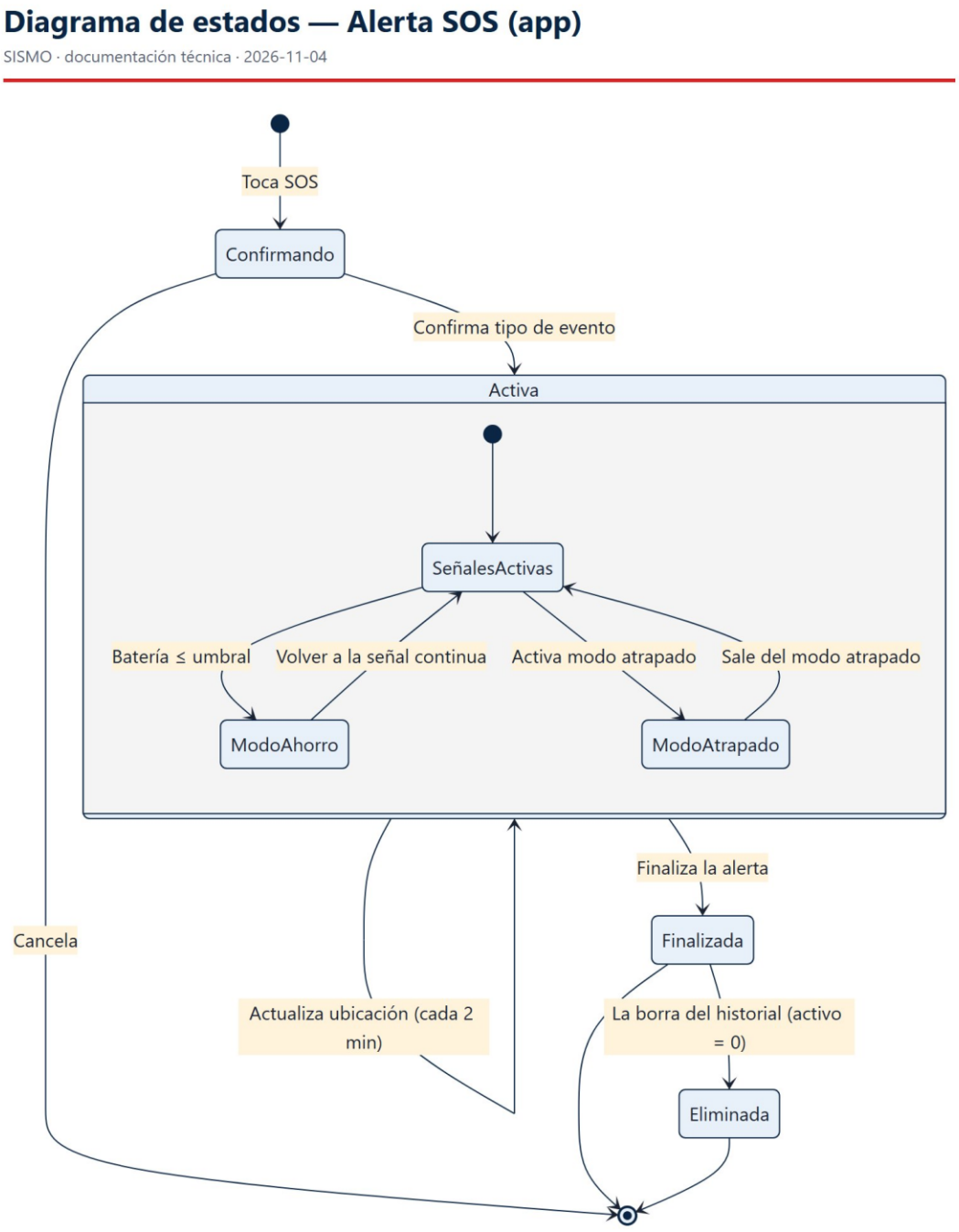

Fuente: elaboración propia.

**Figura 20**
*Diagrama de clases de los servicios de la app*



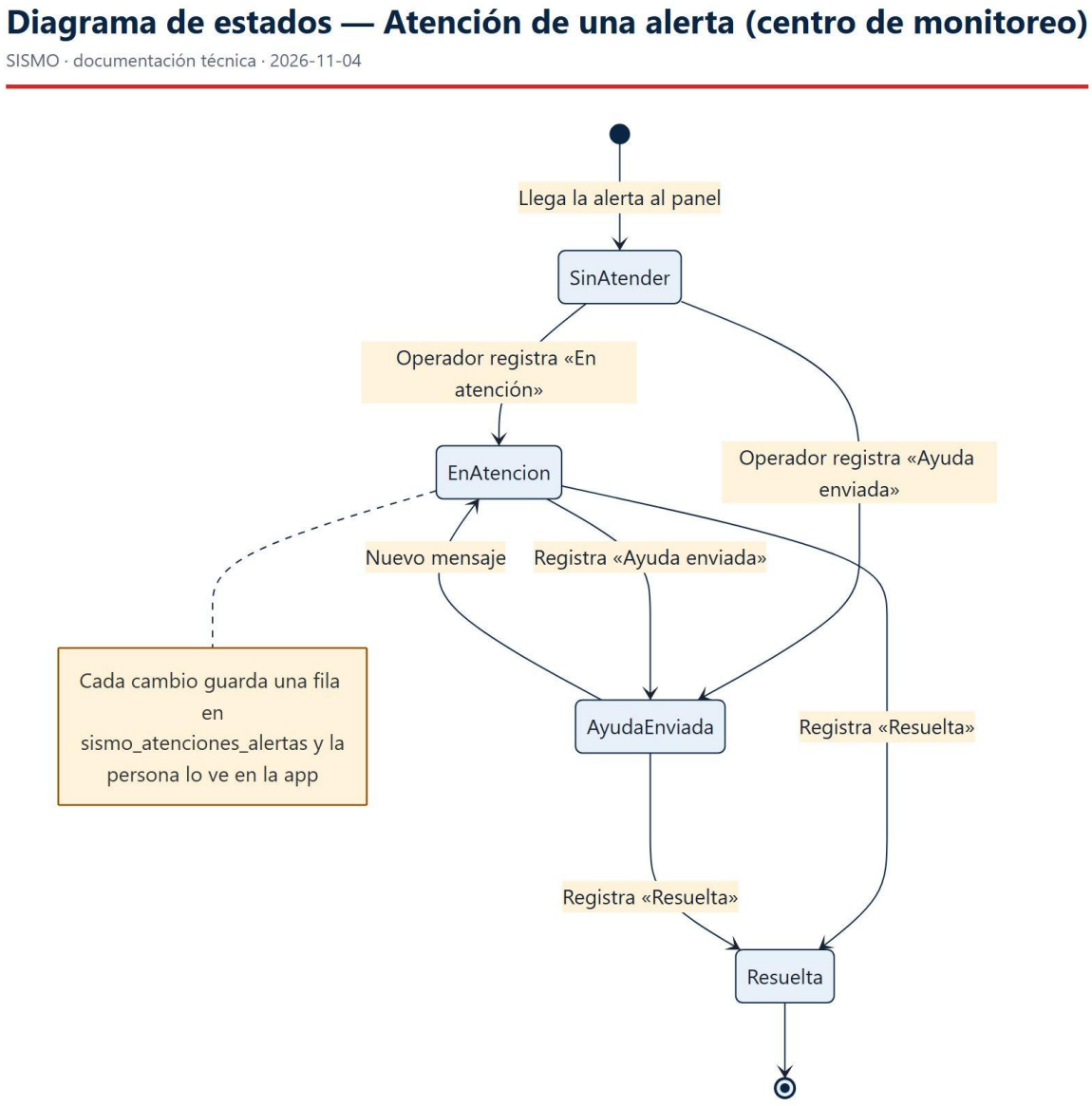

Fuente: elaboración propia.

**Figura 21**
*Diagrama de estados de la alerta SOS*



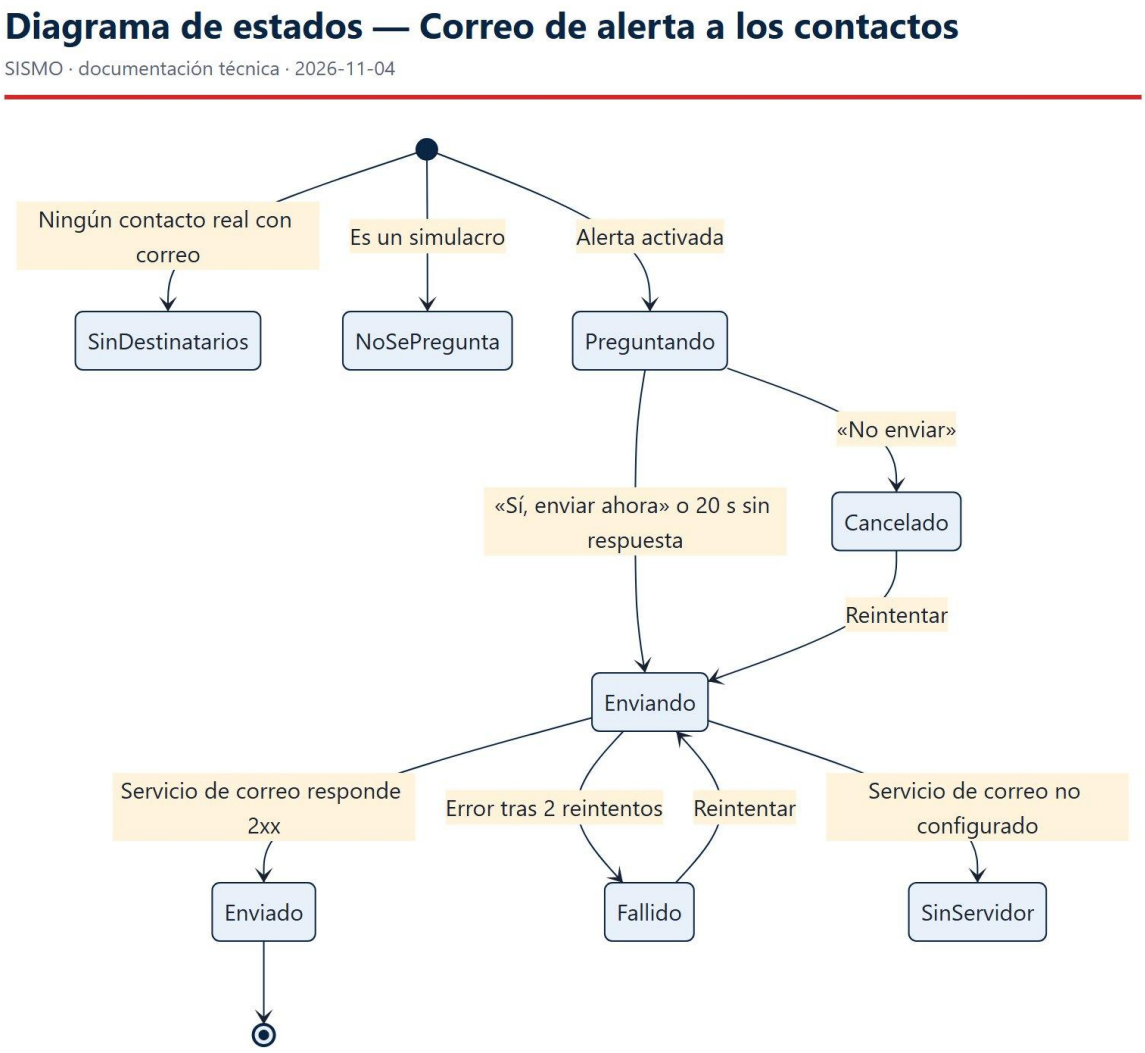

Fuente: elaboración propia.

**Figura 22**
*Diagrama de estados de la atención en el centro de monitoreo*



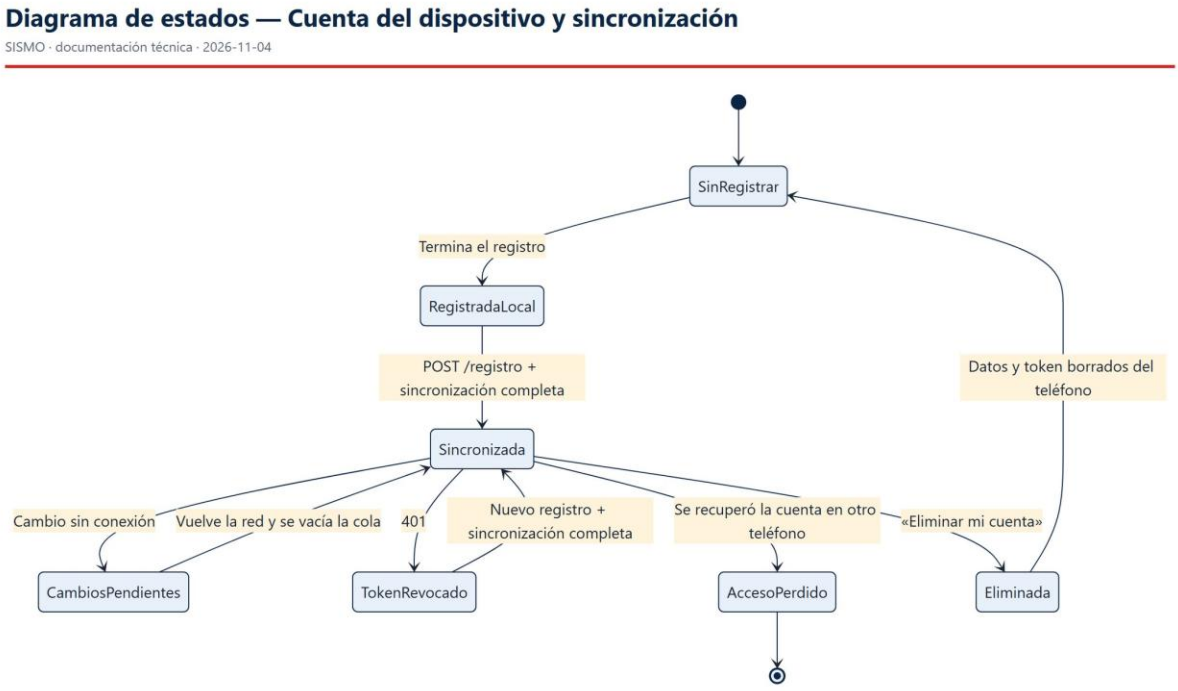

Fuente: elaboración propia.

**Figura 23**
*Diagrama de estados del correo de alerta*



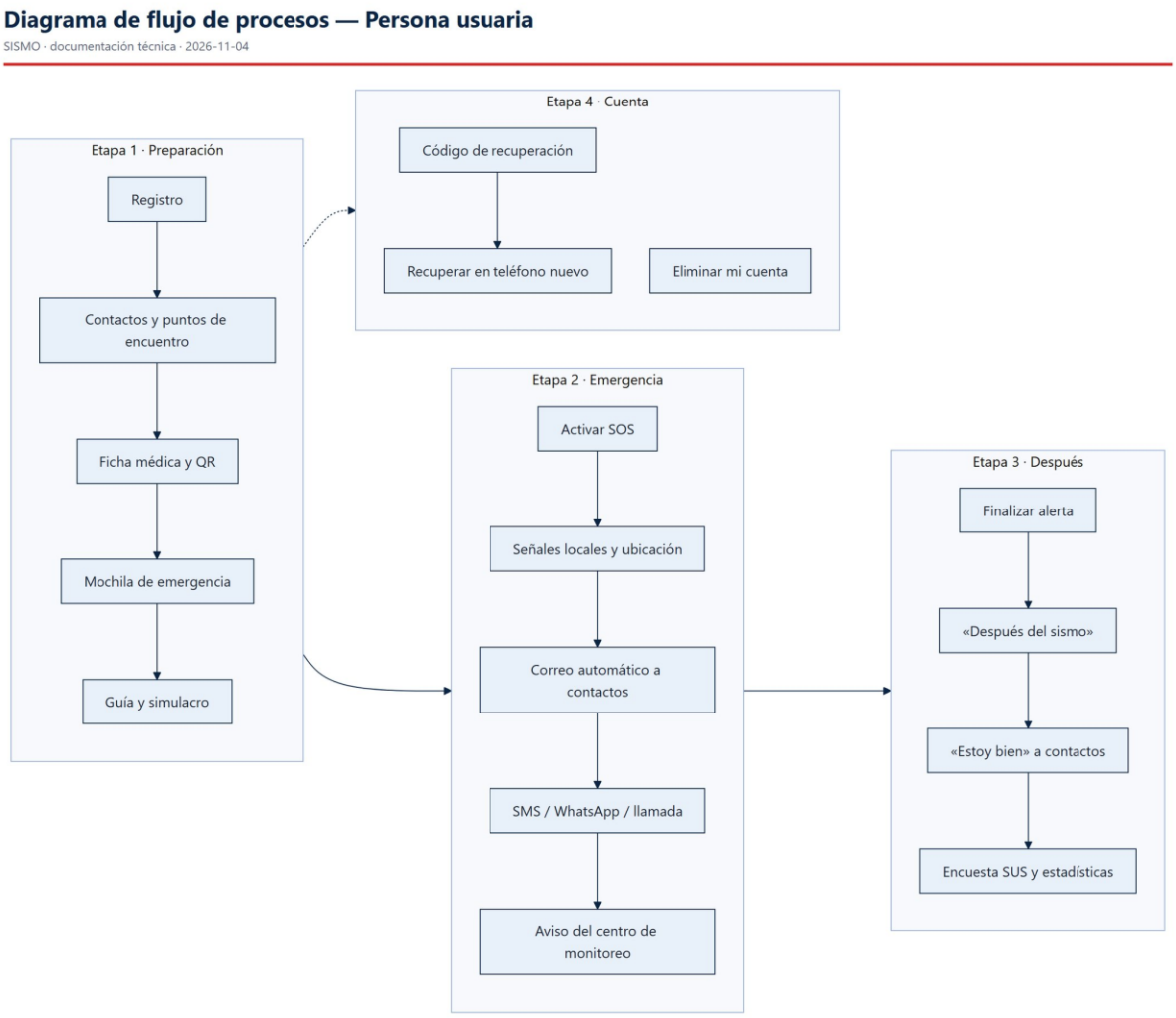

Fuente: elaboración propia.

**Figura 24**
*Diagrama de estados de la cuenta y la sincronización*



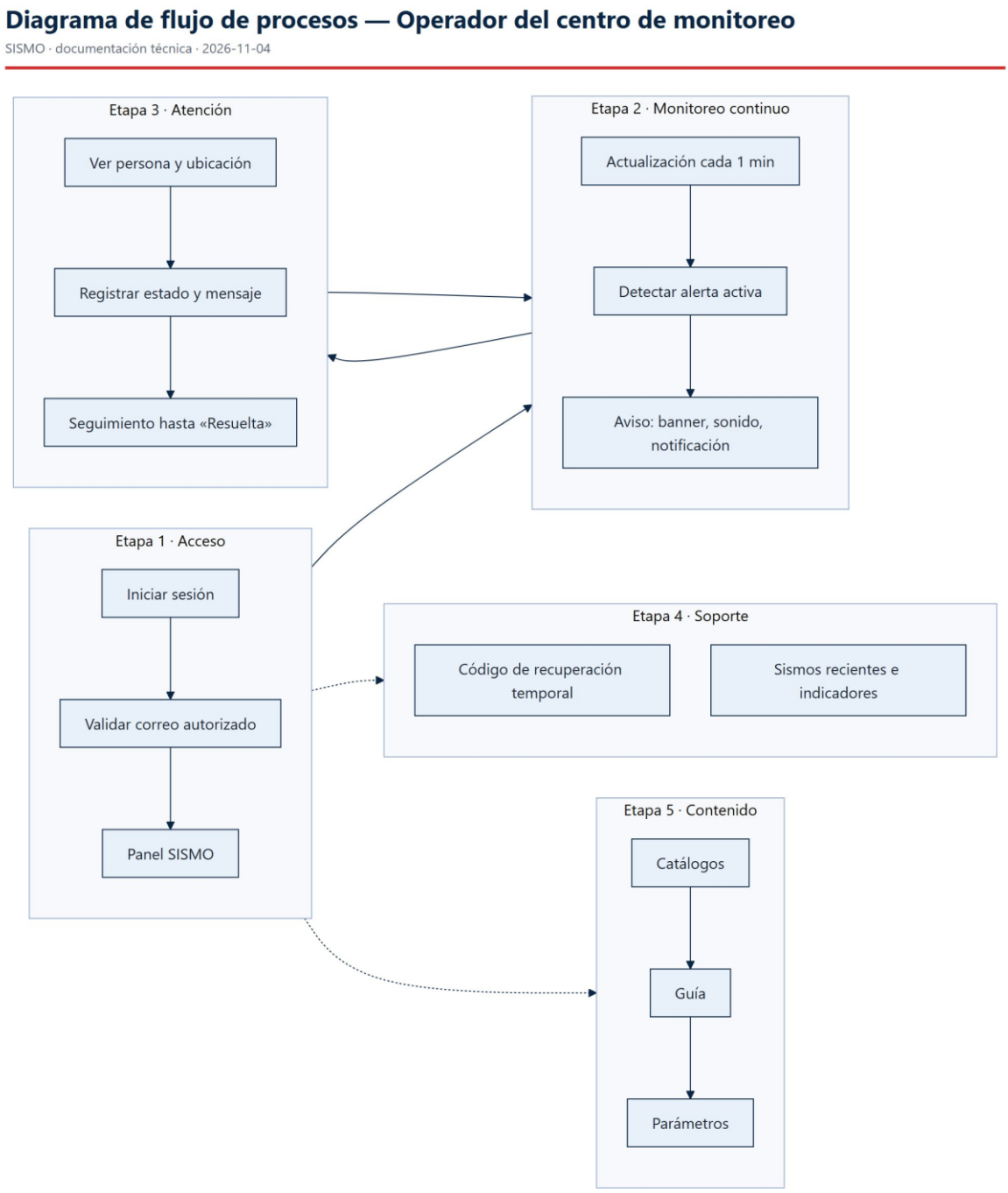

Fuente: elaboración propia.

**Figura 25**
*Flujo de procesos de la persona usuaria*




Fuente: elaboración propia.

**Figura 26**
*Flujo de procesos del operador*



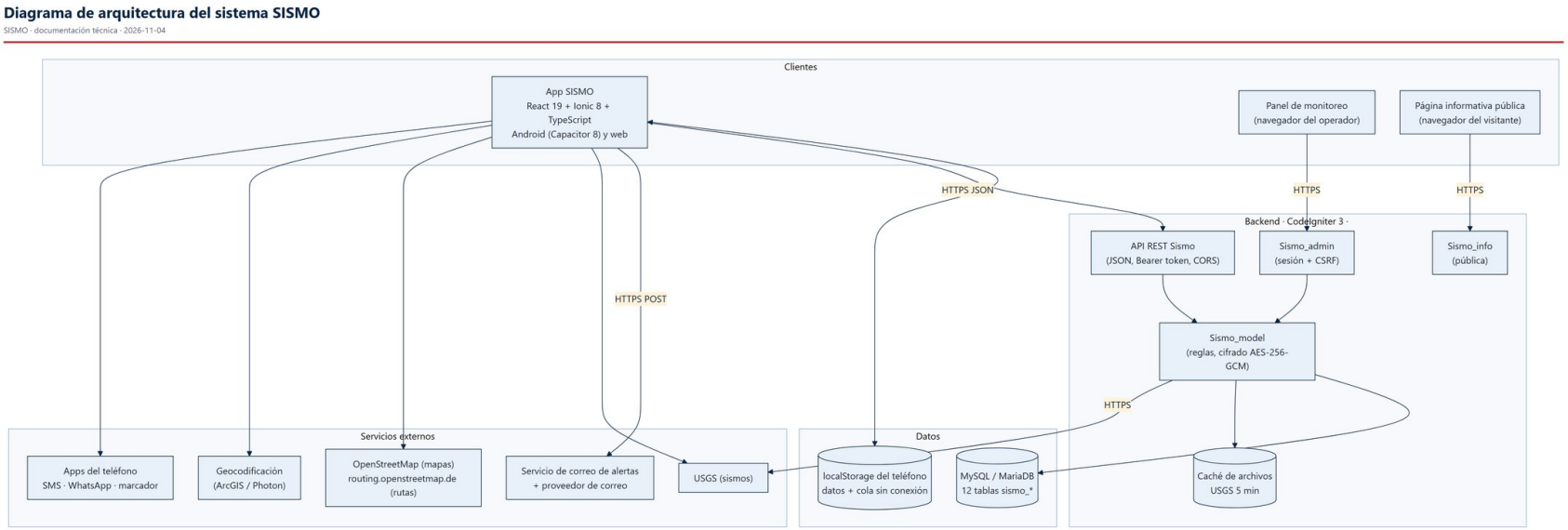

Fuente: elaboración propia.

### 4.5.2 Arquitectura y diseño

La arquitectura implementada tiene tres capas: los clientes (app y panel), el backend en CodeIgniter 3 y los datos (MySQL en el servidor y almacenamiento local en el teléfono), con

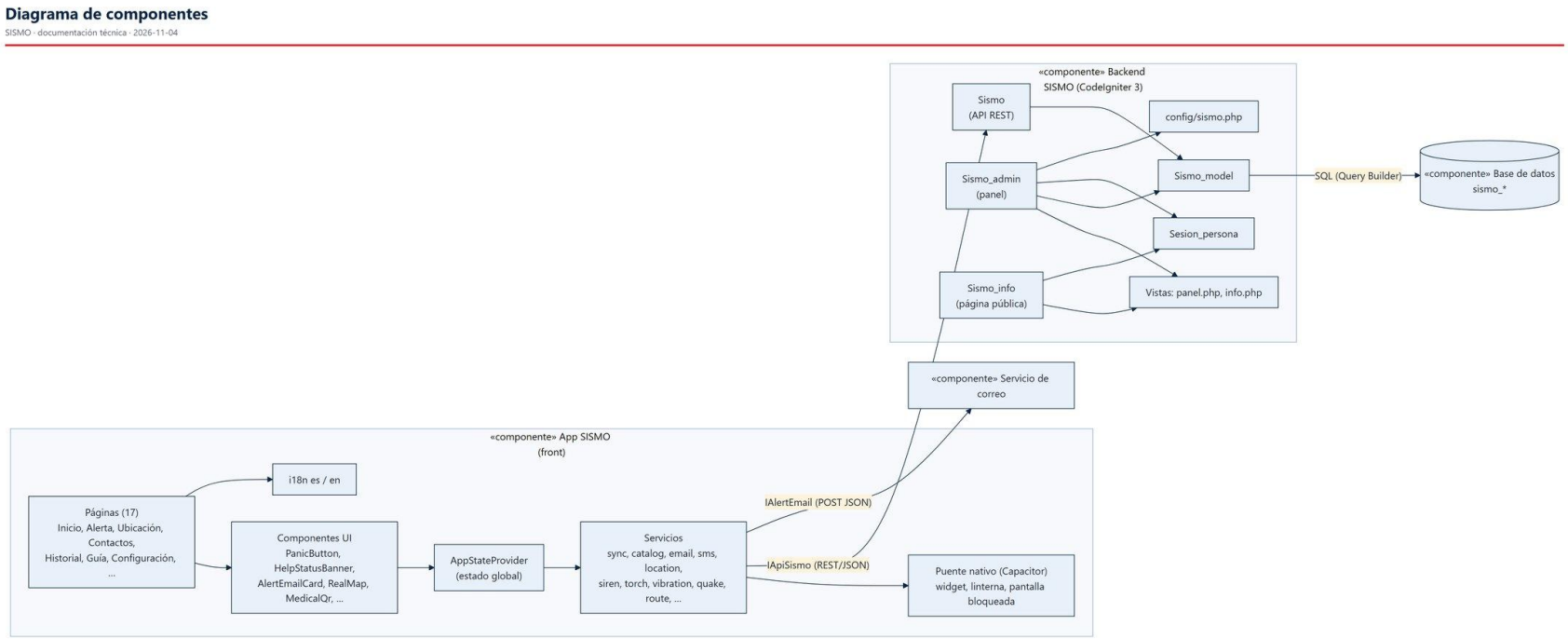
servicios externos para el correo, los sismos y los mapas.

**Figura 27**
*Arquitectura del sistema*



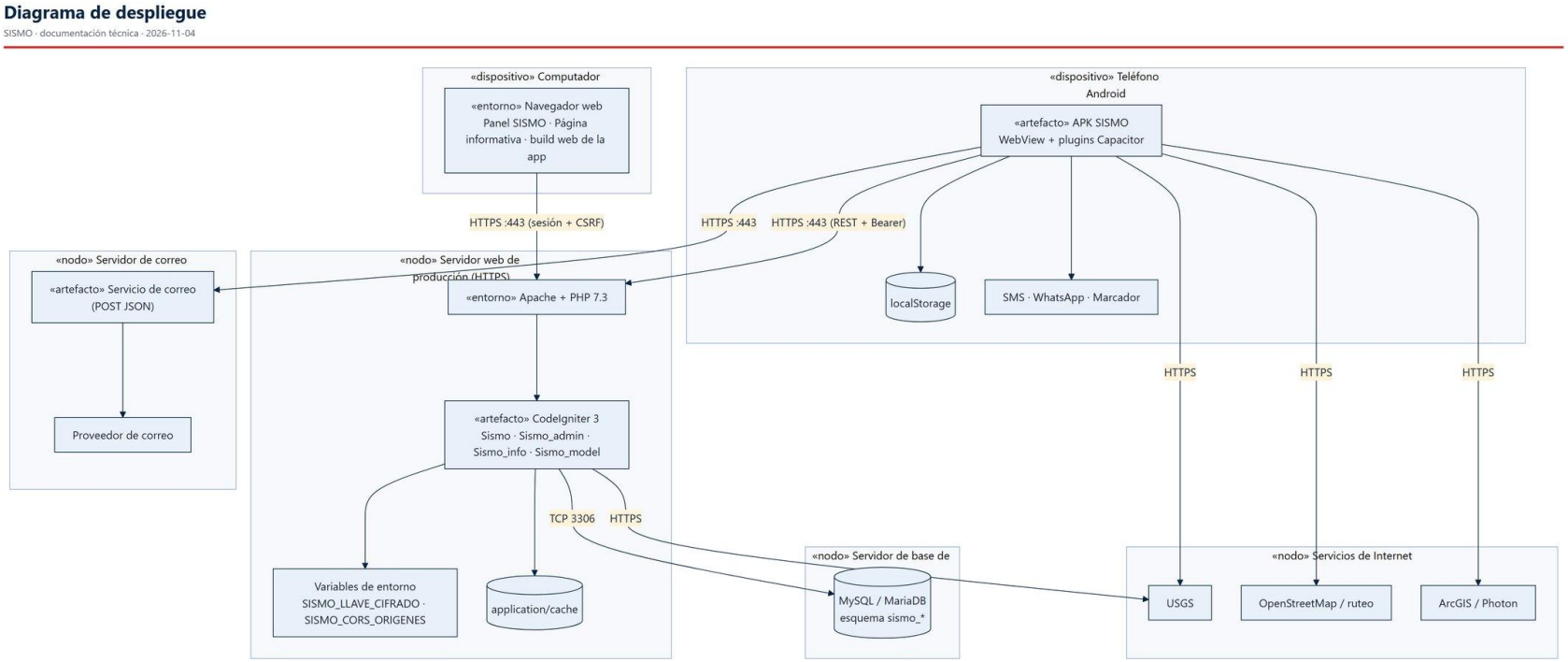

Fuente: elaboración propia.

**Figura 28**
*Diagrama de componentes*



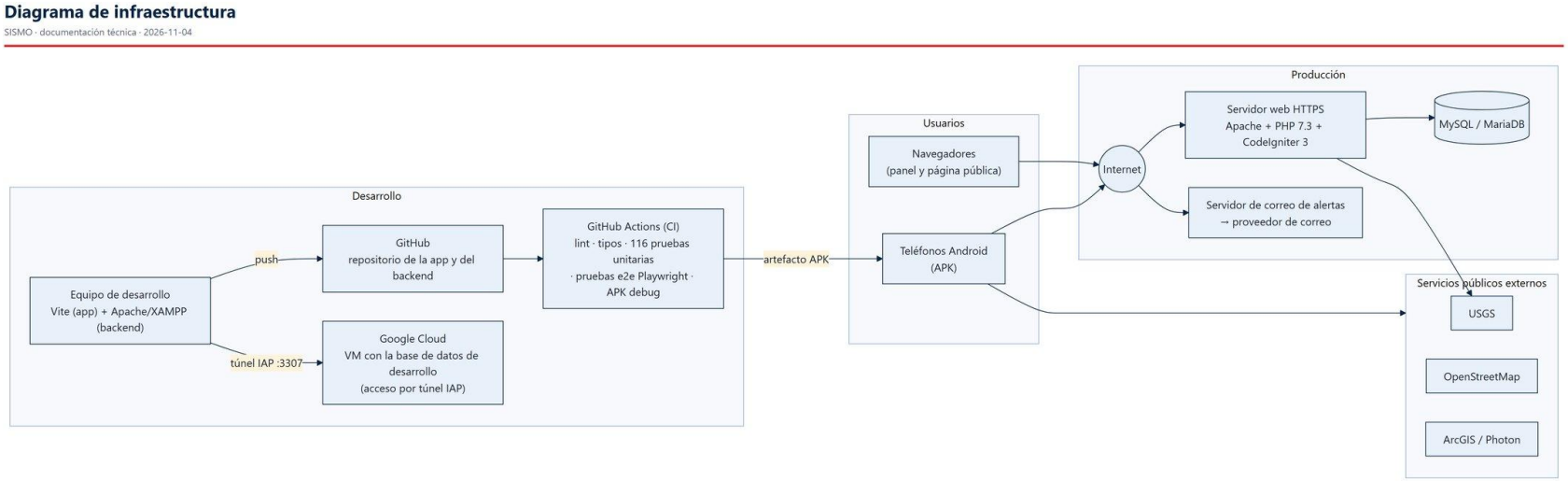

Fuente: elaboración propia.

**Figura 29**
*Diagrama de despliegue*



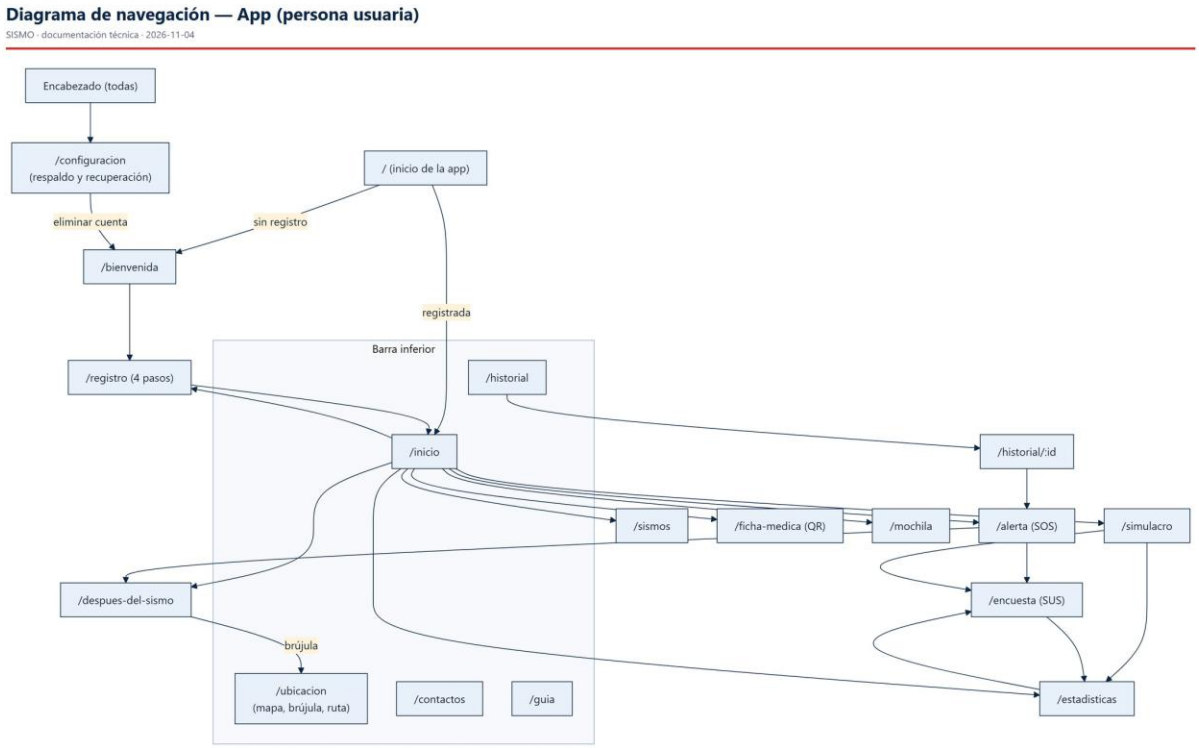

Fuente: elaboración propia.

**Figura 30**
*Infraestructura de desarrollo y producción*



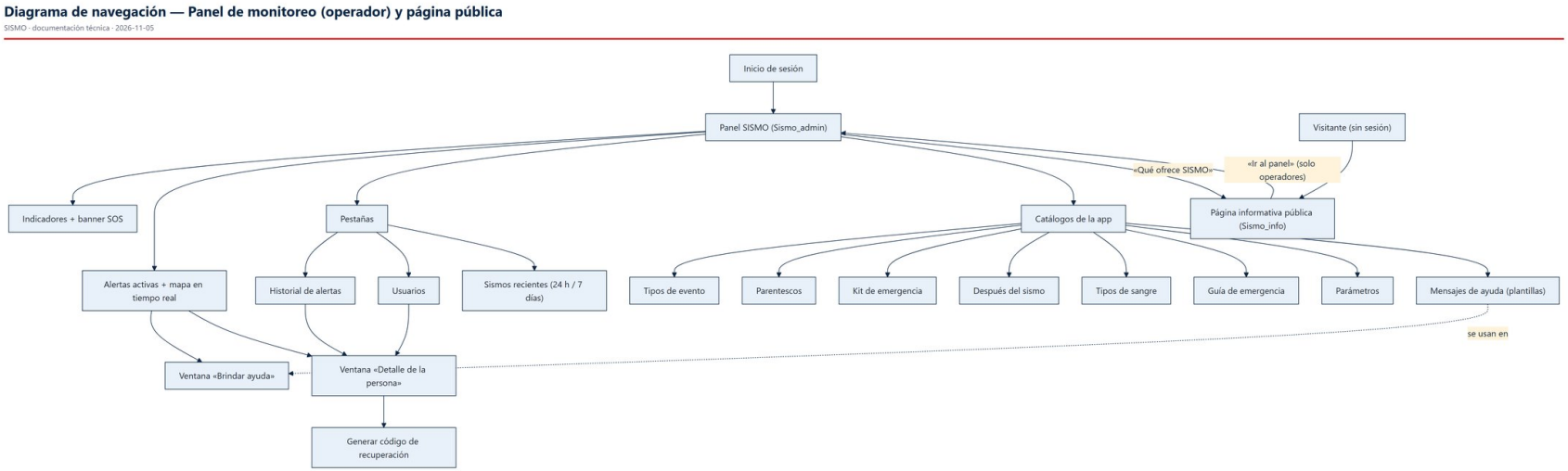

Fuente: elaboración propia.

**Figura 31**
*Mapa de navegación de la app*




Fuente: elaboración propia.

**Figura 32**
*Mapa de navegación del panel y de la página informativa*

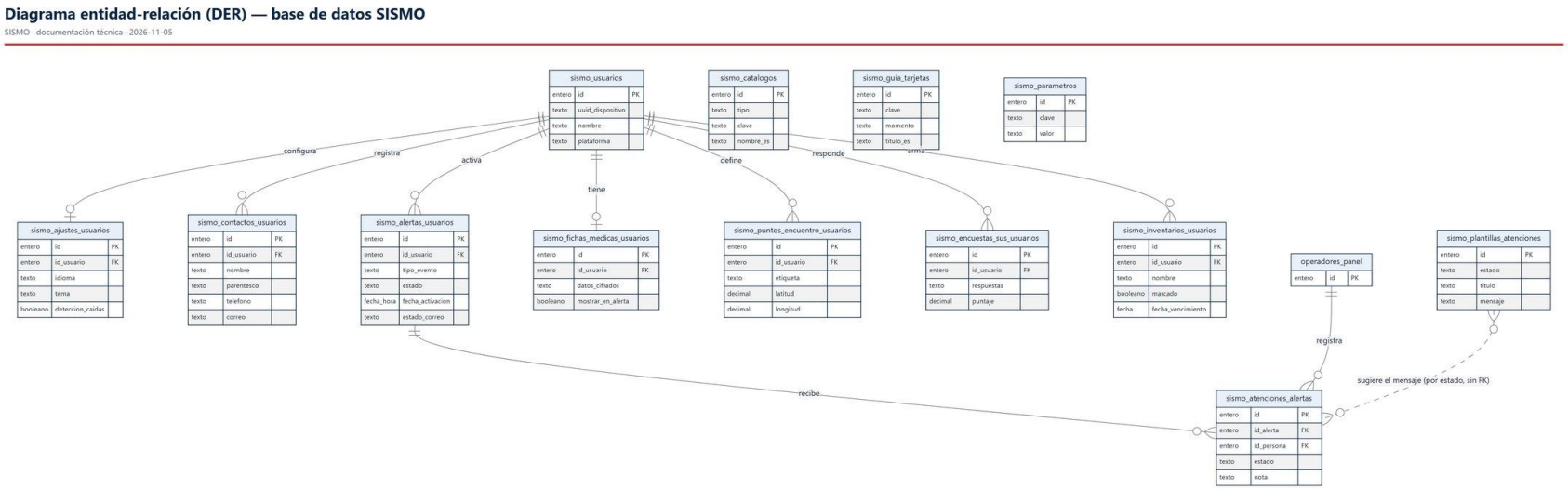



Fuente: elaboración propia.

### 4.5.3 Base de datos

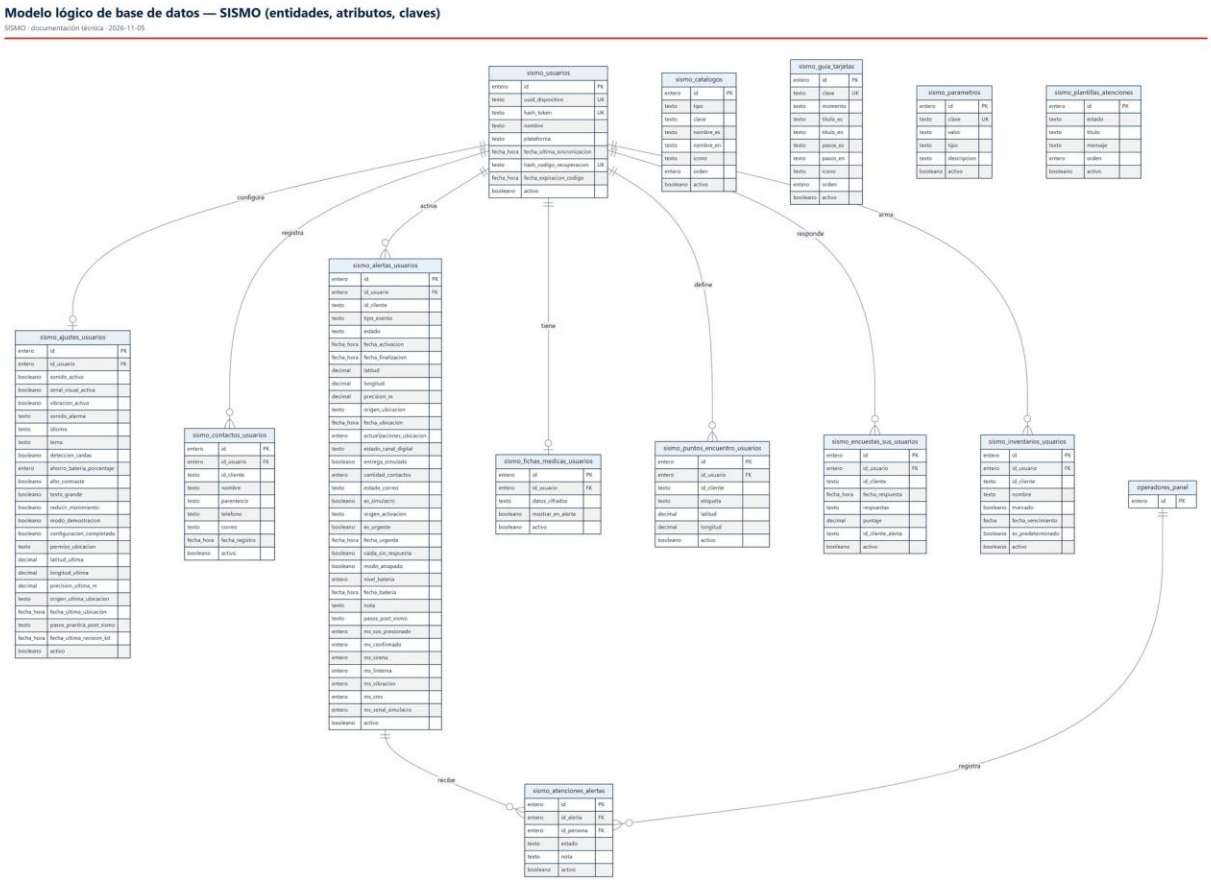
La base de datos sigue una convención de nombres en snake_case, con tablas en plural, comentarios en todas las columnas, claves foráneas con prefijo id_, columnas de auditoría y borrado lógico mediante la columna activo.

**Figura 33**
*Modelo entidad-relación*




Fuente: elaboración propia.

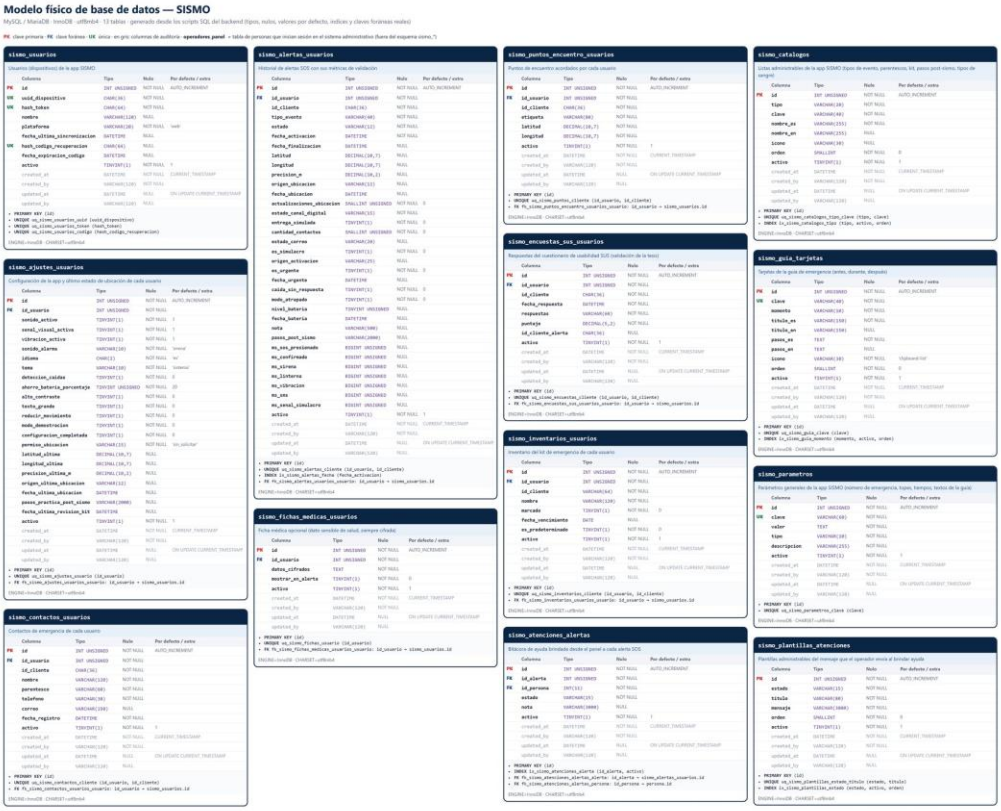
**Figura 34**
*Modelo lógico de la base de datos*




Fuente: elaboración propia.

**Figura 35**

*Modelo físico de la base de datos*




Fuente: elaboración propia.

## 4.6 Resultados de las pruebas

Las pruebas se ejecutaron el 4 de noviembre de 2026 sobre la versión 3.1. Se superó el 100 % de las pruebas automáticas. Los casos manuales que dependen de producción o de un teléfono físico quedaron registrados como pendientes, con su riesgo asociado.

**Tabla 102**

*Resumen de resultados de pruebas al 04/11/2026*

| Nivel | Ejecutadas | Superadas | Fallidas | Resultado |
|---|---|---|---|---|
| **ESLint y TypeScript** | Todo el proyecto | Sin errores | 0 | Aprobado |
| **Unitarias (Vitest)** | 116 | 116 | 0 | 100 % (6,7 s) |
| **Extremo a extremo (Playwright)** | 114 (57 × 2) | 114 | 0 | 100 % (7,5 min) |
| **Manuales del backend (CP-M-01 a 07)** | 7 | 6 | 0 | 1 pendiente en producción |
| **Manuales del panel (CP-M-08 a 11)** | 4 | 1 | 0 | 1 parcial y 2 pendientes |

| Nivel | Ejecutadas | Superadas | Fallidas | Resultado |
|---|---|---|---|---|
| **Otros manuales (CP-M-12 a 14)** | 3 | 1 | 0 | CP-M-12 pendiente; CP-M-13 parcial (emulador) |

Fuente: elaboración propia con base en el informe de resultados de pruebas.

**Tabla 103**
*Defectos encontrados y corregidos durante el ciclo*

| ID | Defecto | Severidad | Corrección (todos cerrados) |
|---|---|---|---|
| **D-01** | Cuatro e2e de «Después del sismo» esperaban el avance por posición | Media | Pruebas actualizadas al guardado por clave de paso |
| **D-02** | El texto de la cuenta regresiva del correo se salía del diálogo | Baja | Tamaño de frase y espacio que no se parte |
| **D-03** | Preflight CORS sin la cabecera Authorization | Alta | Preflight atendido antes del constructor base |
| **D-04** | Error fatal por un método con nivel de acceso distinto al del controlador base | Alta | Método renombrado |
| **D-05** | 401 en ajustes después de registrarse | Media | Nuevo registro inmediato y subida completa |
| **D-06** | Pantalla en blanco por pérdida del contexto en la recarga en caliente | Baja | Contexto global estable y pantalla de error |
| **D-07** | La cuenta regresiva del correo no avanzaba porque la pantalla se redibujaba cada segundo | Alta | Temporizador independiente de los redibujos |

Fuente: elaboración propia.

## 4.7 Indicadores alcanzados frente a las metas

La tabla compara los indicadores de la sección 1.7 con los valores alcanzados al 9 de noviembre de 2026.

**Tabla 104**
*Indicadores de evaluación: meta y valor alcanzado al 09/11/2026*

| Indicador | Meta | Valor alcanzado | Estado |
|---|---|---|---|
| **Cobertura de requisitos** | ≥ 90 % de los requisitos priorizados | 48 de 48 RF implementados; 40 verificados (83 %) | Implementación cumplida; verificación en curso |
| **Ejecución del flujo principal** | 100 % de los casos aprobados | 114 de 114 e2e y 116 de 116 unitarias | Cumplida |
| **Tiempo hasta «alerta activa»** | ≤ 2 s en el 95 % de las ejecuciones | Métrica instrumentada en la app (CP-E-16) | Medición en un teléfono físico pendiente (fase 4) |

| Indicador | Meta | Valor alcanzado | Estado |
|---|---|---|---|
| **Claridad del estado** | Cada estado con respuesta visual comprensible | Estados de la alerta, el correo y la sincronización visibles | Cumplida |
| **Manejo de permisos** | Informar si la ubicación no está disponible | CP-E-13 y CP-E-14 superados | Cumplida |
| **Robustez ante fallos** | No mostrar como enviado lo no confirmado | CP-E-10, CP-E-33 y pruebas de la cola superados | Cumplida |
| **Casos manuales** | 14 casos ejecutados | 8 superados, 2 parciales y 4 pendientes | En curso |
| **Usabilidad (SUS)** | ≥ 68 | Encuesta implementada (CP-E-20) | Planeada en la fase 5 |

Fuente: elaboración propia.

## 4.8 Documentación técnica producida

Además del código, la segunda entrega incluye la documentación técnica y funcional del sistema, mantenida en el repositorio junto con las evidencias.

**Tabla 105**

*Documentación técnica y funcional de la segunda entrega*

| N.º | Documento | Contenido |
|---|---|---|
| **19** | Requisitos funcionales | 48 requisitos por módulo, rol y prioridad |
| **20** | Requisitos no funcionales | 33 requisitos con su forma de cumplimiento |
| **21** | Historias de usuario | Historias por rol con criterios de aceptación |
| **22** | Diccionario de datos | Tablas y columnas de la base de datos |
| **23** | API y endpoints | Rutas, métodos, parámetros y respuestas de la API |
| **24** | Manual técnico | Instalación, configuración, compilación y despliegue |
| **25** | Manual de usuario | Uso de la app y del panel |
| **26** | Plan y casos de prueba | Niveles, entorno, criterios y casos |
| **27** | Informe de resultados de pruebas | Resultados, casos manuales y defectos |
| **28** | Registro de riesgos | 17 riesgos con probabilidad, impacto y mitigación |
| **29** | Evidencias visuales | Inventario de 162 evidencias por fecha |
| **30** | Registro de cambios y versiones | Cambios por versión y actividad del plan |

Fuente: elaboración propia.

# 5. DISCUSIÓN Y CONCLUSIONES

## 5.1 Discusión

La propuesta concentra la interacción principal en un botón de pánico y organiza las responsabilidades del software para que la señalización local no dependa necesariamente de la comunicación digital. Esto permite diseñar una respuesta visible en el teléfono aun cuando no se pueda confirmar un envío remoto. La utilidad para avisar a otra persona depende, sin embargo, de integraciones y condiciones reales de conectividad.

La geolocalización y la mensajería no deben tratarse como funciones infalibles. La ubicación puede faltar o ser imprecisa y un mensaje puede quedar sin enviar. La solución debe informar los estados con transparencia. El desarrollo incremental permite verificar primero la experiencia principal y luego incorporar capacidades externas sin confundir funciones planeadas con implementadas. La segunda entrega confirmó este enfoque: la cola de sincronización sin conexión, los estados explícitos del correo y la regla de no mostrar «enviado» sin confirmación hacen que la app informe con transparencia lo que pasa con cada canal.

## 5.2 Cumplimiento de objetivos

Al 9 de noviembre de 2026 los objetivos específicos 1 y 2 están cumplidos: se documentaron 48 requisitos funcionales, 33 no funcionales y 17 riesgos, y se elaboraron 33 diagramas. Los objetivos 3 y 4 están implementados y verificados con pruebas automáticas; falta medir los tiempos en un teléfono físico y probar el correo y el cifrado en producción (fase 4). El objetivo 5 avanza con el 100 % de las pruebas automáticas superadas y 8 de 14 casos manuales; la validación con usuarios mediante la encuesta SUS está planeada del 30 de noviembre al 4 de diciembre (fase 5).

## 5.3 Limitaciones

- La aplicación depende de las capacidades y configuración del teléfono, incluida batería y permisos.
- La conectividad puede ser insuficiente o inexistente durante o después de un sismo.
- La geolocalización no garantiza precisión en interiores o bajo escombros.
- La entrega digital requiere integración y respuesta verificable del servicio.
- Las pruebas controladas no garantizan desempeño en un desastre real.
- El prototipo no reemplaza líneas oficiales de emergencia ni procedimientos de autoprotección.

## 5.4 Conclusiones

SISMO propone una aplicación móvil enfocada en facilitar la activación de una alerta personal ante una emergencia sísmica. Su valor como proyecto de ingeniería está en convertir una necesidad en requisitos verificables, definir una arquitectura modular y planear pruebas con escenarios normales y de fallo.

El diseño debe priorizar la simplicidad, la retroalimentación del estado y el manejo responsable de permisos y datos. Las señales locales y la comunicación digital cumplen funciones distintas y son canales complementarios. La aplicación no puede asegurar recepción de mensajes ni localización en todas las condiciones.

La metodología incremental y la trazabilidad permitieron documentar qué se implementó, qué se probó y qué sigue pendiente. En la segunda entrega el proyecto pasó del diseño a un sistema funcional (app, backend, panel y página informativa), adelantado frente al cronograma original; las conclusiones definitivas se actualizarán después de las pruebas en producción y de la validación con usuarios.

## 5.5 Trabajo futuro

- Poner el sistema en producción: base de datos, llave de cifrado y despliegue (09/11 – 13/11/2026).
- Probar la app en un teléfono físico y ajustar las funciones nativas (09/11 – 20/11/2026).
- Probar el correo de alertas en producción con contactos ficticios (16/11 – 20/11/2026).
- Obtener la aprobación del tratamiento de datos según la Ley 1581 de 2012 (09/11 – 27/11/2026).
- Validar la app con al menos 5 usuarios mediante simulacros y la encuesta SUS (30/11 – 04/12/2026).
- Ajustar según los resultados y entregar el informe final (07/12 – 11/12/2026).

# REFERENCIAS


Allen, R. M., & Melgar, D. (2019). Earthquake early warning: Advances, scientific challenges, and societal needs. *Annual Review of Earth and Planetary Sciences, 47*, 361–388. https://doi.org/10.1146/annurev-earth-053018-060457

Bangor, A., Kortum, P. T., & Miller, J. T. (2008). An empirical evaluation of the System Usability Scale. *International Journal of Human–Computer Interaction, 24*(6), 574–594. https://doi.org/10.1080/10447310802205776

Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R. C., Mellor, S., Schwaber, K., Sutherland, J., & Thomas, D. (2001). *Manifiesto por el desarrollo ágil de software*. https://agilemanifesto.org/iso/es/manifesto.html

Brooke, J. (1996). SUS: A «quick and dirty» usability scale. En P. W. Jordan, B. Thomas, B. A. Weerdmeester y I. L. McClelland (Eds.), *Usability evaluation in industry* (pp. 189–194). Taylor & Francis.

Capacitor. (s.f.). *Geolocation Capacitor plugin API*. Recuperado el 24 de octubre de 2026, de https://capacitorjs.com/docs/apis/geolocation

Congreso de Colombia. (2012a, 24 de abril). *Ley 1523 de 2012, por la cual se adopta la política nacional de gestión del riesgo de desastres y se establece el Sistema Nacional de Gestión del Riesgo de Desastres*.

Congreso de Colombia. (2012b, 17 de octubre). *Ley Estatutaria 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales*.

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Tesis doctoral, University of California, Irvine].

Finazzi, F. (2020). The Earthquake Network Project: A platform for earthquake early warning, rapid impact assessment, and search and rescue. *Frontiers in Earth Science, 8*, Artículo 243. https://doi.org/10.3389/feart.2020.00243

International Organization for Standardization. (2023). *ISO/IEC 25010:2023 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model*.

Ionic. (s.f.). *Ionic React overview*. Recuperado el 24 de octubre de 2026, de https://ionicframework.com/docs/react

Kelley, J. E., & Walker, M. R. (1959). Critical-path planning and scheduling. En *Proceedings of the Eastern Joint Computer Conference* (pp. 160–173). Association for Computing Machinery.

Kong, Q., Allen, R. M., Schreier, L., & Kwon, Y.-W. (2016). MyShake: A smartphone seismic network for earthquake early warning and beyond. *Science Advances, 2*(2), Artículo e1501055. https://doi.org/10.1126/sciadv.1501055

Matsumoto, A., Miyazako, H., & Nara, T. (2026). Localization system for urban search and rescue using smartphone microphones. *JASA Express Letters, 6*(6), Artículo 062801. https://doi.org/10.1121/10.0044095

Nielsen, J. (1994). Enhancing the explanatory power of usability heuristics. En *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (pp. 152–158). Association for Computing Machinery.

OWASP Foundation. (2024). *OWASP Mobile Top 10 2024*. https://owasp.org/www-project-mobile-top-10/

Presidencia de la República de Colombia. (2013, 27 de junio). *Decreto 1377 de 2013, por el cual se reglamenta parcialmente la Ley 1581 de 2012*.

Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9.ª ed.). McGraw-Hill Education.

Reuter, C., & Kaufhold, M.-A. (2018). Fifteen years of social media in emergencies: A retrospective review and future directions for crisis informatics. *Journal of Contingencies and Crisis Management, 26*(1), 41–57. https://doi.org/10.1111/1468-5973.12196

Schwaber, K., & Sutherland, J. (2020). *La guía de Scrum*. https://scrumguides.org/

Servicio Geológico Colombiano. (s.f.). *Amenaza sísmica*. Dirección de Geoamenazas. Recuperado el 24 de octubre de 2026, de https://www2.sgc.gov.co/Direcciones-Tecnicas/geoamenazas/amenaza-sismica/Paginas/default.aspx

Servicio Geológico Colombiano. (2026a, 17 de julio). *Colombia es un país expuesto a la amenaza sísmica*. https://www2.sgc.gov.co/Noticias/Paginas/Colombia-es-un-pais-expuesto-a-la-amenaza-sismica.aspx

Servicio Geológico Colombiano. (2026b, 10 de agosto). *Servicio Geológico Colombiano actualiza la información sobre el sismo ocurrido en San José del Palmar, Chocó*. https://www2.sgc.gov.co/Noticias/Paginas/SGC-actualiza-la-informacion-sobre-el-sismo-ocurrido-en-San-Jose-del-Palmar-Choco.aspx

Sommerville, I. (2016). *Software engineering* (10.ª ed.). Pearson.

Tan, M. L., Prasanna, R., Stock, K., Hudson-Doyle, E., Leonard, G., & Johnston, D. (2017). Mobile applications in crisis informatics literature: A systematic review. *International Journal of Disaster Risk Reduction, 24*, 297–311. https://doi.org/10.1016/j.ijdrr.2017.06.009

United Nations Office for Disaster Risk Reduction. (2015). *Marco de Sendai para la Reducción del Riesgo de Desastres 2015-2030*. https://www.undrr.org/publication/sendai-framework-disaster-risk-reduction-2015-2030

University of California, Berkeley. (s.f.). *MyShake*. Recuperado el 24 de octubre de 2026, de https://myshake.berkeley.edu/

U.S. Geological Survey. (s.f.-a). *Earthquake Hazards Program*. Recuperado el 24 de octubre de 2026, de https://www.usgs.gov/programs/earthquake-hazards

U.S. Geological Survey. (s.f.-b). *GeoJSON summary format*. Recuperado el 2 de noviembre de 2026, de https://earthquake.usgs.gov/earthquakes/feed/v1.0/geojson.php

Wiegers, K., & Beatty, J. (2013). *Software requirements* (3.ª ed.). Microsoft Press.

World Health Organization. (s.f.). *Emergencies*. Recuperado el 24 de octubre de 2026, de https://www.who.int/health-topics/emergencies

World Wide Web Consortium. (2023). *Web Content Accessibility Guidelines (WCAG) 2.2*. https://www.w3.org/TR/WCAG22/

# ANEXO A. MATRIZ RESUMIDA DE TRAZABILIDAD

La matriz se actualizó con los requisitos detallados de la segunda entrega y con los casos de prueba que los verifican.

**Tabla 106**
*Matriz resumida de trazabilidad entre requisitos, casos de prueba y estado*

| Grupo de requisitos | Casos de prueba | Estado al 09/11/2026 |
|---|---|---|
| **RF-RE · Registro** | CP-E-01 a CP-E-06, CP-M-01 | Verificado |
| **RF-AL · Alerta SOS** | CP-E-09, 11, 16 a 19, 22, 44 a 46, CP-M-13 | Verificado; CP-M-13 parcial (falta teléfono físico) |
| **RF-CO · Correo** | CP-E-29 a CP-E-31, emailService.test.ts, CP-M-12 | Verificado en la app; CP-M-12 pendiente en producción |
| **RF-CT · Contactos y puntos** | CP-E-12, 37, 38, 41 a 43 | Verificado |
| **RF-IN · Información** | CP-E-20, 21, 23 a 25, 32, 40 | Verificado |
| **RF-CF · Configuración y cuenta** | CP-E-15, 34, 35, syncService.test.ts, CP-M-06, CP-M-07 | Verificado |
| **RF-SI · Sincronización** | CP-E-33, syncService.test.ts, catalogService.test.ts, CP-M-02 | Verificado |
| **RF-PA · Panel** | CP-M-08 a CP-M-11 | CP-M-08 verificado; CP-M-10 parcial; CP-M-09 y CP-M-11 pendientes |
| **RF-PI · Página informativa** | CP-M-14 | Verificado |
| **RNF** | CP-M-01 a CP-M-07, ESLint, TypeScript e integración continua | Verificado; CP-M-05 pendiente en producción |

Fuente: elaboración propia.

# ANEXO B. FORMATO DE REGISTRO DE PRUEBAS

**Tabla 107**
*Formato de registro de pruebas*

| Campo | Información |
|---|---|
| Identificador | CP-__ |
| Fecha y hora | _____ |
| Versión del prototipo | _____ |
| Dispositivo y sistema operativo | _____ |
| Precondiciones | _____ |
| Pasos ejecutados | _____ |
| Resultado esperado | _____ |

| | |
|---|---|
| Resultado observado | _____ |
| Estado | Aprobado / Fallido / Bloqueado / Pendiente |
| Evidencia asociada | _____ |
| Observaciones | _____ |

Fuente: elaboración propia.

# ANEXO C. DEFINICIONES OPERATIVAS

**Tabla 108**
*Definiciones operativas*

| Término | Uso en el proyecto |
|---|---|
| Alerta local | Señal o cambio de interfaz producido en el teléfono. |
| Alerta digital | Solicitud transmitida mediante un servicio externo implementado. |
| Contacto de confianza | Persona configurada para recibir información si se implementa el canal. |
| Geolocalización disponible | Coordenadas que el dispositivo puede proporcionar; no implica precisión garantizada. |
| Envío confirmado | Estado mostrado solo cuando el servicio devuelve confirmación verificable. |
| Prototipo | Versión parcial para evaluar flujos y requisitos, no necesariamente lista para uso real. |

Fuente: elaboración propia.

# ANEXO D. EVIDENCIAS VISUALES Y TÉCNICAS DEL PROTOTIPO (PRIMERA ENTREGA)

Las siguientes evidencias documentan las pantallas del prototipo y algunos archivos del código fuente. Las vistas identificadas como modo de demostración utilizan datos ficticios. La simulación no demuestra el envío real de mensajes ni la integración con servicios oficiales de emergencia.

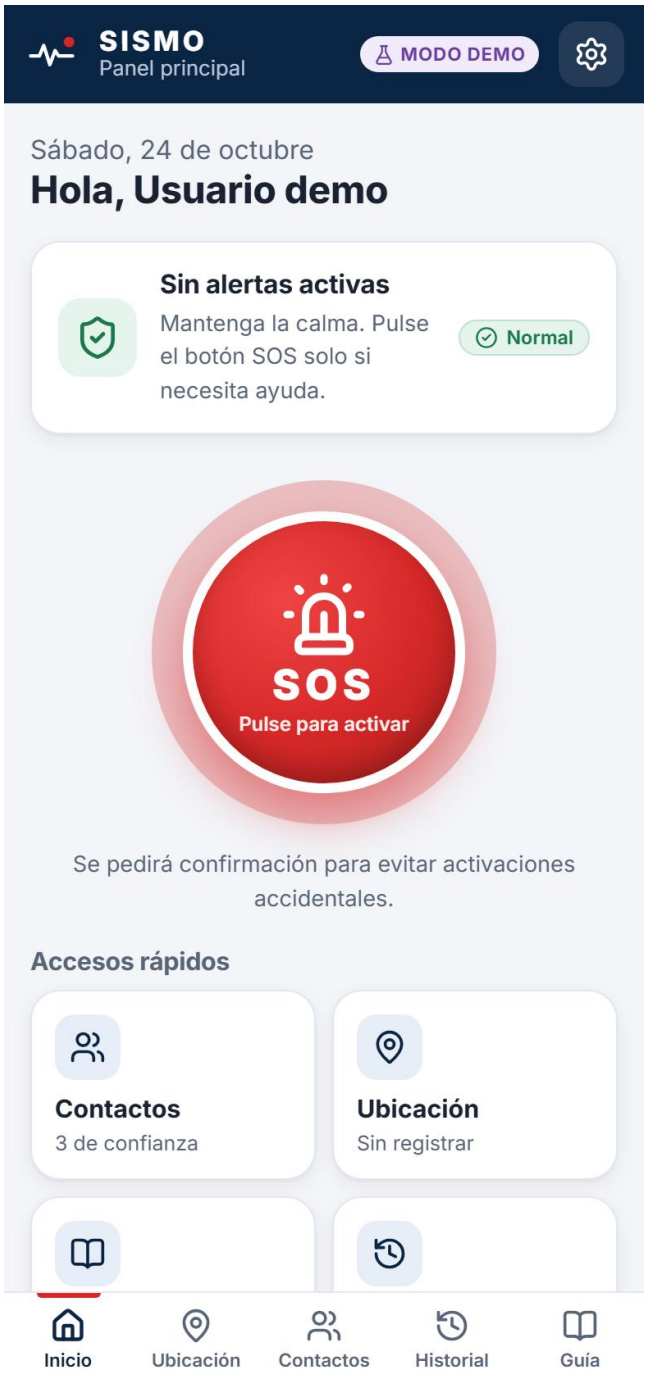
**Figura 36**

*Pantalla de bienvenida y alcance del prototipo*

Fuente: elaboración propia (captura del 24/10/2026, primera entrega).

**Figura 37**

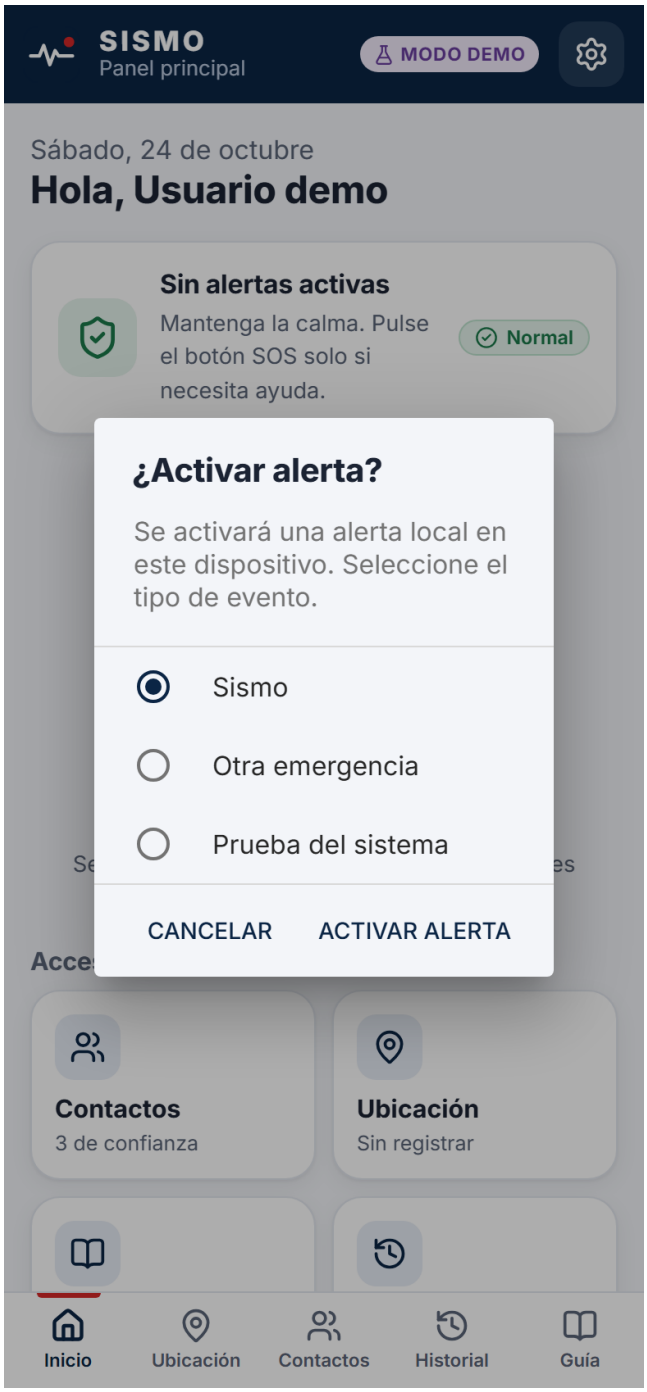
*Panel principal y botón SOS*




Fuente: elaboración propia (captura del 24/10/2026, primera entrega).

**Figura 38**

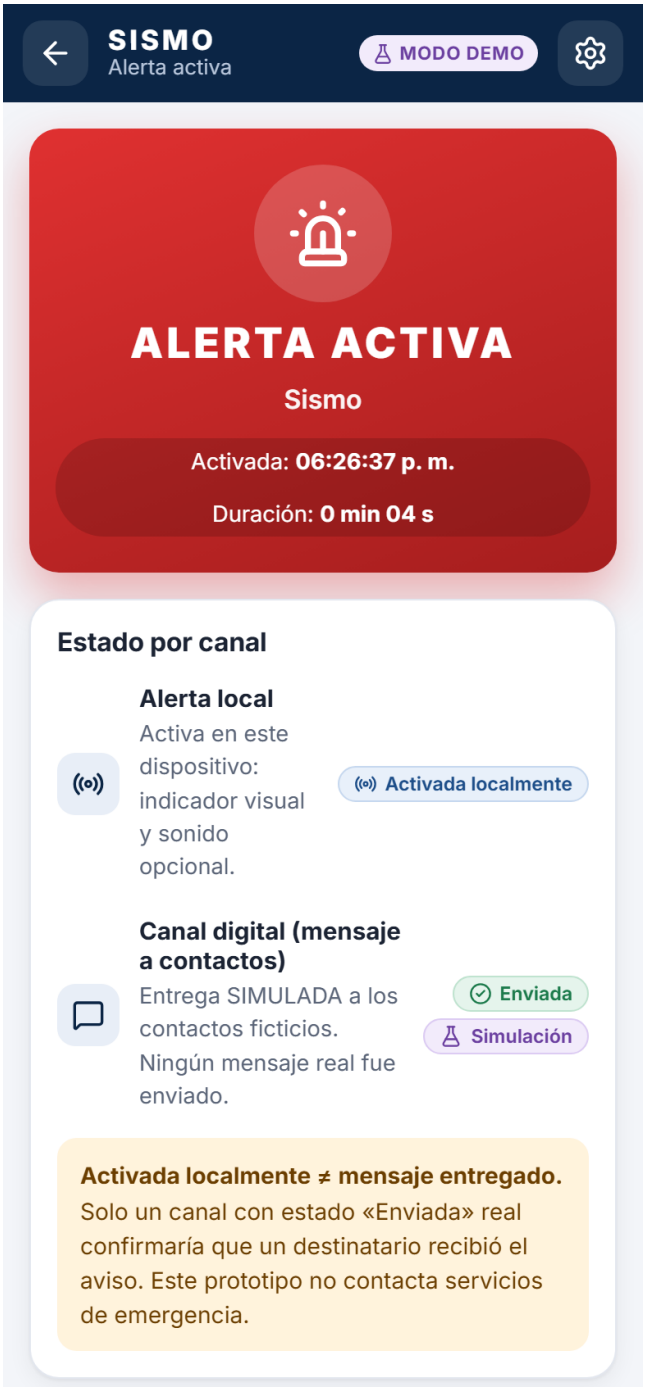
*Confirmación del tipo de alerta*




Fuente: elaboración propia (captura del 24/10/2026, primera entrega).

**Figura 39**

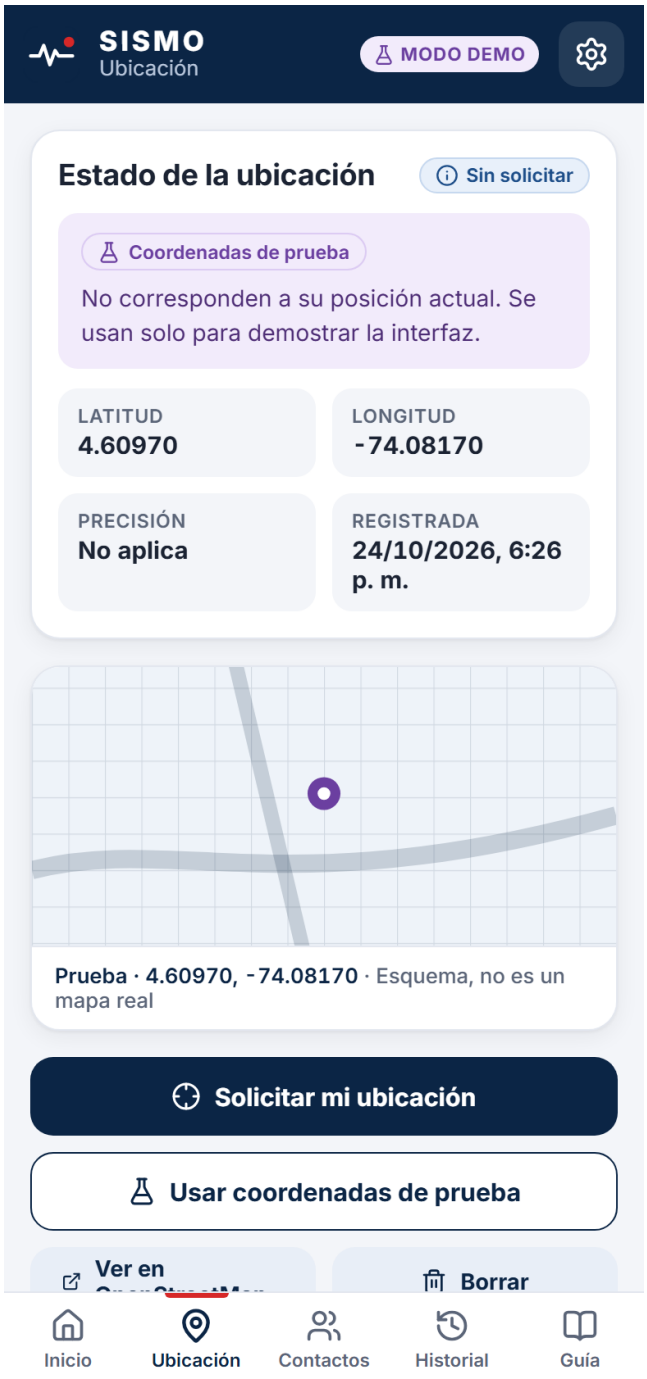
*Estado de alerta activa y canales en modo de demostración*




Fuente: elaboración propia (captura del 24/10/2026, primera entrega).

**Figura 40**

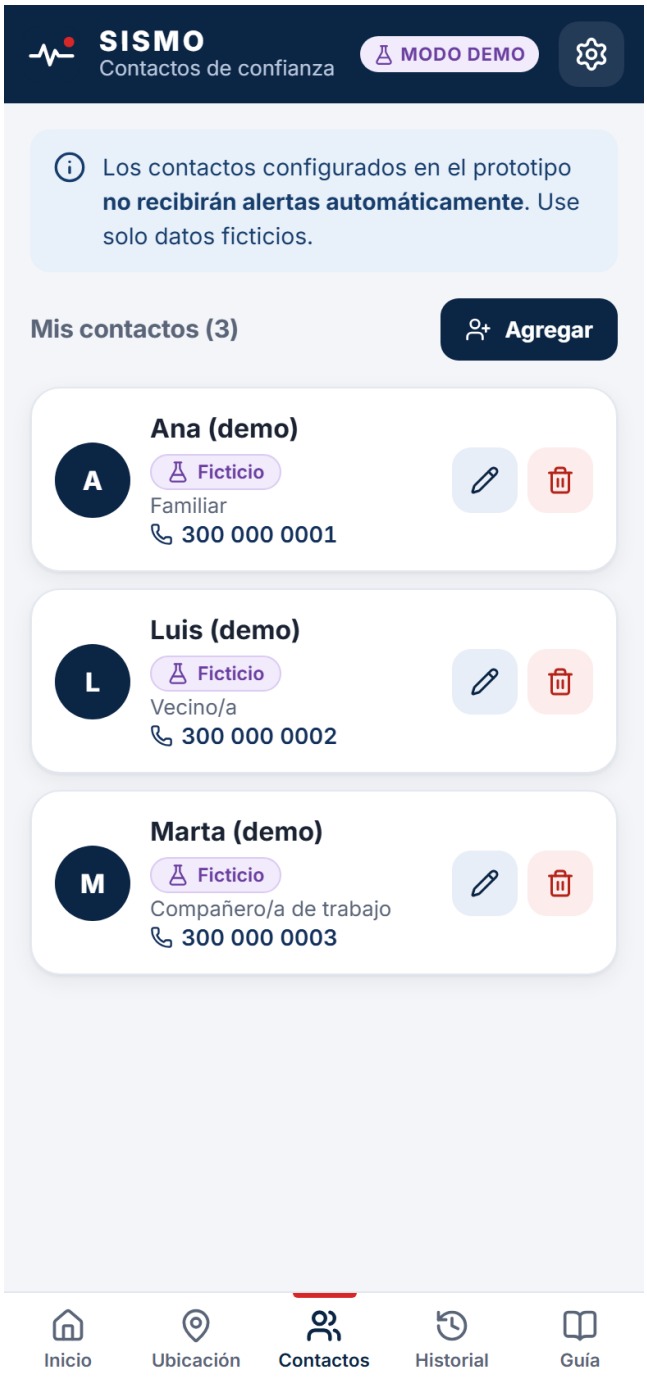
*Estado de ubicación y coordenadas de prueba*




Fuente: elaboración propia (captura del 24/10/2026, primera entrega).

**Figura 41**

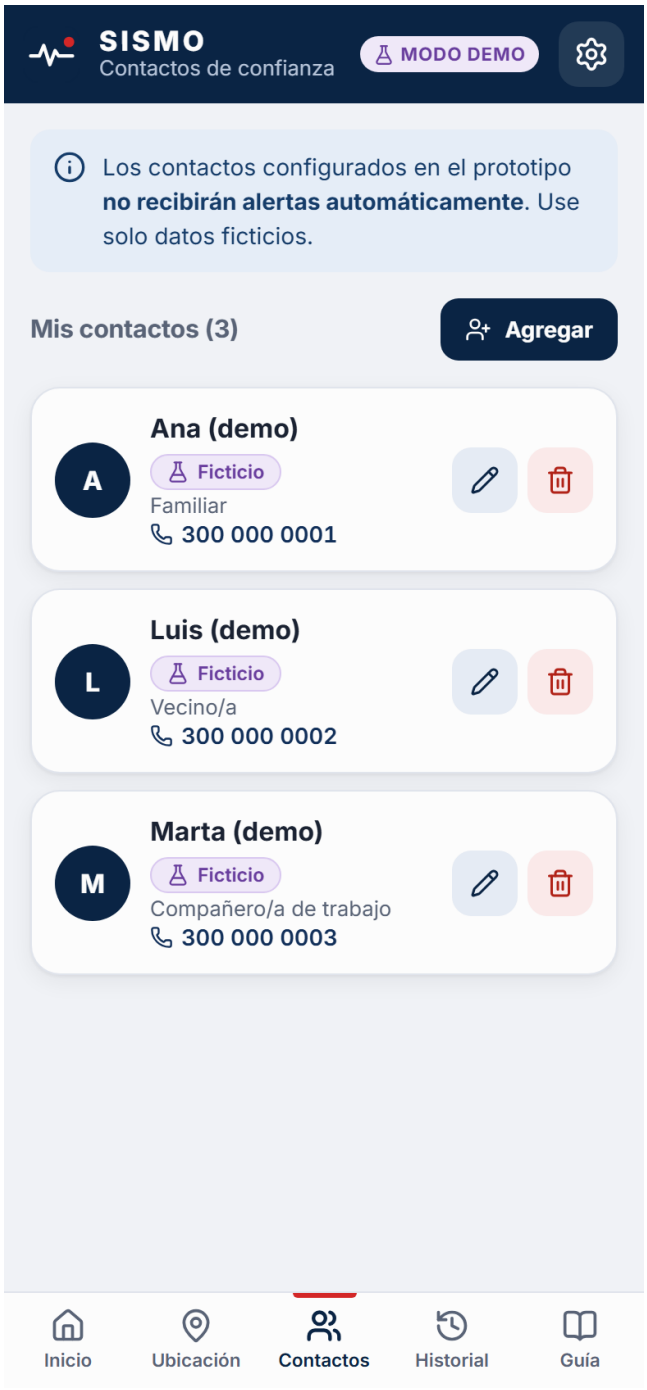
*Listado de contactos de confianza ficticios*




Fuente: elaboración propia (captura del 24/10/2026, primera entrega).

**Figura 42**

*Formulario para agregar un contacto ficticio*




Fuente: elaboración propia (captura del 24/10/2026, primera entrega).

**Figura 43**

*Historial de alertas de ejemplo*

Fuente: elaboración propia (captura del 24/10/2026, primera entrega).

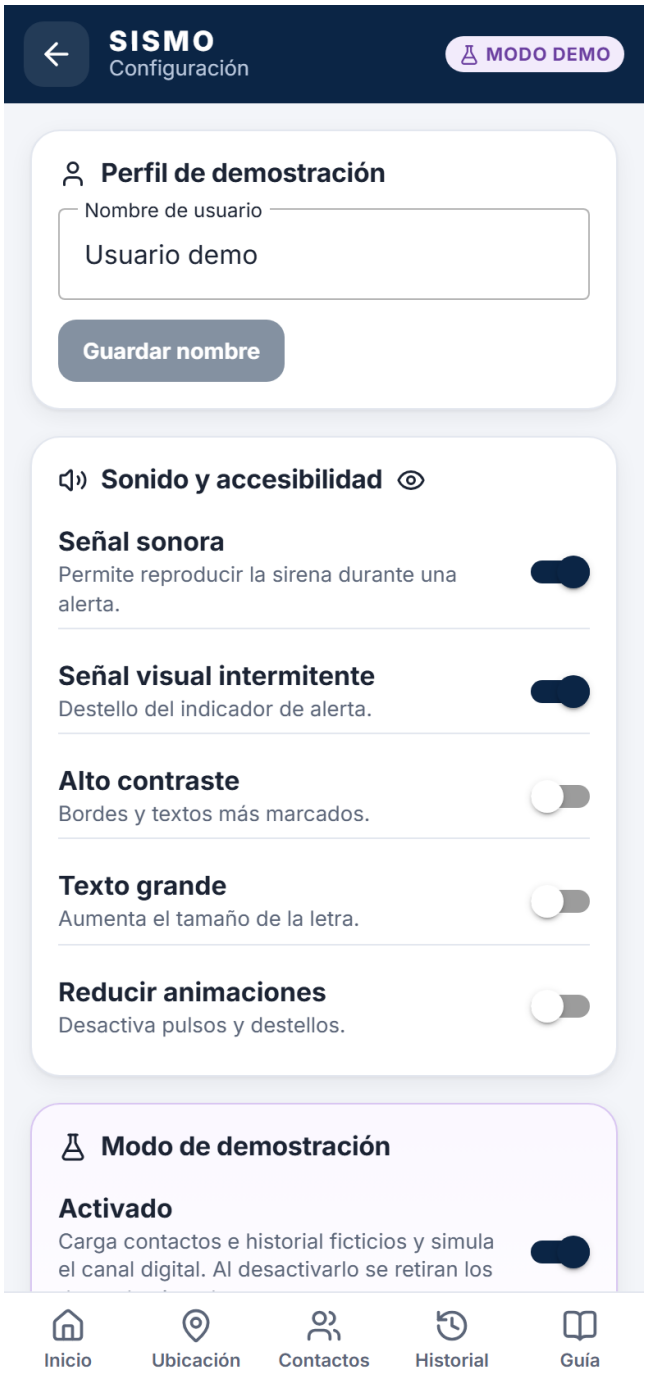
**Figura 44**

*Guía de recomendaciones para emergencias sísmicas*

Fuente: elaboración propia (captura del 24/10/2026, primera entrega).

**Figura 45**

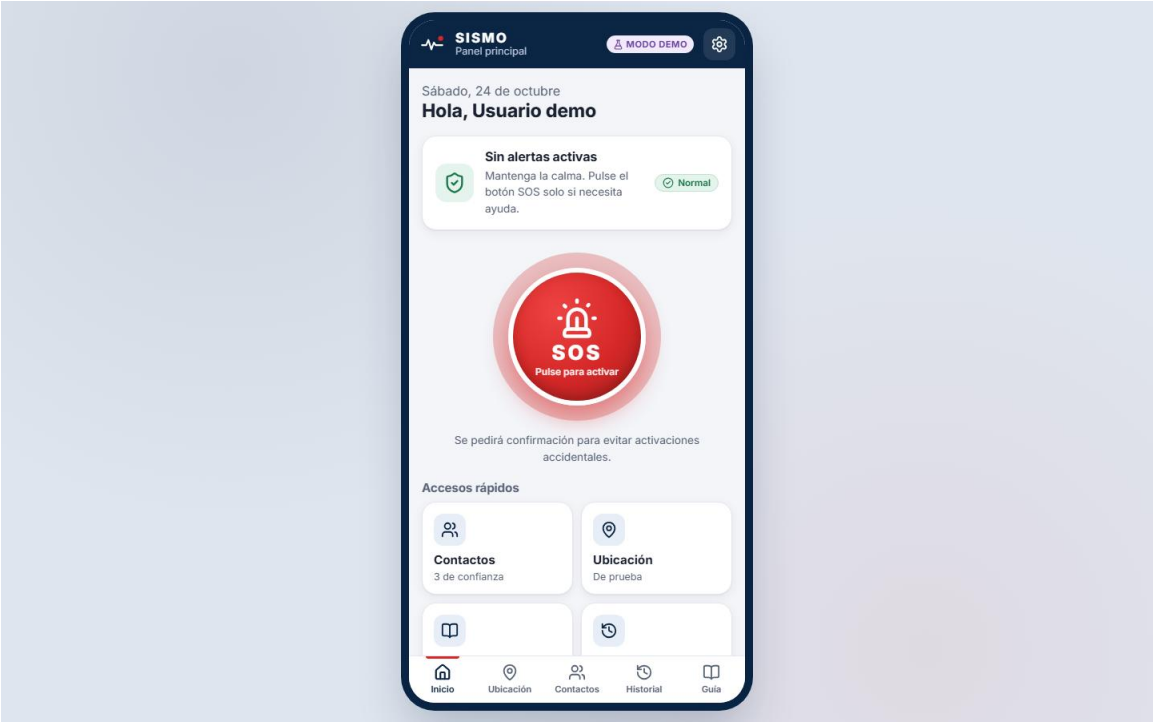
*Ajustes de sonido, accesibilidad y modo de demostración*




Fuente: elaboración propia (captura del 24/10/2026, primera entrega).

**Figura 46**

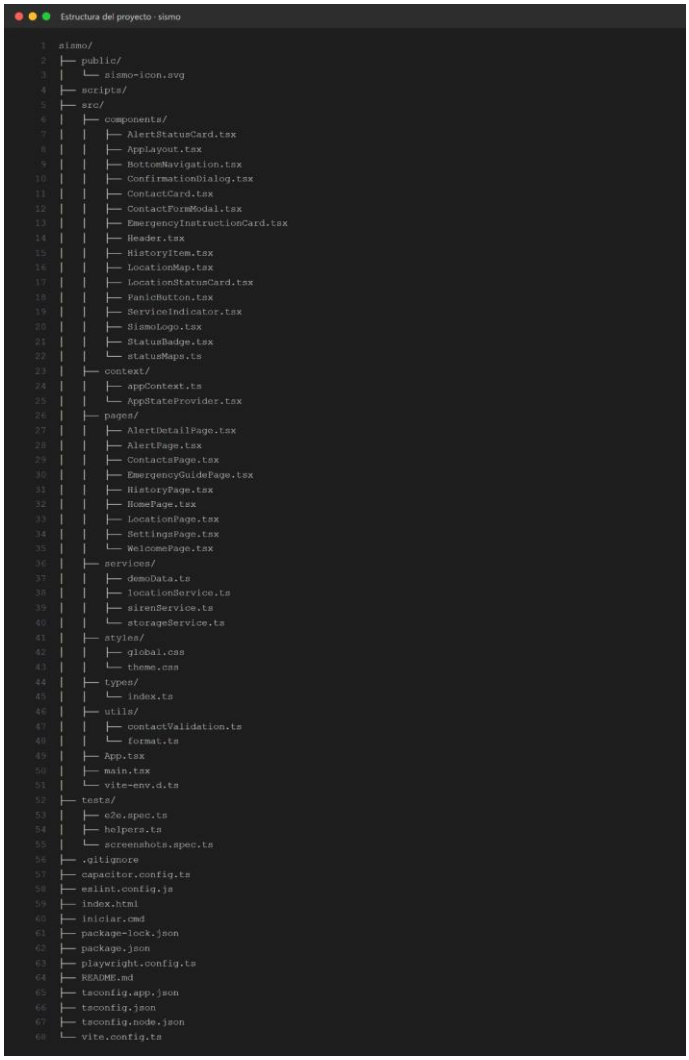
*Vista de escritorio del prototipo*




Fuente: elaboración propia (captura del 24/10/2026, primera entrega).

**Figura 47**

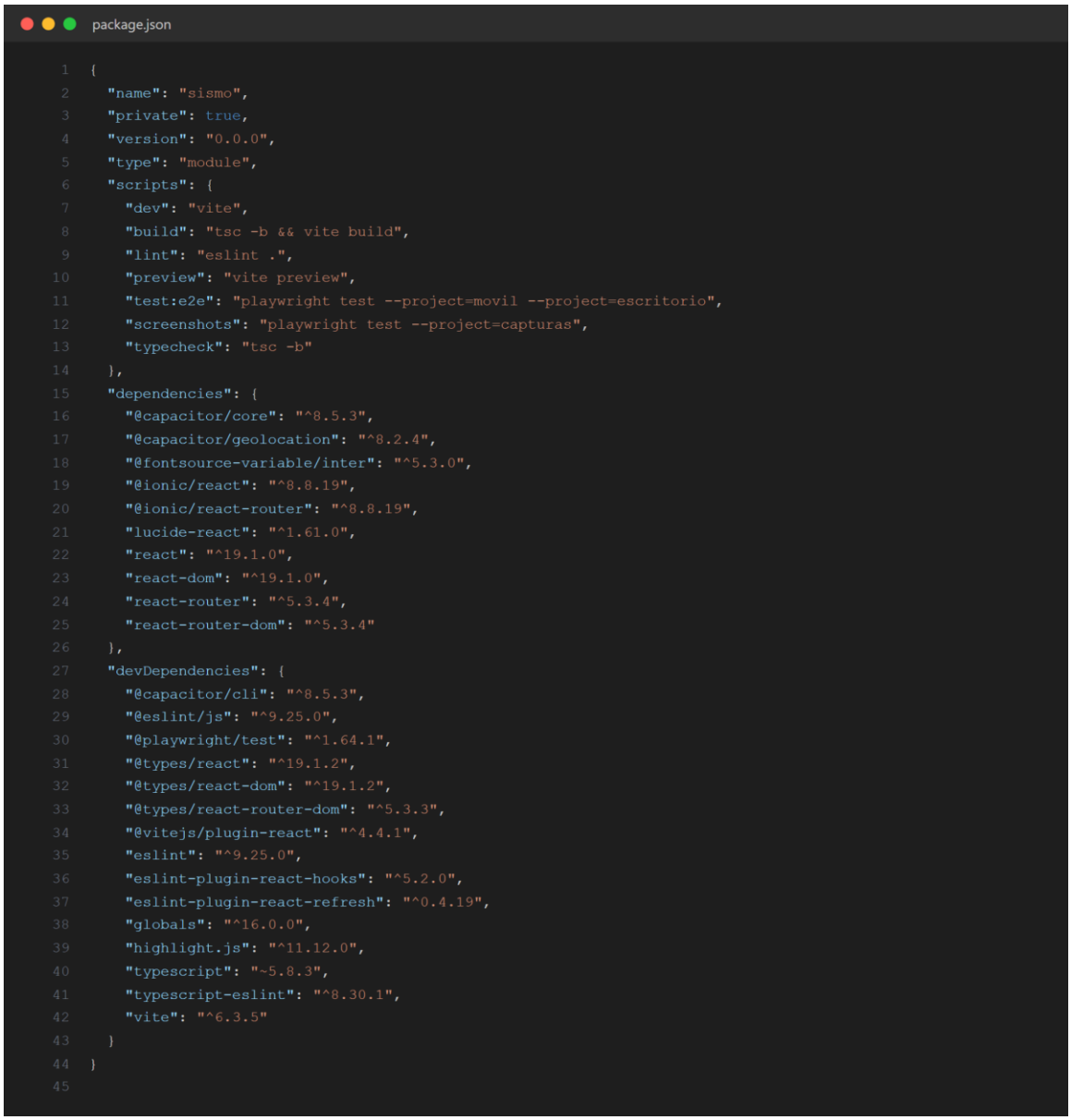
*Estructura de carpetas y archivos*




Fuente: elaboración propia (captura del 24/10/2026, primera entrega).

**Figura 48**

*Dependencias y comandos del proyecto*




Fuente: elaboración propia (captura del 24/10/2026, primera entrega).

**Figura 49**

*Rutas de navegación de la aplicación*

src/App.tsx — rutas (React Router + IonRouterOutlet)

```tsx
import { IonApp, IonRouterOutlet, setupIonicReact } from '@ionic/react';
import { IonReactRouter } from '@ionic/react-router';
import { Redirect, Route } from 'react-router-dom';
import { AppStateProvider } from './context/AppStateProvider';
import { useAppState } from './context/appContext';
import AlertDetailPage from './pages/AlertDetailPage';
import AlertPage from './pages/AlertPage';
import ContactsPage from './pages/ContactsPage';
import EmergencyGuidePage from './pages/EmergencyGuidePage';
import HistoryPage from './pages/HistoryPage';
import HomePage from './pages/HomePage';
import LocationPage from './pages/LocationPage';
import SettingsPage from './pages/SettingsPage';
import WelcomePage from './pages/WelcomePage';

setupIonicReact({ mode: 'md' });

function Routes() {
  const { data } = useAppState();
  return (
    <IonRouterOutlet>
      <Route exact path="/bienvenida" component={WelcomePage} />
      <Route exact path="/inicio" component={HomePage} />
      <Route exact path="/alerta" component={AlertPage} />
      <Route exact path="/ubicacion" component={LocationPage} />
      <Route exact path="/contactos" component={ContactsPage} />
      <Route exact path="/historial" component={HistoryPage} />
      <Route exact path="/historial/:id" component={AlertDetailPage} />
      <Route exact path="/guia" component={EmergencyGuidePage} />
      <Route exact path="/configuracion" component={SettingsPage} />
      <Route exact path="/">
        <Redirect to={data.settings.onboarded ? '/inicio' : '/bienvenida'} />
      </Route>
    </IonRouterOutlet>
  );
}

export default function App() {
  return (
    <IonApp>
      <AppStateProvider>
        <IonReactRouter>
          <Routes />
        </IonReactRouter>
      </AppStateProvider>
    </IonApp>
  );
}

```

Fuente: elaboración propia (captura del 24/10/2026, primera entrega).

**Figura 50**

*Componente del botón de pánico*

src/components/PanicButton.tsx

```tsx
import { Siren } from 'lucide-react';

interface PanicButtonProps {
  onPress: () => void;
  active?: boolean;
}

export function PanicButton({ onPress, active = false }: PanicButtonProps) {
  return (
    <div className={`panic${active ? ' panic--active' : ''}`}>
      <span className="panic__ring" aria-hidden="true" />
      <span className="panic__ring panic__ring--2" aria-hidden="true" />
      <button className="panic__button" onClick={onPress} aria-label={active ? 'Ver alerta activa' : 'Activar alerta de emergencia'}>
        <Siren size={54} strokeWidth={2} aria-hidden="true" />
        <strong>{active ? 'ALERTA ACTIVA' : 'SOS'}</strong>
        <small>{active ? 'Toque para ver' : 'Pulse para activar'}</small>
      </button>
    </div>
  );
}
```

Fuente: elaboración propia (captura del 24/10/2026, primera entrega).

**Figura 51**

*Lógica de activación y simulación de alertas*

Fuente: elaboración propia (captura del 24/10/2026, primera entrega).

## Nota sobre las evidencias

Las capturas ilustran la interfaz y fragmentos visibles del código. Para acreditar compilación o pruebas superadas, deben adjuntarse registros reales de ejecución. Las coordenadas marcadas como prueba no son una ubicación verificada del usuario.

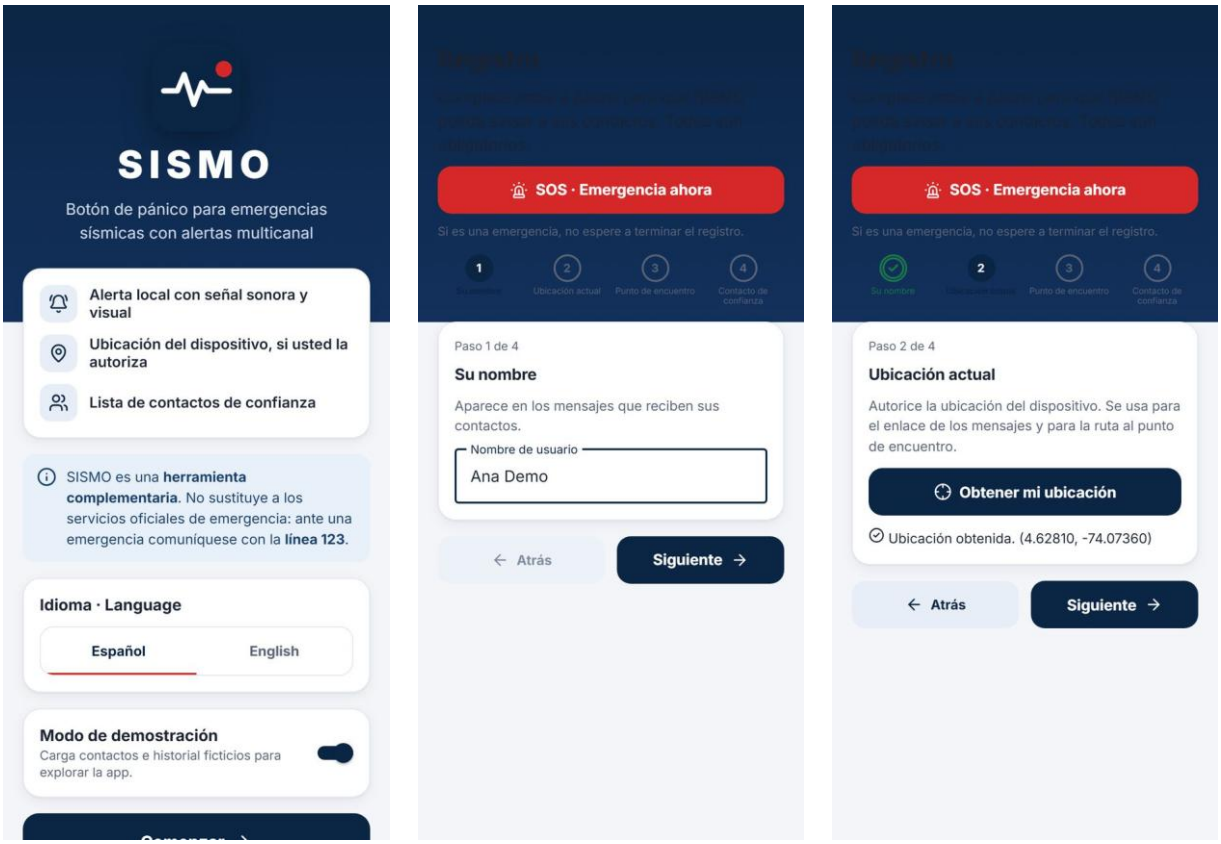
# ANEXO E. EVIDENCIAS VISUALES DE LA APLICACIÓN MÓVIL (SEGUNDA ENTREGA)

Capturas de la app tomadas en las fechas de cada versión, con datos ficticios (nombres «Demo», teléfonos de prueba y correos @example.com). Las capturas de Android se tomaron en el emulador Android 16.

## E.1 Versión 2: app completa (02/11/2026)

**Figura 52**

*App (v2): Bienvenida; registro, paso 1: nombre; registro, paso 2: ubicación*




(a) Bienvenida (b) Registro, paso 1: nombre (c) Registro, paso 2: ubicación

**(a) Bienvenida.** Primera pantalla de la app. Presenta SISMO, lo que hace (alerta local, ubicación y contactos de confianza), aclara que no sustituye a la línea 123, permite elegir el idioma y activar el modo de demostración antes de tocar «Comenzar».

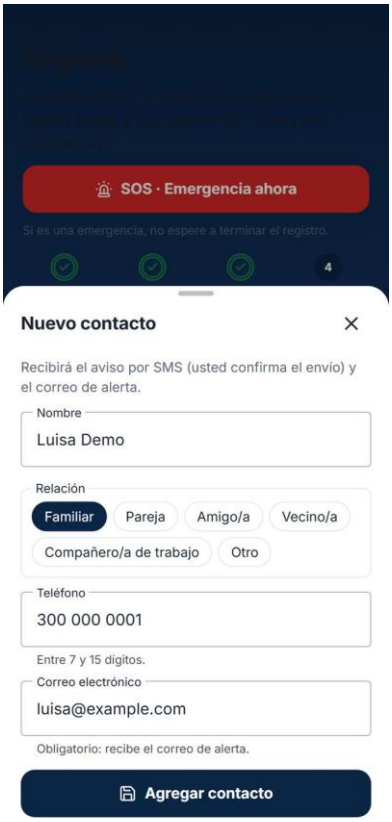
**(b) Registro, paso 1: nombre.** Paso 1 del registro obligatorio: la persona escribe el nombre que verán sus contactos en los mensajes. El botón «SOS · Emergencia ahora» permanece arriba para pedir ayuda sin terminar el registro.

**(c) Registro, paso 2: ubicación.** Paso 2: la persona autoriza la ubicación del dispositivo con «Obtener mi ubicación»; la app confirma las coordenadas obtenidas, que se usan en los mensajes y en la ruta al punto de encuentro.

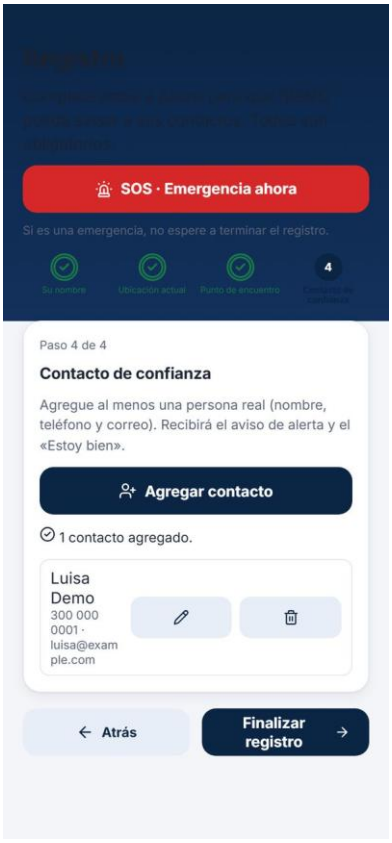
Fuente: elaboración propia (captura del 02/11/2026).

**Figura 53**
*App (v2): Registro, paso 3: puntos de encuentro; registro, paso 4: nuevo contacto; registro, paso 4: contactos*

(a) Registro, paso 3: puntos de encuentro




(b) Registro, paso 4: nuevo contacto




(c) Registro, paso 4: contactos

**(a) Registro, paso 3: puntos de encuentro.** Paso 3: se registra al menos un punto de encuentro desde la ubicación actual o buscando una dirección; la lista muestra los puntos guardados (hasta 5) con la opción de eliminarlos.

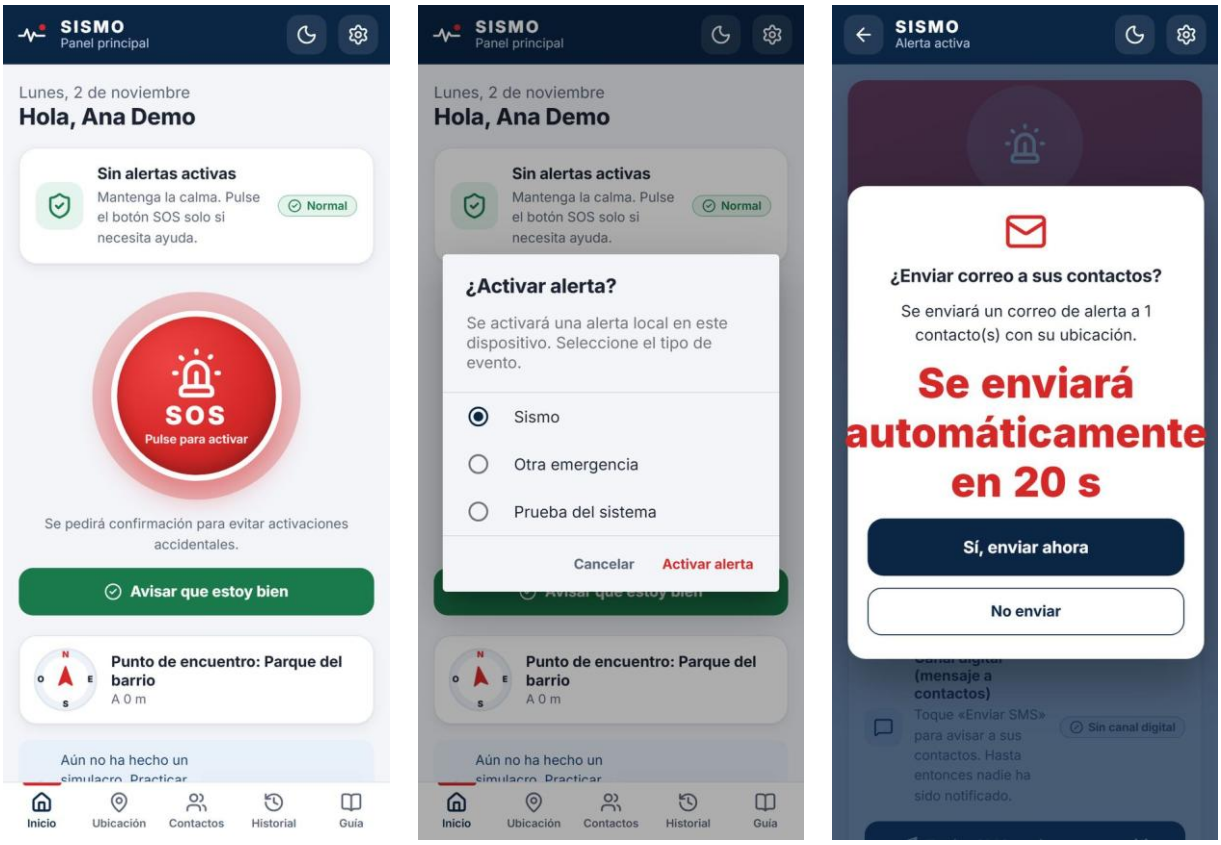
**(b) Registro, paso 4: nuevo contacto.** Formulario «Nuevo contacto» del paso 4: nombre, relación (botones del catálogo), teléfono y correo, con validación de cada campo antes de «Agregar contacto».

**(c) Registro, paso 4: contactos.** Paso 4 completo: el contacto agregado aparece con opciones para editarlo o eliminarlo y se habilita «Finalizar registro».

Fuente: elaboración propia (captura del 02/11/2026).

**Figura 54**

*App (v2): Inicio; sOS tipo de evento; alerta pregunta correo*




(a) Inicio (b) SOS tipo de evento (c) Alerta pregunta correo

**(a) Inicio.** Inicio de la app: saludo, estado «Sin alertas activas», botón SOS grande que pide confirmación, botón «Avisar que estoy bien» y el punto de encuentro con la brújula.

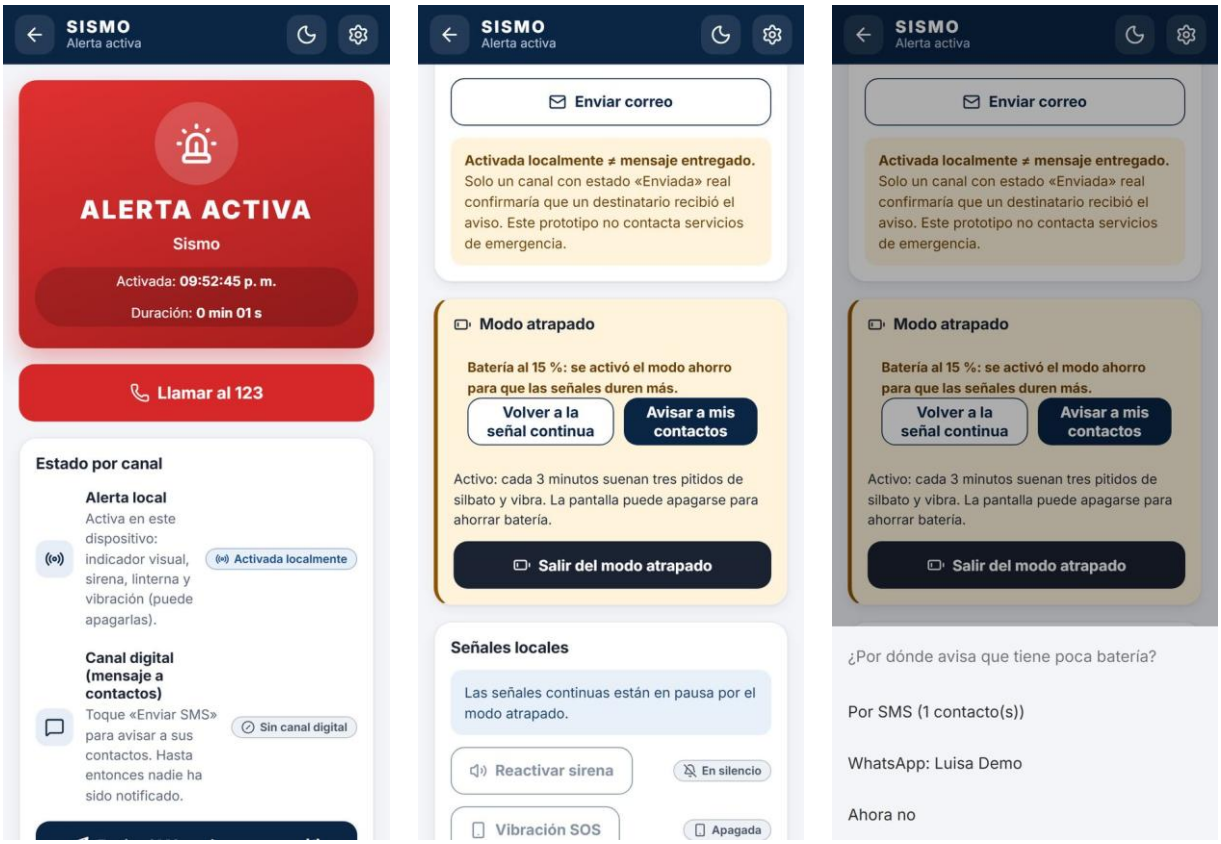
**(b) SOS tipo de evento.** Al tocar SOS la app pide el tipo de evento (sismo, otra emergencia o prueba del sistema) y una confirmación, para evitar activaciones accidentales.

**(c) Alerta pregunta correo.** Con la alerta activa, la app pregunta si envía un correo a los contactos y muestra la cuenta regresiva de 20 segundos; si nadie responde, el correo se envía solo.

Fuente: elaboración propia (captura del 02/11/2026).

**Figura 55**

*App (v2): Alerta activa; alerta ahorro batería; alerta avisar batería baja*




(a) Alerta activa (b) Alerta ahorro batería (c) Alerta avisar batería baja

**(a) Alerta activa.** Pantalla de alerta activa: tipo de evento, hora y duración, botón «Llamar al 123» y el estado de cada canal (alerta local activa, canal digital sin enviar todavía).

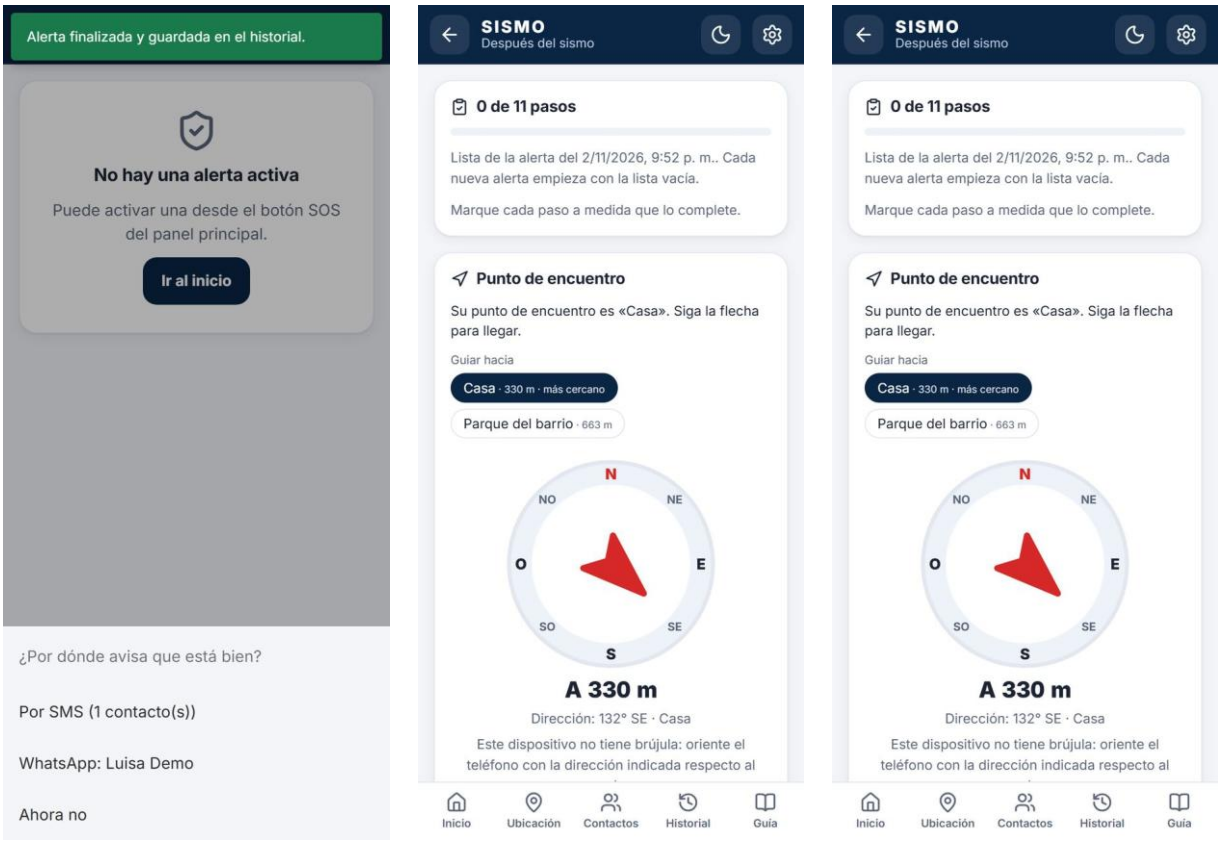
**(b) Alerta ahorro batería.** Con la batería baja, la alerta pasa sola al modo ahorro: las señales suenan por intervalos y se ofrece volver a la señal continua o avisar a los contactos.

**(c) Alerta avisar batería baja.** Hoja «¿Por dónde avisa que tiene poca batería?»: la persona elige enviar el aviso por SMS o por WhatsApp a un contacto, o no avisar.

Fuente: elaboración propia (captura del 02/11/2026).

**Figura 56**

*App (v2): Estoy bien canal; después del sismo; después del sismo completo*




(a) Estoy bien canal (b) Después del sismo (c) Después del sismo completo

**(a) Estoy bien canal.** Al finalizar la alerta, esta queda en el historial y la app pregunta por qué canal avisar «Estoy bien» (SMS o WhatsApp por contacto).

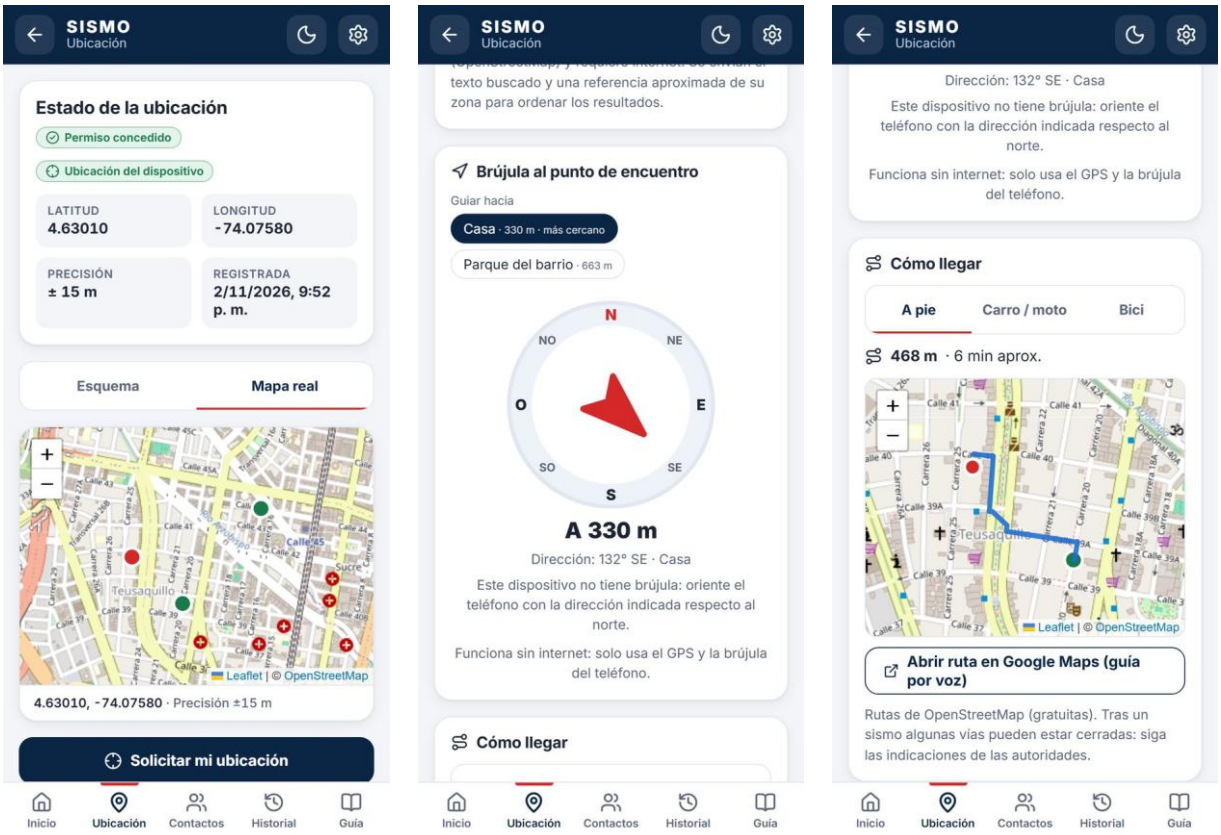
**(b) Después del sismo.** Lista «Después del sismo» con los pasos por completar, el punto de encuentro y la brújula que guía hacia el punto más cercano.

**(c) Después del sismo completo.** Vista completa de «Después del sismo»: el avance se guarda por paso y cada nueva alerta empieza la lista vacía.

Fuente: elaboración propia (captura del 02/11/2026).

**Figura 57**

*App (v2): Ubicación mapa real; brújula varios puntos; ruta al punto de encuentro*




(a) Ubicación mapa real (b) Brújula varios puntos (c) Ruta al punto de encuentro

**(a) Ubicación mapa real.** Estado de la ubicación con permiso concedido: latitud, longitud, precisión y hora, y el mapa real de OpenStreetMap con los puntos de encuentro.

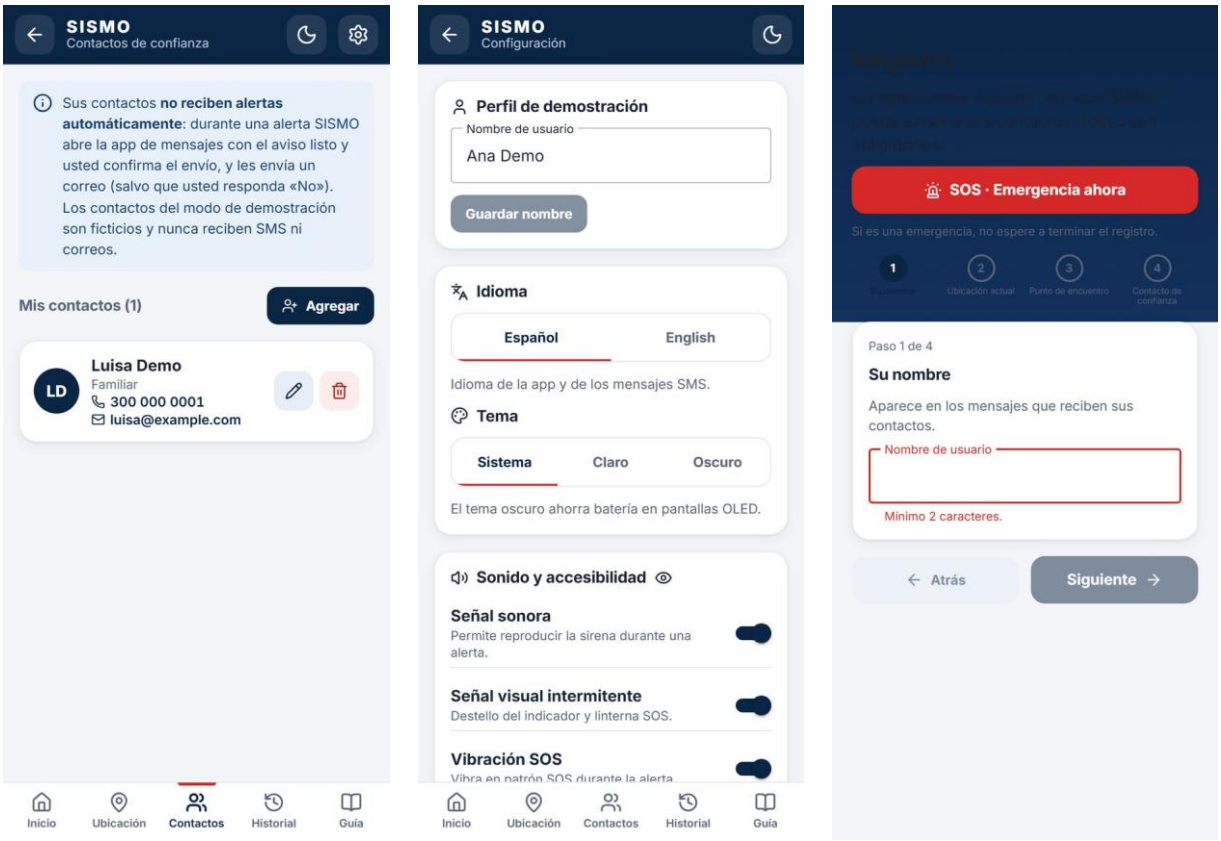
**(b) Brújula varios puntos.** Brújula hacia el punto de encuentro elegido (por defecto el más cercano), con distancia y dirección; funciona sin internet con el GPS y la brújula del teléfono.

**(c) Ruta al punto de encuentro.** «Cómo llegar»: ruta a pie, en carro o moto o en bicicleta con distancia y tiempo estimado, y enlace para abrir la ruta en Google Maps.

Fuente: elaboración propia (captura del 02/11/2026).

**Figura 58**

*App (v2): Contactos; configuración completa; registro SOS disponible*




(a) Contactos (b) Configuración completa (c) Registro SOS disponible

**(a) Contactos.** Lista de contactos de confianza con su relación, teléfono y correo; aclara que la persona confirma cada envío de SMS y permite agregar, editar o eliminar contactos.

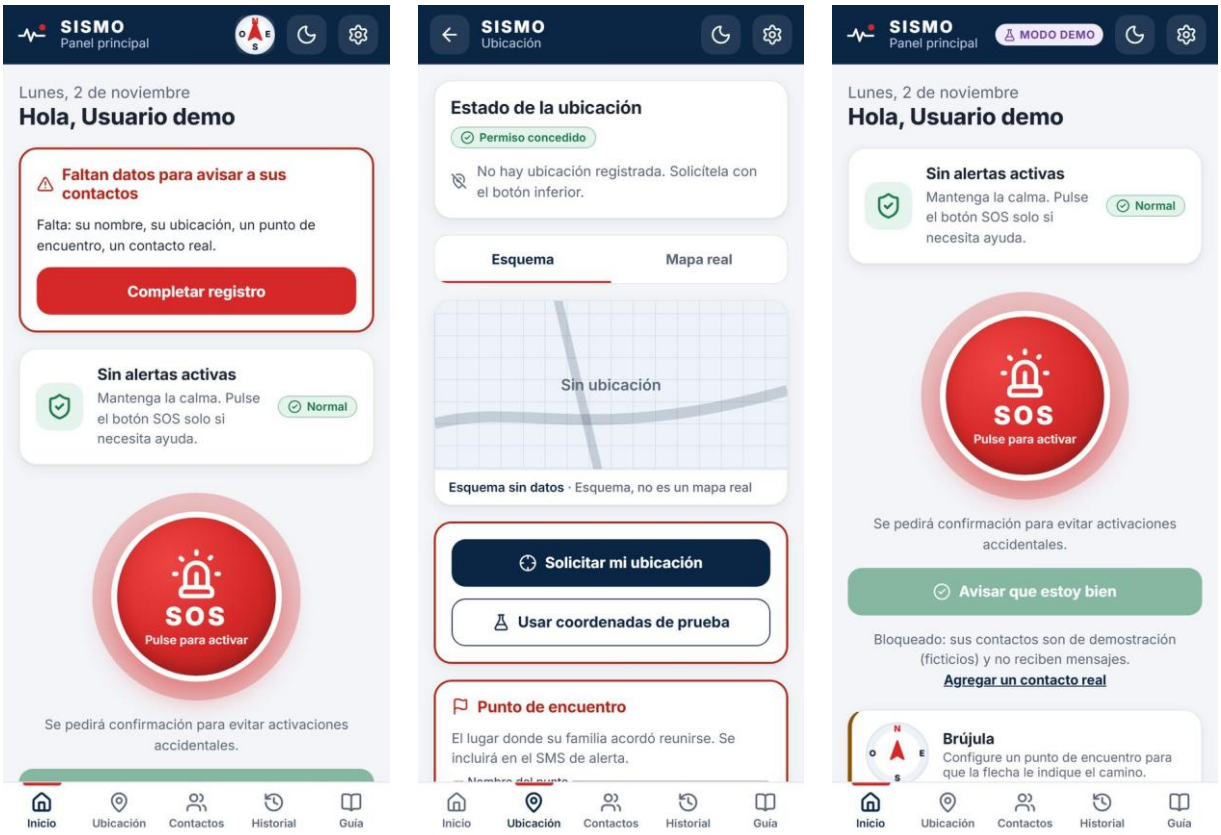
**(b) Configuración completa.** Configuración: nombre de usuario, idioma, tema (sistema, claro u oscuro) y las señales de la alerta (sirena, destello y vibración).

**(c) Registro SOS disponible.** Registro con un dato inválido: el campo se marca en rojo con el mensaje de error, y el botón SOS sigue disponible durante el registro.

Fuente: elaboración propia (captura del 02/11/2026).

**Figura 59**

*App (v2): Inicio datos faltantes; ubicación datos faltantes; inicio demostración*




(a) Inicio datos faltantes (b) Ubicación datos faltantes (c) Inicio demostración

**(a) Inicio datos faltantes.** Inicio con datos obligatorios faltantes: el aviso rojo «Faltan datos para avisar a sus contactos» lista lo que falta y lleva a «Completar registro».

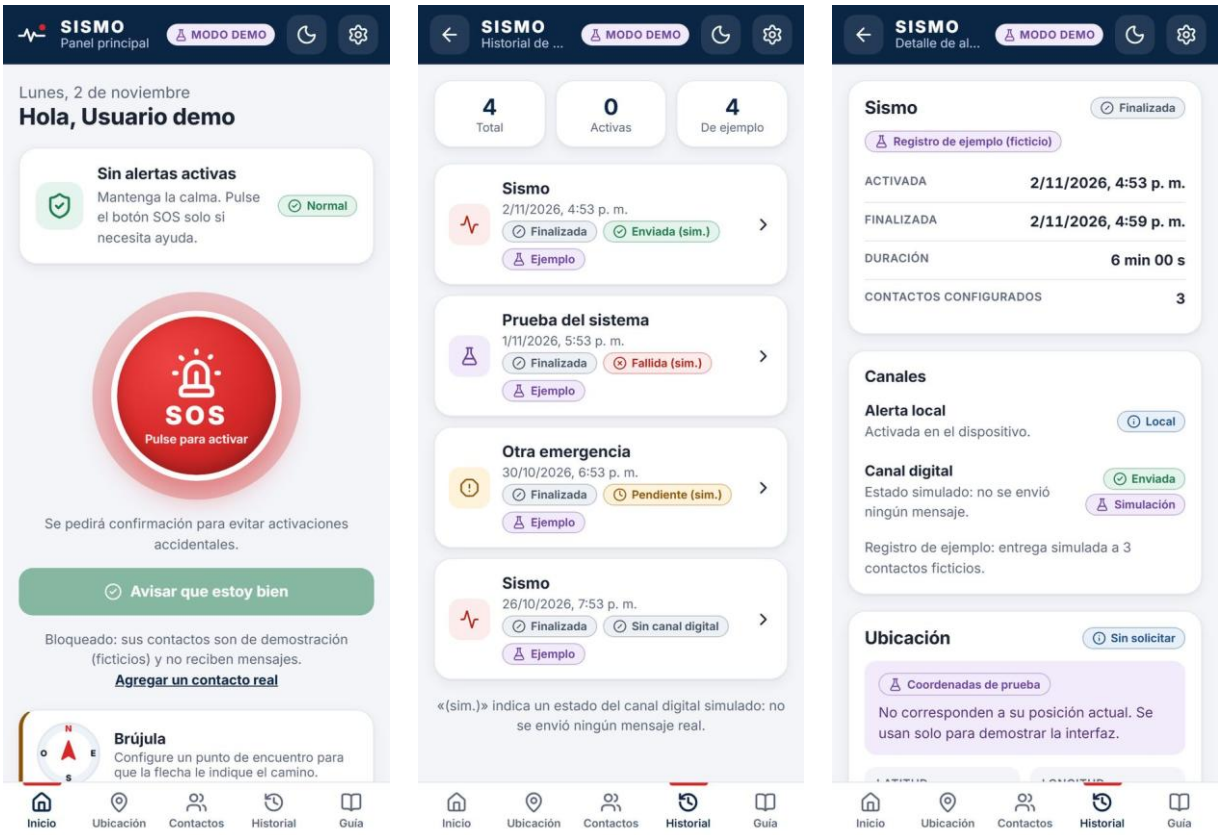
**(b) Ubicación datos faltantes.** Ubicación sin datos: tarjetas con borde rojo para solicitar la ubicación o usar coordenadas de prueba, y para registrar el punto de encuentro.

**(c) Inicio demostración.** Inicio en modo de demostración (solo en desarrollo): datos ficticios y el botón «Avisar que estoy bien» bloqueado porque los contactos de demostración no reciben mensajes.

Fuente: elaboración propia (captura del 02/11/2026).

**Figura 60**

*App (v2): Inicio completo; historial; detalle de alerta*




(a) Inicio completo (b) Historial (c) Detalle de alerta

**(a) Inicio completo.** Inicio completo en modo de demostración, con la brújula y las tarjetas de preparación debajo del botón SOS.

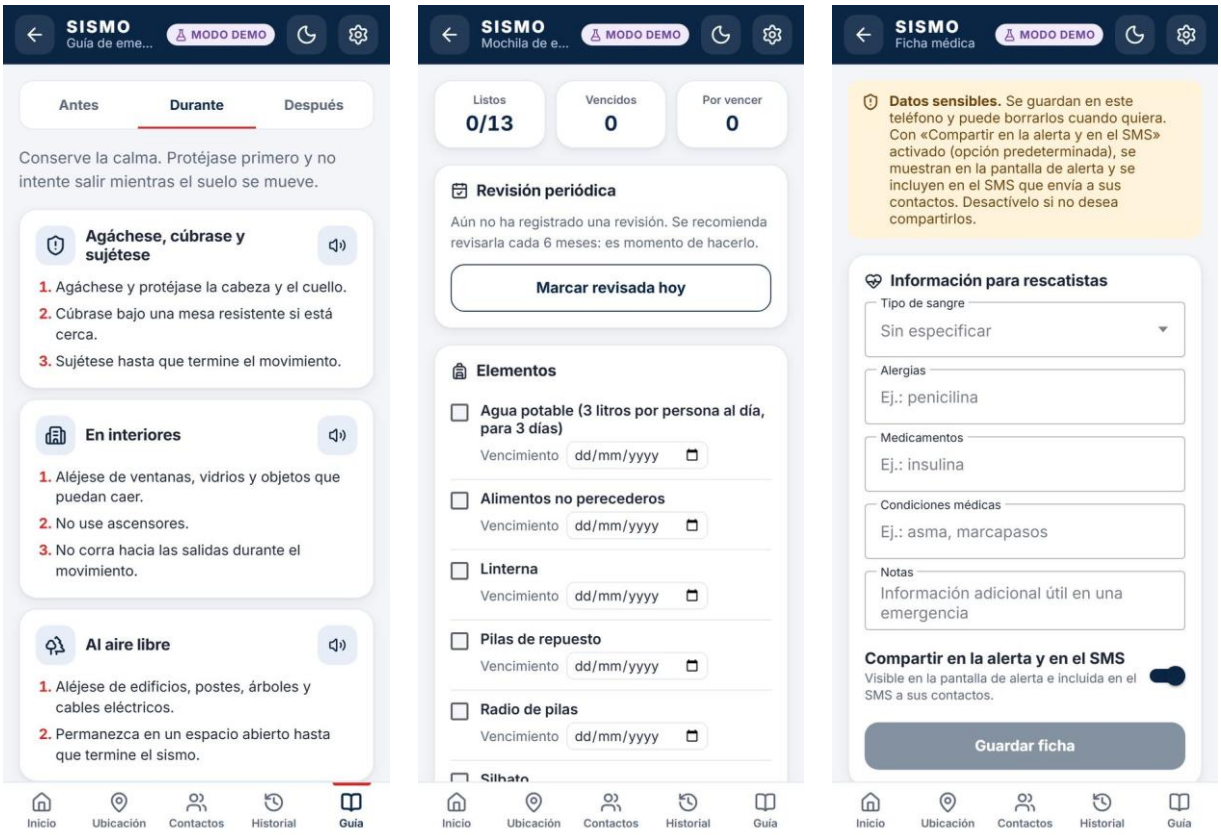
**(b) Historial.** Historial de alertas con totales (total, activas y de ejemplo) y el estado de cada alerta y de su canal digital.

**(c) Detalle de alerta.** Detalle de una alerta: horas de activación y finalización, duración, contactos configurados, estado de cada canal y la ubicación registrada.

Fuente: elaboración propia (captura del 02/11/2026).

**Figura 61**

*App (v2): Guía de emergencia; mochila de emergencia; ficha médica*




(a) Guía de emergencia (b) Mochila de emergencia (c) Ficha médica

**(a) Guía de emergencia.** Guía de emergencia en tres pestañas (antes, durante y después), con recomendaciones por situación y botón para escucharlas en voz alta.

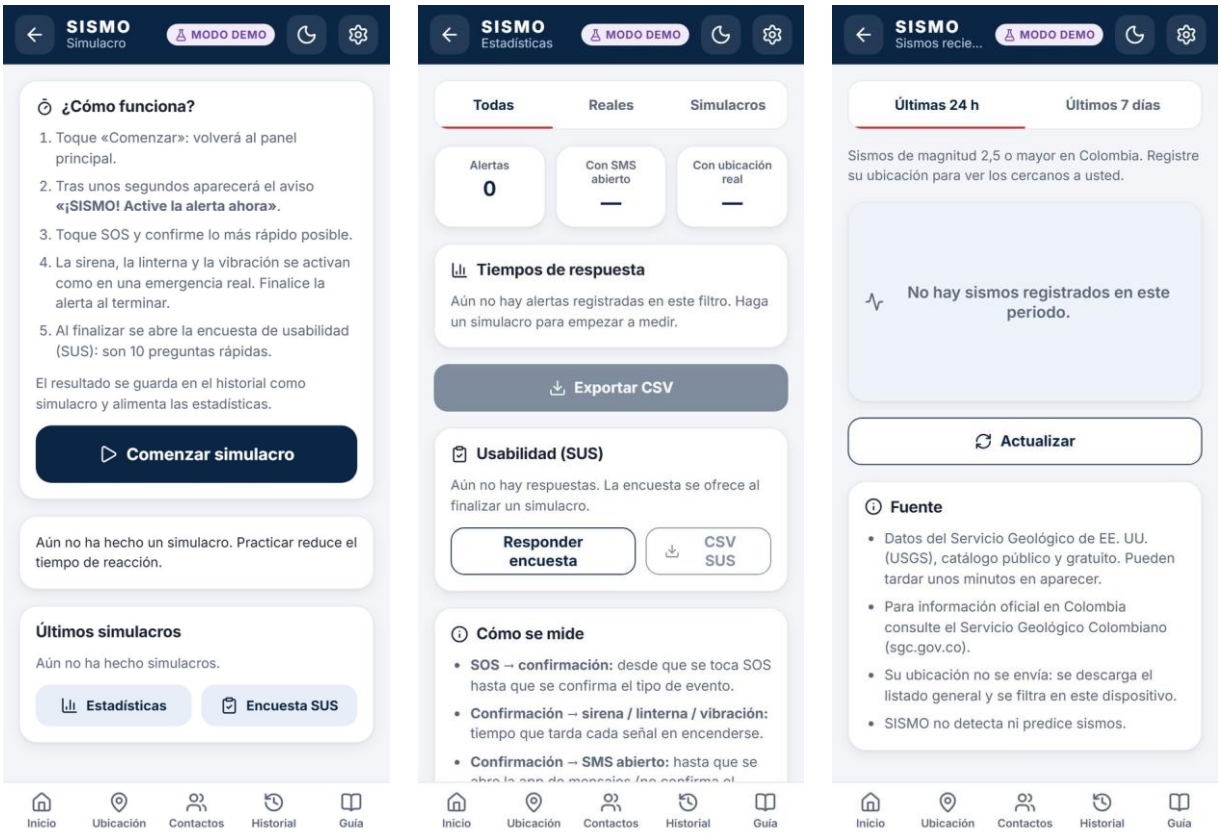
**(b) Mochila de emergencia.** Mochila de emergencia: elementos sugeridos con fecha de vencimiento, contadores de listos, vencidos y por vencer, y recordatorio de revisión periódica.

**(c) Ficha médica.** Ficha médica para rescatistas (tipo de sangre, alergias, medicamentos, condiciones y notas) con el interruptor para compartirla en la alerta y en el SMS.

Fuente: elaboración propia (captura del 02/11/2026).

**Figura 62**

*App (v2): Simulacro; estadísticas; sismos recientes*




(a) Simulacro (b) Estadísticas (c) Sismos recientes

**(a) Simulacro.** Simulacro: explica cómo funciona, mide el tiempo de reacción al activar el SOS y al final ofrece la encuesta de usabilidad SUS.

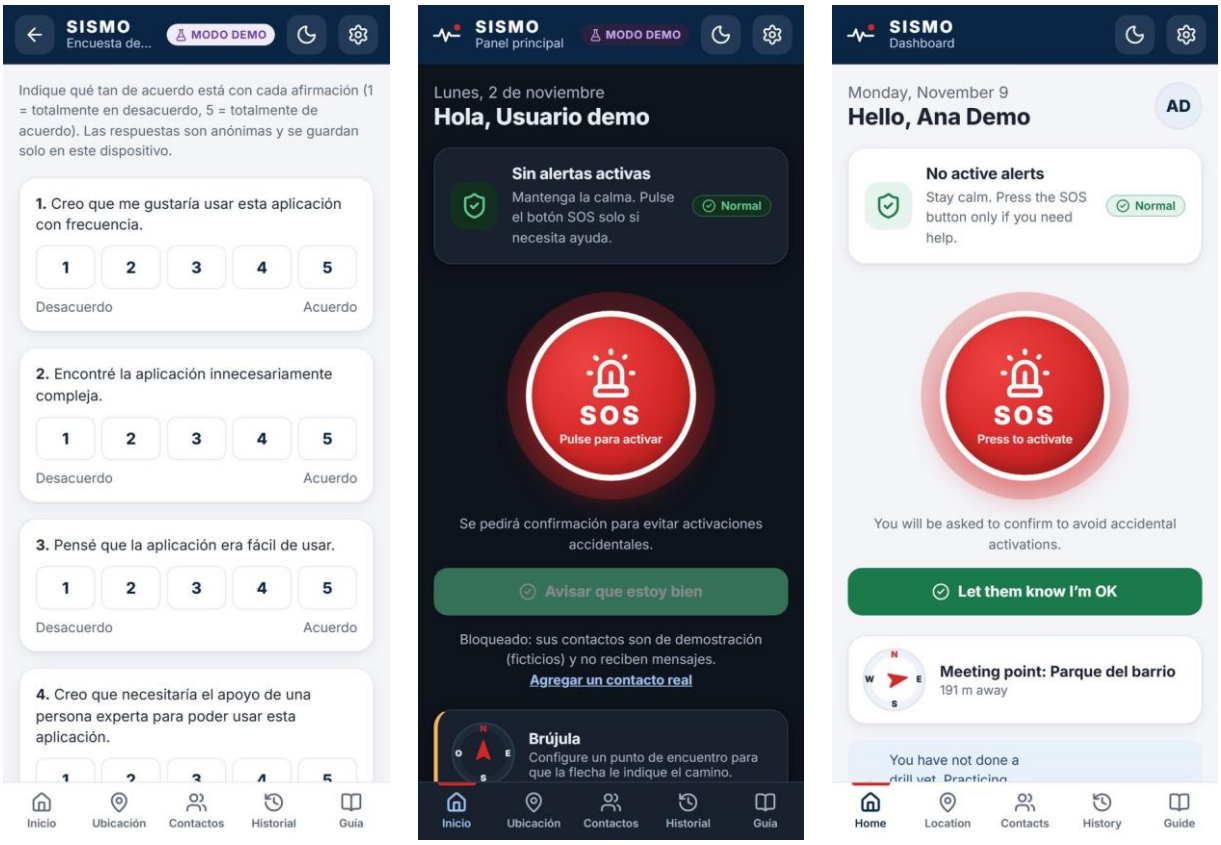
**(b) Estadísticas.** Estadísticas: tiempos de respuesta de las alertas y simulacros, puntaje SUS y exportación a CSV sin datos personales.

**(c) Sismos recientes.** Sismos recientes de las últimas 24 horas o 7 días, tomados del USGS, con botón para actualizar y la fuente de los datos.

Fuente: elaboración propia (captura del 02/11/2026).

**Figura 63**

*App (v2): Encuesta SUS; inicio tema oscuro; inicio inglés*




(a) Encuesta SUS (b) Inicio tema oscuro (c) Inicio inglés

**(a) Encuesta SUS.** Encuesta SUS de diez afirmaciones calificadas de 1 a 5; las respuestas son anónimas y se guardan en el dispositivo.

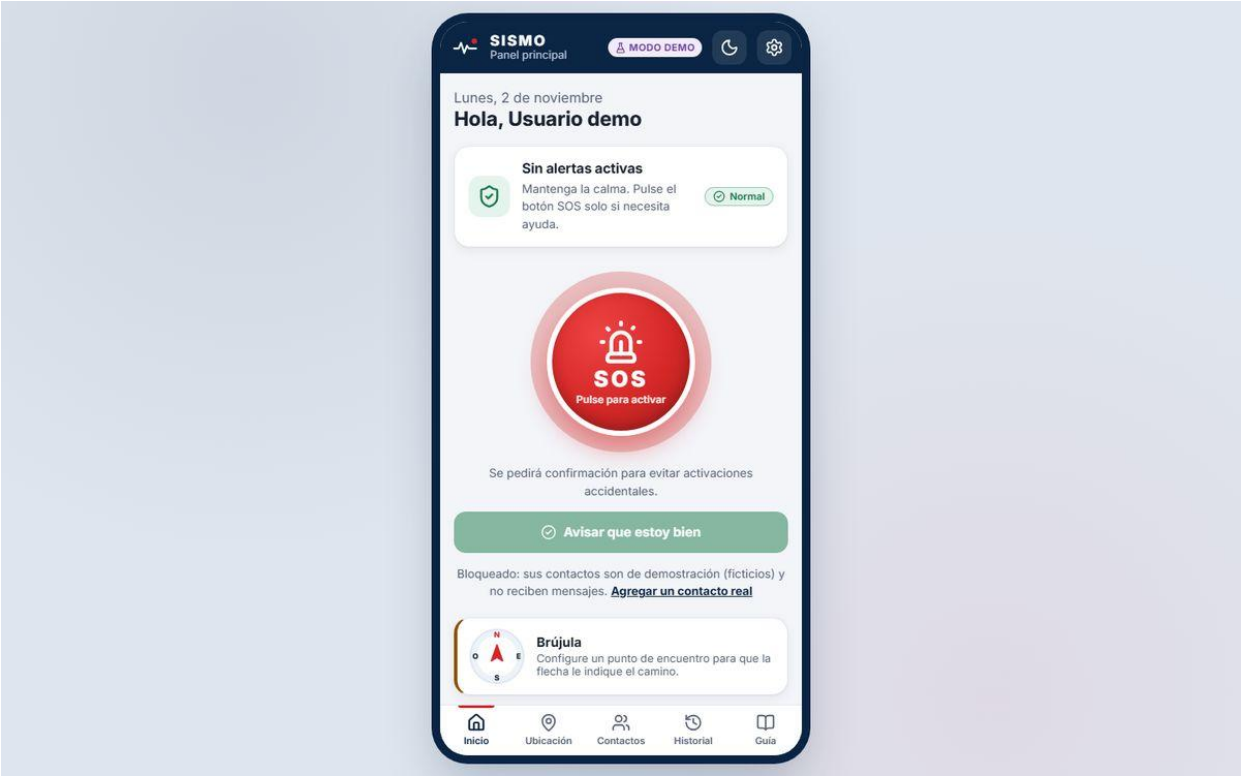
**(b) Inicio tema oscuro.** Inicio con el tema oscuro, que se elige en Configuración o con el botón del encabezado.

**(c) Inicio inglés.** Inicio con el idioma inglés: todas las pantallas y los mensajes cambian de idioma.

Fuente: elaboración propia (captura del 02/11/2026; captura retomada el 09/11/2026 porque la original del 02/11 había quedado en español).

## E.2 Versión 2: vista de escritorio (02/11/2026)

**Figura 64**
*App (v2): vista escritorio*



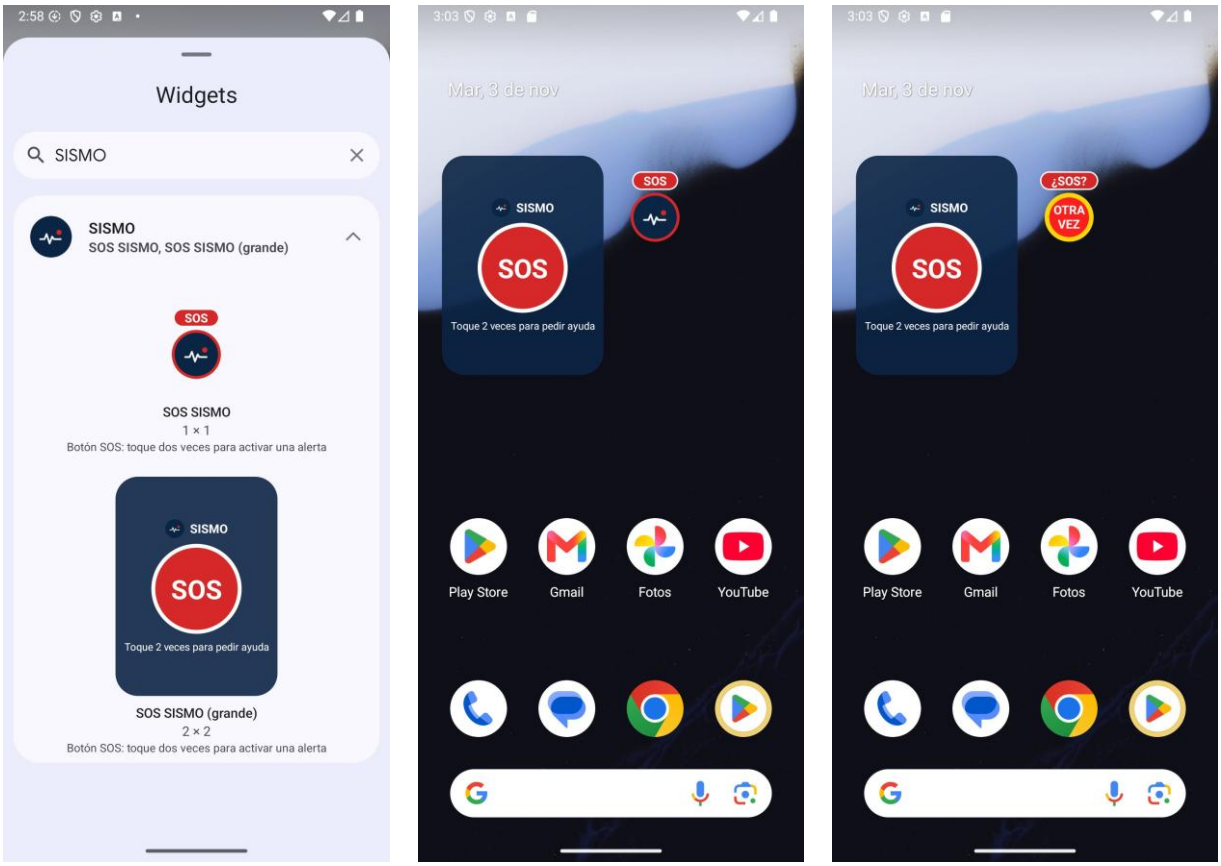

**Descripción.** Vista de escritorio: en una pantalla grande la app se muestra centrada con el ancho de un teléfono, con las mismas funciones.

Fuente: elaboración propia (captura del 02/11/2026).

## E.3 Versión 2: Android, widget y notificación (02/11/2026)

**Figura 65**

*Android (v2): Selector de widgets; widgets en pantalla de inicio; widget 1x1 primer toque*




(a) Selector de widgets (b) Widgets en pantalla de inicio (c) Widget 1x1 primer toque

**(a) Selector de widgets.** Selector de widgets de Android con los dos widgets de SISMO: el botón SOS de 1 × 1 y el de 2 × 2.

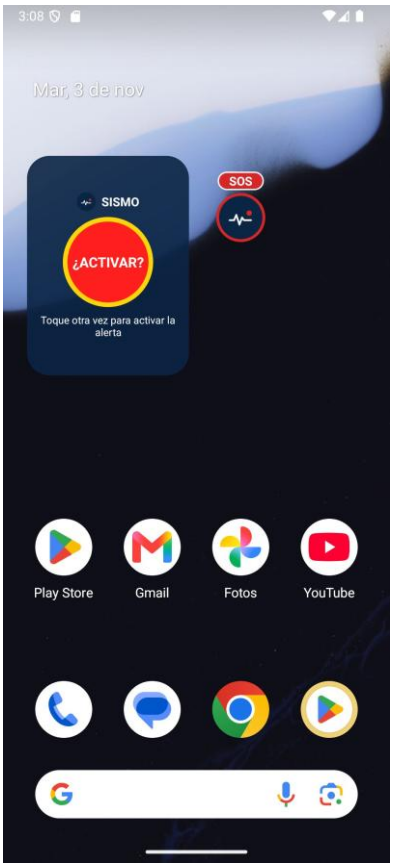
**(b) Widgets en pantalla de inicio.** Los dos widgets en la pantalla de inicio del teléfono, listos para activar una alerta sin abrir la app.

**(c) Widget 1x1 primer toque.** Primer toque en el widget de 1 × 1: cambia a «¿Otra vez?» y solo activa la alerta con el segundo toque, para evitar activaciones accidentales.

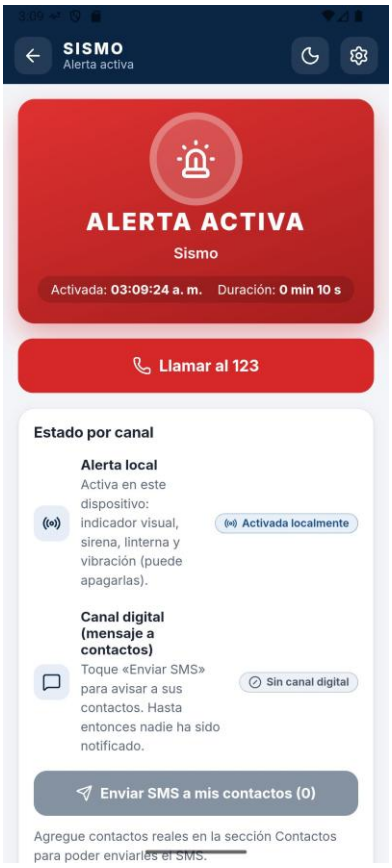
Fuente: elaboración propia (captura del 02/11/2026).

**Figura 66**

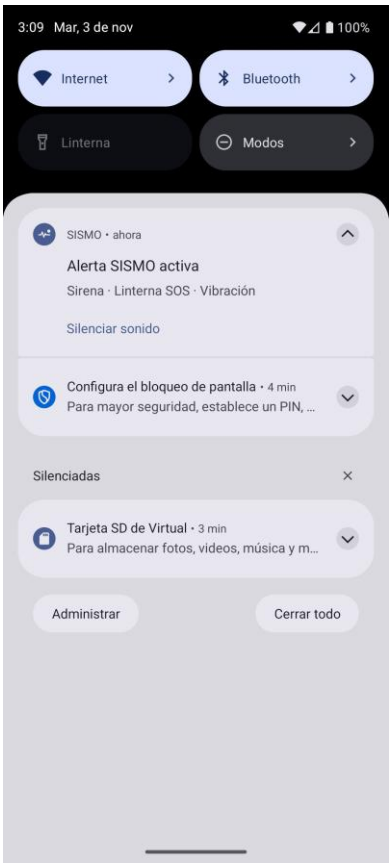
*Android (v2): Widget 2x2 primer toque; alerta activada desde el widget; notificación alerta activa*




(a) Widget 2x2 primer toque




(b) Alerta activada desde el widget




(c) Notificación alerta activa

**(a) Widget 2x2 primer toque.** Primer toque en el widget de 2 × 2: muestra «¿Activar?» y pide tocar otra vez para activar la alerta.

**(b) Alerta activada desde el widget.** Alerta activada desde el widget: la app se abre directamente en la pantalla de alerta activa.

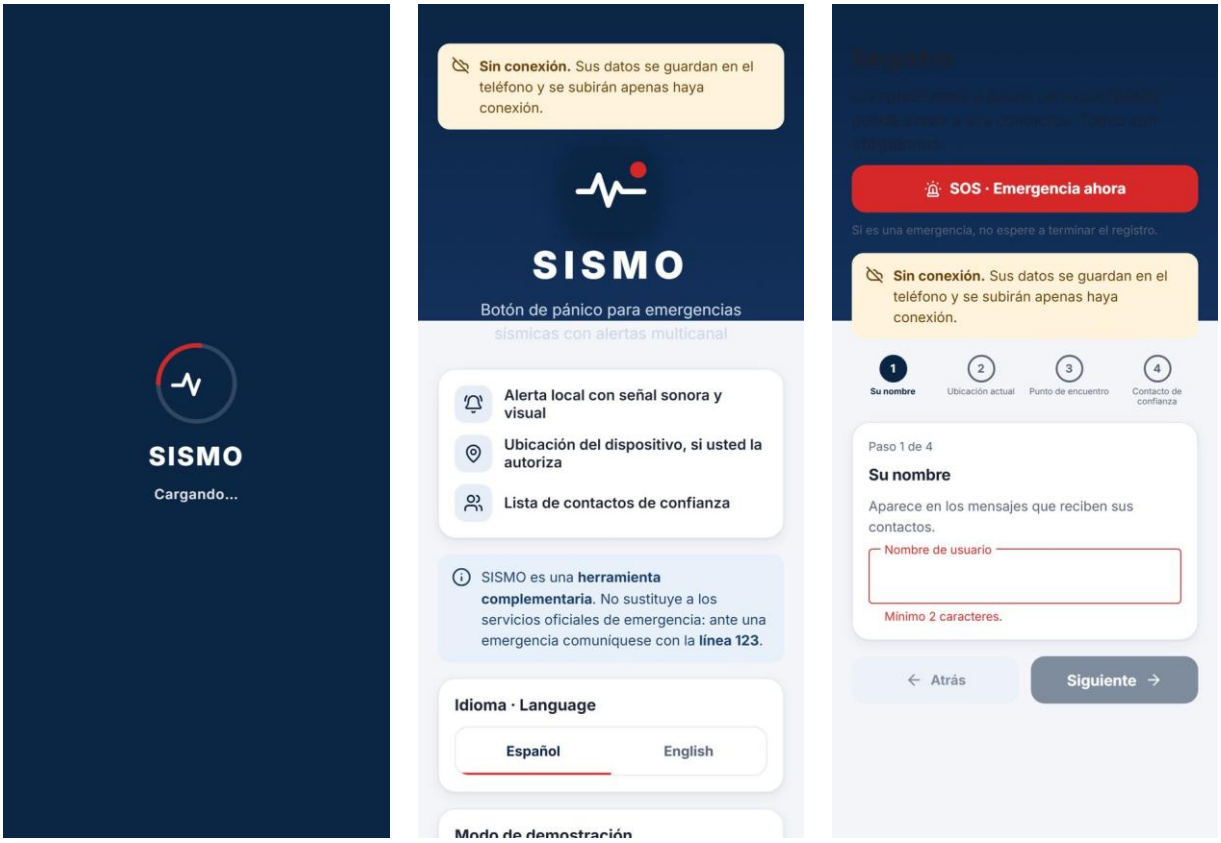
**(c) Notificación alerta activa.** Notificación permanente «Alerta SISMO activa» con sirena, linterna SOS y vibración, y la opción de silenciar el sonido.

Fuente: elaboración propia (captura del 02/11/2026).

## E.4 Versión 3: sincronización, recuperación y ayuda del centro de monitoreo (04/11/2026)

**Figura 67**

*App (v3): Pantalla de carga; bienvenida sin conexión; registro sin conexión*




(a) Pantalla de carga (b) Bienvenida sin conexión (c) Registro sin conexión

**(a) Pantalla de carga.** Pantalla de carga al abrir la app (entre 0,6 y 1,5 segundos) antes de ir a la Bienvenida o a Inicio.

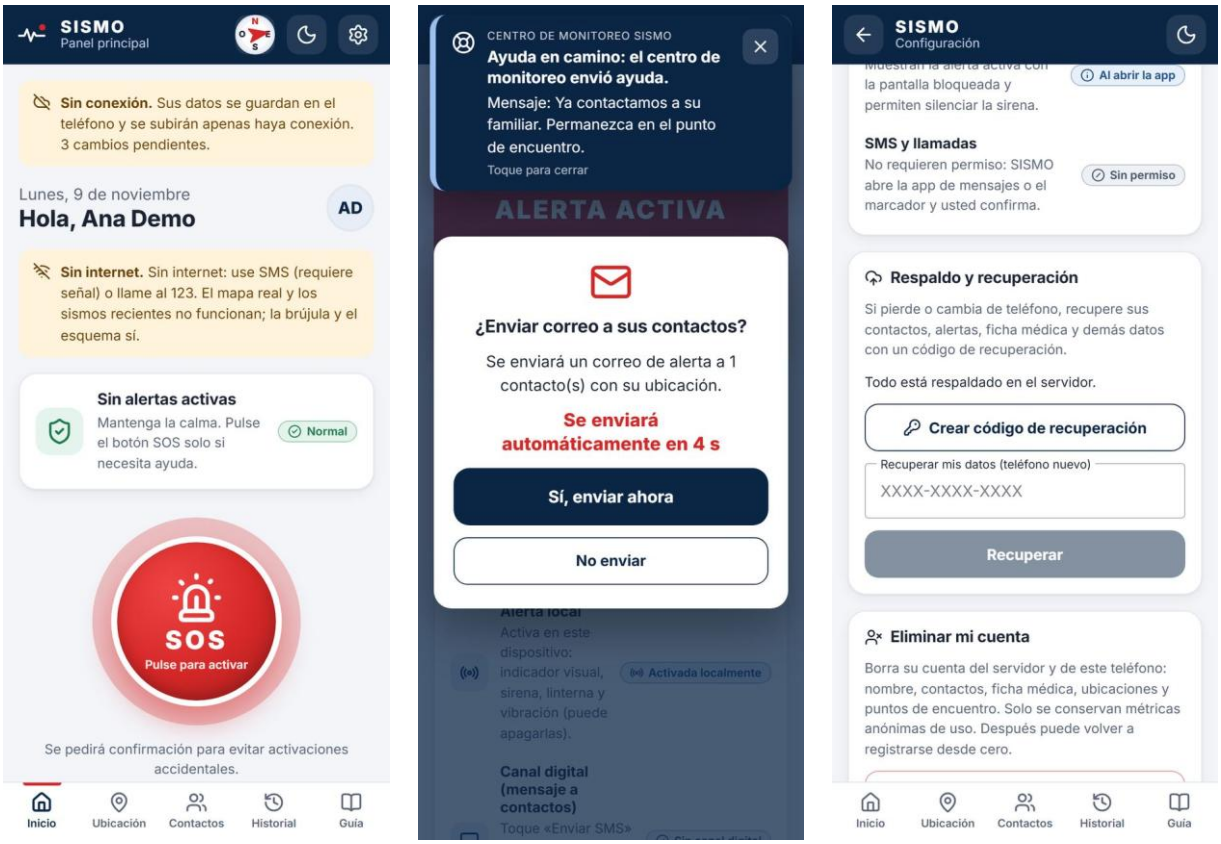
**(b) Bienvenida sin conexión.** Bienvenida sin conexión: el aviso amarillo indica que los datos se guardan en el teléfono y se subirán cuando haya conexión.

**(c) Registro sin conexión.** Registro sin conexión: se puede completar igual; los datos esperan en la cola y el botón SOS sigue disponible.

Fuente: elaboración propia (captura del 04/11/2026).

**Figura 68**

*App (v3): Inicio sin conexión cambios pendientes; alerta ayuda del centro de monitoreo; configuración respaldo y recuperación*




(a) Inicio sin conexión cambios pendientes

(b) Alerta ayuda del centro de monitoreo

(c) Configuración respaldo y recuperación

**(a) Inicio sin conexión cambios pendientes.** Inicio sin conexión: avisa que los datos se guardan en el teléfono e indica cuántos cambios esperan subirse al servidor («3 cambios pendientes»).

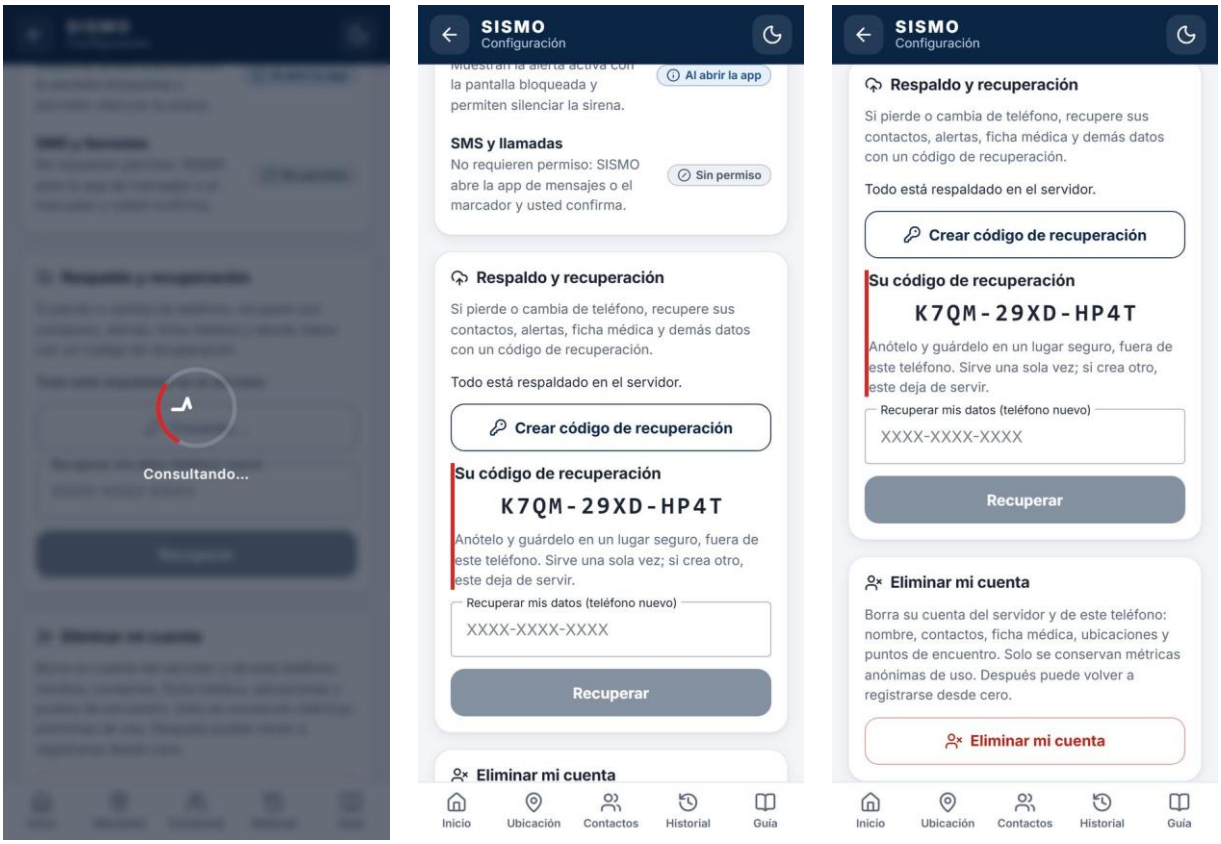
**(b) Alerta ayuda del centro de monitoreo.** Aviso del centro de monitoreo sobre la alerta: «Ayuda en camino» con el mensaje del operador, mientras sigue la pregunta del correo a los contactos.

**(c) Configuración respaldo y recuperación.** Configuración, sección «Respaldo y recuperación»: crear un código para recuperar los datos en un teléfono nuevo o escribir un código recibido.

Fuente: elaboración propia (captura del 04/11/2026; captura retomada el 09/11/2026 porque la original del 04/11 quedó en blanco).

**Figura 69**

*App (v3): Loader de solicitud; código de recuperación; configuración eliminar cuenta*




(a) Loader de solicitud (b) Código de recuperación (c) Configuración eliminar cuenta

**(a) Loader de solicitud.** Loader «Consultando…» que aparece mientras la app espera la respuesta del servidor en una acción iniciada por la persona.

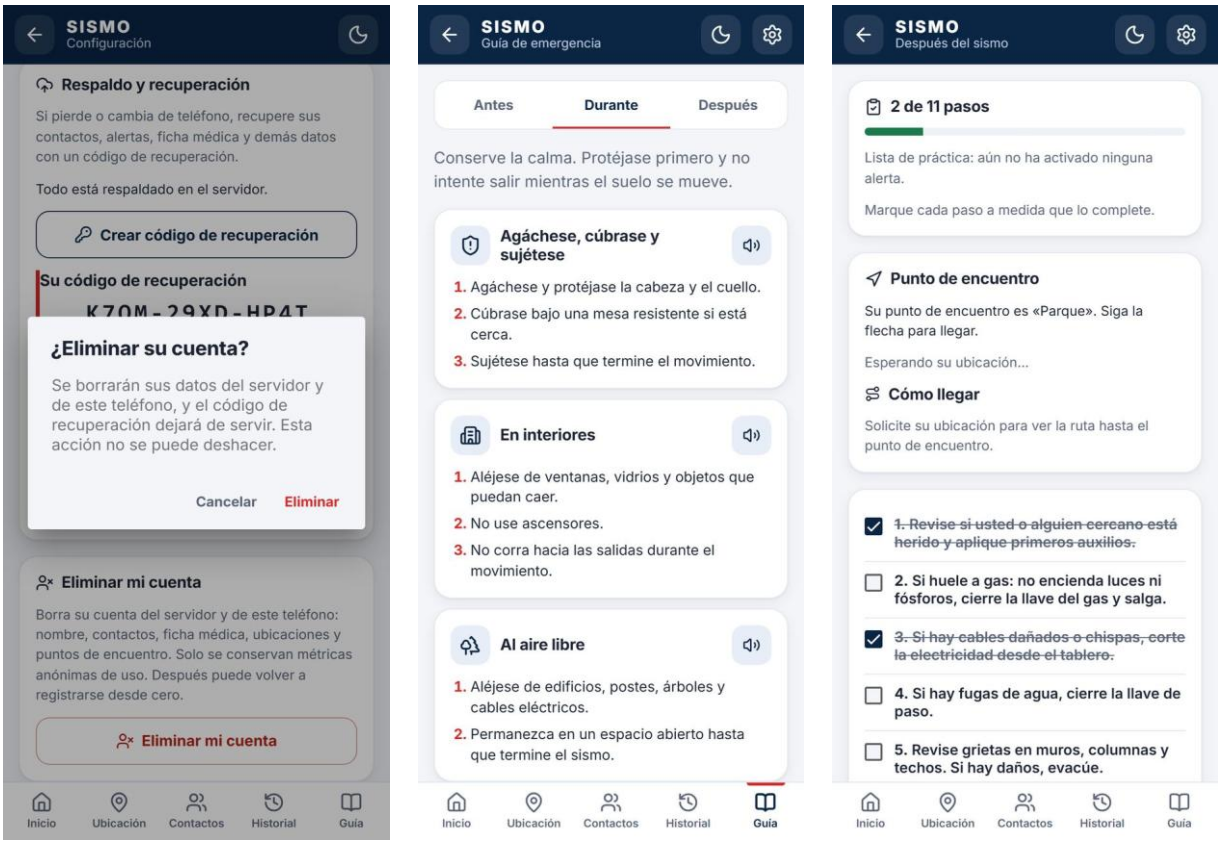
**(b) Código de recuperación.** Código de recuperación generado (formato XXXX-XXXX-XXXX), con la indicación de guardarlo fuera del teléfono porque es de un solo uso.

**(c) Configuración eliminar cuenta.** Sección «Eliminar mi cuenta»: explica qué datos se borran del servidor y del teléfono y que solo quedan métricas anónimas.

Fuente: elaboración propia (captura del 04/11/2026).

**Figura 70**

*App (v3): Confirmar eliminar cuenta; guía desde catalogo; después del sismo por clave*




(a) Confirmar eliminar cuenta (b) Guía desde catalogo (c) Después del sismo por clave

**(a) Confirmar eliminar cuenta.** Confirmación antes de eliminar la cuenta: la acción no se puede deshacer y el código de recuperación deja de servir.

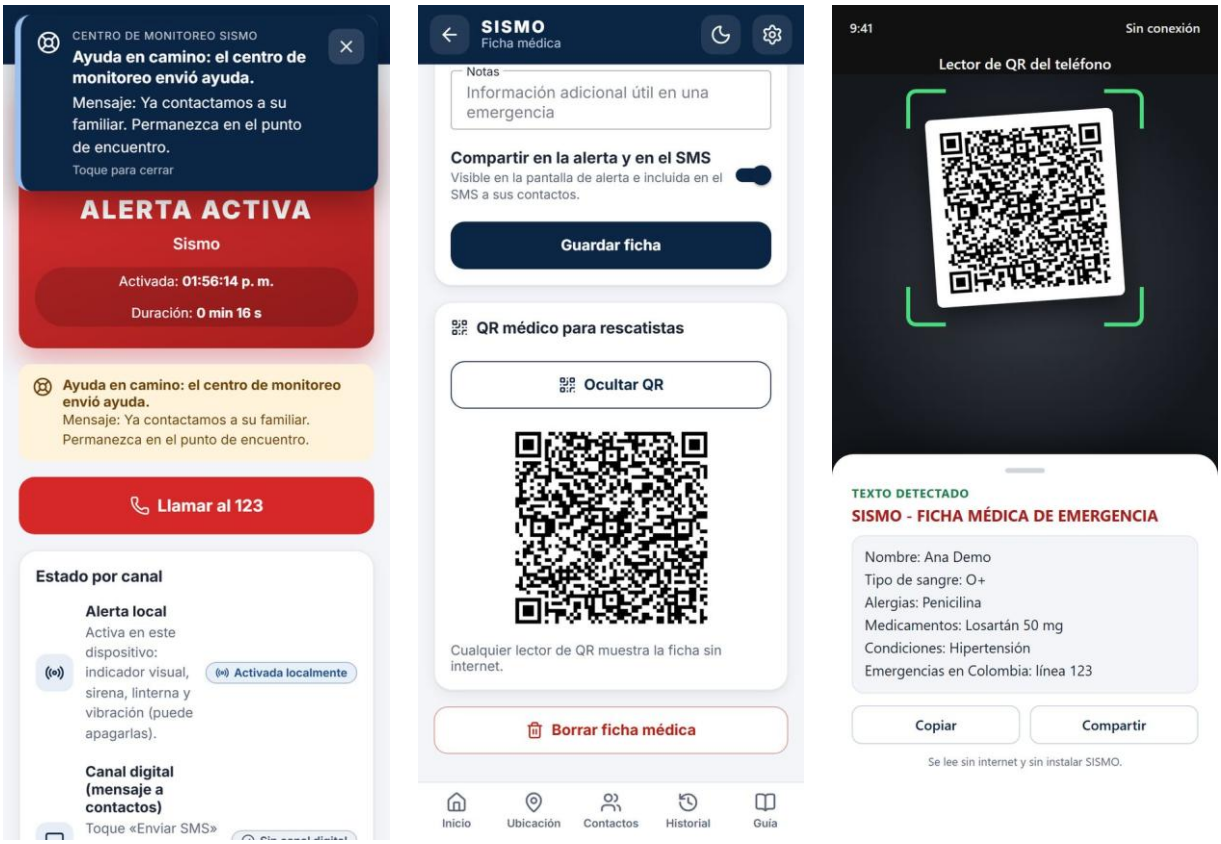
**(b) Guía desde catalogo.** Guía de emergencia cargada desde el catálogo que el operador administra en el panel.

**(c) Después del sismo por clave.** Lista «Después del sismo» con el avance guardado por clave de paso (2 de 11), los pasos tachados y el punto de encuentro.

Fuente: elaboración propia (captura del 04/11/2026).

**Figura 71**

*App (v3): Alerta aviso del centro; ficha médica QR; qR escaneado por un rescatista*




(a) Alerta aviso del centro (b) Ficha médica QR (c) QR escaneado por un rescatista

**(a) Alerta aviso del centro.** Aviso fijo del centro de monitoreo en la pantalla de alerta: la persona ve el estado y el mensaje del operador (consulta cada 15 s).

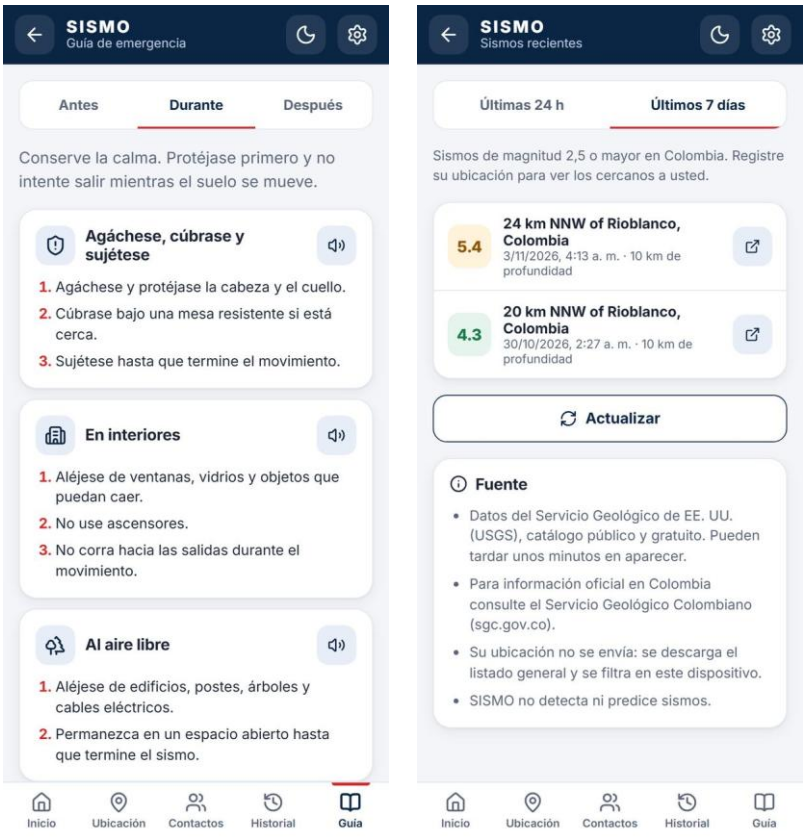
**(b) Ficha médica QR.** Ficha médica con el QR médico generado; cualquier lector de QR muestra la ficha sin internet y sin instalar la app.

**(c) QR escaneado por un rescatista.** QR escaneado por un rescatista con el lector del teléfono, sin conexión: aparece la ficha médica en texto plano.

Fuente: elaboración propia (captura del 04/11/2026).

**Figura 72**

*App (v3): Guía de emergencia; sismos recientes*




(a) Guía de emergencia (b) Sismos recientes

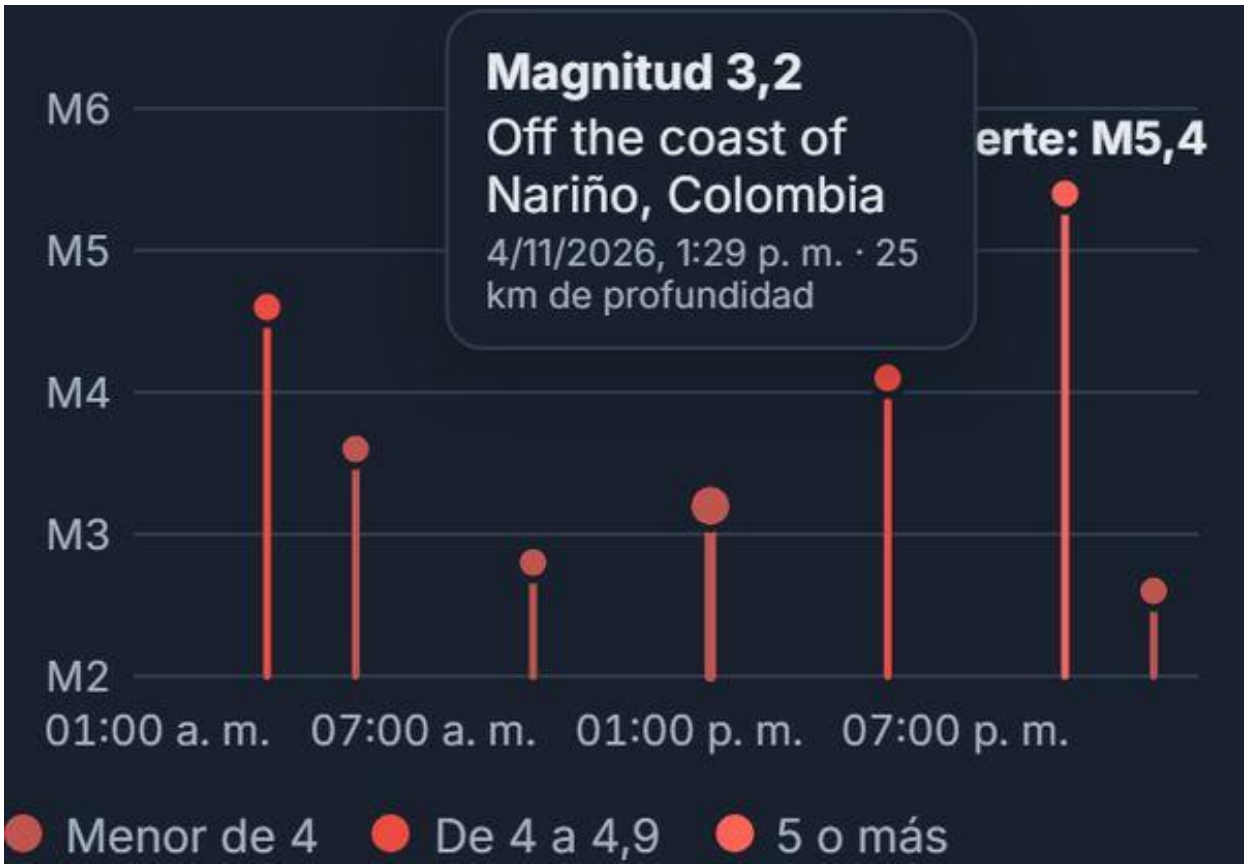
**(a) Guía de emergencia.** Guía de emergencia con lectura en voz alta para cada situación (agáchese, cúbrase y sujétese; en interiores; al aire libre).

**(b) Sismos recientes.** Sismos recientes de los últimos 7 días con magnitud, lugar, hora y profundidad, y enlace al detalle de cada sismo.

Fuente: elaboración propia (captura del 04/11/2026).

## E.5 Versión 3.2: sismos con gráfica de picos y mapa con ondas (05/11/2026)

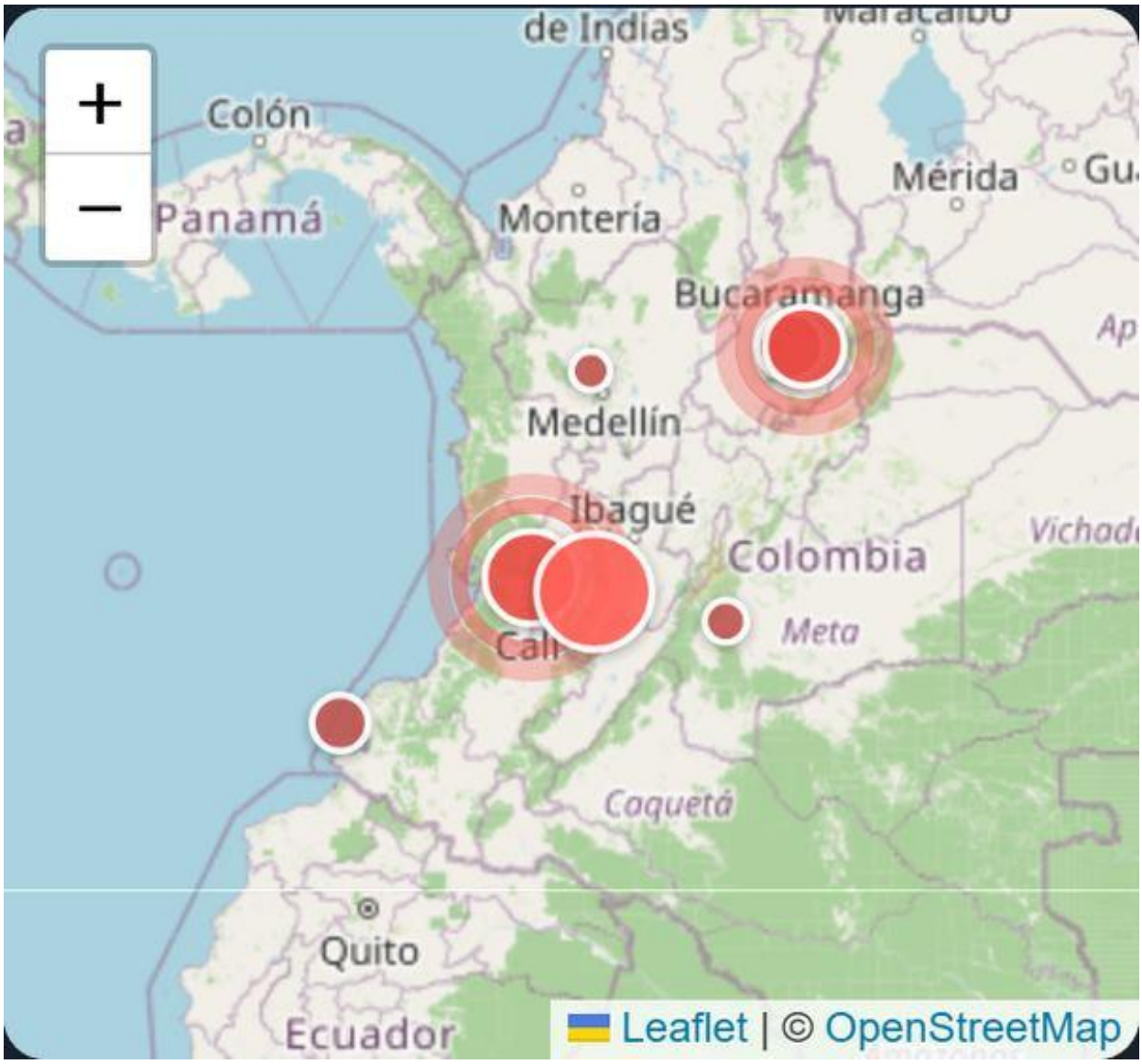
**Figura 73**
*App (v3.2): sismos gráfica de picos*




**Descripción.** Gráfica de picos de los sismos recientes: cada pico es un sismo y su altura indica la magnitud; al pasar el cursor se ve el detalle.

Fuente: elaboración propia (captura del 05/11/2026).

**Figura 74**
*App (v3.2): sismos mapa con ondas*




**Descripción.** Mapa de sismos con ondas: entre más intenso el sismo, más grande el punto y más ondas lo rodean.

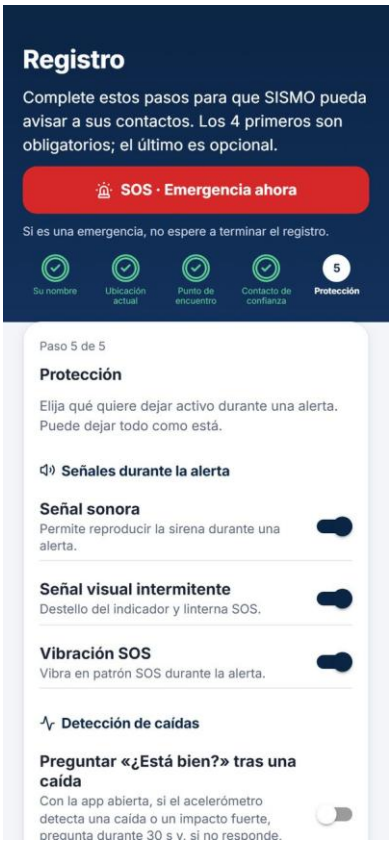
Fuente: elaboración propia (captura del 05/11/2026).

## E.6 Versión 3.2: prioridad, ayuda urgente y protección (05/11/2026)

**Figura 75**
*App (v3.2): Aviso de alerta activa en otra pantalla; estoy a salvo fijo abajo; registro, paso 5: protección*

(a) Aviso de alerta activa en otra pantalla

(b) Estoy a salvo fijo abajo




(c) Registro, paso 5: protección

**(a) Aviso de alerta activa en otra pantalla.** Aviso de alerta activa en otra pantalla: una franja roja recuerda que hay una alerta en curso y permite volver a ella con un toque.

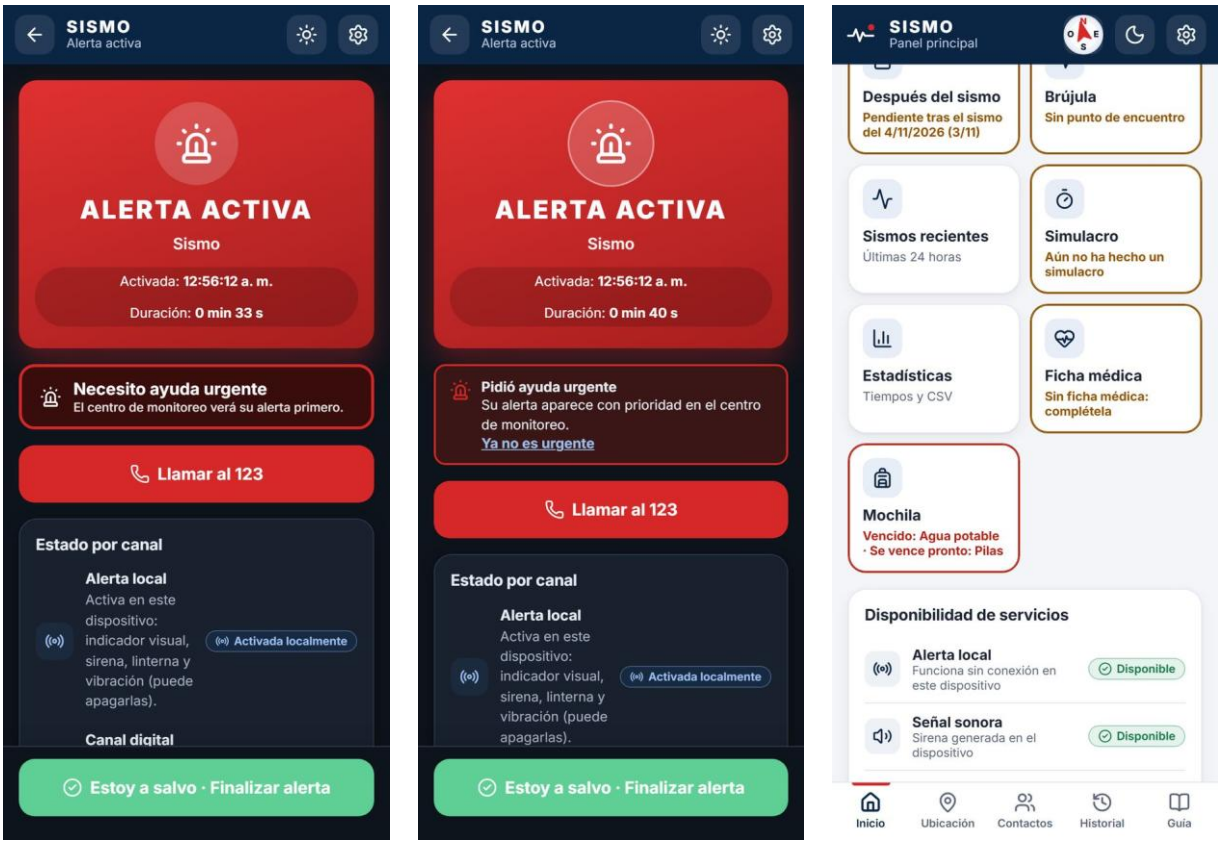
**(b) Estoy a salvo fijo abajo.** Botón «Estoy a salvo · Finalizar alerta» fijo en la parte inferior de la pantalla de alerta, siempre a mano.

**(c) Registro, paso 5: protección.** Paso 5 del registro, «Protección» (opcional): elegir las señales de la alerta, la detección de caídas y el ahorro de batería.

Fuente: elaboración propia (captura del 05/11/2026).

**Figura 76**

*App (v3.2): Alerta botón necesito ayuda urgente; alerta ayuda urgente solicitada; inicio tarjetas por atender*




(a) Alerta botón necesito ayuda urgente

(b) Alerta ayuda urgente solicitada

(c) Inicio tarjetas por atender

**(a) Alerta botón necesito ayuda urgente.** Botón «Necesito ayuda urgente» en la alerta activa: marca la alerta como prioritaria para el centro de monitoreo.

**(b) Alerta ayuda urgente solicitada.** Ayuda urgente solicitada: la app confirma que la alerta aparece con prioridad en el centro de monitoreo y permite indicar que ya no es urgente.

**(c) Inicio tarjetas por atender.** Inicio con tarjetas por atender en amarillo (Después del sismo, brújula, simulacro, ficha médica) y la mochila en rojo con lo vencido.

Fuente: elaboración propia (captura del 05/11/2026).

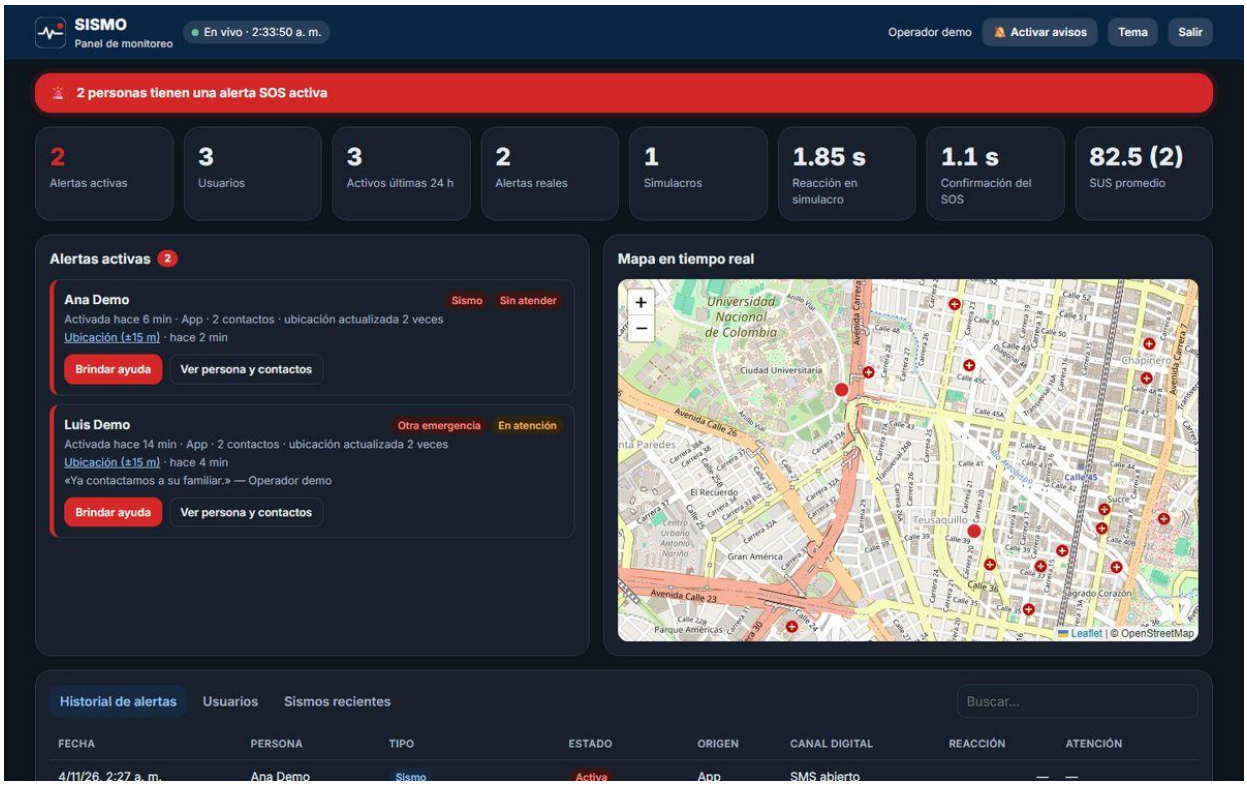
# ANEXO F. EVIDENCIAS VISUALES DEL PANEL DE MONITOREO Y DE LA PÁGINA INFORMATIVA

Capturas del panel del centro de monitoreo y de la página informativa pública, con datos ficticios. Las capturas de la página informativa en escritorio muestran solo el fragmento superior de la página.

## F.1 Versión 3: panel de monitoreo (04/11/2026)

**Figura 77**

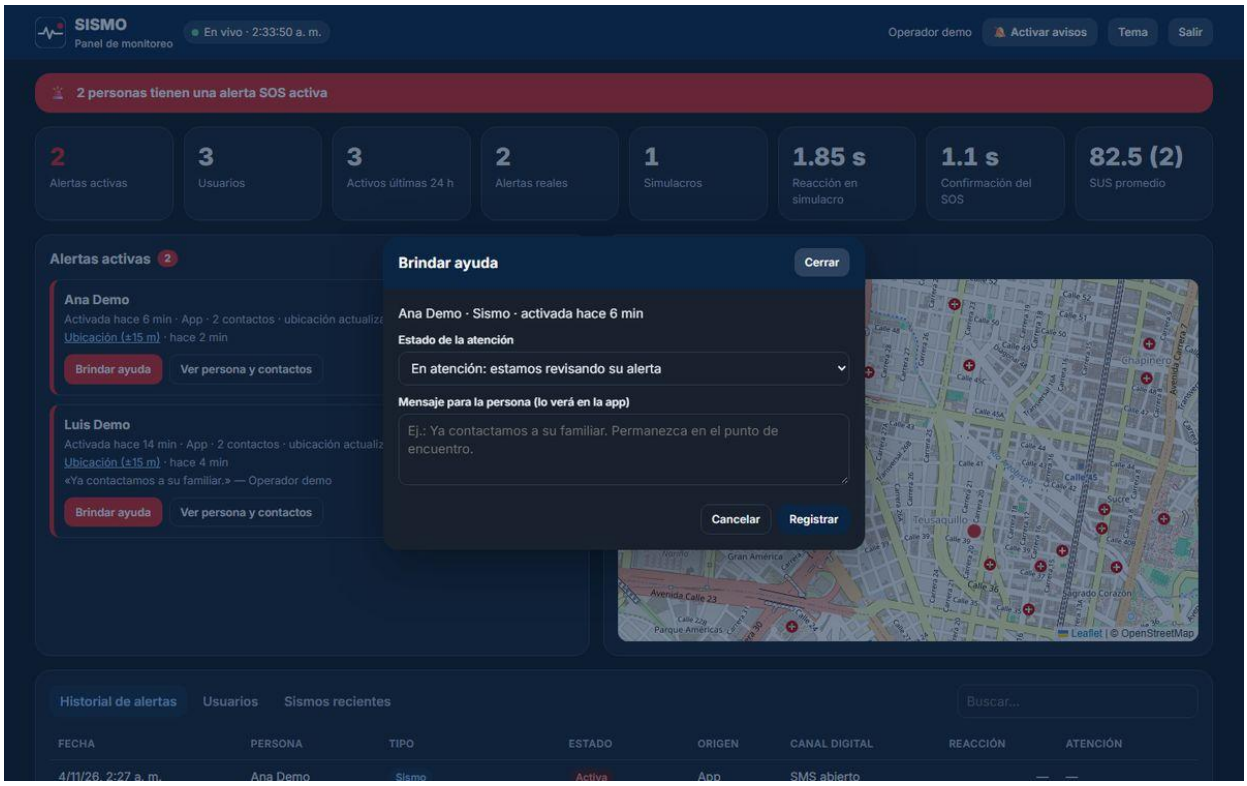
*Panel (v3): alertas activas y mapa*




**Descripción.** Panel del centro de monitoreo: indicadores, alertas activas con la persona, el tipo y el estado de atención, y el mapa en tiempo real con la ubicación de cada alerta.

Fuente: elaboración propia (captura del 04/11/2026).

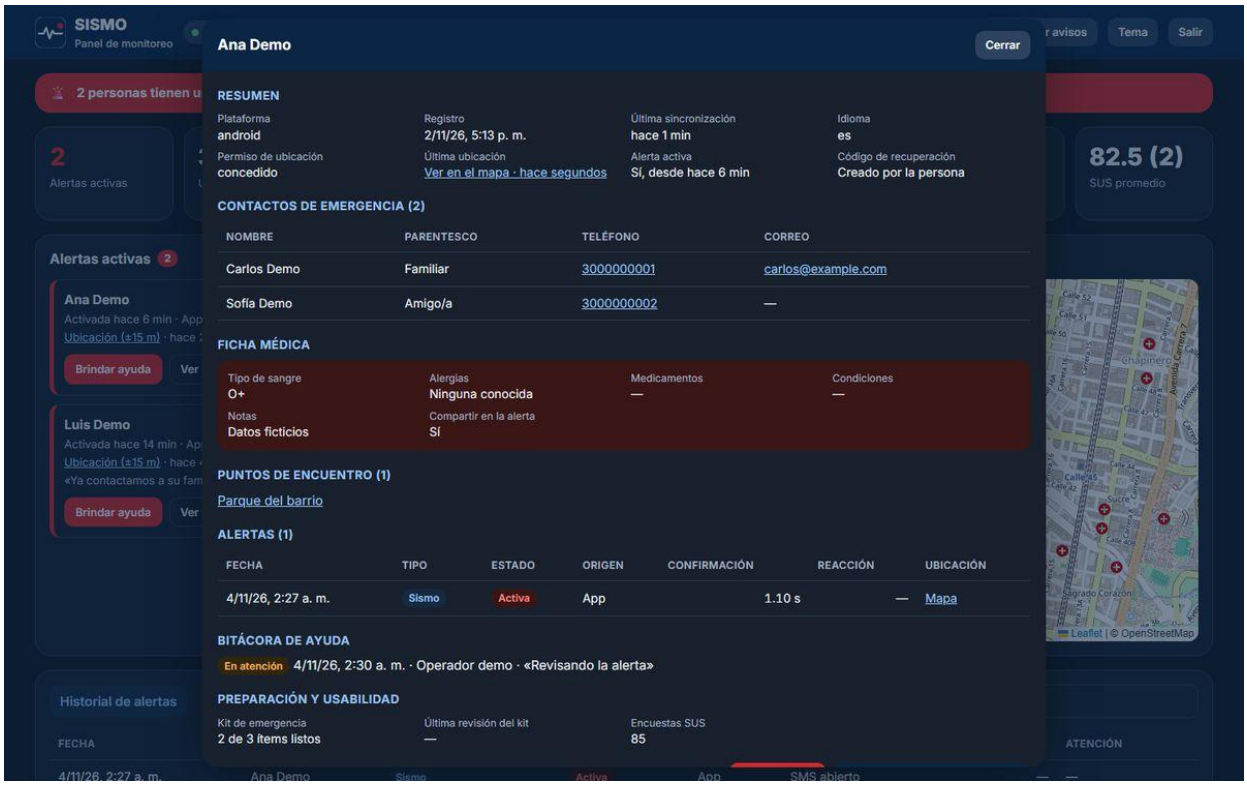
**Figura 78**
*Panel (v3): brindar ayuda*




**Descripción.** Ventana «Brindar ayuda»: el operador elige el estado de la atención (en atención, ayuda enviada o resuelta) y escribe el mensaje que verá la persona.

Fuente: elaboración propia (captura del 04/11/2026).

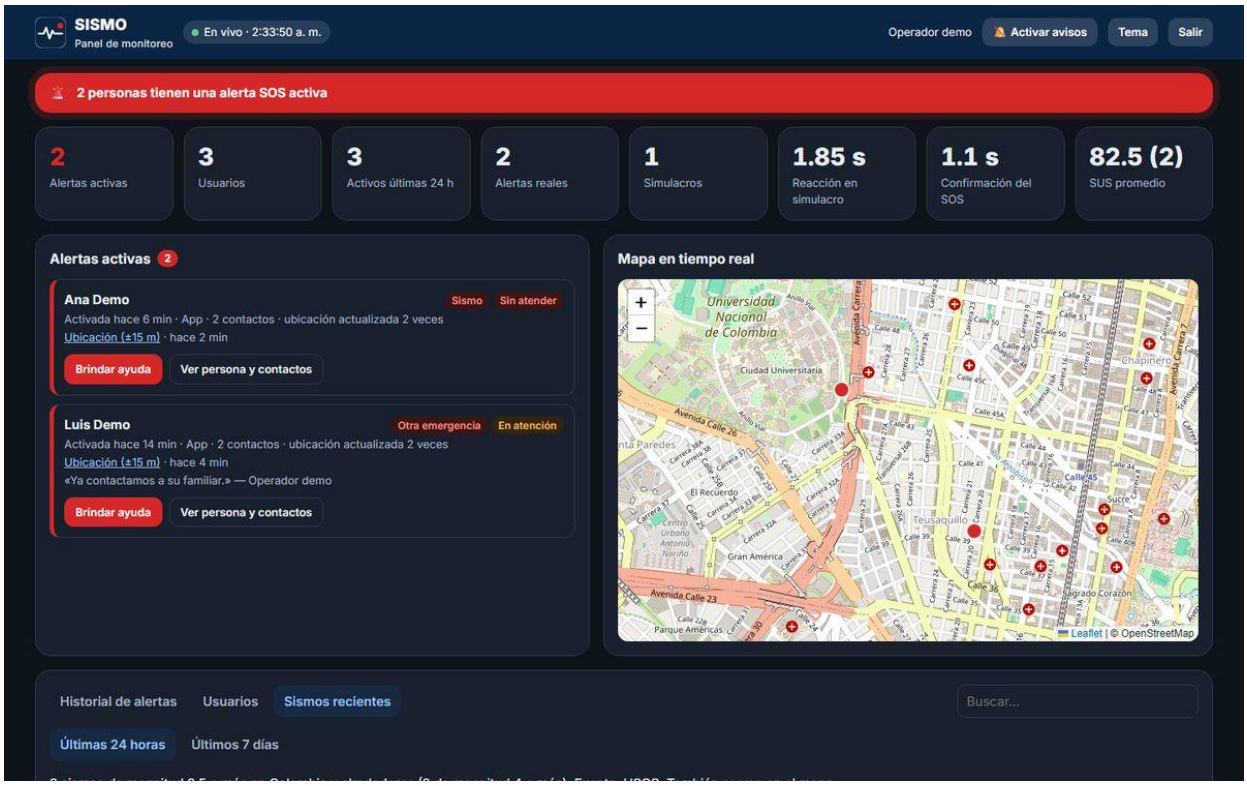
**Figura 79**
*Panel (v3): detalle de la persona*




**Descripción.** Detalle de la persona: datos del dispositivo, contactos, ficha médica descifrada, puntos de encuentro, alertas y bitácora de ayuda; cada consulta queda en el log.

Fuente: elaboración propia (captura del 04/11/2026).

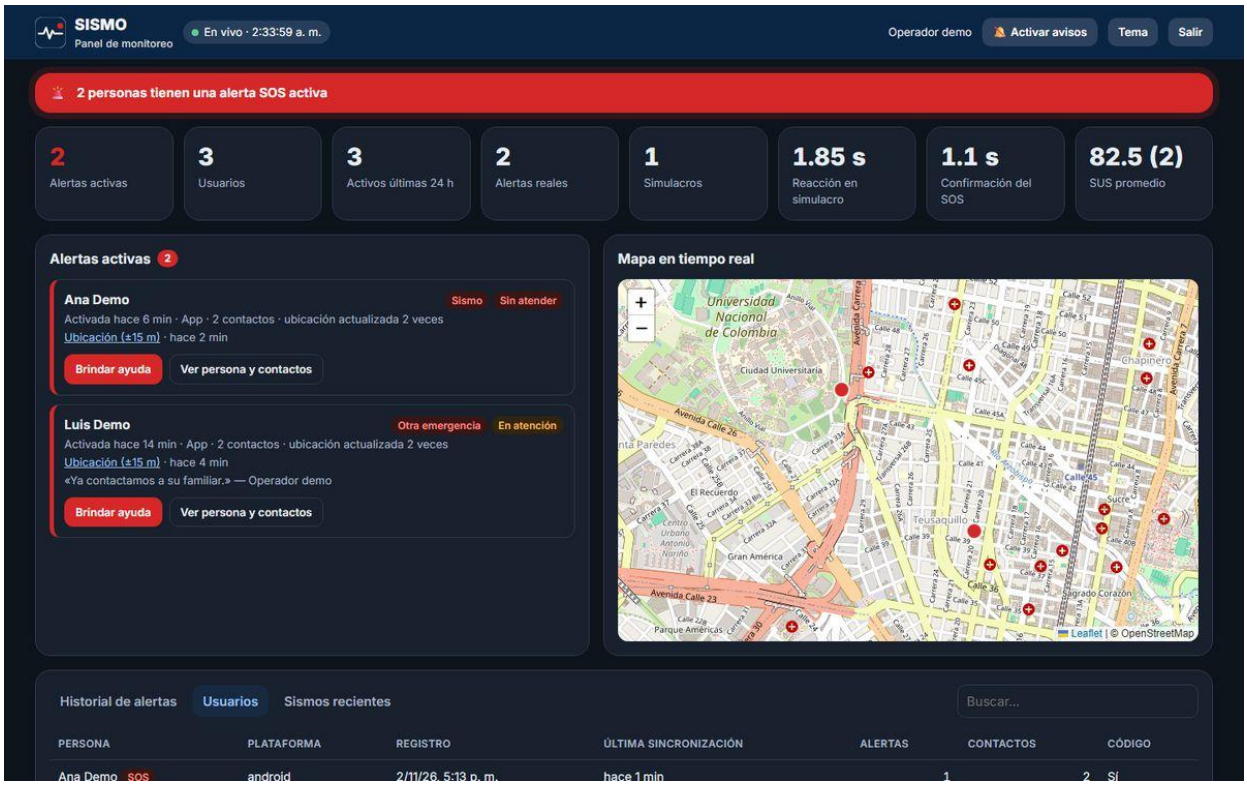
**Figura 80**
*Panel (v3): sismos recientes*




**Descripción.** Pestaña de sismos recientes (24 horas o 7 días) dentro del panel.

Fuente: elaboración propia (captura del 04/11/2026).

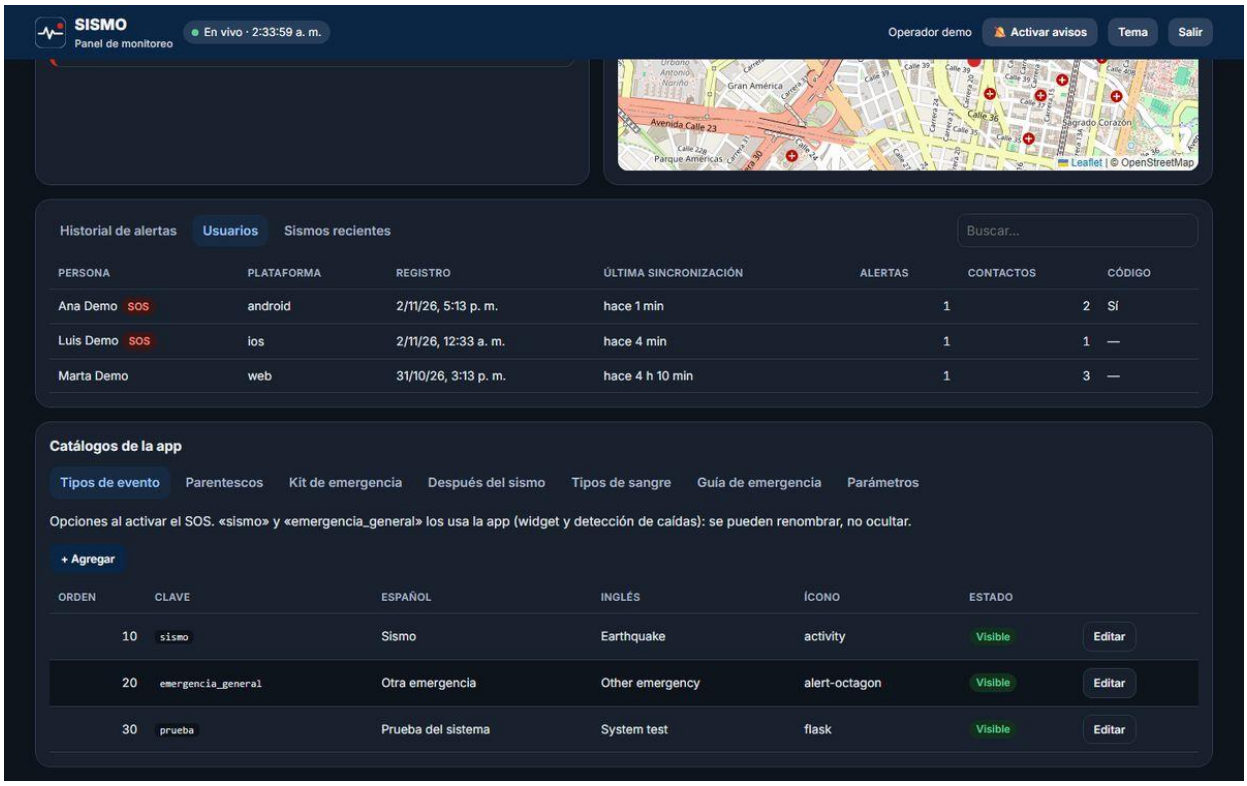
**Figura 81**
*Panel (v3): usuarios*




**Descripción.** Lista de usuarios con plataforma, fecha de registro, última sincronización, número de alertas y contactos, y buscador.

Fuente: elaboración propia (captura del 04/11/2026).

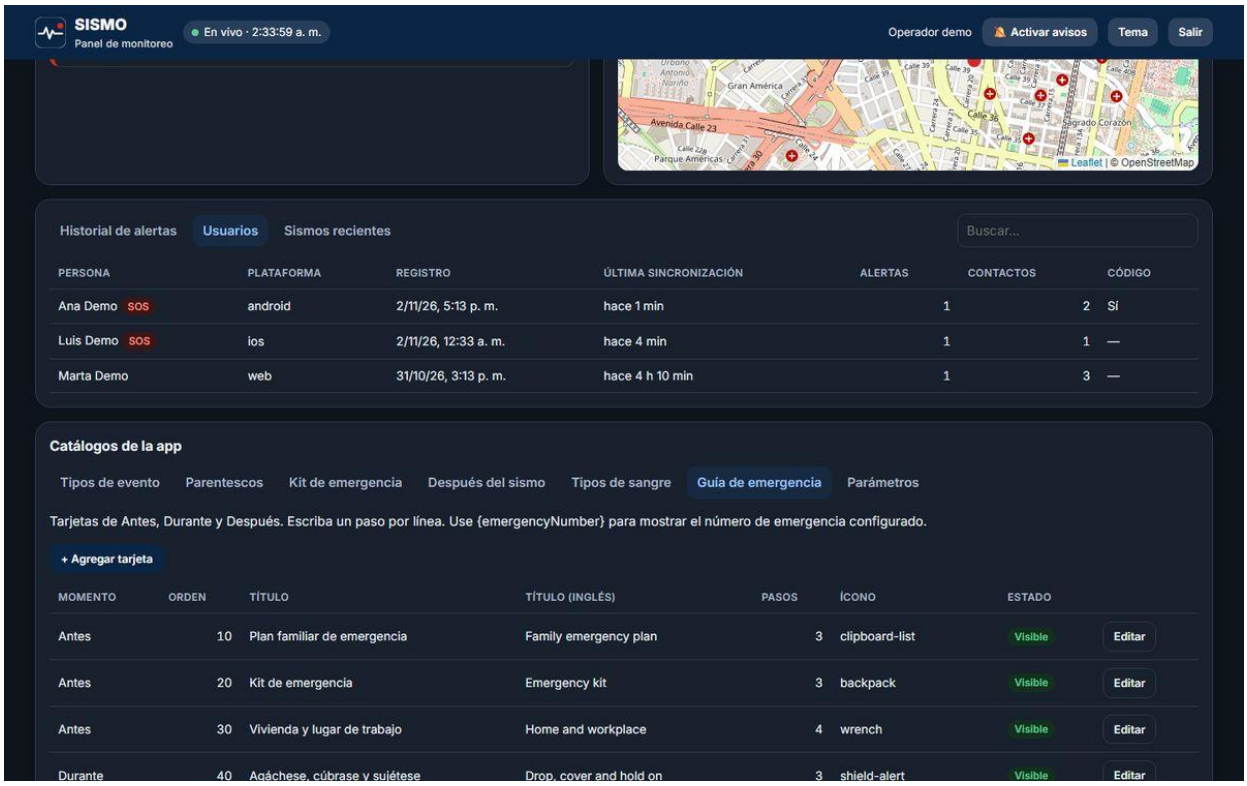
**Figura 82**
*Panel (v3): catálogos tipos de evento*




**Descripción.** Catálogo de tipos de evento: el operador agrega o edita los tipos que la app ofrece al activar el SOS (sismo y otra emergencia no se pueden ocultar).

Fuente: elaboración propia (captura del 04/11/2026).

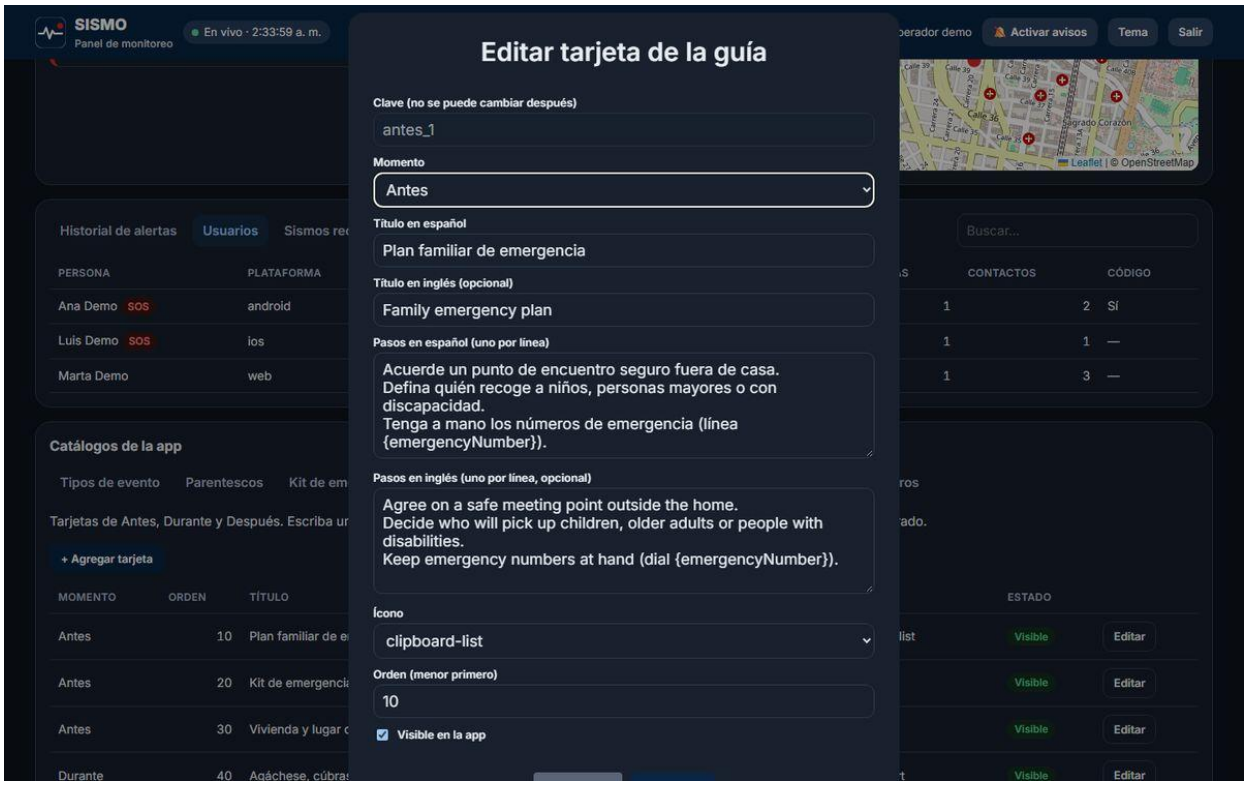
**Figura 83**
*Panel (v3): catálogos guía*




**Descripción.** Catálogo de la guía de emergencia: tarjetas por momento (antes, durante y después) en español e inglés.

Fuente: elaboración propia (captura del 04/11/2026).

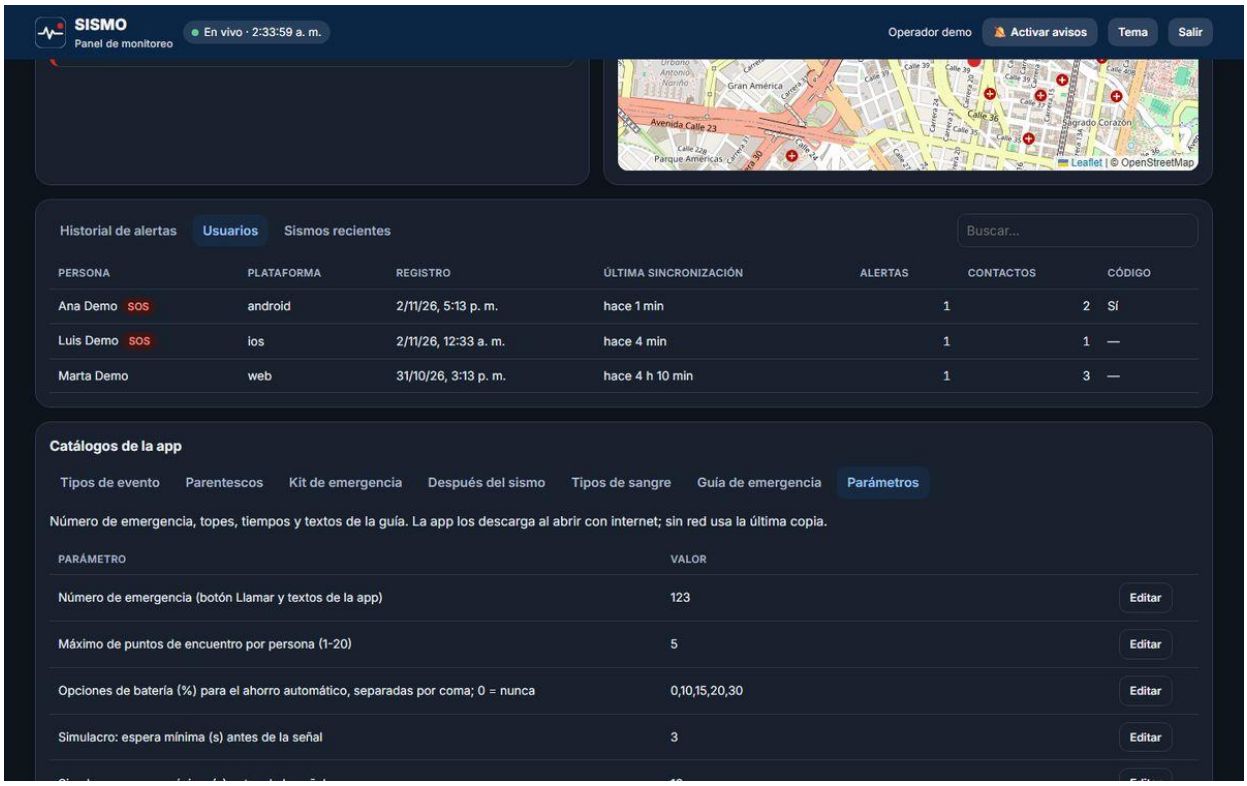
**Figura 84**
*Panel (v3): editar tarjeta de la guía*




**Descripción.** Edición de una tarjeta de la guía: momento, título y pasos en ambos idiomas, ícono, orden y visibilidad en la app.

Fuente: elaboración propia (captura del 04/11/2026).

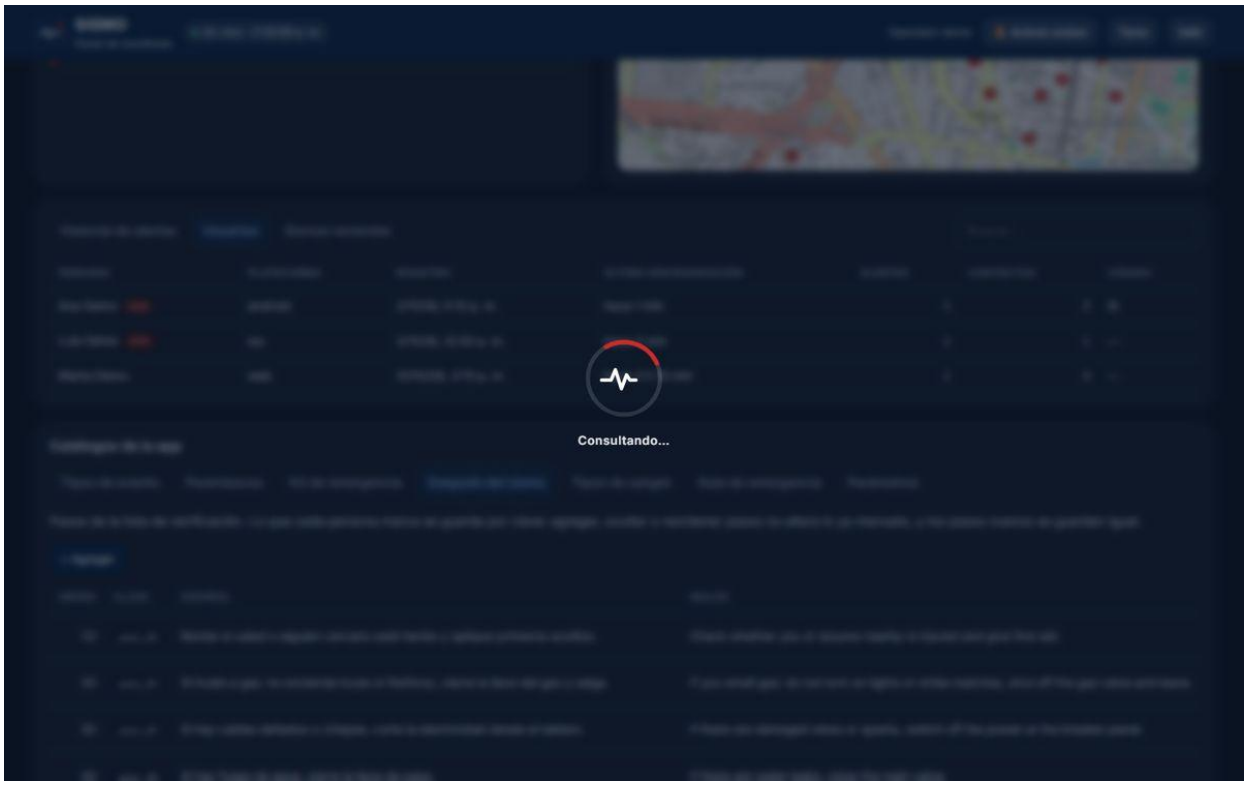
**Figura 85**
*Panel (v3): parámetros*




**Descripción.** Parámetros administrables: número de emergencia, máximo de puntos de encuentro, opciones de batería y tiempos del simulacro.

Fuente: elaboración propia (captura del 04/11/2026).

**Figura 86**
*Panel (v3): loader*




**Descripción.** Loader del panel mientras se consulta el servidor en una acción del operador.

Fuente: elaboración propia (captura del 04/11/2026).

**Figura 87**
*Panel (v3): página informativa escritorio*

**Descripción.** Página informativa pública en escritorio (fragmento superior): presenta SISMO, sus beneficios y lo que aún está pendiente, sin iniciar sesión.

Fuente: elaboración propia (captura del 04/11/2026).

## F.2 Versión 3: página informativa en el celular (04/11/2026)

**Figura 88**

*Página informativa (v3): Página informativa celular*

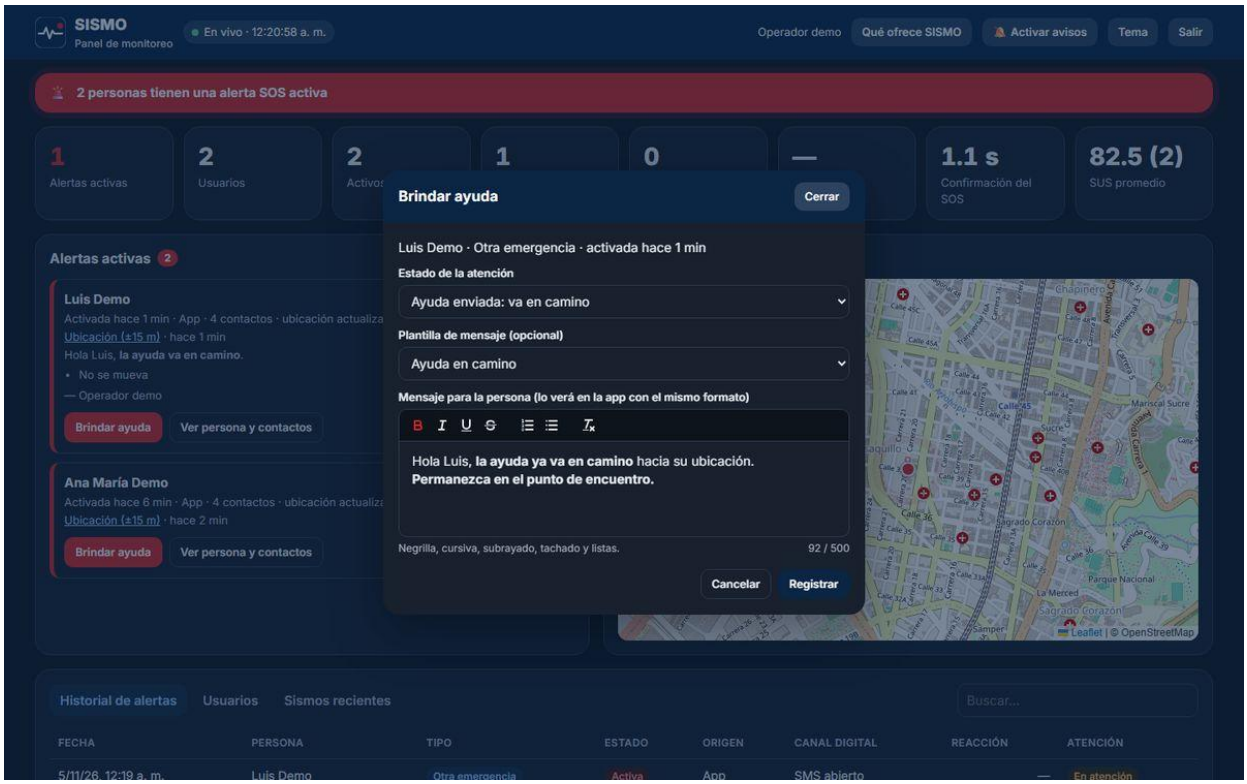
(a) Página informativa celular

**(a) Página informativa celular.** Página informativa en el celular: el mismo contenido adaptado a la pantalla pequeña.

Fuente: elaboración propia (captura del 04/11/2026).

## F.3 Versión 3.2: plantillas, prioridad y evento masivo (05/11/2026)

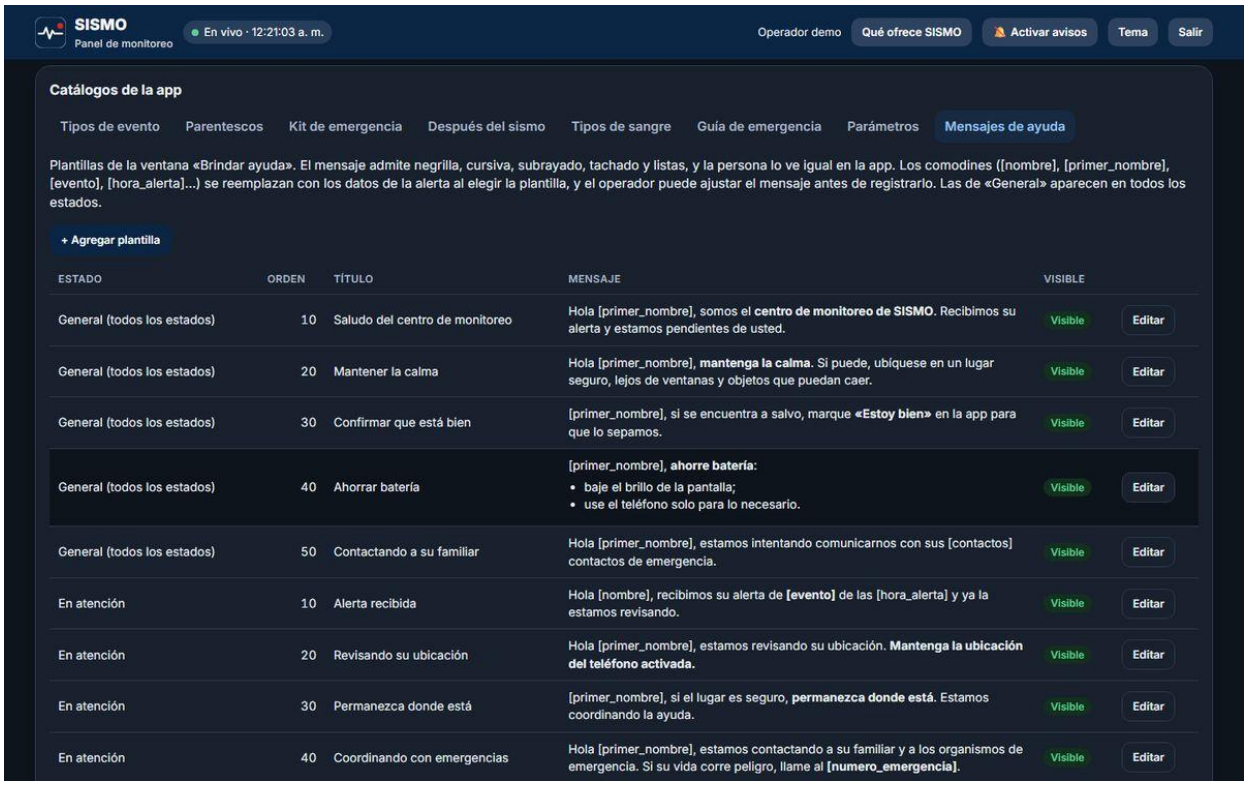
**Figura 89**
*Panel (v3.2): brindar ayuda con plantilla*




**Descripción.** «Brindar ayuda» con una plantilla de mensaje elegida y el editor de texto enriquecido (negrita, cursiva, listas) antes de registrar la ayuda.

Fuente: elaboración propia (captura del 05/11/2026).

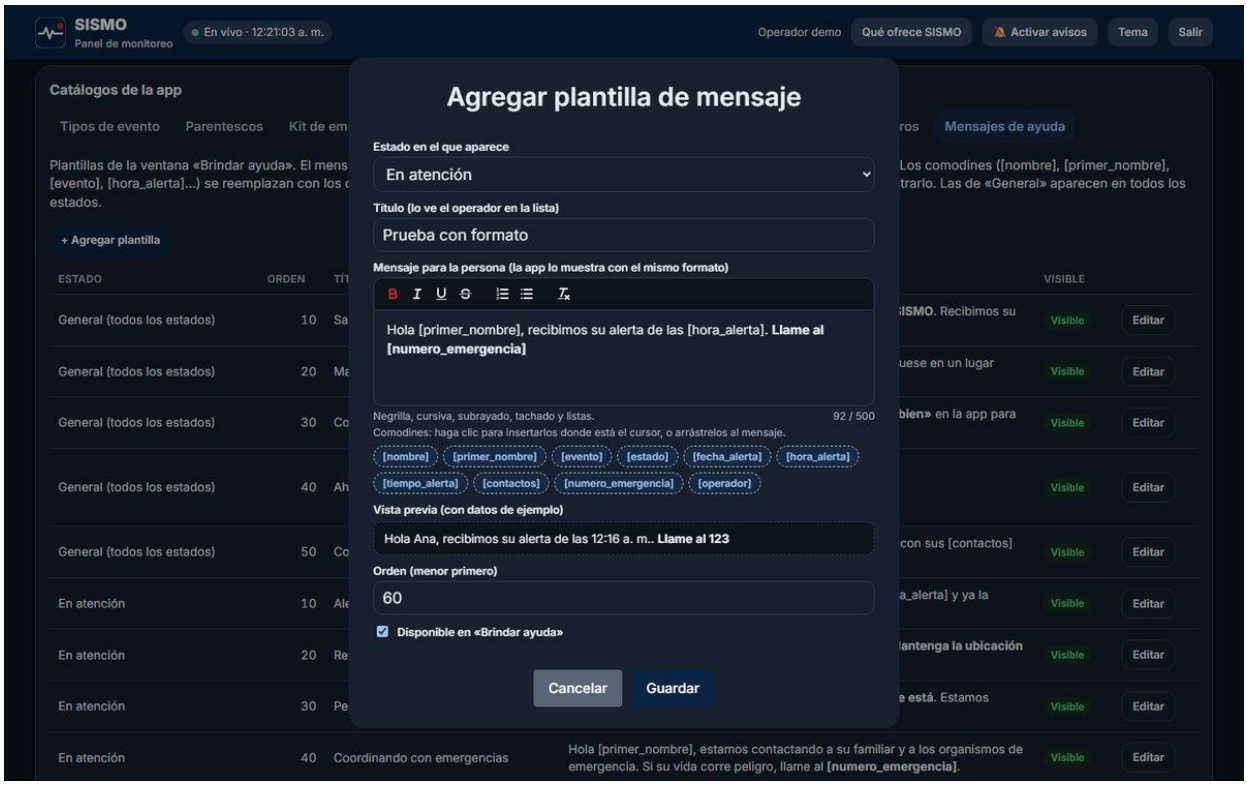
**Figura 90**
*Panel (v3.2): catálogos mensajes de ayuda*




**Descripción.** Catálogo «Mensajes de ayuda»: plantillas generales y por estado con comodines como [primer_nombre] o [evento] que se reemplazan al usarlas.

Fuente: elaboración propia (captura del 05/11/2026).

**Figura 91**
*Panel (v3.2): editar plantilla de mensaje*

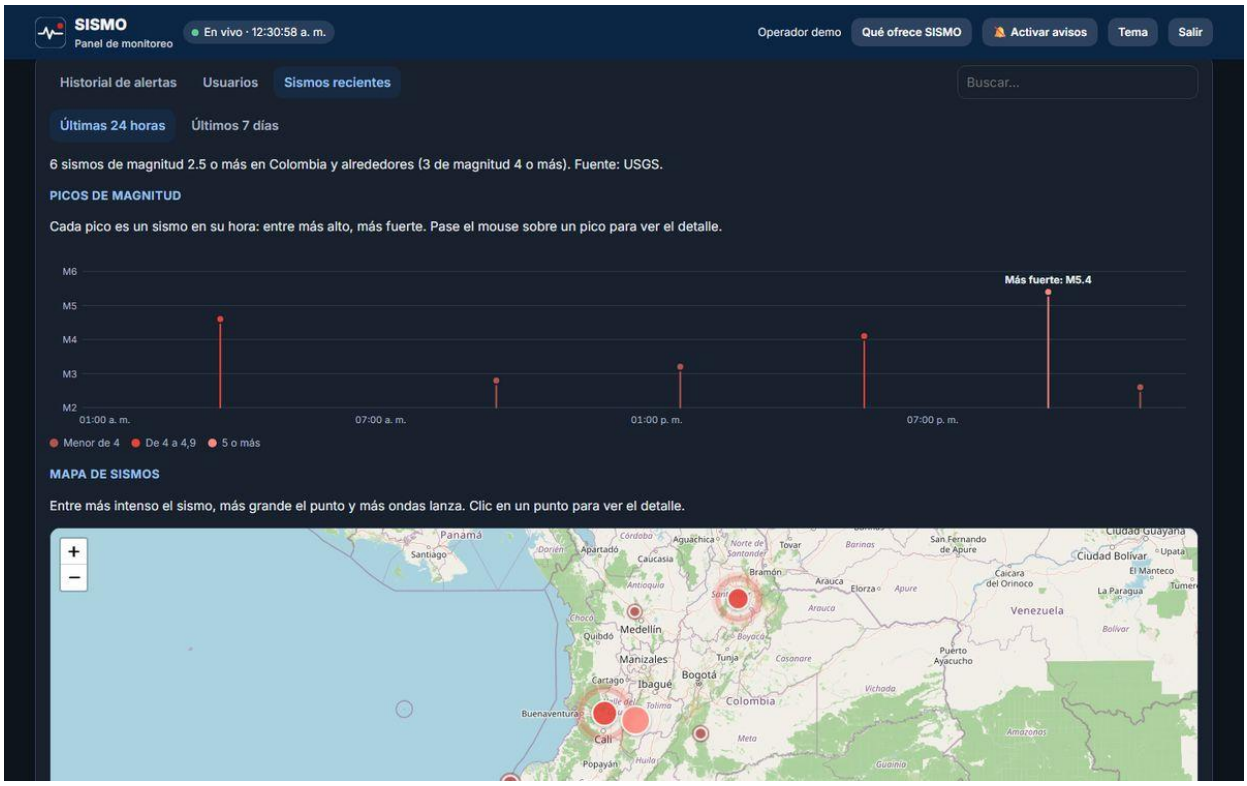



**Descripción.** Edición de una plantilla: estado al que aplica, título, mensaje con formato, comodines que se insertan con un clic y vista previa.

Fuente: elaboración propia (captura del 05/11/2026).

**Figura 92**

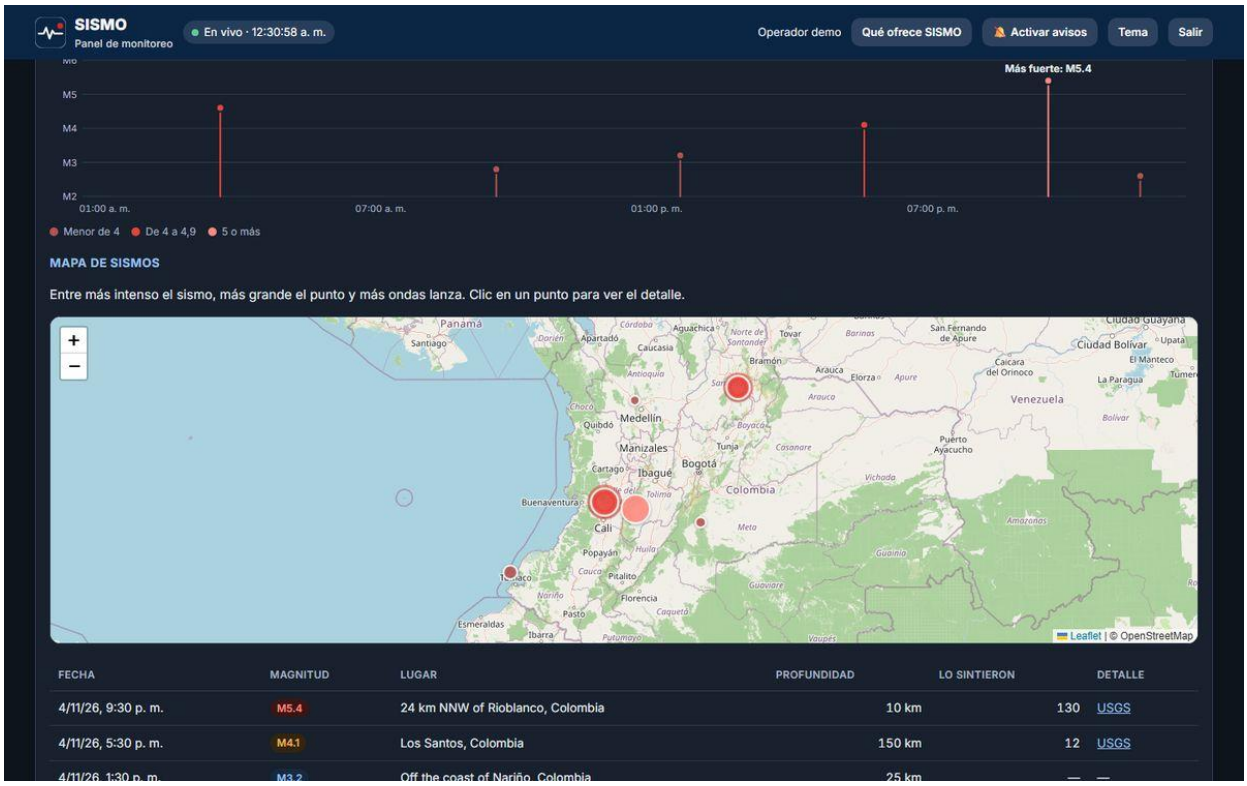
*Panel (v3.2): sismos gráfica de picos*




**Descripción.** Sismos en el panel con la gráfica de picos por magnitud y el mapa de sismos.

Fuente: elaboración propia (captura del 05/11/2026).

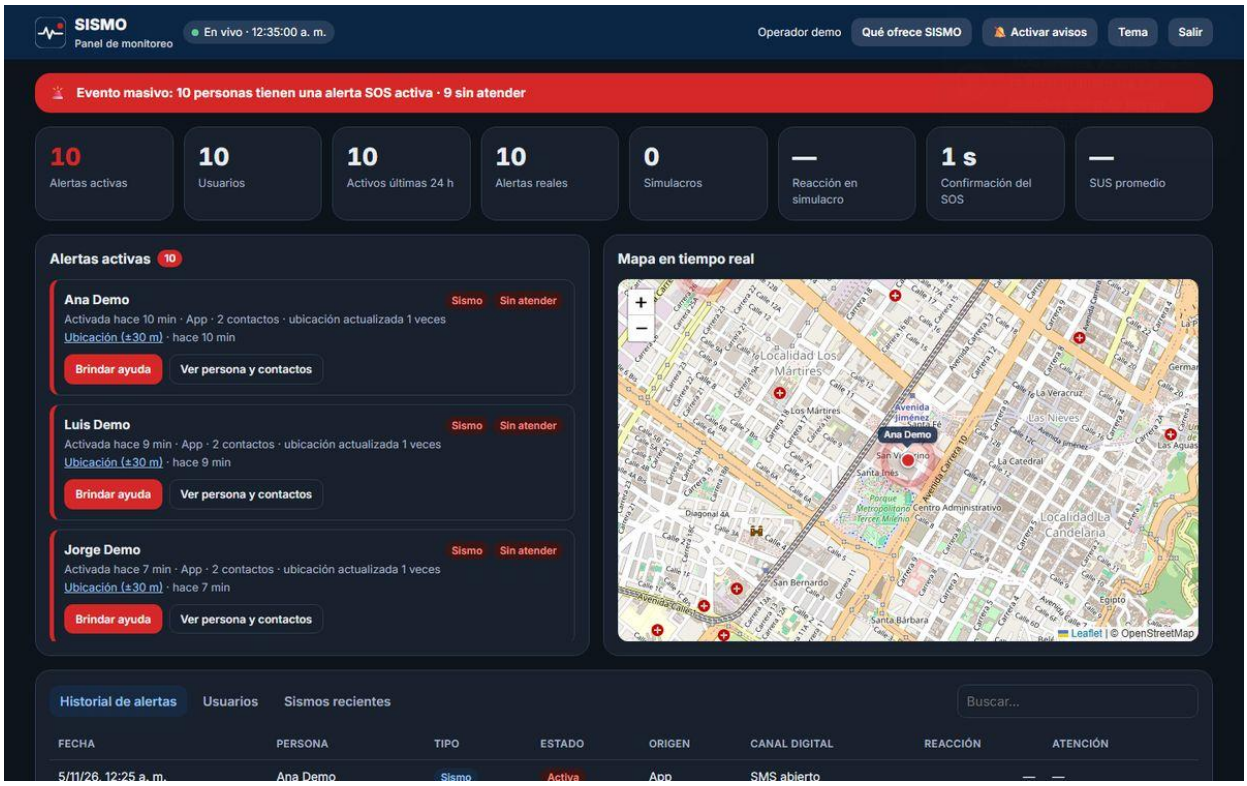
**Figura 93**
*Panel (v3.2): sismos mapa con ondas*




**Descripción.** Mapa de sismos con ondas y la tabla con fecha, magnitud, lugar y profundidad de cada sismo.

Fuente: elaboración propia (captura del 05/11/2026).

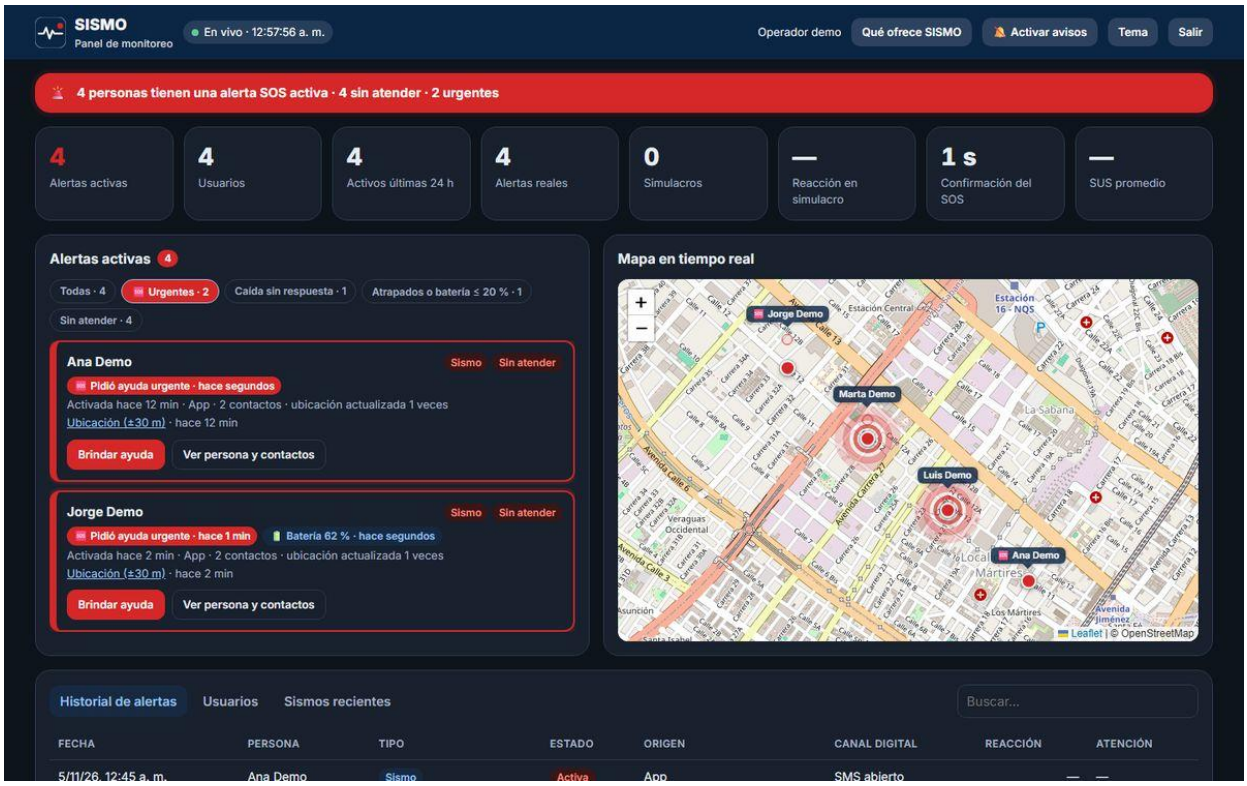
**Figura 94**
*Panel (v3.2): evento masivo*




**Descripción.** Evento masivo: con muchas alertas a la vez los avisos se agrupan en una sola franja, se ordenan por triaje y el mapa sigue siendo legible.

Fuente: elaboración propia (captura del 05/11/2026).

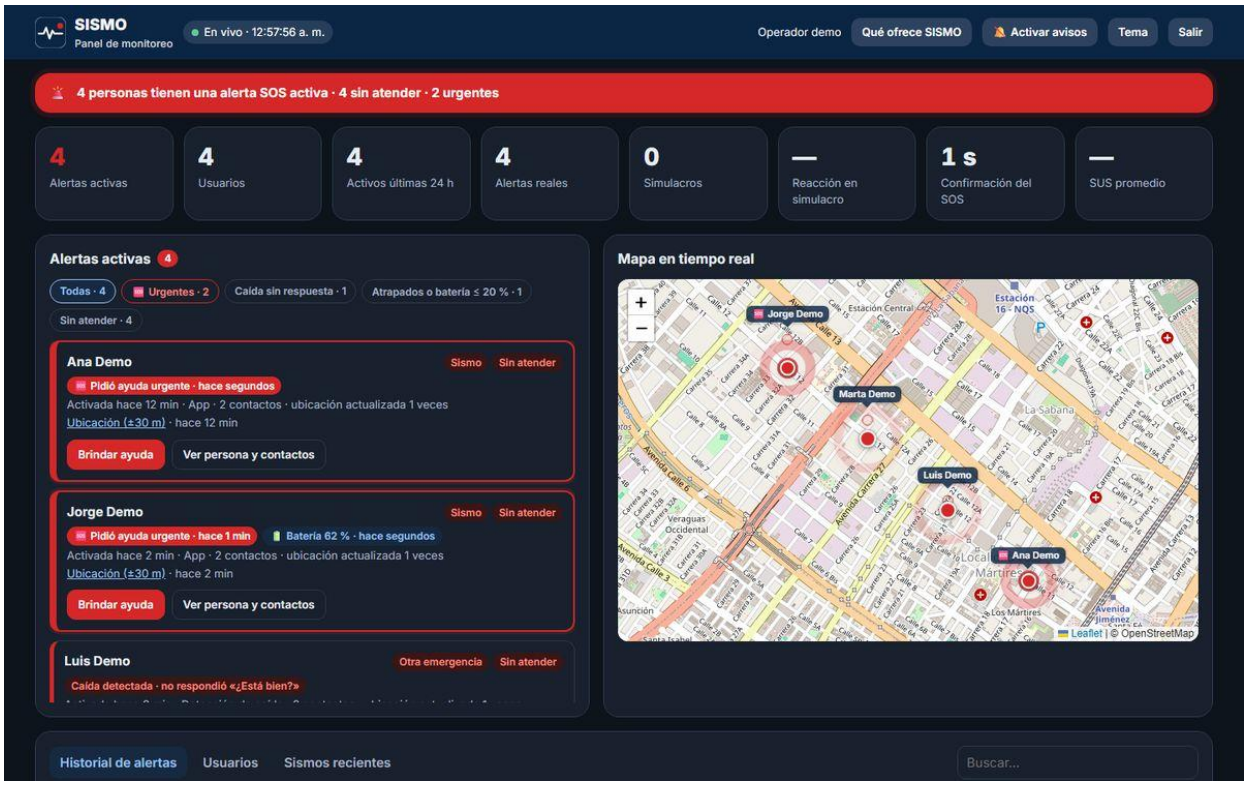
**Figura 95**
*Panel (v3.2): prioridad de alertas*




**Descripción.** Prioridad de las alertas: cada tarjeta muestra si la persona pidió ayuda urgente, si hubo una caída sin respuesta o si tiene batería baja.

Fuente: elaboración propia (captura del 05/11/2026).

**Figura 96**
*Panel (v3.2): filtros por criticidad*




**Descripción.** Filtros por criticidad: el operador ve solo las alertas urgentes, con caída o con batería baja para atender primero las más graves.

Fuente: elaboración propia (captura del 05/11/2026).

**Figura 97**

*Panel (v3.2): página informativa actualizada*

**Descripción.** Página informativa actualizada el 05/11 (fragmento superior), con las nuevas funciones de prioridad, preparación y sismos.

Fuente: elaboración propia (captura del 05/11/2026).

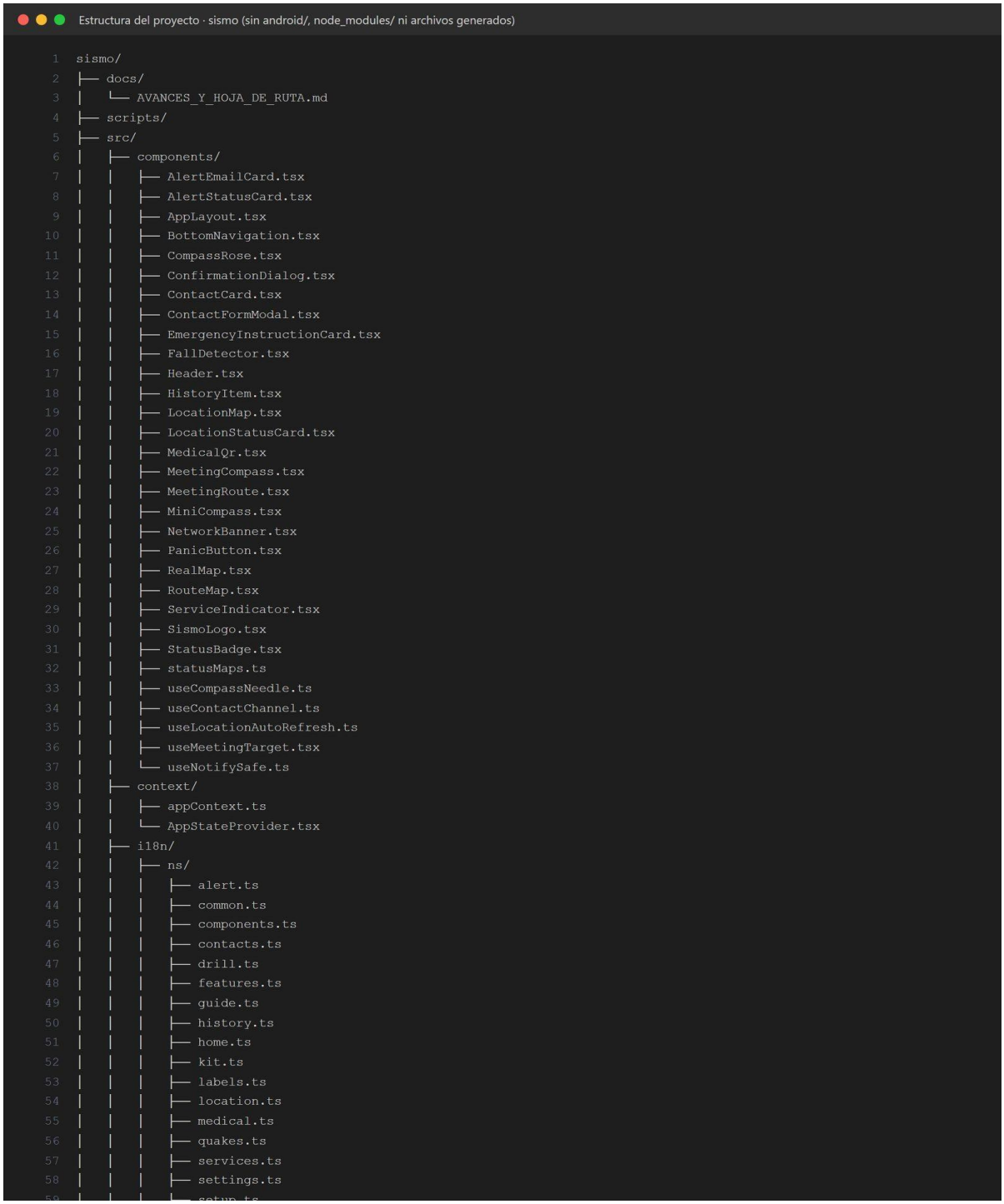
# ANEXO G. EVIDENCIAS DEL CÓDIGO FUENTE

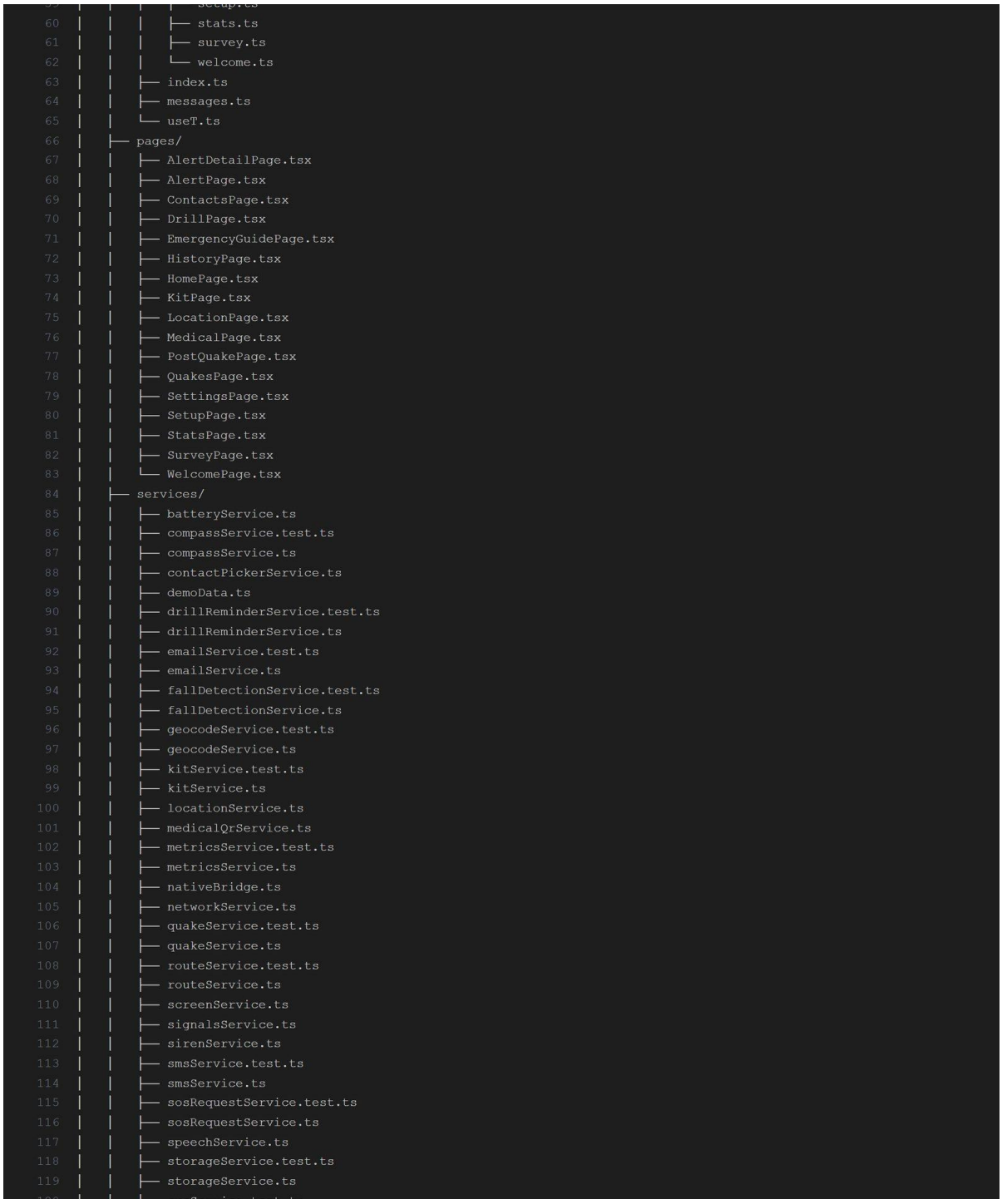
Fragmentos del código de la app y del backend que implementan los requisitos de la segunda entrega. No contienen credenciales: las llaves y las URL de los servicios se leen de variables de entorno.

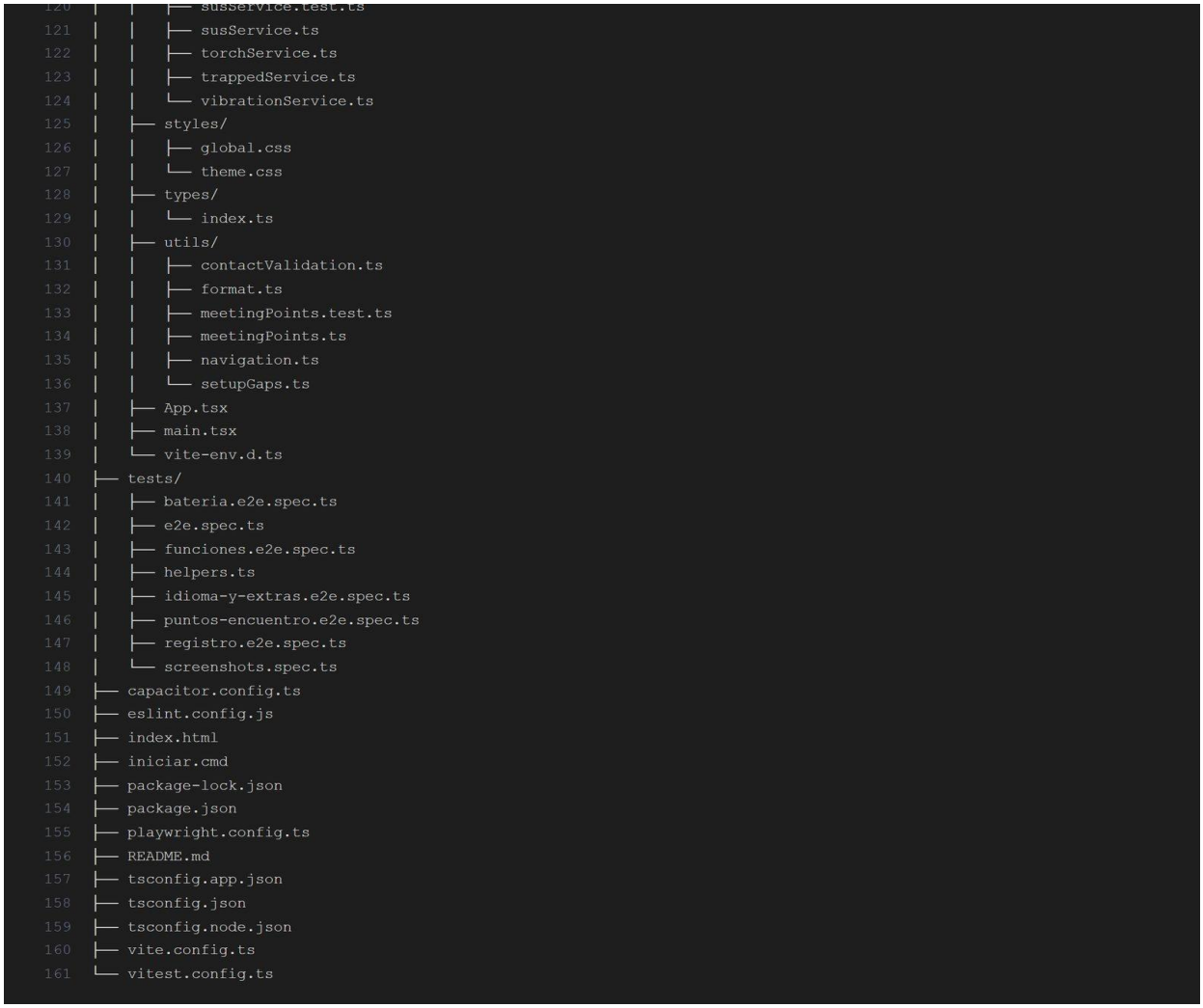
## G.1 App, versión 2 (02/11/2026)

**Figura 98**

*Código de la app (v2): estructura del proyecto*



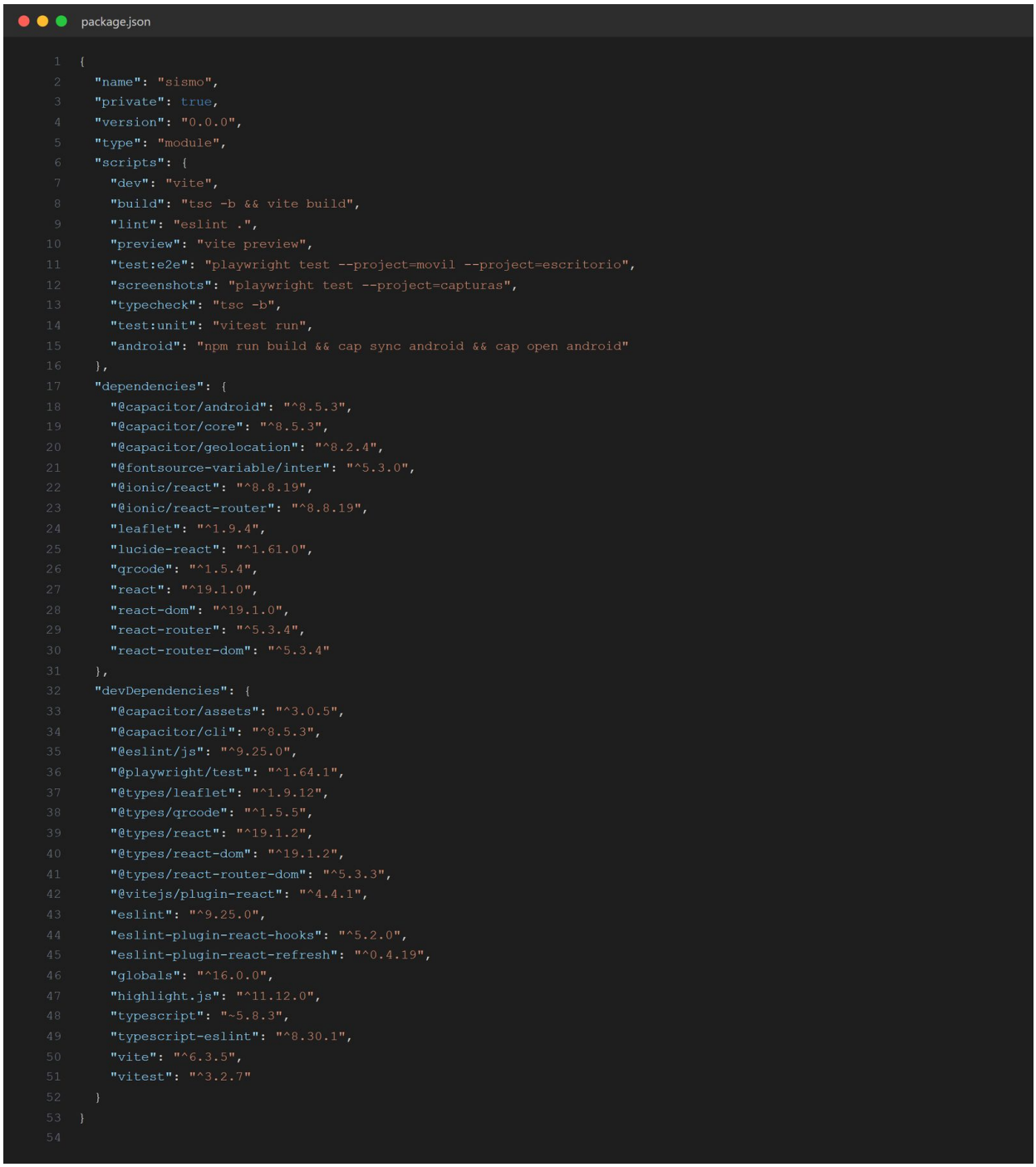

setup.ts
stats.ts
survey.ts
welcome.ts
index.ts
messages.ts
useT.ts
pages/
AlertDetailPage.tsx
AlertPage.tsx
ContactsPage.tsx
DrillPage.tsx
EmergencyGuidePage.tsx
HistoryPage.tsx
HomePage.tsx
KitPage.tsx
LocationPage.tsx
MedicalPage.tsx
PostQuakePage.tsx
QuakesPage.tsx
SettingsPage.tsx
SetupPage.tsx
StatsPage.tsx
SurveyPage.tsx
WelcomePage.tsx
services/
batteryService.ts
compassService.test.ts
compassService.ts
contactPickerService.ts
demoData.ts
drillReminderService.test.ts
drillReminderService.ts
emailService.test.ts
emailService.ts
fallDetectionService.test.ts
fallDetectionService.ts
geocodeService.test.ts
geocodeService.ts
kitService.test.ts
kitService.ts
locationService.ts
medicalQrService.ts
metricsService.test.ts
metricsService.ts
nativeBridge.ts
networkService.ts
quakeService.test.ts
quakeService.ts
routeService.test.ts
routeService.ts
screenService.ts
signalsService.ts
sirenService.ts
smsService.test.ts
smsService.ts
sosRequestService.test.ts
sosRequestService.ts
speechService.ts
storageService.test.ts
storageService.ts




**Descripción.** Árbol de carpetas del proyecto de la app (sin android/, node_modules/ ni archivos generados): componentes, páginas, servicios, contexto, traducciones (i18n), utilidades y pruebas, separados por responsabilidad.

Fuente: elaboración propia (captura del 02/11/2026).

**Figura 99**
*Código de la app (v2): package JSON*




**Descripción.** Archivo package.json: scripts para desarrollo, compilación, lint, pruebas unitarias (Vitest), e2e (Playwright) y Android, y las dependencias principales (Ionic React, Capacitor, Leaflet, React Router y qrcode).

Fuente: elaboración propia (captura del 02/11/2026).

**Figura 100**
*Código de la app (v2): navegación rutas*

src/App.tsx — rutas (React Router + IonRouterOutlet)

```
      <IonRouterOutlet>
        <Route exact path="/bienvenida" component={WelcomePage} />
        <Route exact path="/registro" component={SetupPage} />
        <Route exact path="/inicio" component={HomePage} />
        <Route exact path="/alerta" component={AlertPage} />
        <Route exact path="/ubicacion" component={LocationPage} />
        <Route exact path="/contactos" component={ContactsPage} />
        <Route exact path="/historial" component={HistoryPage} />
        <Route exact path="/historial/:id" component={AlertDetailPage} />
        <Route exact path="/guia" component={EmergencyGuidePage} />
        <Route exact path="/configuracion" component={SettingsPage} />
        <Route exact path="/estadisticas" component={StatsPage} />
        <Route exact path="/simulacro" component={DrillPage} />
        <Route exact path="/sismos" component={QuakesPage} />
        <Route exact path="/ficha-medica" component={MedicalPage} />
        <Route exact path="/encuesta" component={SurveyPage} />
        <Route exact path="/mochila" component={KitPage} />
        <Route exact path="/despues-del-sismo" component={PostQuakePage} />
        <Route exact path="/">
          <Redirect to={data.settings.onboarded ? '/inicio' : '/bienvenida'} />
        </Route>
```

**Descripción.** src/App.tsx: rutas de la app con IonRouterOutlet (bienvenida, registro, inicio, alerta, ubicación, contactos, historial, guía, configuración y demás); la ruta raíz lleva a Inicio o a Bienvenida según si la persona ya se registró.

Fuente: elaboración propia (captura del 02/11/2026).

**Figura 101**
*Código de la app (v2): componente botón pánico*

src/components/PanicButton.tsx

```
import { Siren } from 'lucide-react';
import { useT } from '../i18n/useT';

interface PanicButtonProps {
  onPress: () => void;
  active?: boolean;
}

export function PanicButton({ onPress, active = false }: PanicButtonProps) {
  const t = useT().components.panic;
  return (
    <div className={`panic${active ? ' panic--active' : ''}`}>
      <span className="panic__ring" aria-hidden="true" />
      <span className="panic__ring panic__ring--2" aria-hidden="true" />
      <button className="panic__button" onClick={onPress} aria-label={active ? t.viewActive : t.activate}>
        <Siren size={54} strokeWidth={2} aria-hidden="true" />
        <strong>{active ? t.active : t.sos}</strong>
        <small>{active ? t.tapToView : t.pressToActivate}</small>
      </button>
    </div>
  );
}
```

**Descripción.** src/components/PanicButton.tsx: componente del botón SOS con sus anillos animados, etiqueta accesible (aria-label) y texto que cambia cuando la alerta está activa.

Fuente: elaboración propia (captura del 02/11/2026).

**Figura 102**
*Código de la app (v2): registro obligatorio*

src/pages/SetupPage.tsx — registro obligatorio de 4 pasos

```tsx
const STEPS = ['name', 'location', 'meeting', 'contact'] as const;

/**
 * Registro obligatorio cuando se empieza sin modo de demostración: nombre, ubicación real del dispositivo,
 * punto de encuentro y al menos un contacto real. «Siguiente» se habilita solo con el paso completo y la app
 * no queda marcada como configurada (`onboarded`) hasta terminar. El SOS sigue disponible en todo momento.
 */
export default function SetupPage() {
  const router = useIonRouter();
  const { data, updateSettings, setLocation, setLocationPermission, addMeetingPoint, removeMeetingPoint, addContact, updateContact, deleteContact } = useAppState();
  const { setup: t, settings: ts, location: tl, contacts: tc } = useT();
  const [presentToast, dismissToast] = useIonToast();
  const [imported, setImported] = useState<ContactInput | null>(null);
  const [step, setStep] = useState(0);
  // El nombre por defecto («Usuario demo») no cuenta: la persona debe escribir el suyo.
  const [name, setName] = useState(data.settings.userName === defaultSettings.userName ? '' : data.settings.userName);
  const [locating, setLocating] = useState(false);
  const [locationMessage, setLocationMessage] = useState('');
  const [label, setLabel] = useState(tl.meeting.defaultLabel);
  const [address, setAddress] = useState('');
  const [results, setResults] = useState<AddressResult[]>([]);
  const [searching, setSearching] = useState(false);
  const [searchMessage, setSearchMessage] = useState('');
  const [formOpen, setFormOpen] = useState(false);
  const [editing, setEditing] = useState<Contact | null>(null);
  const [toDelete, setToDelete] = useState<Contact | null>(null);

  const { lastLocation, meetingPoints } = data;
  const deviceLocation = lastLocation?.source === 'dispositivo' ? lastLocation : null;
  const realContacts = data.contacts.filter((c) => !c.isSeed);
  const nameError = name.trim().length < 2 ? ts.profile.min : name.trim().length > 30 ? ts.profile.max : '';
  const labelError = label.trim().length < 2 ? tl.meeting.min : '';

  const done = [!nameError, !!deviceLocation, meetingPoints.length > 0, realContacts.length > 0];
  const key = STEPS[step];
  const isLast = step === STEPS.length - 1;

  const next = () => {
    if (!done[step]) return;
    if (key === 'name') {
      updateSettings({ userName: name.trim() });
      toast(ts.profile.saved);
    }
    if (!isLast) {
      setStep(step + 1);
      return;
    }
    updateSettings({ onboarded: true });
    router.push('/inicio', 'root', 'replace');
  };

```

**Descripción.** src/pages/SetupPage.tsx: registro obligatorio de 4 pasos (nombre, ubicación, punto de encuentro y contacto); cada paso valida su dato antes de habilitar «Siguiente» y el SOS sigue disponible mientras tanto.

Fuente: elaboración propia (captura del 02/11/2026).

**Figura 103**
*Código de la app (v2): datos faltantes*

src/utils/setupGaps.ts — datos necesarios que faltan (borde rojo)

```ts
import { defaultSettings } from '../services/storageService';
import type { AppData } from '../types';

export type SetupItem = 'name' | 'location' | 'meeting' | 'contact';

/**
 * Datos necesarios para que SISMO pueda avisar a los contactos que faltan (los mismos 4 del registro).
 * Ocurre si se dio SOS durante el registro sin terminarlo, o si después se borraron datos. En el modo de
 * demostración no se marca nada (los datos de ejemplo son ficticios a propósito).
 */
export function setupGaps(data: AppData): SetupItem[] {
  if (data.settings.demoMode) return [];
  const name = data.settings.userName.trim();
  const gaps: SetupItem[] = [];
  if (!name || name === defaultSettings.userName) gaps.push('name');
  if (data.lastLocation?.source !== 'dispositivo') gaps.push('location');
  if (data.meetingPoints.length === 0) gaps.push('meeting');
  if (!data.contacts.some((c) => !c.isSeed)) gaps.push('contact');
  return gaps;
}

```

**Descripción.** src/utils/setupGaps.ts: calcula qué datos obligatorios faltan (nombre, ubicación real, punto de encuentro y contacto real) para mostrarlos con borde rojo en Inicio; en modo de demostración no marca nada.

Fuente: elaboración propia (captura del 02/11/2026).

**Figura 104**
*Código de la app (v2): puntos de encuentro*

src/utils/meetingPoints.ts — varios puntos y el más cercano

```ts
import { distanceKm } from '../services/quakeService';
import type { MeetingPoint } from '../types';

/** Tope de puntos de encuentro: todos van en el SMS, que debe seguir siendo corto. */
export const MAX_MEETING_POINTS = 5;

/**
 * Punto de encuentro más cercano a una posición: es el que usan la brújula y la ruta por defecto.
 * Sin posición conocida, el primero de la lista.
 */
export function nearestMeetingPoint(
  points: MeetingPoint[],
  from: { latitude: number; longitude: number } | null | undefined,
): MeetingPoint | null {
  if (points.length === 0) return null;
  if (!from) return points[0];
  const dist = (p: MeetingPoint) => distanceKm(from.latitude, from.longitude, p.latitude, p.longitude);
  return points.reduce((best, p) => (dist(p) < dist(best) ? p : best));
}

```

**Descripción.** src/utils/meetingPoints.ts: tope de 5 puntos de encuentro y función que elige el punto más cercano a la posición actual, que es el que usan la brújula y la ruta por defecto.

Fuente: elaboración propia (captura del 02/11/2026).

**Figura 105**
*Código de la app (v2): mensajes SMS WhatsApp*

src/services/smsService.ts — aviso de alerta, «Estoy bien» y batería baja

```ts
export function buildAlertMessage(
  userName: string,
  eventType: EventType,
  location: LocationSnapshot | null,
  meetingPoints: MeetingPoint[] = [],
  medical: MedicalInfo | null = null,
  batteryLevel: number | null = null,
): string {
  const t = getT().services.sms;
  const who = userName.trim() || t.defaultWho;
  const lines = [t.alertHeader(who, eventTypeLabel[eventType]), locationLine(location), ...meetingLines(meetingPoints)];
  const med = medicalSummary(medical);
  if (med) lines.push(t.medical(med));
  // Les dice a los contactos cuánto tiempo queda para comunicarse.
  if (batteryLevel != null) lines.push(t.battery(batteryLevel));
  lines.push(t.callToAction);
  return lines.join('\n');
}

/** Línea con la ubicación (enlace a Google Maps y precisión), compartida por el aviso de alerta y «Estoy bien». */
function locationLine(location: LocationSnapshot | null): string {
  const t = getT().services.sms;
  if (!location) return t.noLocation;
  const precision = location.accuracy != null ? ` (±${Math.round(location.accuracy)} m)` : '';
  const link = mapsLink(location.latitude, location.longitude);
  return location.source === 'prueba' ? t.testLocation(link) : t.location(precision, link);
}

/** Una línea por punto de encuentro, cada una con su enlace a Google Maps. */
function meetingLines(meetingPoints: MeetingPoint[]): string[] {
  const t = getT().services.sms;
  return meetingPoints.map((p) => t.meetingPoint(p.label, mapsLink(p.latitude, p.longitude)));
}

/** Datos opcionales del aviso «Estoy bien»: dónde está la persona, los puntos de encuentro y la batería. */
export interface SafeDetails {
  location?: LocationSnapshot | null;
  meetingPoints?: MeetingPoint[];
  batteryLevel?: number | null;
}

/**
 * Aviso de que la persona está a salvo: al finalizar una alerta (con `eventType`) o en cualquier momento
 * desde el panel principal, por ejemplo después de un temblor (sin `eventType`).
 */
export function buildSafeMessage(userName: string, eventType?: EventType, at: Date = new Date(), details: SafeDetails = {}): string {
  const t = getT().services.sms;
  const who = userName.trim() || t.defaultSafeWho;
  const hour = at.toLocaleTimeString(getLocale(), { hour: '2-digit', minute: '2-digit' });
  const lines = [eventType ? t.safeAfterAlert(who, eventTypeLabel[eventType], hour) : t.safeNow(who, hour)];
  // Mensaje completo: dónde está, los puntos de encuentro y la batería (si se conocen).
  if (details.location !== undefined) lines.push(locationLine(details.location));
  lines.push(...meetingLines(details.meetingPoints ?? []));
  if (details.batteryLevel != null) lines.push(t.battery(details.batteryLevel));
  return lines.join('\n');
}

/**
 * Aviso de batería baja durante una alerta (cuando el teléfono pasa solo al modo ahorro): nivel, última
 * ubicación conocida y puntos de encuentro, por si la persona se queda sin comunicación.
 */
export function buildLowBatteryMessage(
  userName: string,
  level: number,
  location: LocationSnapshot | null,
  meetingPoints: MeetingPoint[] = [],
): string {
  const t = getT().services.sms;
  const who = userName.trim() || t.defaultWho;
  return [t.lowBattery(who, level), locationLine(location), ...meetingLines(meetingPoints), t.callToAction].join('\n');
}

/**
 * Número en el formato que exige WhatsApp (wa.me): código de país y solo dígitos, sin «+».
 * Si se guardó un celular colombiano sin indicativo (10 dígitos que empiezan por 3), se antepone 57;
 * por eso «3203498531» y «+57 320 349 8531» abren el mismo chat.
 */
```

**Descripción.** src/services/smsService.ts: arma el texto del aviso de alerta con el nombre, el tipo de evento, la ubicación con enlace a Google Maps, los puntos de encuentro, la ficha médica (si se comparte) y la batería.

Fuente: elaboración propia (captura del 02/11/2026).

**Figura 106**
*Código de la app (v2): ahorro batería automático*

src/pages/AlertPage.tsx — ahorro de batería automático

```
82   // Ahorro automático: con la batería en el umbral de Configuración pasa sola al modo ahorro (una sola vez por
83   // alerta: si la persona vuelve a la señal continua, se respeta). No aplica en simulacros.
84   const [lowBattery, setLowBattery] = useState<number | null>(null);
85   const threshold = data.settings.lowBatterySaver;
86   const alertId = activeAlert && !activeAlert.isDrill ? activeAlert.id : null;
87   useEffect(() => {
88     if (!alertId || threshold <= 0) return;
89     const check = async () => {
90       if (autoSaverDone.has(alertId) || isTrapped()) return;
91       const level = await getBatteryLevel();
92       if (level == null || level > threshold || autoSaverDone.has(alertId)) return;
93       autoSaverDone.add(alertId);
94       setFlash(false);
95       await startTrapped();
96       setLowBattery(level);
97       tick((n) => n + 1);
98     };
99     void check();
100    const t = window.setInterval(() => void check(), BATTERY_CHECK_MS);
101    return () => window.clearInterval(t);
102  }, [alertId, threshold]);
103
104  const notifyLowBattery = () => {
105    if (!activeAlert || lowBattery == null) return;
106    const location = activeAlert.location ?? data.lastLocation;
107    chooseChannel(ta.lowBattery.sheetHeader, async () =>
108      buildLowBatteryMessage(data.settings.userName, (await getBatteryLevel()) ?? lowBattery, location, data.meetingPoints),
109    );
110  };
111
```

**Descripción.** src/pages/AlertPage.tsx: ahorro de batería automático; revisa la carga periódicamente y, al llegar al umbral de Configuración, pasa una sola vez la alerta al modo ahorro y ofrece avisar a los contactos.

Fuente: elaboración propia (captura del 02/11/2026).

**Figura 107**
*Código de la app (v2): ubicación cada 5 minutos*

src/components/useLocationAutoRefresh.ts — ubicación cada 5 minutos

```
import { useEffect, useRef } from 'react';
import { useAppState } from '../context/appContext';
import { requestDeviceLocation } from '../services/locationService';

/** Cada cuánto se actualiza la ubicación mientras la app está en pantalla. */
export const LOCATION_REFRESH_MS = 5 * 60 * 1000;

/**
 * Mantiene la última ubicación al día: cada 5 minutos y al volver a la app si la última tiene más de 5 minutos.
 * Solo con la app abierta y visible (sin permiso de ubicación en segundo plano ni gasto de batería con la
 * pantalla apagada), solo si el permiso ya está concedido (nunca muestra el diálogo del sistema) y nunca
 * reemplaza las coordenadas de prueba que el usuario eligió a propósito.
 */
export function useLocationAutoRefresh() {
  const { data, setLocation } = useAppState();
  const state = useRef({ data, busy: false });
  state.current.data = data;

  useEffect(() => {
    const refresh = async (onlyIfStale: boolean) => {
      const { data: d, busy } = state.current;
      if (busy || document.visibilityState !== 'visible' || !d.settings.onboarded) return;
      if (d.locationPermission !== 'concedido' || d.lastLocation?.source === 'prueba') return;
      const age = d.lastLocation ? Date.now() - new Date(d.lastLocation.timestamp).getTime() : Infinity;
      if (onlyIfStale && age < LOCATION_REFRESH_MS) return;
      state.current.busy = true;
      try {
        const result = await requestDeviceLocation();
        // Si falla (sin señal), se conserva la última ubicación conocida.
        if (result.snapshot) setLocation(result.snapshot, result.permission);
      } finally {
        state.current.busy = false;
      }
    };
    const timer = window.setInterval(() => void refresh(false), LOCATION_REFRESH_MS);
    const onVisible = () => void refresh(true);
    document.addEventListener('visibilitychange', onVisible);
    void refresh(true);
    return () => {
      window.clearInterval(timer);
      document.removeEventListener('visibilitychange', onVisible);
    };
  }, [setLocation]);
}

```

**Descripción.** src/hooks/useLocationAutoRefresh.ts: actualiza la ubicación cada 5 minutos solo mientras la app está abierta y visible, sin permiso de ubicación en segundo plano y sin reemplazar coordenadas de prueba elegidas a propósito.

Fuente: elaboración propia (captura del 02/11/2026).

**Figura 108**
*Código de la app (v2): widget doble toque*

co/edu/poli/sismo/SosWidgetProvider.java — widget SOS con doble toque

```java
/**
 * Widget SOS 1×1 que activa la alerta con doble toque, sin abrir diálogos:
 * - Primer toque: el widget se «arma» y muestra «Toque otra vez» durante ARM_WINDOW_MS.
 * - Segundo toque (dentro de ese tiempo): abre SISMO con SismoNativePlugin.SOS_NOW_URI; MainActivity enciende
 *   de inmediato sirena, linterna y vibración, y la app crea la alerta (tipo «Sismo») y muestra su pantalla.
 * - Si no hay segundo toque, vuelve solo al estado normal.
 * El segundo toque abre la actividad directamente (PendingIntent de actividad), porque Android no deja
 * abrir pantallas desde un receptor en segundo plano.
 */
public class SosWidgetProvider extends AppWidgetProvider {

    static final String ACTION_ARM = "co.edu.poli.sismo.WIDGET_ARM";
    static final long ARM_WINDOW_MS = 4000;

    /** Hasta cuándo está armado cada widget (reloj de arranque). Se pierde si el proceso muere: queda desarmado. */
    private static final java.util.Map<Integer, Long> armedUntil = new java.util.HashMap<>();

    /** true en el widget grande (2×2). */
    protected boolean isLarge() {
        return false;
    }

    @Override
    public void onUpdate(Context context, AppWidgetManager manager, int[] widgetIds) {
        for (int id : widgetIds) manager.updateAppWidget(id, buildViews(context, id, isArmed(id)));
    }

    @Override
    public void onReceive(Context context, Intent intent) {
        super.onReceive(context, intent);
        if (!ACTION_ARM.equals(intent.getAction())) return;
        int id = intent.getIntExtra(AppWidgetManager.EXTRA_APPWIDGET_ID, AppWidgetManager.INVALID_APPWIDGET_ID);
        if (id == AppWidgetManager.INVALID_APPWIDGET_ID) return;

        AppWidgetManager manager = AppWidgetManager.getInstance(context);
        long until = SystemClock.elapsedRealtime() + ARM_WINDOW_MS;
        synchronized (armedUntil) {
            armedUntil.put(id, until);
        }
        manager.updateAppWidget(id, buildViews(context, id, true));

        // Pasado el tiempo sin segundo toque, vuelve al estado normal.
        final PendingResult pending = goAsync();
        final Context app = context.getApplicationContext();
        new Handler(Looper.getMainLooper()).postDelayed(() -> {
            try {
                if (!isArmed(id)) manager.updateAppWidget(id, buildViews(app, id, false));
            } finally {
                pending.finish();
            }
        }, ARM_WINDOW_MS + 100);
    }

    private static boolean isArmed(int id) {
        synchronized (armedUntil) {
            Long until = armedUntil.get(id);
            return until != null && SystemClock.elapsedRealtime() < until;
        }
    }

```

**Descripción.** SosWidgetProvider.java (Android): widget SOS con doble toque; el primer toque lo «arma» unos segundos y el segundo abre la app directamente en la alerta activa, para evitar activaciones accidentales.

Fuente: elaboración propia (captura del 02/11/2026).

**Figura 109**
*Código de la app (v2): widget diseño 2x2*

res/layout/widget_sos_large.xml — diseño del widget 2×2

```xml
<!-- Nota de Transparencia: Código generado/refactorizado con asistencia de IA Generativa (Claude Code) bajo la Política ODTI012 del CCS. Requiere supervisión y validación humana permanente. -->
<!-- Widget SOS 2×2: logo y nombre arriba, botón SOS redondo grande y una línea de estado.
     Primer toque: el botón pasa a «¿ACTIVAR?» con borde amarillo; segundo toque: alerta activada. -->
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/widget_sos_button"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@drawable/widget_sos_large_bg"
    android:contentDescription="@string/widget_sos_description"
    android:gravity="center"
    android:orientation="vertical"
    android:padding="8dp">

    <LinearLayout
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:gravity="center_vertical"
        android:importantForAccessibility="no"
        android:orientation="horizontal">

        <ImageView
            android:layout_width="20dp"
            android:layout_height="20dp"
            android:importantForAccessibility="no"
            android:src="@mipmap/ic_launcher_round" />

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginStart="6dp"
            android:importantForAccessibility="no"
            android:text="@string/app_name"
            android:textColor="#FFFFFF"
            android:textSize="13sp"
            android:textStyle="bold" />
    </LinearLayout>

    <TextView
        android:id="@+id/widget_sos_circle"
        android:layout_width="96dp"
        android:layout_height="96dp"
        android:layout_marginTop="6dp"
        android:layout_marginBottom="6dp"
        android:background="@drawable/widget_sos_circle"
        android:gravity="center"
        android:importantForAccessibility="no"
        android:text="@string/widget_sos_big"
        android:textColor="#FFFFFF"
        android:textSize="28sp"
        android:textStyle="bold" />

    <TextView
        android:id="@+id/widget_sos_status"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:gravity="center"
        android:importantForAccessibility="no"
        android:maxLines="2"
        android:text="@string/widget_sos_status_idle"
        android:textColor="#FFFFFF"
        android:textSize="11sp" />
</LinearLayout>

```

**Descripción.** Diseño XML del widget de 2 × 2 de Android: fondo, logo, botón circular SOS y texto «Toque 2 veces para pedir ayuda».

Fuente: elaboración propia (captura del 02/11/2026).

**Figura 110**
*Código de la app (v2): paleta de colores*

src/styles/theme.css — paleta de colores (tema claro)

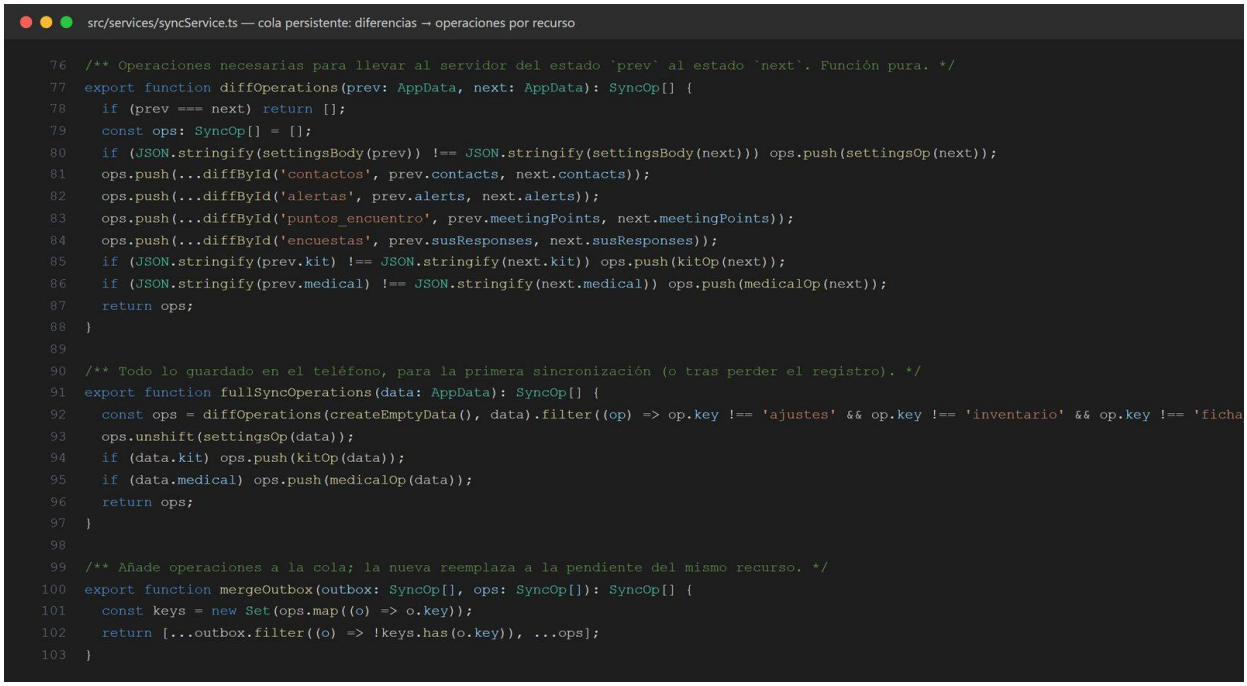
```
:root {
  color-scheme: light;
  /* Azul de marca para fondos (encabezado, botón principal): igual en claro y oscuro. */
  --brand: #0b2545;
  /* --navy es el azul de textos e íconos: en oscuro se aclara para que se lea sobre fondo oscuro. */
  --navy: #0b2545;
  --frame-bg: #dfe5ee;
  --surface-2: #eef3f9;
  --btn-dark: #1b2333;
  --on-info: #163d6b;
  --on-warning: #6b3e00;
  --on-demo: #4d2c75;
  --navy-2: #13315c;
  --navy-soft: #e8eef7;
  --red: #d62828;
  --red-dark: #a61e1e;
  --bg: #f3f5f9;
  --card: #ffffff;
  --text: #1b2333;
  --muted: #5b6578;
  --border: #e3e7ef;

  --success: #1b7a4b;
  --success-bg: #e5f4ec;
  --warning: #8f5300;
  --warning-bg: #fff3dc;
  --danger: #b42318;
  --danger-bg: #fdecec;
  --info: #1d4e89;
  --info-bg: #e8f0fa;
  --neutral: #475467;
  --neutral-bg: #eef1f5;
  --demo: #6b3fa0;
  --demo-bg: #f2ebfb;

  --radius: 18px;
  --radius-sm: 12px;
  --shadow: 0 1px 2px rgba(16, 24, 40, 0.06), 0 4px 14px rgba(16, 24, 40, 0.06);

  --ion-font-family: 'Inter Variable', system-ui, -apple-system, 'Segoe UI', Roboto, sans-serif;
  --ion-background-color: var(--bg);
  --ion-text-color: var(--text);

```

**Descripción.** src/styles/theme.css: paleta de colores del tema claro como variables CSS (azul de marca, rojo de alerta y estados de éxito, advertencia, peligro e información) usadas en toda la app.

Fuente: elaboración propia (captura del 02/11/2026).

## G.2 App, versión 3 (04/11/2026)

**Figura 111**
*Código de la app (v3): cola sin conexión*




**Descripción.** src/services/syncService.ts: compara el estado anterior con el nuevo y genera operaciones PUT o DELETE por recurso (ajustes, contactos, alertas, puntos, encuestas, kit y ficha médica) que se guardan en la cola persistente.

Fuente: elaboración propia (captura del 04/11/2026).

**Figura 112**
*Código de la app (v3): envío y reintentos*

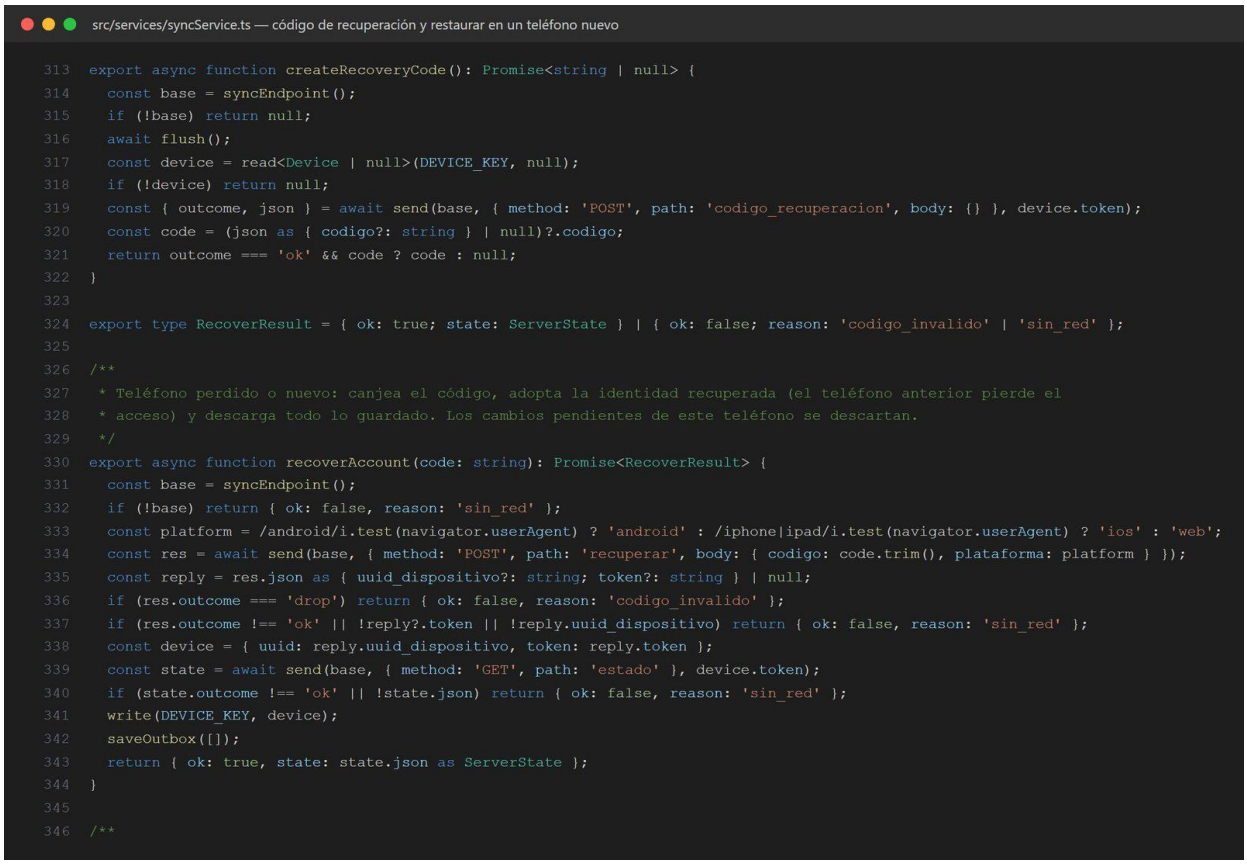
src/services/syncService.ts — envío al volver la red, reintentos y nuevo registro tras 401

```
/** Envía la cola en orden. Se detiene en el primer fallo de red y lo reintenta en el siguiente ciclo. */
export function flush(): Promise<void> {
  const base = syncEndpoint();
  if (!base) return Promise.resolve();
  if (typeof navigator !== 'undefined' && navigator.onLine === false) return Promise.resolve();
  if (flushing) return flushing;
  flushing = (async () => {
    let device = await ensureDevice(base);
    if (!device) return;
    let reRegistered = false;
    while (loadOutbox().length > 0) {
      const op = loadOutbox()[0];
      const { outcome } = await send(base, op, device.token);
      if (outcome === 'retry') return;
      if (outcome === 'unauthorized') {
        // Token revocado, cuenta eliminada o base vaciada: se registra de nuevo de inmediato (una vez por ciclo)
        // y se resube todo lo del teléfono.
        write(DEVICE_KEY, null);
        if (reRegistered) return;
        reRegistered = true;
        device = await ensureDevice(base);
        if (!device) return;
        continue;
      }
      if (outcome === 'drop') console.warn('[sismo-sync] operación rechazada por el servidor', op.method, op.path);
      // Solo se quita si la operación sigue siendo la primera (pudo reemplazarse mientras viajaba).
      const current = loadOutbox();
      if (current[0] === op) saveOutbox(current.slice(1));
    }
  })().finally(() => {
    flushing = null;
  });
  return flushing;
}

/**
```

**Descripción.** syncService.ts, función flush: envía la cola en orden al volver la red, se detiene ante el primer fallo para reintentar en el siguiente ciclo y, si el token deja de ser válido (401), registra el dispositivo de nuevo.

Fuente: elaboración propia (captura del 04/11/2026).

**Figura 113**
*Código de la app (v3): recuperar datos*




**Descripción.** syncService.ts: creación del código de recuperación y recuperación de la cuenta en un teléfono nuevo; recibe un token nuevo, descarga todo el estado guardado y descarta la cola local.

Fuente: elaboración propia (captura del 04/11/2026).

**Figura 114**
*Código de la app (v3): eliminar cuenta*

src/services/syncService.ts — eliminar cuenta (servidor + teléfono)

```ts
export async function deleteAccount(): Promise<'ok' | 'sin_red' | 'error'> {
  const base = syncEndpoint();
  const device = read<Device | null>(DEVICE_KEY, null);
  const forget = () => {
    write(DEVICE_KEY, null);
    saveOutbox([]);
  };
  // Sin servidor o sin registrar: no hay nada que borrar en el servidor.
  if (!base || !device) {
    forget();
    return 'ok';
  }
  const { outcome } = await send(base, { method: 'POST', path: 'eliminar_cuenta', body: {} }, device.token);
  // 401: la cuenta ya no existía o el token fue revocado; para el teléfono es igual que eliminada.
  if (outcome === 'ok' || outcome === 'unauthorized') {
    forget();
    return 'ok';
  }
  return outcome === 'retry' ? 'sin_red' : 'error';
}
```

**Descripción.** syncService.ts, función deleteAccount: pide al servidor eliminar la cuenta y borra el token y la cola del teléfono; si la cuenta ya no existía, el resultado es el mismo.

Fuente: elaboración propia (captura del 04/11/2026).

**Figura 115**
*Código de la app (v3): catálogos administrables*

src/services/catalogService.ts — catálogos con valores por defecto y copia sin red

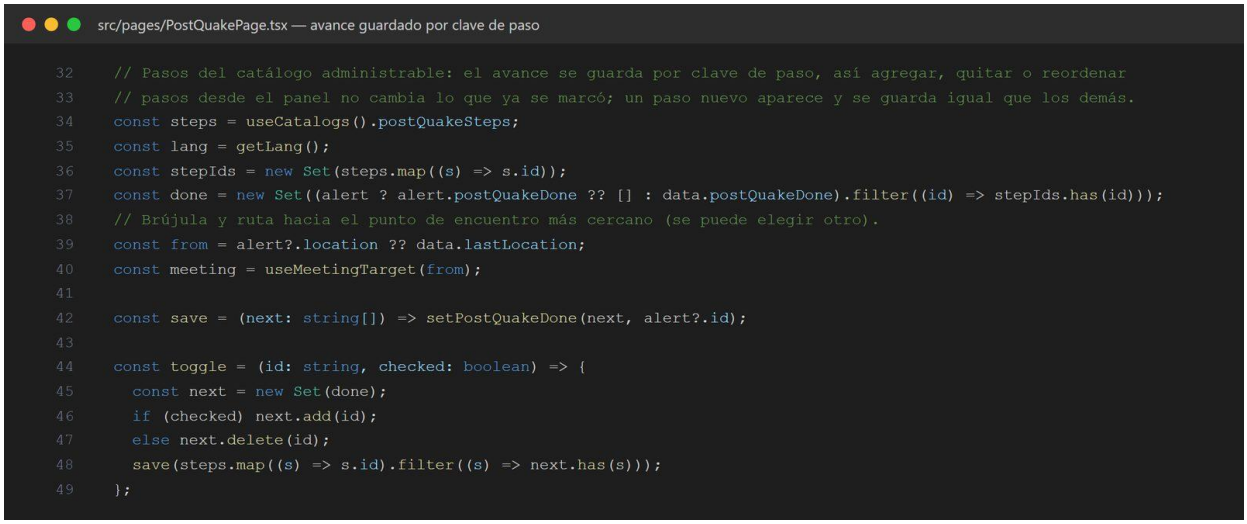
```ts
/** Valores por defecto: salen de los textos de la app (es/en). También se usan para sembrar la base de datos. */
export function defaultCatalogs(): Catalogs {
  const es = messages;
  const ev = (k: 'sismo' | 'emergencia_general' | 'prueba') => ({ es: es.labels.es.eventType[k], en: es.labels.en.eventType[k] });
  const rel = es.contacts.es.relationships;
  const relEn = es.contacts.en.relationships;
  const g = es.guide.es;
  const gEn = es.guide.en;
  return {
    version: 'local',
    eventTypes: [
      { id: 'sismo', label: ev('sismo'), icon: 'activity' },
      { id: 'emergencia_general', label: ev('emergencia_general'), icon: 'alert-octagon' },
      { id: 'prueba', label: ev('prueba'), icon: 'flask' },
    ],
    relationships: Object.keys(rel).map((id) => ({ id, label: { es: rel[id], en: relEn[id] ?? rel[id] } })),
    kitItems: pair(es.services.es.kit.defaultItems, es.services.en.kit.defaultItems, 'kit'),
    postQuakeSteps: pair(es.features.es.postQuake.steps, es.features.en.postQuake.steps, 'paso'),
    bloodTypes: ['O+', 'O-', 'A+', 'A-', 'B+', 'B-', 'AB+', 'AB-'].map((id) => ({ id, label: { es: id, en: id } }))
      .concat([{ id: BLOOD_UNKNOWN, label: { es: es.medical.es.bloodUnknown, en: es.medical.en.bloodUnknown } }]),
    guide: {
      intro: Object.fromEntries(MOMENTS.map((m) => [m, { es: g.sections[m].intro, en: gEn.sections[m].intro }])) as Record<Moment, L10n>,
      disclaimer: {
        es: `${g.disclaimerPre}${g.disclaimerStrong}${g.disclaimerPost}`,
        en: `${gEn.disclaimerPre}${gEn.disclaimerStrong}${gEn.disclaimerPost}`,
      },
      cards: MOMENTS.flatMap((m) => g.sections[m].cards.map((c, i): GuideCard => ({
        id: `${m}_${i + 1}`,
        moment: m,
        title: { es: c.title, en: gEn.sections[m].cards[i]?.title ?? c.title },
        steps: { es: c.steps, en: gEn.sections[m].cards[i]?.steps ?? c.steps },
        icon: GUIDE_ICONS[m][i] ?? 'clipboard-list',
      }))),
    },
    params: { emergencyNumber: '123', maxMeetingPoints: 5, lowBatteryOptions: [0, 10, 15, 20, 30], drillDelayMinS: 3, drillDelayMaxS: 10, alertLocationRefreshMin: 2 },
  };
}
```

**Descripción.** src/services/catalogService.ts: valores por defecto de los catálogos (tipos de evento, parentescos, kit, pasos posteriores al sismo, tipos de sangre, guía y parámetros) que

se usan sin conexión o si el servidor no responde.

Fuente: elaboración propia (captura del 04/11/2026).

**Figura 116**
*Código de la app (v3): después del sismo por clave*




**Descripción.** src/pages/PostQuakePage.tsx: la lista «Después del sismo» guarda el avance por clave de paso, de modo que agregar, quitar o reordenar pasos desde el panel no cambia lo que la persona ya marcó.

Fuente: elaboración propia (captura del 04/11/2026).

**Figura 117**
*Código de la app (v3): número emergencia configurable*

src/i18n/index.ts — número de emergencia configurable en todos los textos

```ts
 * Número de emergencia configurable (lo fija el catálogo descargado del servidor). Los textos lo llevan como
 * `{emergencyNumber}` y se reemplaza al construir los textos del idioma.
 */
let emergencyNumber = '123';

export function setEmergencyNumber(value: string): void {
  const next = value.trim();
  if (!next || next === emergencyNumber) return;
  emergencyNumber = next;
  for (const key of Object.keys(cache) as Lang[]) delete cache[key];
}

export function getEmergencyNumber(): string {
  return emergencyNumber;
}

/** Reemplaza los marcadores (`{emergencyNumber}`) de un texto. */
export function fillTokens(text: string): string {
  return text.split('{emergencyNumber}').join(emergencyNumber);
}

/** Aplica fillTokens a todos los textos (también a los que devuelven las funciones de traducción). */
function withTokens<T>(value: T): T {
  if (typeof value === 'string') return fillTokens(value) as T;
  if (typeof value === 'function') {
    const fn = value as unknown as (...args: unknown[]) => unknown;
    return ((...args: unknown[]) => withTokens(fn(...args))) as T;
  }
  if (Array.isArray(value)) return value.map(withTokens) as T;
  if (value && typeof value === 'object') {
    return Object.fromEntries(Object.entries(value).map(([k, v]) => [k, withTokens(v)])) as T;
  }
  return value;
}
```

**Descripción.** src/i18n/index.ts: número de emergencia configurable; el valor que llega en los catálogos reemplaza el marcador {emergencyNumber} en todos los textos de la app.

Fuente: elaboración propia (captura del 04/11/2026).

**Figura 118**
*Código de la app (v3): pantalla de carga y loader*

src/components/AppLoader.tsx — pantalla de carga (nunca tapa el SOS) y loader de solicitudes

```tsx
/** Pantalla de carga al abrir la app. `skip`: la app se abrió para una alerta. */
export function SplashScreen({ skip }: { skip: boolean }) {
  const t = useT().common.loader;
  const [state, setState] = useState<'visible' | 'saliendo' | 'oculta'>(skip ? 'oculta' : 'visible');

  useEffect(() => {
    if (skip) return;
    let done = false;
    const start = Date.now();
    const finish = () => {
      if (done) return;
      done = true;
      window.setTimeout(() => setState('saliendo'), Math.max(0, SPLASH_MIN_MS - (Date.now() - start)));
    };
    const max = window.setTimeout(finish, SPLASH_MAX_MS);
    void ensureFirstRefresh().finally(finish);
    return () => window.clearTimeout(max);
  }, [skip]);

  useEffect(() => {
    if (state !== 'saliendo') return;
    const timer = window.setTimeout(() => setState('oculta'), 250);
    return () => window.clearTimeout(timer);
  }, [state]);

  if (state === 'oculta') return null;
  return (
    <div className={`app-loader app-loader--splash${state === 'saliendo' ? ' app-loader--out' : ''}`} role="status" aria-live="polite">
      <LoaderMark />
      <strong className="app-loader__brand">SISMO</strong>
      <span className="app-loader__text">{t.loading}</span>
    </div>
  );
}
```

**Descripción.** src/components/AppLoader.tsx: pantalla de carga al abrir la app con duración mínima y máxima, que nunca tapa una alerta activa.

Fuente: elaboración propia (captura del 04/11/2026).

**Figura 119**
*Código de la app (v3): aviso sin conexión*

src/components/SyncStatusBanner.tsx — «sus datos se guardan en el teléfono»

```tsx
/**
 * Aviso en todas las pantallas cuando los datos no se pueden subir:
 * - sin internet: «Sus datos se guardan en el teléfono y se subirán apenas haya conexión»;
 * - con internet pero sin respuesta del servidor: mismo mensaje, indicando que es el servidor.
 * Nada se pierde: los cambios esperan en la cola persistente (syncService). Sin servidor configurado no se muestra.
 */
export function SyncStatusBanner() {
  const t = useT().components.sync;
  const online = useOnline();
  const status = useSyncExternalStore(subscribeSyncStatus, getSyncStatus);
  if (!syncEnabled()) return null;
  const [pendingText, reachable] = status.split('|');
  const pending = Number(pendingText);
  const serverDown = online && reachable === '0';
  if (online && !serverDown) return null;
  return (
    <p className="callout callout--warning callout--row small sync-banner" role="status" aria-live="polite">
      <CloudOff size={18} aria-hidden="true" />
      <span>
        <strong>{online ? t.serverTitle : t.offlineTitle}</strong> {online ? t.server : t.offline}
        {pending > 0 && <> {t.pending(pending)}</>}
      </span>
    </p>
  );
}
```

**Descripción.** src/components/SyncStatusBanner.tsx: aviso que aparece en todas las pantallas cuando no hay internet o el servidor no responde, con el número de cambios pendientes de subir.

Fuente: elaboración propia (captura del 04/11/2026).

**Figura 120**
*Código de la app (v3): alerta ayuda fija*

src/components/HelpStatusBanner.tsx — aviso fijo del centro de monitoreo

```tsx
  // Clave de la actualización: si cambia el estado, el mensaje o la hora, es una novedad y se vuelve a mostrar.
  const key = help ? `${help.alertId}|${help.status}|${help.at}|${help.note}` : '';
  const [dismissed, setDismissed] = useState<string>(() => readDismissed());
  const close = () => {
    setDismissed(key);
    writeDismissed(key);
  };

  if (!help) return null;
  const tone = help.status === 'resuelta' ? 'help-toast--ok' : help.status === 'ayuda_enviada' ? 'help-toast--sent' : '';
  const toast = key !== dismissed
    ? createPortal(
      <div className={`help-toast ${tone}`} role="alert" aria-live="assertive" onClick={close}>
        <LifeBuoy size={24} aria-hidden="true" className="help-toast__icon" />
        <div className="help-toast__body">
          <small className="help-toast__title">{t.title}</small>
          <strong>{t[help.status]}</strong>
          {help.note && <p>{t.note}: {help.note}</p>}
          <small className="help-toast__hint">{t.tapToClose}</small>
        </div>
        <button type="button" className="help-toast__close" aria-label={t.dismiss} onClick={(e) => { e.stopPropagation(); close(); }}>
          <X size={20} aria-hidden="true" />
        </button>
      </div>,
      document.body,
    )
    : null;

  return (
    <>
      {toast}
    <div className={`callout callout--row ${help.status === 'resuelta' ? 'callout--info' : 'callout--warning'}`} role="status" aria-live="polite">
      <LifeBuoy size={20} aria-hidden="true" />
      <div>
        <strong>{t[help.status]}</strong>
        {help.note && <p className="small">{t.note}: {help.note}</p>}
      </div>
    </div>
```

**Descripción.** src/components/HelpStatusBanner.tsx: aviso fijo con el estado y el mensaje del centro de monitoreo en la pantalla de alerta; solo se vuelve a mostrar si el estado o el mensaje cambian.

Fuente: elaboración propia (captura del 04/11/2026).

**Figura 121**
*Código de la app (v3): modo demo solo desarrollo*

src/utils/env.ts y AppStateProvider.tsx — modo demostración solo en desarrollo

```tsx
/**
 * Modo de demostración (datos ficticios, entrega simulada, accesos para pruebas) disponible SOLO en desarrollo
 * (`npm run dev`). En producción (`npm run build` y el APK) no aparece en ninguna pantalla y queda siempre apagado.
 * Se puede forzar en una compilación de pruebas con VITE_DEMO=true.
 */
export const DEMO_ENABLED: boolean = import.meta.env.DEV || import.meta.env.VITE_DEMO === 'true';

// src/context/AppStateProvider.tsx
/** En producción no existe el modo de demostración: se apaga y se quitan los datos ficticios que hubiera. */
function loadInitialData(): AppData {
  const loaded = loadData();
  if (DEMO_ENABLED || (!loaded.settings.demoMode && !loaded.contacts.some((c) => c.isSeed) && !loaded.alerts.some((a) => a.isSeed))) return loaded;
  const clean = withoutDemoSeeds(loaded);
  return { ...clean, settings: { ...clean.settings, demoMode: false } };
}
```

**Descripción.** src/utils/env.ts y AppStateProvider.tsx: el modo de demostración solo existe en desarrollo; en la versión de producción se apaga y se quitan los datos ficticios que hubiera.

Fuente: elaboración propia (captura del 04/11/2026).

**Figura 122**
*Código de la app (v3): backend en la app*

src/context/AppStateProvider.tsx — sincronización y catálogos al abrir

```
  // Sincronización con el backend: cada cambio se encola (cola persistente) y se sube apenas haya red.
  const prevDataRef = useRef(data);
  const dataRef = useRef(data);
  dataRef.current = data;
  // Tras restaurar desde el servidor no se vuelve a subir lo que se acaba de descargar.
  const skipSyncRef = useRef(false);
  useEffect(() => {
    if (skipSyncRef.current) skipSyncRef.current = false;
    else queueChanges(prevDataRef.current, data);
    prevDataRef.current = data;
  }, [data]);
  useEffect(() => startSync(() => dataRef.current), []);

  // Catálogos administrables: se descargan al abrir y al recuperar la red; sin red se usa la última copia.
  // Suscribirse aquí vuelve a dibujar toda la app cuando llegan catálogos nuevos.
  const catalogs = useCatalogs();
  useEffect(() => {
    void ensureFirstRefresh();
    const onOnline = () => void refreshCatalogs();
    window.addEventListener('online', onOnline);
    return () => window.removeEventListener('online', onOnline);
  }, []);
```

**Descripción.** src/context/AppStateProvider.tsx: cada cambio del estado se encola para el servidor y los catálogos se descargan al abrir la app y al recuperar la conexión.

Fuente: elaboración propia (captura del 04/11/2026).

## G.3 Backend, versión 3 (04/11/2026)

**Figura 123**
*Código del backend (v3): tablas sismo*

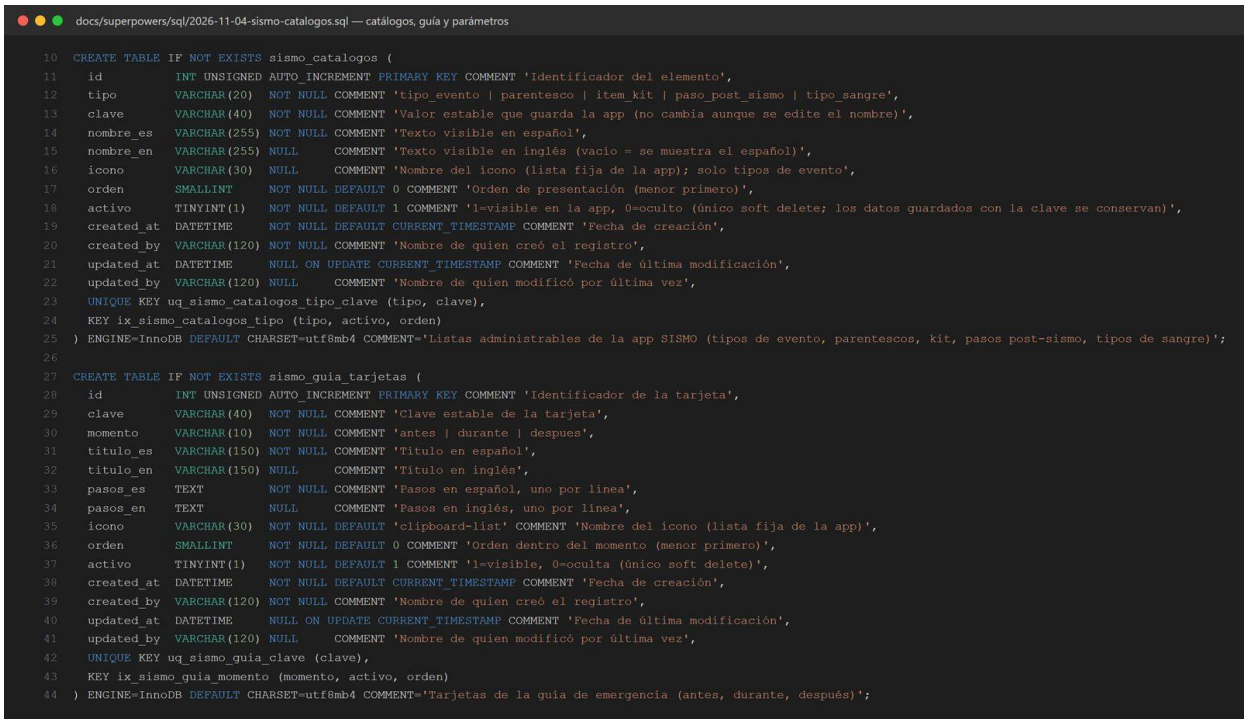
docs/superpowers/sql/2026-11-04-sismo.sql — usuarios y alertas (convención CCS)

```
CREATE TABLE IF NOT EXISTS sismo_usuarios (
  id                       INT UNSIGNED AUTO_INCREMENT PRIMARY KEY COMMENT 'Identificador del usuario (un dispositivo)',
  uuid_dispositivo         CHAR(36)     NOT NULL COMMENT 'UUID v4 público del dispositivo, generado por el servidor al registrarse',
  hash_token               CHAR(64)     NOT NULL COMMENT 'SHA-256 del token de acceso (el token nunca se guarda en claro)',
  nombre                   VARCHAR(120) NULL     COMMENT 'Nombre que la persona escribió en la app (Configuración)',
  plataforma               VARCHAR(20)  NOT NULL DEFAULT 'web' COMMENT 'android | ios | web',
  fecha_ultima_sincronizacion DATETIME  NULL     COMMENT 'Última vez que la app envió datos',
  hash_codigo_recuperacion CHAR(64)     NULL     COMMENT 'SHA-256 del código de recuperación vigente (un solo uso); NULL = sin código',
  fecha_expiracion_codigo  DATETIME     NULL     COMMENT 'Vencimiento (UTC) del código de recuperación; NULL = no vence (código creado por la persona)',
  activo                   TINYINT(1)   NOT NULL DEFAULT 1 COMMENT '1=activo, 0=revocado (único soft delete)',
  created_at               DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT 'Fecha de creación',
  created_by               VARCHAR(120) NOT NULL COMMENT 'Nombre de quien creó el registro',
  updated_at               DATETIME     NULL ON UPDATE CURRENT_TIMESTAMP COMMENT 'Fecha de última modificación',
  updated_by               VARCHAR(120) NULL     COMMENT 'Nombre de quien modificó por última vez',
  UNIQUE KEY uq_sismo_usuarios_uuid (uuid_dispositivo),
  UNIQUE KEY uq_sismo_usuarios_token (hash_token),
  UNIQUE KEY uq_sismo_usuarios_codigo (hash_codigo_recuperacion)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='Usuarios (dispositivos) de la app SISMO';
```

**Descripción.** Script SQL de la tabla sismo_usuarios: un registro por dispositivo, con el hash SHA-256 del token y del código de recuperación, columnas comentadas, auditoría completa y activo como único borrado lógico.

Fuente: elaboración propia (captura del 04/11/2026).

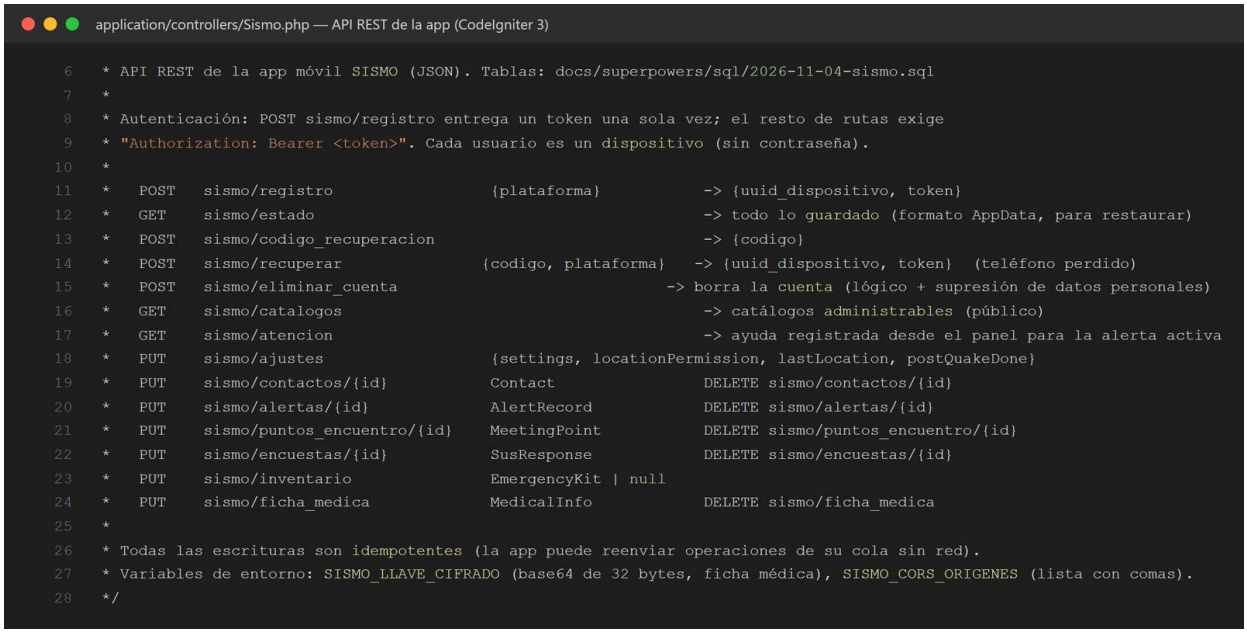
**Figura 124**
*Código del backend (v3): tablas catálogos*




**Descripción.** Script SQL de las tablas de catálogos y de la guía de emergencia, con textos en español e inglés, orden, visibilidad e índices únicos por clave.

Fuente: elaboración propia (captura del 04/11/2026).

**Figura 125**
*Código del backend (v3): aPI rutas*




**Descripción.** application/controllers/Sismo.php: documentación de la API REST de la app; lista

cada ruta (registro, estado, recuperación, eliminación, catálogos, atención y los PUT y DELETE por recurso) y explica la autenticación con token por dispositivo.

Fuente: elaboración propia (captura del 04/11/2026).

**Figura 126**
*Código del backend (v3): aPI CORS y validación*

application/controllers/Sismo.php — validación central, CORS y logs JSON

```php
 * Valida método, autenticación y cuerpo; ejecuta la acción y responde JSON. Centraliza errores y logs.
 *
 * @param string[] $metodos   Métodos HTTP permitidos (minúsculas)
 * @param bool     $con_token Si exige token
 * @param callable $accion    fn(array|null $cuerpo): [int $codigo, array $respuesta]
 */
private function ejecutar(array $metodos, $con_token, callable $accion) {
        $metodo = $this->input->method();
        if (!in_array($metodo, $metodos, true)) {
                return $this->responder(405, ['error' => 'metodo_no_permitido']);
        }
        if ($con_token) {
                $this->usuario = $this->Sismo_model->usuario_por_token($this->token());
                if (!$this->usuario) {
                        return $this->responder(401, ['error' => 'token_invalido']);
                }
        }
        $cuerpo = null;
        if (in_array($metodo, ['post', 'put'], true)) {
                $crudo = file_get_contents('php://input', false, null, 0, self::MAX_CUERPO + 1);
                if (strlen($crudo) > self::MAX_CUERPO) {
                        return $this->responder(413, ['error' => 'cuerpo_muy_grande']);
                }
                $cuerpo = json_decode($crudo, true);
                if ($crudo !== '' && $crudo !== 'null' && $cuerpo === null) {
                        return $this->responder(400, ['error' => 'json_invalido']);
                }
        }
        try {
                list($codigo, $respuesta) = $accion($cuerpo);
        } catch (InvalidArgumentException $e) {
                return $this->responder(422, ['error' => 'dato_invalido', 'campo' => $e->getMessage()]);
        } catch (Exception $e) {
                $this->log('error', 500, ['detalle' => get_class($e)]);
                return $this->responder(500, ['error' => 'error_interno']);
        }
        if ($this->usuario && $codigo < 300) {
                $this->Sismo_model->marcar_sincronizacion($this->usuario->id);
        }
        return $this->responder($codigo, $respuesta);
}

private function responder($codigo, array $datos) {
        $this->log('peticion', $codigo);
        $this->output
                ->set_status_header($codigo)
                ->set_header('X-Execution-Id: ' . $this->execution_id)
                ->set_content_type('application/json', 'utf-8')
                ->set_output(json_encode($datos, JSON_UNESCAPED_UNICODE));
}

/** Log estructurado JSON a stderr (sin datos personales). */
private function log($evento, $codigo, array $extra = []) {
        $linea = json_encode([
                'execution_id' => $this->execution_id,
                'servicio'     => 'sismo',
                'evento'       => $evento,
                'ruta'         => $this->router->fetch_method(),
                'metodo'       => $this->input->method(),
                'codigo'       => $codigo,
                'id_usuario'   => $this->usuario ? (int) $this->usuario->id : null,
                'fecha'        => gmdate('c'),
        ] + $extra);
```

```
406         file_put_contents('php://stderr', $linea . PHP_EOL);
407 }
408
409 /** Cabeceras CORS solo para los orígenes permitidos de la app (vacío si el origen no está en la lista). */
410 private static function cabeceras_cors() {
411         $origen     = isset($_SERVER['HTTP_ORIGIN']) ? $_SERVER['HTTP_ORIGIN'] : '';
412         $permitidos = array_map('trim', explode(',', getenv('SISMO_CORS_ORIGENES') ?: self::ORIGENES_POR_DEFECTO));
413         // En desarrollo (backend en localhost) se acepta cualquier puerto local: Vite cambia de puerto si el suyo está ocupado.
414         $desarrollo = defined('IP_SERVER') && strpos(IP_SERVER, '//localhost') !== false
415                 && preg_match('#^http://(localhost|127\.0\.0\.1)(:\d{1,5})?$#', $origen);
416         if ($origen === '' || (!$desarrollo && !in_array($origen, $permitidos, true))) {
417                 return [];
418         }
419         return [
420                 'Access-Control-Allow-Origin: ' . $origen,
421                 'Vary: Origin',
422                 'Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS',
423                 'Access-Control-Allow-Headers: Content-Type, Authorization',
424                 'Access-Control-Expose-Headers: X-Execution-Id',
425                 'Access-Control-Max-Age: 600',
426         ];
427 }
```

**Descripción.** Sismo.php, método ejecutar: valida el método HTTP, el token y el cuerpo JSON (máximo 64 KB), ejecuta la acción y centraliza los errores (405, 401, 413, 400, 422 y 500) con log en JSON.

Fuente: elaboración propia (captura del 04/11/2026).

**Figura 127**
*Código del backend (v3): escritura idempotente*

```
application/models/Sismo_model.php — escrituras idempotentes por UUID de la app

102  * Crea o actualiza un registro identificado por id_cliente (contactos, alertas, puntos, encuestas).
103  *
104  * @param string $recurso Clave de TABLAS_CLIENTE
105  * @param object $usuario
106  * @param string $id_cliente UUID de la app (ya validado)
107  * @param array  $columnas Columnas ya validadas
108  * @return bool
109  */
110 public function guardar_por_cliente($recurso, $usuario, $id_cliente, array $columnas) {
111         $tabla     = self::TABLAS_CLIENTE[$recurso];
112         $autor     = $this->autor($usuario);
113         $existente = $this->db->select('id')->where('id_usuario', $usuario->id)->where('id_cliente', $id_cliente)
114                 ->get($tabla)->row();
115         if ($existente) {
116                 return $this->db->where('id', $existente->id)->update($tabla, $columnas + ['activo' => 1, 'updated_by' => $autor]);
117         }
118         return $this->db->insert($tabla, $columnas + ['id_usuario' => $usuario->id, 'id_cliente' => $id_cliente, 'created_by' => $autor]);
119 }
120
121 /**
122  * Soft delete de un registro por id_cliente. Idempotente (borrar algo que no existe no es error).
123  */
124 public function desactivar_por_cliente($recurso, $usuario, $id_cliente) {
125         return $this->db->where('id_usuario', $usuario->id)->where('id_cliente', $id_cliente)
126                 ->update(self::TABLAS_CLIENTE[$recurso], ['activo' => 0, 'updated_by' => $this->autor($usuario)]);
127 }
128
129 /**
```

**Descripción.** application/models/Sismo_model.php: escrituras idempotentes; si el registro con ese id_cliente ya existe se actualiza y si no se inserta, y el borrado es lógico (activo = 0).

Fuente: elaboración propia (captura del 04/11/2026).

**Figura 128**
*Código del backend (v3): ficha médica cifrada*

application/models/Sismo_model.php — ficha médica cifrada AES-256-GCM (llave en variable de entorno)

```php
/** Llave AES-256 desde SISMO_LLAVE_CIFRADO (base64 de 32 bytes). Nunca en el código. */
private function llave() {
        $base64 = getenv('SISMO_LLAVE_CIFRADO');
        $llave  = $base64 ? base64_decode($base64, true) : false;
        return ($llave !== false && strlen($llave) === 32) ? $llave : null;
}

/** AES-256-GCM. Devuelve base64(iv[12] | tag[16] | texto) o null si no hay llave. */
private function cifrar($texto) {
        $llave = $this->llave();
        if ($llave === null) {
                return null;
        }
        $iv      = random_bytes(12);
        $tag     = '';
        $cifrado = openssl_encrypt($texto, 'aes-256-gcm', $llave, OPENSSL_RAW_DATA, $iv, $tag);
        return $cifrado === false ? null : base64_encode($iv . $tag . $cifrado);
}

/** Inverso de cifrar(); null si la llave no coincide o el dato está dañado. */
private function descifrar($base64) {
        $llave = $this->llave();
        $crudo = base64_decode((string) $base64, true);
        if ($llave === null || $crudo === false || strlen($crudo) < 29) {
                return null;
        }
        $texto = openssl_decrypt(substr($crudo, 28), 'aes-256-gcm', $llave, OPENSSL_RAW_DATA, substr($crudo, 0, 12), substr($crudo, 12, 16));
        return $texto === false ? null : $texto;
}
}
```

**Descripción.** Sismo_model.php: cifrado de la ficha médica con AES-256-GCM; la llave se lee de una variable de entorno y nunca está en el código ni en la base de datos.

Fuente: elaboración propia (captura del 04/11/2026).

**Figura 129**
*Código del backend (v3): recuperación de cuenta*

application/models/Sismo_model.php — código de recuperación y token nuevo

```php
// ------------------------------------------------------------------ Recuperación (teléfono perdido o cambiado)

/**
 * Genera un código de recuperación nuevo (reemplaza al anterior). Se entrega una sola vez.
 *
 * @param int      $id_usuario
 * @param int|null $horas Vigencia; null = no vence (código que la persona guarda)
 * @param string   $autor Nombre para updated_by
 * @return string Código con formato XXXX-XXXX-XXXX
 */
public function generar_codigo_recuperacion($id_usuario, $horas, $autor) {
        $alfabeto = 'ABCDEFGHJKMNPQRSTUVWXYZ23456789';
        $codigo   = '';
        for ($i = 0; $i < 12; $i++) {
                $codigo .= $alfabeto[random_int(0, strlen($alfabeto) - 1)];
        }
        $this->db->where('id', (int) $id_usuario)->update('sismo_usuarios', [
                'hash_codigo_recuperacion' => hash('sha256', $codigo),
                'fecha_expiracion_codigo'  => $horas === null ? null : gmdate('Y-m-d H:i:s', time() + 3600 * (int) $horas),
                'updated_by'               => $autor,
        ]);
        return implode('-', str_split($codigo, 4));
}

/**
 * Canjea un código de recuperación: entrega un token NUEVO para el mismo usuario. El token del teléfono
 * perdido deja de funcionar y el código queda usado.
 *
 * @param string $codigo Como lo escribió la persona (se ignoran guiones, espacios y minúsculas)
 * @param string $plataforma
 * @return array|null ['uuid_dispositivo', 'token'] o null si el código no es válido o venció
 */
public function recuperar($codigo, $plataforma) {
        $limpio = strtoupper(preg_replace('/[^A-Za-z0-9]/', '', (string) $codigo));
        if (strlen($limpio) !== 12) {
                return null;
        }
        $usuario = $this->db->where('hash_codigo_recuperacion', hash('sha256', $limpio))->where('activo', 1)
                ->get('sismo_usuarios')->row();
        if (!$usuario || ($usuario->fecha_expiracion_codigo && strtotime($usuario->fecha_expiracion_codigo . ' UTC') < time())) {
                return null;
        }
        $token = bin2hex(random_bytes(32));
        $this->db->where('id', $usuario->id)->update('sismo_usuarios', [
                'hash_token'               => hash('sha256', $token),
                'hash_codigo_recuperacion' => null,
                'fecha_expiracion_codigo'  => null,
                'plataforma'               => $plataforma,
                'updated_by'               => $this->autor($usuario),
        ]);
        return ['uuid_dispositivo' => $usuario->uuid_dispositivo, 'token' => $token];
}
```

**Descripción.** Sismo_model.php: genera el código de recuperación de 12 caracteres sin letras ambiguas, guarda solo su SHA-256 y, al usarlo, entrega un token nuevo e invalida el del teléfono anterior.

Fuente: elaboración propia (captura del 04/11/2026).

**Figura 130**
*Código del backend (v3): eliminar cuenta*

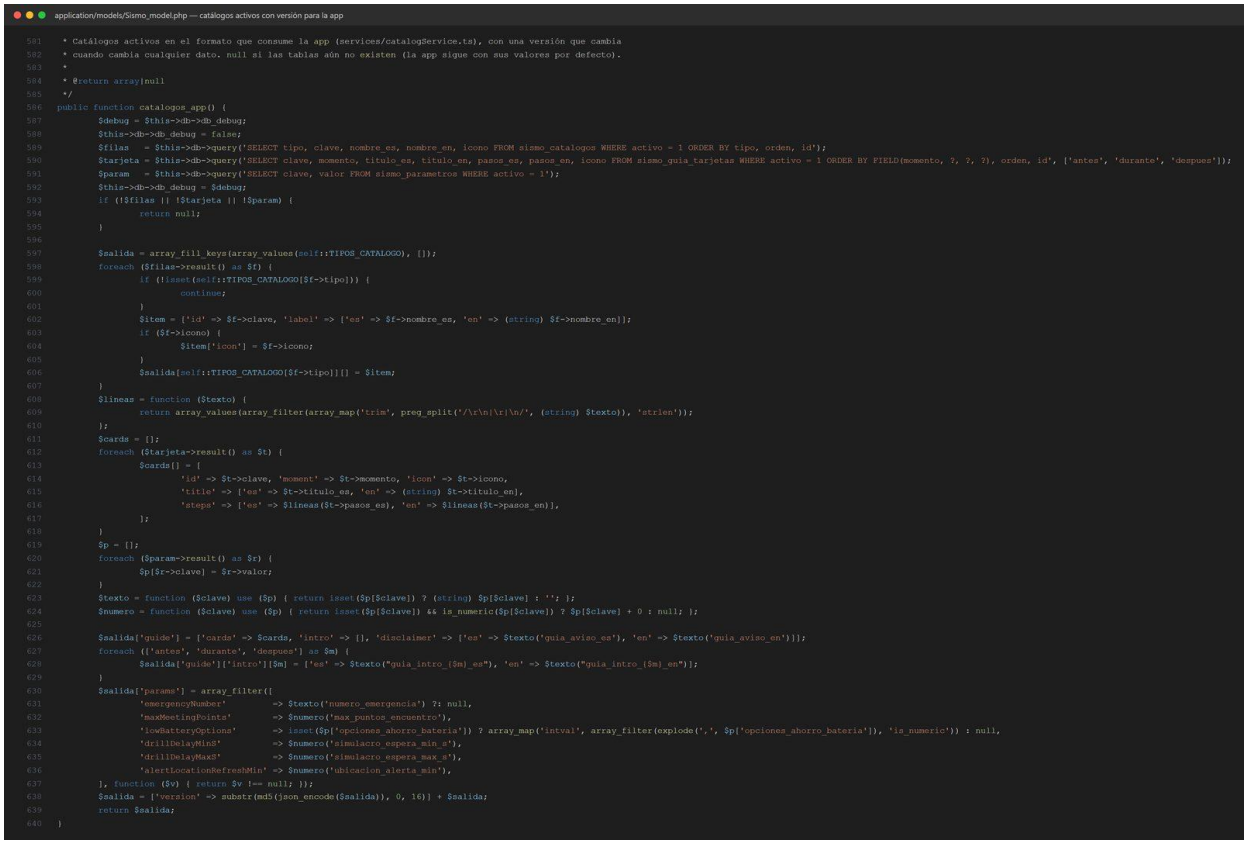
application/models/Sismo_model.php — eliminar cuenta: borrado lógico + supresión de datos

```php
// ------------------------------------------------------------------ Eliminar cuenta (desde la app)

/**
 * Elimina la cuenta en cascada. Convención CCS: borrado lógico (activo = 0) en todas las tablas de la persona,
 * y además supresión de datos personales y de salud (Ley 1581): se vacían nombre, contactos, ficha médica,
 * ubicaciones, puntos de encuentro, notas y mensajes. Se revocan token y código de recuperación, así el teléfono
 * puede volver a registrarse desde cero. Quedan solo métricas anónimas (tiempos, SUS) para la tesis.
 *
 * @param object $usuario
 * @return bool
 */
public function eliminar_cuenta($usuario) {
        $id    = (int) $usuario->id;
        $autor = self::USUARIO_SISTEMA;
        $this->db->trans_start();

        $this->db->where('id_usuario', $id)->update('sismo_contactos_usuarios', [
                'nombre' => '(eliminado)', 'parentesco' => '', 'telefono' => '', 'correo' => null, 'activo' => 0, 'updated_by' => $autor,
        ]);
        $this->db->where('id_usuario', $id)->update('sismo_fichas_medicas_usuarios', [
                'datos_cifrados' => '', 'mostrar_en_alerta' => 0, 'activo' => 0, 'updated_by' => $autor,
        ]);
        $this->db->where('id_usuario', $id)->update('sismo_puntos_encuentro_usuarios', [
                'etiqueta' => '(eliminado)', 'latitud' => 0, 'longitud' => 0, 'activo' => 0, 'updated_by' => $autor,
        ]);
        $this->db->where('id_usuario', $id)->update('sismo_ajustes_usuarios', [
                'latitud_ultima' => null, 'longitud_ultima' => null, 'precision_ultima_m' => null, 'origen_ultima_ubicacion' => null,
                'fecha_ultima_ubicacion' => null, 'activo' => 0, 'updated_by' => $autor,
        ]);
        $this->db->where('id_usuario', $id)->update('sismo_alertas_usuarios', [
                'latitud' => null, 'longitud' => null, 'precision_m' => null, 'origen_ubicacion' => null, 'fecha_ubicacion' => null,
                'nota' => null, 'activo' => 0, 'updated_by' => $autor,
        ]);
        $this->db->where('id_usuario', $id)->update('sismo_encuestas_sus_usuarios', ['activo' => 0, 'updated_by' => $autor]);
        $this->db->where('id_usuario', $id)->update('sismo_inventarios_usuarios', ['activo' => 0, 'updated_by' => $autor]);
        // Mensajes de la bitácora de ayuda (pueden contener datos de la persona).
        $this->db->query(
                'UPDATE sismo_atenciones_alertas t JOIN sismo_alertas_usuarios a ON a.id = t.id_alerta
                 SET t.nota = NULL, t.activo = 0, t.updated_by = ? WHERE a.id_usuario = ?',
                [$autor, $id]
        );
        // Usuario: inactivo, sin nombre y con token y código revocados (el token anterior deja de servir).
        $this->db->where('id', $id)->update('sismo_usuarios', [
                'nombre' => null, 'hash_token' => hash('sha256', 'revocado:' . bin2hex(random_bytes(32))),
                'hash_codigo_recuperacion' => null, 'fecha_expiracion_codigo' => null, 'activo' => 0, 'updated_by' => $autor,
        ]);

        $this->db->trans_complete();
        return $this->db->trans_status();
}
```

**Descripción.** Sismo_model.php, eliminar_cuenta: en una transacción marca como inactivos todos los registros de la persona y vacía sus datos personales y de salud (Ley 1581); solo quedan métricas anónimas.

Fuente: elaboración propia (captura del 04/11/2026).

**Figura 131**
*Código del backend (v3): catálogos para la app*




```
 * Catálogos activos en el formato que consume la app (services/catalogService.ts), con una versión que cambia
 * cuando cambia cualquier dato. null si las tablas aún no existen (la app sigue con sus valores por defecto).
 *
 * @return array|null
 */
public function catalogos_app() {
        $debug = $this->db->db_debug;
        $this->db->db_debug = false;
        $filas   = $this->db->query('SELECT tipo, clave, nombre_es, nombre_en, icono FROM sismo_catalogos WHERE activo = 1 ORDER BY tipo, orden, id');
        $tarjeta = $this->db->query('SELECT clave, momento, titulo_es, titulo_en, pasos_es, pasos_en, icono FROM sismo_guia_tarjetas WHERE activo = 1 ORDER BY FIELD(momento, ?, ?, ?), orden, id', ['antes', 'durante', 'despues']);
        $param   = $this->db->query('SELECT clave, valor FROM sismo_parametros WHERE activo = 1');
        $this->db->db_debug = $debug;
        if (!$filas || !$tarjeta || !$param) {
                return null;
        }

        $salida = array_fill_keys(array_values(self::TIPOS_CATALOGO), []);
        foreach ($filas->result() as $f) {
                if (!isset(self::TIPOS_CATALOGO[$f->tipo])) {
                        continue;
                }
                $item = ['id' => $f->clave, 'label' => ['es' => $f->nombre_es, 'en' => (string) $f->nombre_en]];
                if ($f->icono) {
                        $item['icon'] = $f->icono;
                }
                $salida[self::TIPOS_CATALOGO[$f->tipo]][] = $item;
        }
        $lineas = function ($texto) {
                return array_values(array_filter(array_map('trim', preg_split('/\r\n|\r|\n/', (string) $texto)), 'strlen'));
        };
        $cards = [];
        foreach ($tarjeta->result() as $t) {
                $cards[] = [
                        'id' => $t->clave, 'moment' => $t->momento, 'icon' => $t->icono,
                        'title' => ['es' => $t->titulo_es, 'en' => (string) $t->titulo_en],
                        'steps' => ['es' => $lineas($t->pasos_es), 'en' => $lineas($t->pasos_en)],
                ];
        }
        $p = [];
        foreach ($param->result() as $r) {
                $p[$r->clave] = $r->valor;
        }
        $texto = function ($clave) use ($p) { return isset($p[$clave]) ? (string) $p[$clave] : ''; };
        $numero = function ($clave) use ($p) { return isset($p[$clave]) && is_numeric($p[$clave]) ? $p[$clave] + 0 : null; };

        $salida['guide'] = ['cards' => $cards, 'intro' => [], 'disclaimer' => ['es' => $texto('guia_aviso_es'), 'en' => $texto('guia_aviso_en')]];
        foreach (['antes', 'durante', 'despues'] as $m) {
                $salida['guide']['intro'][$m] = ['es' => $texto("guia_intro_{$m}_es"), 'en' => $texto("guia_intro_{$m}_en")];
        }
        $salida['params'] = array_filter([
                'emergencyNumber'         => $texto('numero_emergencia') ?: null,
                'maxMeetingPoints'        => $numero('max_puntos_encuentro'),
                'lowBatteryOptions'       => isset($p['opciones_ahorro_bateria']) ? array_map('intval', array_filter(explode(',', $p['opciones_ahorro_bateria']), 'is_numeric')) : null,
                'drillDelayMinS'          => $numero('simulacro_espera_min_s'),
                'drillDelayMaxS'          => $numero('simulacro_espera_max_s'),
                'alertLocationRefreshMin' => $numero('ubicacion_alerta_min'),
        ], function ($v) { return $v !== null; });
        $salida = ['version' => substr(md5(json_encode($salida)), 0, 16)] + $salida;
        return $salida;
}
```

**Descripción.** Sismo_model.php: arma los catálogos activos en el formato que consume la app, con versión y textos en ambos idiomas.

Fuente: elaboración propia (captura del 04/11/2026).

**Figura 132**
*Código del backend (v3): sismos recientes USGS*

application/models/Sismo_model.php — sismos de 24 h y 7 días (USGS, caché 5 min)

```php
/** Recuadro de Colombia y zona de influencia (mismo criterio que la app). */
const ZONA_SISMOS = ['minlatitude' => -4.3, 'maxlatitude' => 13.5, 'minlongitude' => -79.1, 'maxlongitude' => -66.8];

/**
 * Sismos de magnitud >= 2.5 del último día (24 h) o de los últimos 7 días en la zona (los mismos periodos de la app), desde el catálogo público del USGS
 * (FDSN). Se guarda en caché 5 minutos para no consultar al USGS en cada actualización del panel.
 *
 * @param int $dias 1 (últimas 24 horas) o 7
 * @return array|null Lista ordenada del más reciente al más antiguo, o null si el USGS no responde
 */
public function sismos_recientes($dias) {
        $dias    = (int) $dias === 1 ? 1 : 7;
        $archivo = APPPATH . 'cache/sismo_usgs_' . $dias . '.json';
        if (is_file($archivo) && filemtime($archivo) > time() - 300) {
                $cache = json_decode((string) file_get_contents($archivo), true);
                if (is_array($cache)) {
                        return $cache;
                }
        }
        $url = 'https://earthquake.usgs.gov/fdsnws/event/1/query?' . http_build_query(self::ZONA_SISMOS + [
                'format' => 'geojson', 'minmagnitude' => 2.5, 'orderby' => 'time', 'limit' => 500,
                'starttime' => gmdate('Y-m-d\TH:i:s', time() - $dias * 86400),
        ]);
        $ch = curl_init($url);
        curl_setopt_array($ch, [CURLOPT_RETURNTRANSFER => true, CURLOPT_TIMEOUT => 12, CURLOPT_CONNECTTIMEOUT => 6, CURLOPT_USERAGENT => 'SISMO-panel']);
        $cuerpo = curl_exec($ch);
        $codigo = (int) curl_getinfo($ch, CURLINFO_HTTP_CODE);
        curl_close($ch);
        $geo = $codigo === 200 ? json_decode((string) $cuerpo, true) : null;
        if (!is_array($geo) || !isset($geo['features']) || !is_array($geo['features'])) {
                // Sin respuesta: se devuelve la última copia aunque esté vencida.
                $viejo = is_file($archivo) ? json_decode((string) file_get_contents($archivo), true) : null;
                return is_array($viejo) ? $viejo : null;
        }
        $lista = [];
        foreach ($geo['features'] as $f) {
                // Los datos externos se validan antes de usarse.
                $p = isset($f['properties']) ? $f['properties'] : null;
                $c = isset($f['geometry']['coordinates']) ? $f['geometry']['coordinates'] : null;
                if (!is_array($p) || !is_array($c) || count($c) < 3 || !is_numeric($p['mag']) || !is_numeric($p['time'])) {
                        continue;
                }
                $enlace = isset($p['url']) && is_string($p['url']) && strpos($p['url'], 'https://earthquake.usgs.gov/') === 0 ? $p['url'] : '';
                $lista[] = [
                        'id'           => is_string($f['id']) ? mb_substr($f['id'], 0, 40) : '',
                        'magnitud'     => round((float) $p['mag'], 1),
                        'lugar'        => is_string($p['place']) ? mb_substr($p['place'], 0, 200) : '',
                        'fecha'        => gmdate('Y-m-d H:i:s', (int) floor($p['time'] / 1000)),
                        'latitud'      => round((float) $c[1], 4),
                        'longitud'     => round((float) $c[0], 4),
                        'profundidad'  => round((float) $c[2], 1),
                        'sentido'      => isset($p['felt']) && is_numeric($p['felt']) ? (int) $p['felt'] : null,
                        'url'          => $enlace,
                ];
        }
        @file_put_contents($archivo, json_encode($lista), LOCK_EX);
        return $lista;
}
```

**Descripción.** Sismo_model.php: consulta los sismos recientes de Colombia y su zona en el servicio del USGS, valida cada dato recibido y guarda una caché de 5 minutos para no saturar el servicio.

Fuente: elaboración propia (captura del 04/11/2026).

**Figura 133**
*Código del backend (v3): panel endpoints*

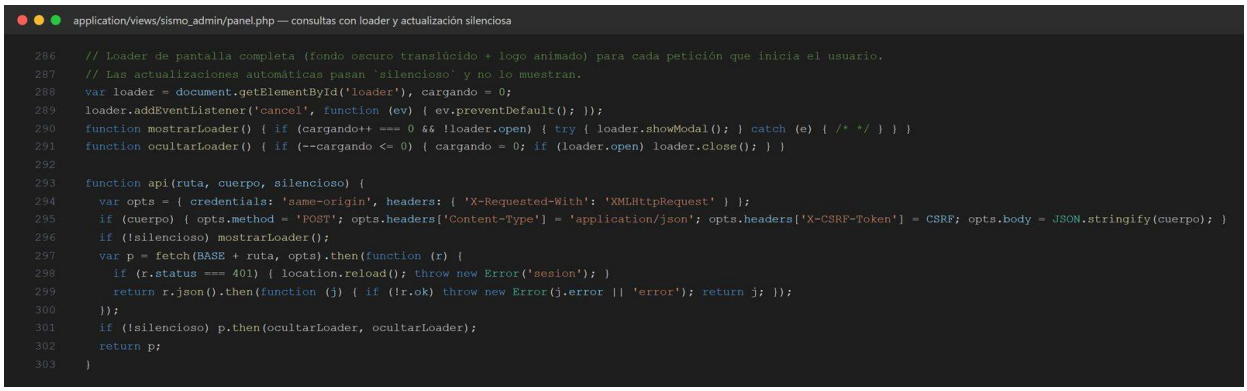
application/controllers/Sismo_admin.php — panel: datos, persona, ayuda y código

```php
/** GET: datos generales para el panel (consulta periódica). */
public function datos() {
        $this->responder_json(200, [
                'resumen'  => $this->Sismo_model->resumen(),
                'activas'  => $this->Sismo_model->alertas_panel(true, 200),
                'alertas'  => $this->Sismo_model->alertas_panel(false, 150),
                'usuarios' => $this->Sismo_model->usuarios_panel(500),
                'ahora'    => gmdate('Y-m-d H:i:s'),
        ]);
}

/** GET ?dias=1|7: sismos de las últimas 24 horas o de los últimos 7 días (M2.5+) en Colombia y alrededores (USGS). */
public function sismos() {
        $dias   = (int) $this->input->get('dias') === 1 ? 1 : 7;
        $sismos = $this->Sismo_model->sismos_recientes($dias);
        if ($sismos === null) {
                return $this->responder_json(502, ['error' => 'usgs_no_disponible']);
        }
        $this->responder_json(200, ['dias' => $dias, 'sismos' => $sismos]);
}

/** GET: detalle completo de un usuario. Queda registrado quién lo consultó (incluye datos de salud). */
public function usuario($id = null) {
        $usuario = ctype_digit((string) $id) ? $this->Sismo_model->usuario_por_id((int) $id) : null;
        if (!$usuario) {
                return $this->responder_json(404, ['error' => 'usuario_no_encontrado']);
        }
        $this->log('consulta_usuario', ['id_usuario' => (int) $usuario->id]);
        $this->responder_json(200, [
                'usuario' => [
                        'id'                        => (int) $usuario->id,
                        'nombre'                    => $usuario->nombre,
                        'plataforma'                => $usuario->plataforma,
                        'created_at'                => $usuario->created_at,
                        'fecha_ultima_sincronizacion' => $usuario->fecha_ultima_sincronizacion,
                        'tiene_codigo'              => $usuario->hash_codigo_recuperacion !== null,
                        'fecha_expiracion_codigo'   => $usuario->fecha_expiracion_codigo,
                ],
                'estado'     => $this->Sismo_model->estado($usuario),
                'atenciones' => $this->Sismo_model->atenciones_usuario($usuario->id),
        ]);
}

/** POST: registra la ayuda brindada a una alerta. */
public function atender() {
        $cuerpo = $this->post_json();
        if ($cuerpo === null) {
                return;
        }
        $alerta = isset($cuerpo['id_alerta']) && is_int($cuerpo['id_alerta']) ? $this->Sismo_model->alerta_por_id($cuerpo['id_alerta']) : null;
        $estado = isset($cuerpo['estado']) ? $cuerpo['estado'] : null;
        $nota   = isset($cuerpo['nota']) && is_string($cuerpo['nota']) ? trim(strip_tags($cuerpo['nota'])) : '';
        if (!$alerta) {
                return $this->responder_json(404, ['error' => 'alerta_no_encontrada']);
        }
        if (!in_array($estado, self::ESTADOS_ATENCION, true) || mb_strlen($nota) > 500) {
                return $this->responder_json(422, ['error' => 'dato_invalido']);
        }
        $persona = $this->sesion_persona->persona();
        $ok = $this->Sismo_model->registrar_atencion($alerta->id, $persona->id, $estado, $nota === '' ? null : $nota, $this->sesion_persona->nombre());
        $this->log('atencion', ['id_alerta' => (int) $alerta->id, 'estado' => $estado]);
        $this->responder_json($ok ? 200 : 500, ['ok' => (bool) $ok]);
}

/** POST: código de recuperación temporal para que la persona recupere sus datos en otro teléfono. */
public function codigo() {
        $cuerpo = $this->post_json();
        if ($cuerpo === null) {
                return;
        }
        $usuario = isset($cuerpo['id_usuario']) && is_int($cuerpo['id_usuario']) ? $this->Sismo_model->usuario_por_id($cuerpo['id_usuario']) : null;
        if (!$usuario) {
                return $this->responder_json(404, ['error' => 'usuario_no_encontrado']);
        }
        $this->config->load('sismo', true, true);
        $horas  = (int) ($this->config->item('sismo_horas_codigo_panel', 'sismo') ?: 48);
        $codigo = $this->Sismo_model->generar_codigo_recuperacion($usuario->id, $horas, $this->sesion_persona->nombre());
        $this->log('codigo_recuperacion', ['id_usuario' => (int) $usuario->id]);
        $this->responder_json(201, ['codigo' => $codigo, 'horas' => $horas]);
}
```

**Descripción.** application/controllers/Sismo_admin.php: endpoints del panel (datos generales, sismos y detalle de la persona); cada consulta del detalle queda registrada en el log.

Fuente: elaboración propia (captura del 04/11/2026).

**Figura 134**
*Código del backend (v3): panel tiempo real y loader*




**Descripción.** Vista del panel (panel.php): función api que hace las peticiones con el token CSRF y muestra el loader solo en las consultas iniciadas por el operador; las actualizaciones automáticas son silenciosas.

Fuente: elaboración propia (captura del 04/11/2026).

## G.4 App, versión 3.2 (05/11/2026)

**Figura 135**
*Código de la app (v3.2): limpiador texto enriquecido*

src/utils/richText.ts — lista blanca del mensaje del centro de monitoreo (sin DOM)

```ts
const ALLOWED: Record<string, string> = {
  p: 'p', br: 'br', strong: 'strong', b: 'strong', em: 'em', i: 'em', u: 'u', s: 's', strike: 's', del: 's', ul: 'ul', ol: 'ol', li: 'li',
};
const DANGEROUS = /<(script|style|iframe|object|embed|svg|math|template|noscript|textarea|title|select|button)\b[^>]*>[\s\S]*?<\/\1\s*>/gi;
const HAS_TAG = /<\/?[a-z][^>]*>/i;
const TOKEN = /<!--[\s\S]*?-->|<\/?([a-z][a-z0-9]*)\b[^>]*>/gi;

const escapeText = (text: string) => text.replace(/</g, '&lt;').replace(/>/g, '&gt;');
const escapeAll = (text: string) => text.replace(/&/g, '&amp;').replace(/</g, '&lt;').replace(/>/g, '&gt;').replace(/"/g, '&quot;');

/** HTML seguro del mensaje. Un texto sin etiquetas (mensajes anteriores) se muestra como texto: un párrafo por línea. */
export function sanitizeRichText(input: string | null | undefined): string {
  const raw = String(input ?? '');
  if (!HAS_TAG.test(raw)) {
    return raw.split(/\r?\n/).map((line) => line.trim()).filter(Boolean).map((line) => `<p>${escapeAll(line)}</p>`).join('');
  }
  const cleaned = raw.replace(DANGEROUS, '');
  let out = '';
  let last = 0;
  for (const match of cleaned.matchAll(TOKEN)) {
    out += escapeText(cleaned.slice(last, match.index));
    last = (match.index ?? 0) + match[0].length;
    const name = match[1]?.toLowerCase();
    const tag = name ? ALLOWED[name] : undefined;
    if (!tag) continue; // comentarios y etiquetas no permitidas: fuera (su texto queda)
    if (tag === 'br') out += '<br>';
    else out += match[0].startsWith('</') ? `</${tag}>` : `<${tag}>`;
  }
  out += escapeText(cleaned.slice(last));
  return out.replace(/ /g, ' ');
}

/** Texto visible (sin etiquetas) del mensaje, p. ej. para lectores de pantalla o comparar cambios. */
export function richTextToPlain(input: string | null | undefined): string {
  return sanitizeRichText(input)
    .replace(/<br>|<\/p>|<\/li>/g, ' ')
    .replace(/<[^>]+>/g, '')
    .replace(/&lt;/g, '<').replace(/&gt;/g, '>').replace(/&quot;/g, '"').replace(/&#0?39;/g, "'").replace(/&amp;/g, '&')
    .replace(/\s+/g, ' ')
    .trim();
}
```

**Descripción.** src/utils/richText.ts: limpia el mensaje con formato que envía el centro de monitoreo con una lista blanca de etiquetas y elimina scripts y cualquier otra etiqueta peligrosa (protección contra XSS).

Fuente: elaboración propia (captura del 05/11/2026).

**Figura 136**
*Código de la app (v3.2): mensaje con formato*

src/components/RichText.tsx — el mensaje se ve con el mismo formato del editor

```tsx
/**
 * Muestra un mensaje del centro de monitoreo con el mismo formato del editor del panel (negrilla, cursiva,
 * subrayado, tachado y listas). El HTML pasa siempre por la lista blanca de `sanitizeRichText`.
 */
export function RichText({ html, className = '' }: { html: string; className?: string }) {
  const safe = useMemo(() => sanitizeRichText(html), [html]);
  if (!safe) return null;
  // HTML limpio con lista blanca estricta (solo p, br, strong, em, u, s, ul, ol, li, sin atributos).
  return <div className={`rich-text ${className}`.trim()} dangerouslySetInnerHTML={{ __html: safe }} />;
}
```

**Descripción.** src/components/RichText.tsx: muestra el mensaje del operador con el mismo formato del editor del panel (negrita, cursiva y listas), siempre después de pasarlo por la limpieza.

Fuente: elaboración propia (captura del 05/11/2026).

**Figura 137**
*Código de la app (v3.2): botón ayuda urgente*

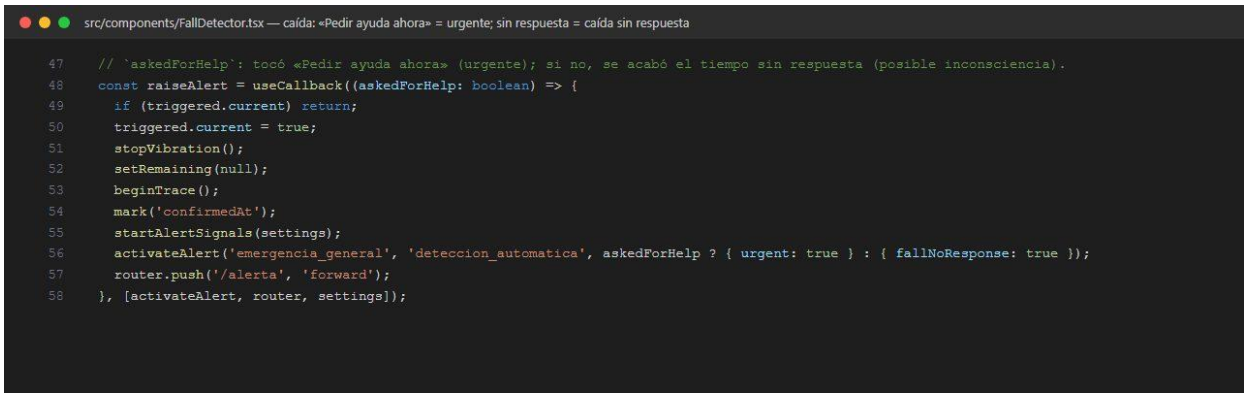
src/pages/AlertPage.tsx — modo atrapado, batería y «Necesito ayuda urgente» viajan con la alerta

```tsx
  // Prioridad para el centro de monitoreo: modo atrapado y batería viajan con la alerta (el panel las muestra).
  const activeId = activeAlert?.id;
  useEffect(() => {
    if (activeId) updateActiveAlert({ trapped });
  }, [activeId, trapped, updateActiveAlert]);
  const savedBattery = activeAlert?.batteryLevel;
  useEffect(() => {
    if (!activeId) return;
    const read = async () => {
      const level = await getBatteryLevel();
      // Solo cuando cambia el porcentaje (no en cada lectura), para no sincronizar de más.
      if (level != null && level !== savedBattery) updateActiveAlert({ batteryLevel: level, batteryAt: new Date().toISOString() });
    };
    void read();
    const t = window.setInterval(() => void read(), BATTERY_CHECK_MS);
    return () => window.clearInterval(t);
  }, [activeId, savedBattery, updateActiveAlert]);

  const askUrgentHelp = (urgent: boolean) => {
    updateActiveAlert(urgent ? { urgent: true, urgentAt: new Date().toISOString() } : { urgent: false });
    if (urgent) presentToast({ message: ta.urgent.toast, duration: 3500, color: 'danger', position: 'top' });
  };
```

**Descripción.** AlertPage.tsx: el modo atrapado, la batería y el botón «Necesito ayuda urgente» se guardan en la alerta y viajan con ella para que el panel los muestre.

Fuente: elaboración propia (captura del 05/11/2026).

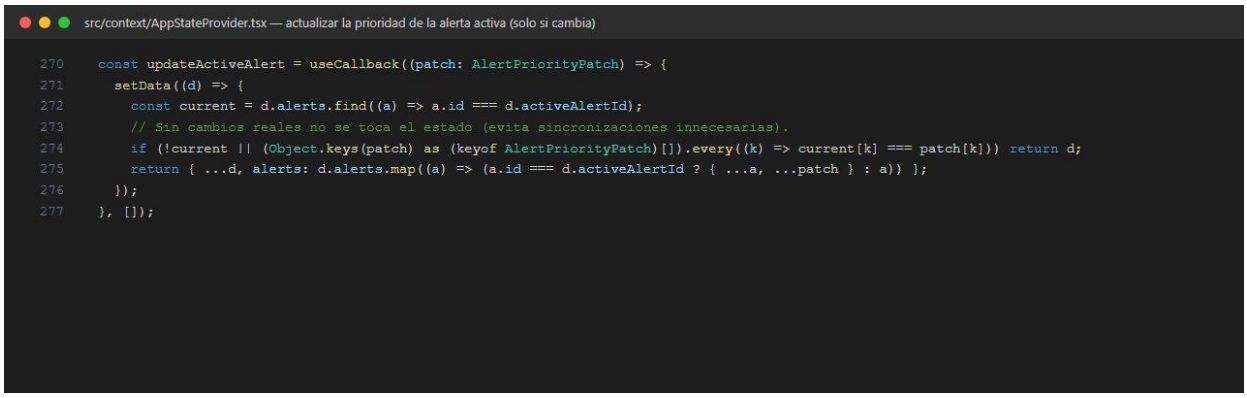
**Figura 138**
*Código de la app (v3.2): caída urgente o sin respuesta*




**Descripción.** src/components/FallDetector.tsx: si la persona toca «Pedir ayuda ahora», la alerta se marca como urgente; si se acaba el tiempo sin respuesta, como caída sin respuesta.

Fuente: elaboración propia (captura del 05/11/2026).

**Figura 139**
*Código de la app (v3.2): prioridad en el estado*




**Descripción.** AppStateProvider.tsx: actualiza la prioridad de la alerta activa solo si algún dato cambió, para no generar sincronizaciones innecesarias.

Fuente: elaboración propia (captura del 05/11/2026).

**Figura 140**
*Código de la app (v3.2): aviso alerta activa*

src/components/ActiveAlertBanner.tsx — aviso fijo de alerta activa en cualquier pantalla

```tsx
/**
 * Aviso fijo arriba de cualquier pantalla mientras hay una alerta SOS activa (la alerta sigue corriendo aunque la
 * persona se mueva a Ubicación, Configuración, etc.). Tocarlo vuelve a la pantalla de alerta. No se muestra en ella.
 */
export function ActiveAlertBanner() {
  const { activeAlert } = useAppState();
  const t = useT().alert.activeBanner;
  const router = useIonRouter();
  const { pathname } = useLocation();
  // Refresca el tiempo transcurrido cada 15 s mientras se ve.
  const [, tick] = useState(0);
  const visible = !!activeAlert && pathname !== '/alerta';
  useEffect(() => {
    if (!visible) return;
    const id = window.setInterval(() => tick((n) => n + 1), 15_000);
    return () => window.clearInterval(id);
  }, [visible]);
  if (!activeAlert || pathname === '/alerta') return null;
  return (
    <button type="button" className={`active-alert-banner${activeAlert.isDrill ? ' active-alert-banner--drill' : ''}`}
      onClick={() => router.push('/alerta', 'forward')} aria-label={`${activeAlert.isDrill ? t.drill : t.title}. ${t.action}`}>
      <Siren size={20} aria-hidden="true" className="active-alert-banner__icon" />
      <span className="active-alert-banner__text">
        <strong>{activeAlert.isDrill ? t.drill : t.title}</strong>
        <small>{t.since(formatDuration(activeAlert.activatedAt))} · {t.action}</small>
      </span>
      <ChevronRight size={20} aria-hidden="true" />
    </button>
  );
}
```

**Descripción.** src/components/ActiveAlertBanner.tsx: aviso fijo de alerta activa en cualquier pantalla con el tiempo transcurrido; al tocarlo vuelve a la pantalla de alerta.

Fuente: elaboración propia (captura del 05/11/2026).

**Figura 141**
*Código de la app (v3.2): inicio que avisa que falta*

src/utils/homeAttention.ts — amarillo (pendiente) y rojo (vencido) en las tarjetas de Inicio

```ts
/**
 * Qué le falta a la persona para estar preparada, para resaltar las tarjetas de Inicio:
 * - 'pendiente' (amarillo): algo sin hacer (mochila vacía, sin ficha médica, sin simulacro, sin punto de encuentro,
 *   «Después del sismo» sin revisar tras un sismo).
 * - 'urgente' (rojo): algo de la mochila vencido o por vencer.
 * Se calcula con los datos del teléfono (los mismos que se sincronizan con la base de datos): funciona sin internet.
 */
export type Attention = 'pendiente' | 'urgente' | null;

export interface HomeAttention {
  kit: { level: Attention; expired: KitItem[]; expiring: KitItem[]; empty: boolean };
  medical: Attention;
  drill: Attention;
  compass: Attention;
  /** Última alerta real de sismo finalizada cuya lista «Después del sismo» quedó incompleta. */
  postQuake: { level: Attention; alert: AlertRecord | null; done: number; total: number };
}

export function homeAttention(data: AppData, postQuakeTotal: number, today: Date = new Date()): HomeAttention {
  const kit = kitStatus(data.kit, today);
  const kitEmpty = !data.kit || kit.checked === 0;
  const kitLevel: Attention = kit.expired.length || kit.expiringSoon.length ? 'urgente' : kitEmpty || kit.reviewDue ? 'pendiente' : null;

  const real = data.alerts.filter((a) => !a.isSeed);
  const didDrill = real.some((a) => a.isDrill);
  const lastQuake = real
    .filter((a) => !a.isDrill && a.eventType === 'sismo' && a.status === 'finalizada')
    .sort((a, b) => (b.endedAt ?? b.activatedAt).localeCompare(a.endedAt ?? a.activatedAt))[0] ?? null;
  const done = lastQuake?.postQuakeDone?.length ?? 0;
  const pendingPostQuake = !!lastQuake && postQuakeTotal > 0 && done < postQuakeTotal;

  return {
    kit: { level: kitLevel, expired: kit.expired, expiring: kit.expiringSoon, empty: kitEmpty },
    medical: data.medical ? null : 'pendiente',
    drill: didDrill ? null : 'pendiente',
    compass: data.meetingPoints.length ? null : 'pendiente',
    postQuake: { level: pendingPostQuake ? 'pendiente' : null, alert: pendingPostQuake ? lastQuake : null, done, total: postQuakeTotal },
  };
}
```

**Descripción.** src/utils/homeAttention.ts: calcula qué le falta a la persona para estar preparada (mochila, ficha médica, simulacro, punto de encuentro y lista «Después del sismo») y si va en amarillo o en rojo.

Fuente: elaboración propia (captura del 05/11/2026).

**Figura 142**
*Código de la app (v3.2): gráfica de picos cálculo*

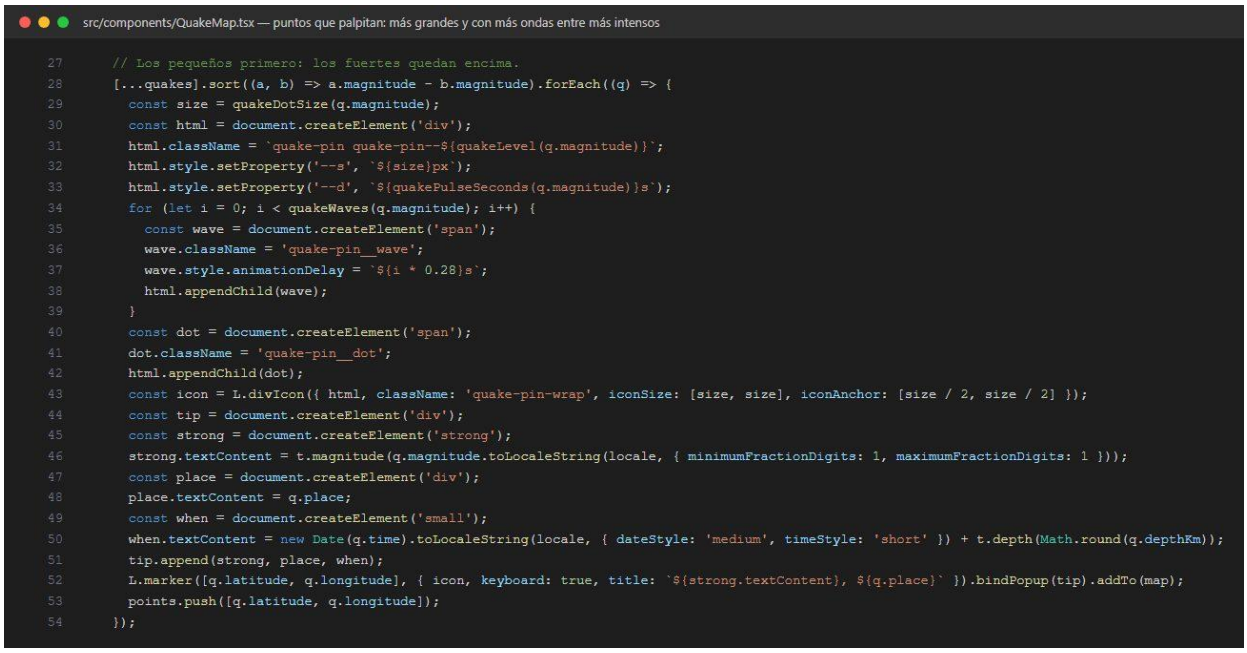
src/utils/quakeViz.ts — intensidad, tamaño del punto, ondas y posiciones de la gráfica de picos

```ts
export type QuakeLevel = 'low' | 'mid' | 'high';

export const quakeLevel = (magnitude: number): QuakeLevel => (magnitude >= 5 ? 'high' : magnitude >= 4 ? 'mid' : 'low');

/** Diámetro del punto en el mapa (px): crece con la magnitud, entre 12 y 46. */
export const quakeDotSize = (magnitude: number): number => Math.round(Math.min(46, Math.max(12, 12 + (magnitude - 2.5) * 8)));

/** Ondas que lanza el punto en cada ráfaga: 1 (baja), 2 (media) o 3 (alta). */
export const quakeWaves = (magnitude: number): number => ({ low: 1, mid: 2, high: 3 })[quakeLevel(magnitude)];

/** Duración (s) de la ráfaga: los más fuertes laten más rápido. */
export const quakePulseSeconds = (magnitude: number): number => ({ low: 3.2, mid: 2.6, high: 2 })[quakeLevel(magnitude)];

export interface SpikeInput {
  id: string;
  magnitude: number;
  time: number;
}

export interface Spike extends SpikeInput {
  x: number;
  y: number;
  level: QuakeLevel;
}

export interface SpikeLayout {
  spikes: Spike[];
  /** Magnitudes enteras para las líneas guía del eje vertical. */
  yTicks: { value: number; y: number }[];
  /** Marcas del eje de tiempo (cada 6 h en 24 h, cada día en 7 días). */
  xTicks: { time: number; x: number }[];
  baseline: number;
  strongest: Spike | null;
}

/**
 * Posiciones de la gráfica de picos: eje x = tiempo del periodo (de `from` a `to`), eje y = magnitud desde 2 hasta
 * el entero superior a la máxima (mínimo 6). Un solo eje, la base en la magnitud 2.
 */
export function layoutSpikes(
  quakes: SpikeInput[],
  opts: { from: number; to: number; width: number; height: number; padLeft: number; padRight: number; padTop: number; padBottom: number },
): SpikeLayout {
  const { from, to, width, height, padLeft, padRight, padTop, padBottom } = opts;
  const yMin = 2;
  const yMax = Math.max(6, Math.ceil(Math.max(0, ...quakes.map((q) => q.magnitude)) + 0.01));
  const plotW = width - padLeft - padRight;
  const plotH = height - padTop - padBottom;
  const baseline = padTop + plotH;
  const xOf = (t: number) => padLeft + (Math.min(Math.max(t, from), to) - from) / Math.max(1, to - from) * plotW;
  const yOf = (m: number) => baseline - (Math.min(Math.max(m, yMin), yMax) - yMin) / (yMax - yMin) * plotH;
  const spikes = [...quakes]
    .sort((a, b) => a.time - b.time)
    .map((q) => ({ ...q, x: xOf(q.time), y: yOf(q.magnitude), level: quakeLevel(q.magnitude) }));
  const yTicks = [];
  for (let v = yMin; v <= yMax; v++) yTicks.push({ value: v, y: yOf(v) });
  const day = 86_400_000;
  const step = to - from > 2 * day ? day : 6 * 3_600_000;
  const xTicks = [];
  for (let t = Math.ceil(from / step) * step; t <= to; t += step) xTicks.push({ time: t, x: xOf(t) });
  const strongest = spikes.reduce<Spike | null>((best, s) => (!best || s.magnitude > best.magnitude ? s : best), null);
  return { spikes, yTicks, xTicks, baseline, strongest };
}
```

**Descripción.** src/utils/quakeViz.ts: cálculo de la gráfica de picos y del mapa; según la magnitud define el nivel, el tamaño del punto, el número de ondas y su velocidad.

Fuente: elaboración propia (captura del 05/11/2026).

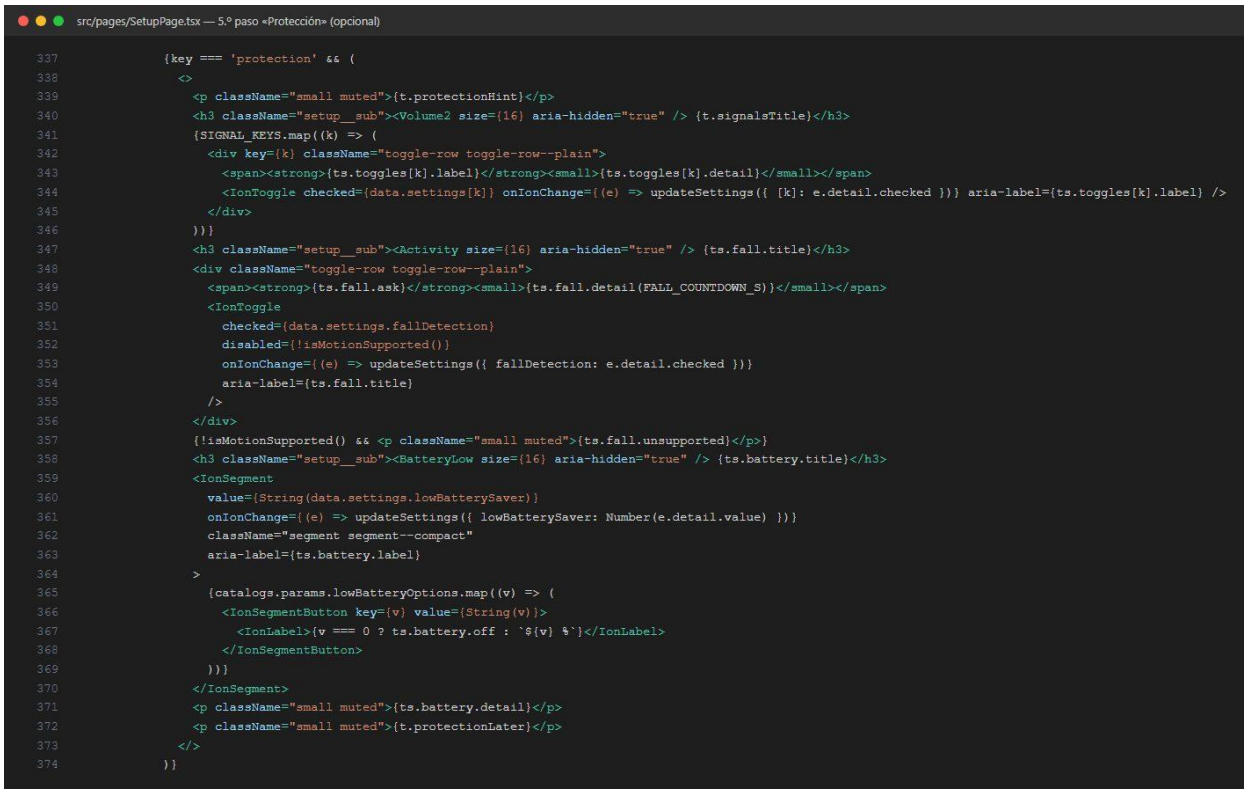
**Figura 143**
*Código de la app (v3.2): mapa de sismos con ondas*




**Descripción.** src/components/QuakeMap.tsx: dibuja cada sismo en el mapa como un punto que palpita, más grande y con más ondas entre más intenso, con una ventana de detalle al tocarlo.

Fuente: elaboración propia (captura del 05/11/2026).

**Figura 144**
*Código de la app (v3.2): registro paso protección*




**Descripción.** SetupPage.tsx: paso 5 del registro, «Protección» (opcional), con interruptores para las señales, la detección de caídas y el umbral de ahorro de batería.

Fuente: elaboración propia (captura del 05/11/2026).

## G.5 Backend, versión 3.2 (05/11/2026)

**Figura 145**
*Código del backend (v3.2): limpieza HTML servidor*

application/libraries/Html_seguro.php — lista blanca del texto enriquecido en el servidor

```php
class Html_seguro {

  const PERMITIDAS = ['p' => 'p', 'br' => 'br', 'strong' => 'strong', 'b' => 'strong', 'em' => 'em', 'i' => 'em',
          'u' => 'u', 's' => 's', 'strike' => 's', 'del' => 's', 'ul' => 'ul', 'ol' => 'ol', 'li' => 'li'];
  const PELIGROSAS = ['script', 'style', 'iframe', 'object', 'embed', 'svg', 'math', 'template', 'noscript', 'head', 'title', 'textarea', 'select', 'button', 'form', 'img', 'video', 'audio', 'link', 'meta'];

  /**
   * Limpia el HTML y mide el texto visible.
   *
   * @param string $entrada HTML del editor (o texto plano)
   * @return array ['html' => string limpio ('' si no hay texto), 'texto' => int caracteres visibles]
   */
  public function limpiar($entrada) {
          $entrada = str_replace(["\r\n", "\r", "\0"], ["\n", "\n", ''], (string) $entrada);
          if (!preg_match('/<\/?[a-z][^>]*>/i', $entrada)) {
                  $lineas = array_values(array_filter(array_map('trim', explode("\n", $entrada)), 'strlen'));
                  $html   = implode('', array_map(function ($l) { return '<p>' . htmlspecialchars($l, ENT_QUOTES, 'UTF-8') . '</p>'; }, $lineas));
                  return ['html' => $html, 'texto' => mb_strlen(implode(' ', $lineas))];
          }
          $dom = new DOMDocument('1.0', 'UTF-8');
          $previo = libxml_use_internal_errors(true);
          $dom->loadHTML('<?xml encoding="utf-8"?><div id="raiz">' . $entrada . '</div>', LIBXML_NONET | LIBXML_HTML_NOIMPLIED | LIBXML_HTML_NODEFDTD);
          libxml_clear_errors();
          libxml_use_internal_errors($previo);
          $raiz = $dom->getElementById('raiz');
          if (!$raiz) {
                  return ['html' => '', 'texto' => 0];
          }
          $html  = $this->hijos($raiz);
          $texto = trim(preg_replace('/\s+/u', ' ', html_entity_decode(strip_tags(str_replace(['<br>', '</p>', '</li>'], ' ', $html)), ENT_QUOTES, 'UTF-8')));
          return ['html' => $texto === '' ? '' : $html, 'texto' => mb_strlen($texto)];
  }

  /** Serializa los hijos de un nodo aplicando la lista blanca. */
  private function hijos(DOMNode $nodo) {
          $salida = '';
          foreach ($nodo->childNodes as $h) {
                  if ($h->nodeType === XML_TEXT_NODE || $h->nodeType === XML_CDATA_SECTION_NODE) {
                          $salida .= htmlspecialchars(str_replace("\xC2\xA0", ' ', $h->nodeValue), ENT_QUOTES, 'UTF-8');
                          continue;
                  }
                  if ($h->nodeType !== XML_ELEMENT_NODE) {
                          continue; // comentarios, instrucciones de proceso
                  }
                  $etiqueta = strtolower($h->nodeName);
                  if (in_array($etiqueta, self::PELIGROSAS, true)) {
                          continue;
                  }
                  if (!isset(self::PERMITIDAS[$etiqueta])) {
                          $salida .= $this->hijos($h);
                          continue;
                  }
                  $t = self::PERMITIDAS[$etiqueta];
                  $salida .= $t === 'br' ? '<br>' : '<' . $t . '>' . $this->hijos($h) . '</' . $t . '>';
          }
          return $salida;
  }
}
```

**Descripción.** application/libraries/Html_seguro.php: limpia en el servidor el HTML de los mensajes con una lista blanca de etiquetas y sin atributos, antes de guardarlo.

Fuente: elaboración propia (captura del 05/11/2026).

**Figura 146**
*Código del backend (v3.2): tabla plantillas*

docs/superpowers/sql/2026-11-04-sismo-plantillas-atenciones.sql — plantillas y mensajes VARCHAR(3000)

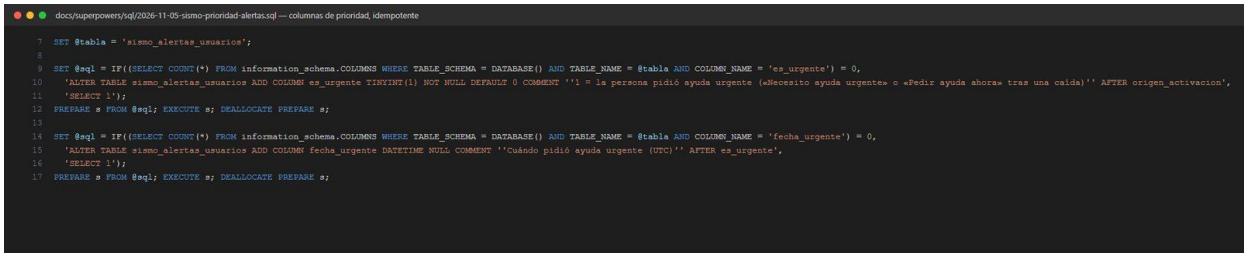
```sql
CREATE TABLE IF NOT EXISTS sismo_plantillas_atenciones (
id          INT UNSIGNED AUTO_INCREMENT PRIMARY KEY COMMENT 'Identificador de la plantilla',
estado      VARCHAR(15)   NOT NULL COMMENT 'general (sirve para cualquier estado) | en_atencion | ayuda_enviada | resuelta',
titulo      VARCHAR(80)   NOT NULL COMMENT 'Nombre corto que ve el operador en la lista',
mensaje     VARCHAR(3000) NOT NULL COMMENT 'Mensaje en HTML limpio (p, br, strong, em, u, s, ul, ol, li; máx. 500 caracteres visibles); admite comodines como [nombre]',
orden       SMALLINT      NOT NULL DEFAULT 0 COMMENT 'Orden dentro del estado (menor primero)',
activo      TINYINT(1)    NOT NULL DEFAULT 1 COMMENT '1=disponible en «Brindar ayuda», 0=oculta (único soft delete)',
created_at  DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT 'Fecha de creación',
created_by  VARCHAR(120)  NOT NULL COMMENT 'Nombre de quien creó el registro',
updated_at  DATETIME      NULL ON UPDATE CURRENT_TIMESTAMP COMMENT 'Fecha de última modificación',
updated_by  VARCHAR(120)  NULL     COMMENT 'Nombre de quien modificó por última vez',
UNIQUE KEY uq_sismo_plantillas_estado_titulo (estado, titulo),
KEY ix_sismo_plantillas_estado (estado, activo, orden)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='Plantillas administrables del mensaje que el operador envía al brindar ayuda';

ALTER TABLE sismo_plantillas_atenciones
MODIFY mensaje VARCHAR(3000) NOT NULL COMMENT 'Mensaje en HTML limpio (p, br, strong, em, u, s, ul, ol, li; máx. 500 caracteres visibles); admite comodines como [nombre]';
ALTER TABLE sismo_atenciones_alertas
MODIFY nota VARCHAR(3000) NULL COMMENT 'Mensaje para la persona en HTML limpio (p, br, strong, em, u, s, ul, ol, li; máx. 500 caracteres visibles) / bitácora del operador';
```

**Descripción.** Script SQL de la tabla sismo_plantillas_atenciones (plantillas de «Brindar ayuda»

por estado) y ampliación de los campos de mensaje para admitir texto con formato.

Fuente: elaboración propia (captura del 05/11/2026).

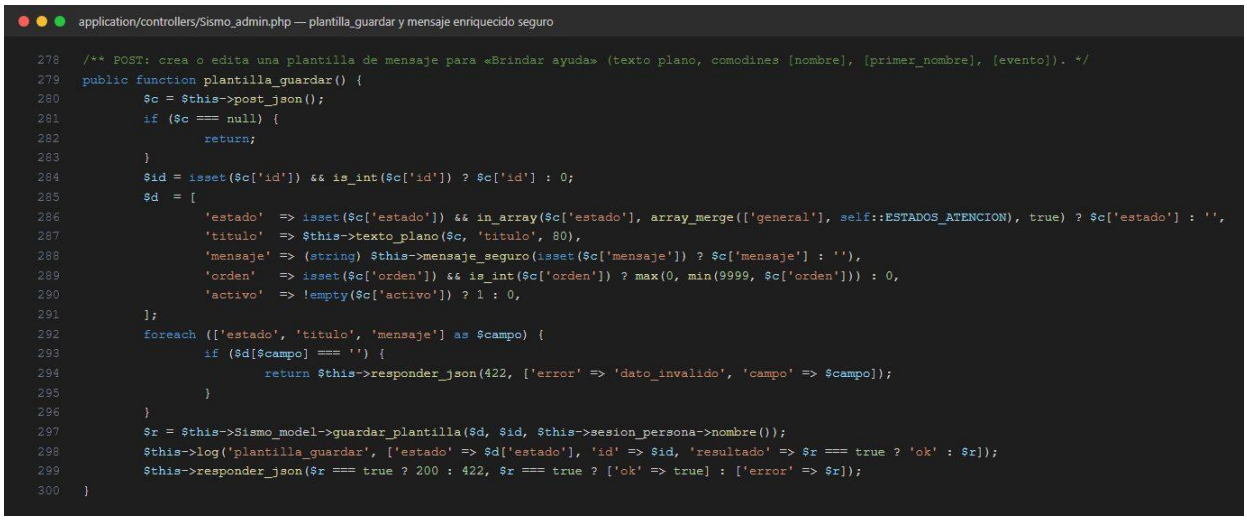
**Figura 147**
*Código del backend (v3.2): columnas de prioridad*




**Descripción.** Script SQL idempotente que agrega a sismo_alertas_usuarios las columnas de prioridad (ayuda urgente, caída sin respuesta, modo atrapado y batería) solo si aún no existen.

Fuente: elaboración propia (captura del 05/11/2026).

**Figura 148**
*Código del backend (v3.2): guardar plantilla y mensaje*




**Descripción.** Sismo_admin.php, plantilla_guardar: valida el estado, el título y el mensaje (limpiado en el servidor) antes de crear o editar una plantilla, y registra la acción en el log.

Fuente: elaboración propia (captura del 05/11/2026).

**Figura 149**
*Código del backend (v3.2): aPI prioridad de la alerta*

application/controllers/Sismo.php — la API recibe urgente, caída, modo atrapado y batería

```php
    * Datos de prioridad de la alerta para el panel: ayuda urgente, caída sin respuesta, modo atrapado y batería.
    * Si aún no se ejecutó docs/superpowers/sql/2026-11-05-sismo-prioridad-alertas.sql, se omiten (la app sigue
    * sincronizando sin error).
    *
    * @param array $c cuerpo de la alerta
    * @return array columnas => valores validados
    */
   private function prioridad_alerta(array $c) {
           if (!$this->Sismo_model->tiene_prioridad_alertas()) {
                   return [];
           }
           return [
                   'es_urgente'          => isset($c['urgent']) ? $this->booleano($c, 'urgent') : 0,
                   'fecha_urgente'       => isset($c['urgentAt']) ? $this->fecha($c['urgentAt'], 'urgentAt') : null,
                   'caida_sin_respuesta' => isset($c['fallNoResponse']) ? $this->booleano($c, 'fallNoResponse') : 0,
                   'modo_atrapado'       => isset($c['trapped']) ? $this->booleano($c, 'trapped') : 0,
                   'nivel_bateria'       => isset($c['batteryLevel']) ? $this->entero($c, 'batteryLevel', 0, 100) : null,
                   'fecha_bateria'       => isset($c['batteryAt']) ? $this->fecha($c['batteryAt'], 'batteryAt') : null,
           ];
   }
```

**Descripción.** Sismo.php, prioridad_alerta: recibe y valida los datos de prioridad de la alerta enviados por la app (ayuda urgente, caída sin respuesta, modo atrapado y nivel de batería).

Fuente: elaboración propia (captura del 05/11/2026).

**Figura 150**
*Código del backend (v3.2): pulso del panel*

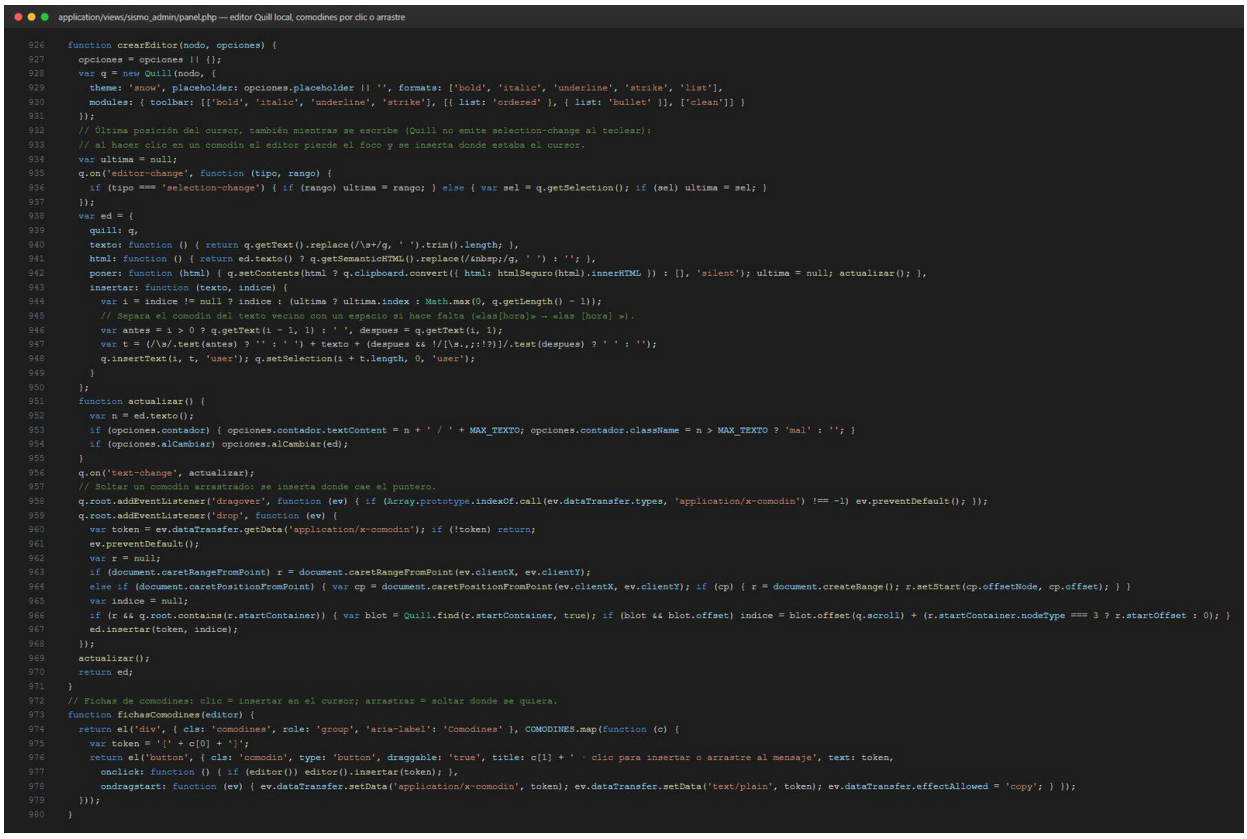
application/models/Sismo_model.php — huella liviana para el pulso de 3 s del panel

```php
   * Huella liviana del panel: cambia si llega o termina una alerta, si se actualiza su ubicación o si se registra
   * una atención. Una sola consulta con agregados (sin traer filas).
   *
   * @return string
   */
  public function firma_panel() {
          $f = $this->db->query(
                  'SELECT
                          (SELECT CONCAT_WS(\'|\', COUNT(*), MAX(id), MAX(COALESCE(updated_at, created_at))) FROM sismo_alertas_usuarios WHERE activo = 1 AND estado = ?) AS activas,
                          (SELECT CONCAT_WS(\'|\', COUNT(*), MAX(id), MAX(COALESCE(updated_at, created_at))) FROM sismo_alertas_usuarios) AS alertas,
                          (SELECT CONCAT_WS(\'|\', COUNT(*), MAX(id)) FROM sismo_atenciones_alertas) AS atenciones,
                          (SELECT CONCAT_WS(\'|\', COUNT(*), MAX(id)) FROM sismo_usuarios WHERE activo = 1) AS usuarios',
                  ['activa']
          )->row_array();
          return substr(md5(implode('#', (array) $f)), 0, 16);
  }

  /**
```

**Descripción.** Sismo_model.php, firma_panel: calcula una huella liviana del estado del panel con una sola consulta de agregados, para saber si algo cambió sin traer todos los datos.

Fuente: elaboración propia (captura del 05/11/2026).

**Figura 151**
*Código del backend (v3.2): editor y comodines*

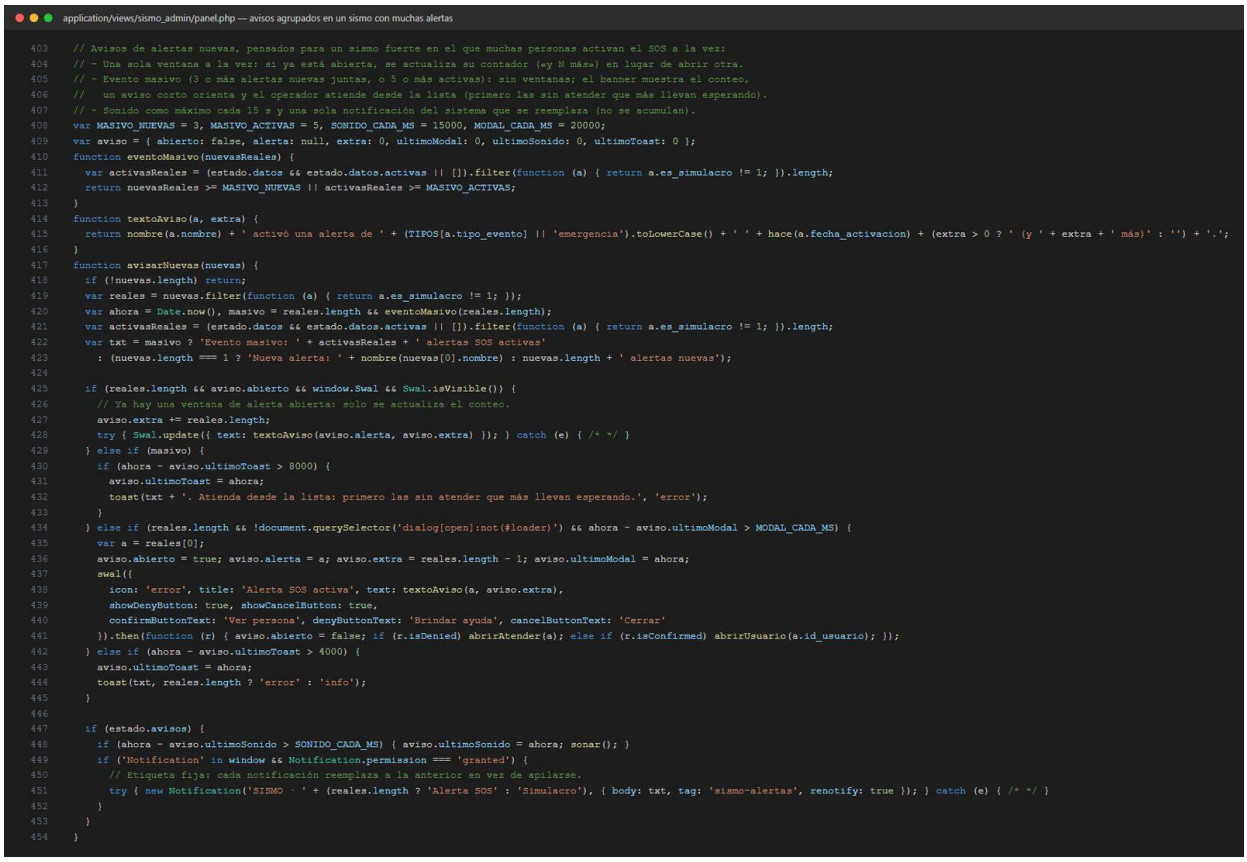


```
application/views/sismo_admin/panel.php — editor Quill local, comodines por clic o arrastre

926  function crearEditor(nodo, opciones) {
927    opciones = opciones || {};
928    var q = new Quill(nodo, {
929      theme: 'snow', placeholder: opciones.placeholder || '', formats: ['bold', 'italic', 'underline', 'strike', 'list'],
930      modules: { toolbar: [['bold', 'italic', 'underline', 'strike'], [{ list: 'ordered' }, { list: 'bullet' }], ['clean']] }
931    });
932    // Última posición del cursor, también mientras se escribe (Quill no emite selection-change al teclear):
933    // al hacer clic en un comodín el editor pierde el foco y se inserta donde estaba el cursor.
934    var ultima = null;
935    q.on('editor-change', function (tipo, rango) {
936      if (tipo === 'selection-change') { if (rango) ultima = rango; } else { var sel = q.getSelection(); if (sel) ultima = sel; }
937    });
938    var ed = {
939      quill: q,
940      texto: function () { return q.getText().replace(/\s+/g, ' ').trim().length; },
941      html: function () { return ed.texto() ? q.getSemanticHTML().replace(/ /g, ' ') : ''; },
942      poner: function (html) { q.setContents(html ? q.clipboard.convert({ html: htmlSeguro(html).innerHTML }) : [], 'silent'); ultima = null; actualizar(); },
943      insertar: function (texto, indice) {
944        var i = indice != null ? indice : (ultima ? ultima.index : Math.max(0, q.getLength() - 1));
945        // Separa el comodín del texto vecino con un espacio si hace falta («las[hora]» → «las [hora] »).
946        var antes = i > 0 ? q.getText(i - 1, 1) : ' ', despues = q.getText(i, 1);
947        var t = (/\s/.test(antes) ? '' : ' ') + texto + (despues && !/[\s.,;:!?)]/.test(despues) ? ' ' : '');
948        q.insertText(i, t, 'user'); q.setSelection(i + t.length, 0, 'user');
949      }
950    };
951    function actualizar() {
952      var n = ed.texto();
953      if (opciones.contador) { opciones.contador.textContent = n + ' / ' + MAX_TEXTO; opciones.contador.className = n > MAX_TEXTO ? 'mal' : ''; }
954      if (opciones.alCambiar) opciones.alCambiar(ed);
955    }
956    q.on('text-change', actualizar);
957    // Soltar un comodín arrastrado: se inserta donde cae el puntero.
958    q.root.addEventListener('dragover', function (ev) { if (Array.prototype.indexOf.call(ev.dataTransfer.types, 'application/x-comodin') !== -1) ev.preventDefault(); });
959    q.root.addEventListener('drop', function (ev) {
960      var token = ev.dataTransfer.getData('application/x-comodin'); if (!token) return;
961      ev.preventDefault();
962      var r = null;
963      if (document.caretRangeFromPoint) r = document.caretRangeFromPoint(ev.clientX, ev.clientY);
964      else if (document.caretPositionFromPoint) { var cp = document.caretPositionFromPoint(ev.clientX, ev.clientY); if (cp) { r = document.createRange(); r.setStart(cp.offsetNode, cp.offset); } }
965      var indice = null;
966      if (r && q.root.contains(r.startContainer)) { var blot = Quill.find(r.startContainer, true); if (blot && blot.offset) indice = blot.offset(q.scroll) + (r.startContainer.nodeType === 3 ? r.startOffset : 0); }
967      ed.insertar(token, indice);
968    });
969    actualizar();
970    return ed;
971  }
972  // Fichas de comodines: clic = insertar en el cursor; arrastrar = soltar donde se quiera.
973  function fichasComodines(editor) {
974    return el('div', { cls: 'comodines', role: 'group', 'aria-label': 'Comodines' }, COMODINES.map(function (c) {
975      var token = '[' + c[0] + ']';
976      return el('button', { cls: 'comodin', type: 'button', draggable: 'true', title: c[1] + ' · clic para insertar o arrastre al mensaje', text: token,
977        onclick: function () { if (editor()) editor().insertar(token); },
978        ondragstart: function (ev) { ev.dataTransfer.setData('application/x-comodin', token); ev.dataTransfer.setData('text/plain', token); ev.dataTransfer.effectAllowed = 'copy'; } });
979    }));
980  }
```

**Descripción.** Vista del panel: editor de texto enriquecido Quill servido desde el servidor (sin CDN), con comodines que se insertan con un clic o arrastrándolos al mensaje.

Fuente: elaboración propia (captura del 05/11/2026).

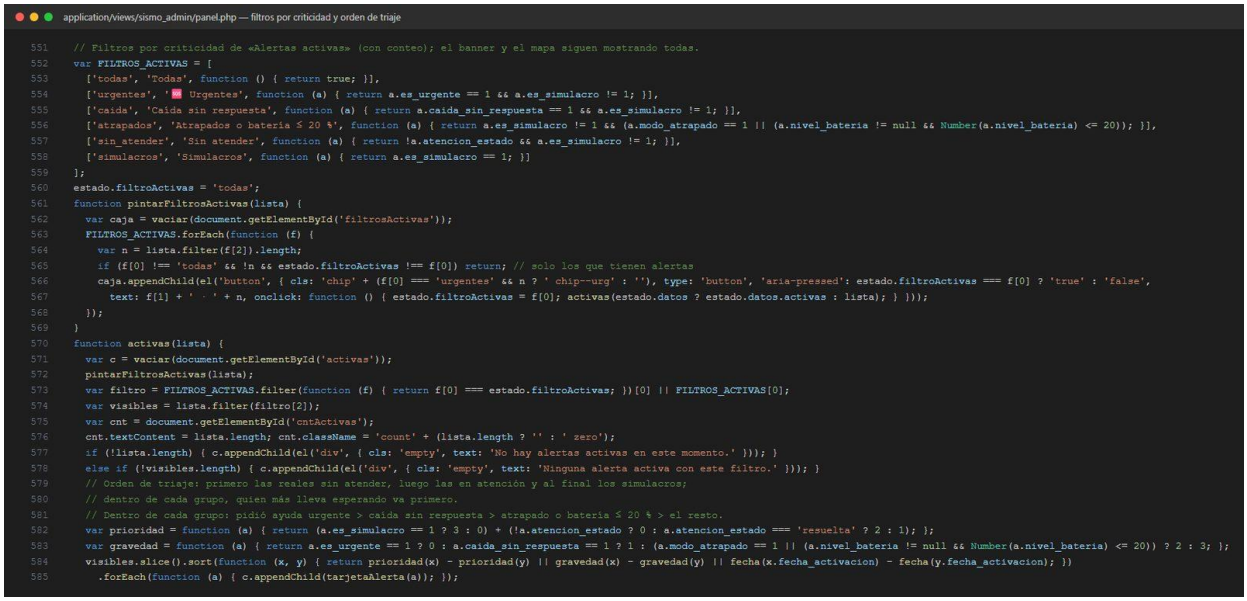
**Figura 152**
*Código del backend (v3.2): evento masivo*




**Descripción.** Vista del panel: evento masivo; cuando llegan muchas alertas a la vez agrupa los avisos en una sola franja, limita el sonido y las notificaciones y ordena las alertas por triaje.

Fuente: elaboración propia (captura del 05/11/2026).

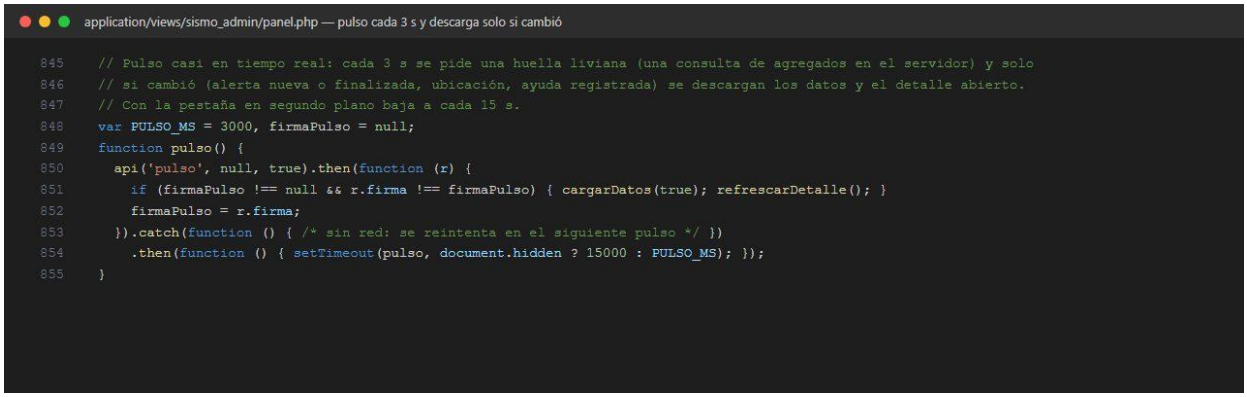
**Figura 153**
*Código del backend (v3.2): prioridad y filtros*




**Descripción.** Vista del panel: filtros por criticidad (urgentes, caída sin respuesta, atrapados, batería baja y sin atender) y orden de las alertas por prioridad y gravedad.

Fuente: elaboración propia (captura del 05/11/2026).

**Figura 154**
*Código del backend (v3.2): pulso casi tiempo real*




**Descripción.** Vista del panel: pulso cada 3 segundos que compara la huella del servidor y solo descarga los datos cuando algo cambió; con la pestaña en segundo plano baja a cada 15 segundos.

Fuente: elaboración propia (captura del 05/11/2026).